ImageSharp

ImageSharp is the high-performance part of the Six Labors stack you reach for when you need to load, inspect, process, and save images entirely in managed .NET code. It gives you one consistent image model whether you are building a thumbnail service, a photo workflow, a web upload pipeline, or a lower-level imaging tool.

This section is written as a guided set of articles rather than a flat feature list. Start with Getting Started if you are new to the library, then branch into loading, processing, formats, or lower-level pixel work as your needs get more specific.

The core model is: choose an image type, load or create pixels, run ordered processing operations, then save with an explicit encoder when output behavior matters. `Image` is convenient when you do not need direct pixel access, while `Image<TPixel>` makes the in-memory pixel format part of the type so high-performance row processing and format-specific work stay explicit.

For production code, the important choices are usually not individual method names. They are whether to identify before decoding, which pixel format to use in memory, how much metadata to preserve, which encoder settings define acceptable output, and when to customize configuration for formats, memory, or security.

License

ImageSharp is licensed under the terms of the Six Labors Split License, Version 1.0. See https://sixlabors.com/pricing for commercial licensing details.

> **IMPORTANT**
>
> Projects that directly reference ImageSharp require a valid Six Labors license key at build time from version 3.2.0 in the previous major and 4.0.0 in the current major. Enforcement applies to direct dependencies only. This is enforcement of the existing license terms, not a license change.

Install ImageSharp

ImageSharp is distributed on NuGet with preview and nightly builds available on Feedz.

Package Manager | .NET CLI | PackageReference | Paket CLI

```
PM > Install-Package SixLabors.ImageSharp -Version VERSION_NUMBER
```

Prerelease versions installed via the Visual Studio NuGet Package Manager require the "include prerelease" checkbox to be checked.

How to use the license file

By default, the build searches from each project directory for `sixlabors.lic`. Place the supplied file in the directory that contains the project file, or in a subdirectory below it. Use the file as supplied; it contains the complete license string required by the build.

> **IMPORTANT**
>
> Assembly-scoped community keys can be committed to a public repository. Contributors can use the included key to build the named assemblies without obtaining their own keys. Unrestricted community keys and commercial keys must remain private. Use environment variables or repository secrets for those keys.

To keep the file elsewhere, including a repository root above the project directory, set `SixLaborsLicenseFile` in your project file or a shared props file:

```
<PropertyGroup>
  <!-- Set the location of the supplied license file. -->
  <SixLaborsLicenseFile>path/to/sixlabors.lic</SixLaborsLicenseFile>
</PropertyGroup>
```

To use an environment variable instead, set this project property:

```
<PropertyGroup>
  <!-- Read the complete license string from the environment. -->
  <SixLaborsLicenseKey>$(SIXLABORS_LICENSE_KEY)</SixLaborsLicenseKey>
</PropertyGroup>
```

Set `SIXLABORS_LICENSE_KEY` to the complete contents of your issued license, not only its `Key` field. Use the original value without escaping. Replace the entire example value below with your issued license contents.

PowerShell:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
$env:SIXLABORS_LICENSE_KEY = 'Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

Bash:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
export SIXLABORS_LICENSE_KEY='Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

If you pass the license directly through `-p:SixLaborsLicenseKey`, escape every semicolon as `%253B`. MSBuild treats semicolons as property separators even inside a quoted argument. The pipe between assembly signatures stays unchanged.

PowerShell:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
dotnet publish "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
```

Bash:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
dotnet publish "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
```

You do not need to reference the licensing package directly. Projects that directly reference a Six Labors package carry its build-time validation.

Community keys for public projects

For an open-source or source-available application, provide the exact `AssemblyName` of every project that directly references a Six Labors package, including test projects. Names are case-sensitive.

For example, if `MyProject.csproj` sets `<AssemblyName>MyProject.Core</AssemblyName>`, enter `MyProject.Core`. If it does not set `AssemblyName`, enter `MyProject`, the project filename without `.csproj`. If a shared props file sets the name, use that value. Do not include `.dll` or `.csproj`.

Existing community license holders can request one early replacement with an assembly-scoped key. These upgrade requests are approved automatically, without another eligibility review. You do not need to wait for your current key to enter its renewal window.

Apply at licensing.sixlabors.com. Select open-source or source-available, use the same email address, and provide the assembly names. Later requests follow the normal renewal window: the final 90 days before the expiry date in your license. Commercial licenses are not eligible for this early replacement.

Security support

The latest major version of each library is eligible for security updates. Each major remains eligible for 12 months after the first stable release of its successor. A later major release does not shorten an existing support window.

Install the latest available patch or minor release within a supported major to receive security fixes. Older releases within that major are not maintained separately. This policy covers security fixes only. It does not include feature backports or extend Microsoft's support for the underlying .NET version. Security fixes remain at Six Labors' discretion.

Start Here

- Getting Started walks through the core image types and the first end-to-end processing workflow.
- Loading, Identifying, and Saving covers file, stream, and buffer-based APIs plus encoder selection.
- Working with Metadata explains how to read and preserve EXIF, ICC, IPTC, XMP, and format-specific metadata.
- Color Profiles and Color Conversion covers ICC and CICP metadata, decode-time profile handling, and explicit working-space conversion.
- Image Formats explains format detection, encoders, decoders, and format registration.
- Processing Images introduces `Mutate()` and `Clone()` pipelines.
- Quantization, Palettes, and Dithering explains `Quantize()`, palette-based encoders, and dithering tradeoffs.
- Pixel Formats and Working with Pixel Buffers cover direct pixel access and advanced processing.

- Interop and Raw Memory covers `LoadPixelData(...)`, `WrapMemory(...)`, and contiguous-buffer interop.
- Configuration, Memory Management, and Security Considerations cover production-focused setup.
- Troubleshooting covers the common failure modes around format detection, streams, memory, and disposal.
- Migrating from System.Drawing maps common GDI-style workflows to ImageSharp APIs.
- Migrating from SkiaSharp maps common SkiaSharp image workflows to ImageSharp APIs.
- Recipes provides copy-pasteable solutions for common tasks.

Implicit Usings

Set `UseImageSharp` in your project file to automatically import the most common ImageSharp namespaces:

```
<PropertyGroup>
  <UseImageSharp>true</UseImageSharp>
</PropertyGroup>
```

When enabled, ImageSharp adds implicit `global using` directives for:

- `SixLabors.ImageSharp`
- `SixLabors.ImageSharp.PixelFormats`
- `SixLabors.ImageSharp.Processing`

You can turn this off by removing the property or setting it to `false`.

How to Use These Docs

- Start with loading, identifying, processing, and saving if you are new to ImageSharp.
- Move to formats, metadata, color profiles, and security before accepting untrusted images in production.
- Use pixel-buffer and interop pages when you need direct memory access rather than normal processors.
- Read the migration pages when replacing APIs whose image model differs from ImageSharp's typed pixel model.

Getting Started

ImageSharp is easiest to learn if you think in terms of a simple flow: load or identify an image, optionally process it, then save it in the format you need. This page introduces the handful of types that show up most often in that flow so the rest of the docs feel familiar more quickly.

The main types you will run into first are:

- `Image` is the format-agnostic image container used by the main loading, processing, and saving APIs.
- `Image<TPixel>` is the generic image container to use when you know the pixel format and want direct pixel access. See Pixel Formats for more detail.
- `ImageFrame` and `ImageFrame<TPixel>` represent individual frames in multi-frame images such as GIF and WebP.
- `ImageInfo` gives you dimensions, pixel information, and metadata without fully decoding the image.

Load, Process, and Save an Image

The most common ImageSharp workflow is to load an image, apply a processing pipeline, and save it again:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Resize(800, 600));

image.Save("output.jpg");
```

This example shows the core workflow:

- `Image.Load()` detects the input format from the image data.
- `Mutate()` applies processors to the current image in order.
- `Save()` picks an encoder from the output path unless you pass one explicitly.

For more detail, see Loading, Identifying, and Saving, Processing Images, and Image Formats.

Read Image Information Without Decoding Pixels

If you only need image dimensions, pixel information, or metadata, use `Image.Identify()` instead of `Image.Load()`:

```
using SixLabors.ImageSharp;

ImageInfo imageInfo = Image.Identify("input.jpg");

Console.WriteLine($"Width: {imageInfo.Width}");
Console.WriteLine($"Height: {imageInfo.Height}");
Console.WriteLine($"Frames: {imageInfo.FrameCount}");
```

This is usually much faster and allocates less memory because the full pixel buffer is never materialized.

Create a New Image

You can also create images directly in memory:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = new(640, 480, Color.White.ToPixel<Rgba32>());
```

Use `Image<TPixel>` when the pixel format matters to your workflow, for example when you need direct access to pixel rows or want to interoperate with a known buffer format.

Mutate or Clone?

ImageSharp exposes two primary processing entry points:

- `Mutate()` changes the current image in place.
- `Clone()` creates a deep copy and applies the processors to that copy.

Use `Mutate()` when you want to transform the current image, and `Clone()` when you need to keep the original unchanged.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");
using Image thumbnail = image.Clone(x => x.Resize(160, 160));
```

Dispose Images Promptly

ImageSharp images own pooled memory buffers and should be disposed as soon as you are done with them. Prefer `using` declarations or `using` blocks around `Image` and `Image<TPixel>` instances.

See Memory Management for production guidance around pooling, contiguous buffers, and diagnostics.

Next Steps

- Loading, Identifying, and Saving
- Working with Metadata
- Processing Images
- Pixel Formats
- Working with Pixel Buffers

Loading, Identifying, and Saving

Most ImageSharp applications start here. Whether images come from disk, streams, or upload buffers, the same load, identify, and save model applies, which makes it easy to move from a quick sample to a production pipeline without relearning the API surface.

The core idea is straightforward: use `Image.Load()` when you need pixels, `Image.Identify()` when you only need dimensions or metadata, and `Image.DetectFormat()` when you only need to know what kind of file you were given.

The Three Questions

Loading APIs answer different questions:

- `DetectFormat(...)` asks which registered detector recognizes the encoded bytes.
- `Identify(...)` asks what the decoder can learn from the headers and metadata without allocating the full pixel buffer.
- `Load(...)` asks the decoder to materialize pixel data into an `Image` or `Image<TPixel>`.

Those operations are intentionally separate. A file extension can be wrong, a header can be recognizable while the image data is still corrupt, and a small compressed file can decode into a large pixel buffer. Production code should choose the cheapest operation that answers the current question, then still handle failure at the later decode boundary.

Load Images

You can load images from a file path, a stream, or an in-memory byte buffer:

```
using SixLabors.ImageSharp;

using Image fromFile = Image.Load("input.webp");

using FileStream stream = File.OpenRead("input.webp");
using Image fromStream = Image.Load(stream);

byte[] buffer = File.ReadAllBytes("input.webp");
using Image fromBytes = Image.Load(buffer);
```

All of these overloads inspect the image data to determine which decoder to use.

If you know the target pixel format you want in memory, use the generic overloads such as `Image.Load<Rgba32>()`.

Use Async APIs for I/O-Bound Work

ImageSharp also exposes async load and save methods for file and stream based workflows:

```
using SixLabors.ImageSharp;

await using FileStream input = File.OpenRead("input.png");
using Image image = await Image.LoadAsync(input);

await image.SaveAsync("output.webp");
```

Use the async overloads when your application already uses asynchronous I/O, for example in ASP.NET Core or background processing pipelines.

Identify Without Decoding Pixel Data

Use `Image.Identify()` when you only need dimensions, pixel information, metadata, or a quick decoded memory estimate:

```
using SixLabors.ImageSharp;

ImageInfo imageInfo = Image.Identify("input.jpg");

Console.WriteLine($"{imageInfo.Width}x{imageInfo.Height}");
Console.WriteLine($"Bits per pixel: {imageInfo.PixelType.BitsPerPixel}");
Console.WriteLine($"Frames: {imageInfo.FrameCount}");
Console.WriteLine($"Estimated pixel memory: {imageInfo.GetPixelMemorySize():N0} bytes");
```

This avoids allocating the full pixel buffer and is usually the right choice for validation, metadata extraction, thumbnail planning, and rejecting images whose decoded pixel budget is too large for your workload.

Detect the Encoded Format

Use `Image.DetectFormat()` when you need to know what encoded format a source contains before loading it:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

IImageFormat format = Image.DetectFormat("input.bin");

Console.WriteLine(format.Name);
```

This is useful when files arrive without a trustworthy extension or when you want to route work based on the encoded format.

Save Images

When you save by path, ImageSharp selects an encoder from the file extension:

```
using SixLabors.ImageSharp;

using Image image = Image.Load("input.jpg");

image.Save("output.png");
```

If you save by path, ImageSharp already chooses the encoder from the destination file extension. Use `DecodedImageFormat` when you want to explicitly save to the originally decoded format, especially when writing to a stream:

```
using SixLabors.ImageSharp;

using Image image = Image.Load("input.jpg");

if (image.Metadata.DecodedImageFormat is not null)
{
    using FileStream output = File.Create("output.jpg");
    image.Save(output, image.Metadata.DecodedImageFormat);
}
```

`DecodedImageFormat` is only populated for images that were decoded from an existing source. Images created from scratch do not have an original encoded format to preserve.

Preserving the original format is not always the right choice. Choose the output format based on the job: JPEG or WebP for photographic delivery, PNG for lossless graphics or transparency, GIF/APNG/WebP for animations, TIFF or OpenEXR for workflows that need richer image data.

Choose Encoders Explicitly

When you need control over output settings, pass an encoder directly:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Formats.Png;

using Image image = Image.Load("input.jpg");

image.Save("output.jpg", new JpegEncoder { Quality = 85 });
image.Save("output.png", new PngEncoder());
```

See Image Formats for a deeper look at encoder and decoder behavior.

Control Decoding with DecoderOptions

Use `DecoderOptions` to customize decoding behavior:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

DecoderOptions options = new()
{
    MaxFrames = 1,
    SkipMetadata = true,
    TargetSize = new Size(1200, 1200)
};

using Image image = Image.Load(options, "animated.webp");
```

These options let you limit decoded frames, skip metadata work, or decode directly to a target size when the format supports it.

Use `DecoderOptions` at trust boundaries. For upload validation, background queues, and web requests, it is better to decide frame limits, metadata policy, color-profile handling, and target decode size before allocating a full image.

Practical Guidance

For production code, decide how much information you need before you decode pixels. `DetectFormat(...)` is the cheapest useful step when the only question is "which decoder would handle this?". `Identify(...)` is the better preflight when routing, validation, or policy depends on dimensions, frame count, encoded pixel type, or metadata. `Load(...)` should be the point where you have already decided the image is worth decoding.

Streams must remain open and readable until the load operation completes. In web and queue-based systems, prefer the async overloads so image I/O follows the rest of the application's asynchronous flow. Once the image is decoded, treat it as a significant resource: decoded pixel buffers can be much larger than the source file, especially for high-resolution photos and multi-frame formats.

Saving deserves the same deliberate boundary. Save by extension for quick tools and samples; pass an explicit encoder when output quality, metadata, color profiles, animation settings, or compression tradeoffs are part of the contract. If a file crosses an API boundary, is cached publicly, or is compared in tests, the encoder settings should usually be visible in code.

Related Topics

- Working with Metadata
- Image Formats
- Processing Images

Working with Metadata

Metadata is where ImageSharp stores the information around the pixels: resolution, format details, EXIF, ICC profiles, and other auxiliary data. For newcomers, the key idea is that you can inspect or preserve this information without treating it as part of the pixel-processing pipeline itself.

ImageSharp exposes that data through `ImageMetadata`. You can access it from a fully decoded image through `image.Metadata`, or from `ImageInfo` when using `Image.Identify()`.

Read Metadata from a File

Use `Image.Identify()` when you only need metadata and dimensions:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Metadata;

ImageInfo imageInfo = Image.Identify("photo.jpg");
ImageMetadata metadata = imageInfo.Metadata;

Console.WriteLine(metadata.DecodedImageFormat?.Name);
Console.WriteLine(metadata.HorizontalResolution);
Console.WriteLine(metadata.VerticalResolution);
```

If you are already loading the image for processing, use `image.Metadata` instead.

Common Metadata Profiles

Depending on the source format, `ImageMetadata` can expose several common profiles:

- `ExifProfile` for camera, orientation, and capture metadata.
- `IccProfile` for embedded color profile data.
- `IptcProfile` for editorial and descriptive metadata.
- `XmpProfile` for extensible structured metadata.
- `CicpProfile` for coding-independent code points metadata when present.

These profile properties are nullable because not every image carries every kind of metadata.

Those profiles serve different purposes. EXIF often contains camera settings, timestamps, orientation, thumbnails, and sometimes GPS data. ICC profiles describe how color values should

be interpreted. CICP metadata can carry color coding information used by some modern image and video workflows. IPTC and XMP often contain editorial, rights, authoring, and workflow data.

That means metadata policy is not simply "keep" or "strip." A public thumbnail service may want to apply orientation and remove personal data. A print or archival pipeline may need to preserve color profiles and selected descriptive metadata. A conversion tool may need to translate what the destination format can represent and drop what it cannot.

Work with Format-Specific Metadata

In addition to the common profiles, ImageSharp exposes format-specific metadata helpers:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Formats.Png;

using Image image = Image.Load("photo.jpg");

JpegMetadata jpegMetadata = image.Metadata.GetJpegMetadata();
PngMetadata pngMetadata = image.Metadata.GetPngMetadata();
```

Similar helpers exist for other built-in formats, including EXR, GIF, TIFF, and WebP.

Access Frame Metadata

Multi-frame formats can also expose per-frame metadata:

```
using SixLabors.ImageSharp;

ImageInfo imageInfo = Image.Identify("animation.webp");

Console.WriteLine($"Frame count: {imageInfo.FrameMetadataCollection.Count}");
```

This is useful when inspecting animated formats without decoding every frame into pixel memory.

Frame metadata matters for animation. Delay, blend mode, disposal mode, frame dimensions, and format-specific values can change how a multi-frame image plays even when the decoded pixels look reasonable in isolation.

Strip Metadata Before Saving

If you do not want to preserve the original metadata, clear the profiles before saving:

```
using SixLabors.ImageSharp;

using Image image = Image.Load("photo.jpg");

image.Metadata.ExifProfile = null;
image.Metadata.IccProfile = null;
image.Metadata.IptcProfile = null;
image.Metadata.XmpProfile = null;
image.Metadata.CicpProfile = null;

image.Save("photo-stripped.jpg");
```

This is a common step when reducing file size, removing personal data, or normalizing exported assets.

Preserve Metadata Intentionally

ImageSharp preserves metadata by default when the decoder and encoder both support that metadata. If metadata is important to your workflow, keep these points in mind:

- `Image.Identify()` lets you inspect metadata without paying for a full decode.
- `DecodedImageFormat` tells you which encoded format was originally loaded.
- Saving to a different format may change which metadata can be represented in the output.

For deeper guidance on loading and saving workflows, see Loading, Identifying, and Saving. For ICC and CICP-specific guidance, see Color Profiles and Color Conversion.

Practical Guidance

Inspect metadata before decoding pixels when routing or validation only needs headers and profiles. Preserve ICC or CICP data when color interpretation matters, or convert to a known output profile before stripping it. Apply `AutoOrient()` before removing EXIF orientation if the visual orientation must remain correct. Treat metadata as user data in privacy-sensitive workflows; EXIF, IPTC, and XMP can contain identifying information.

Color Profiles and Color Conversion

Color management in ImageSharp has two related but separate parts:

- Embedded color metadata, such as `IccProfile` and `CicpProfile`, which describes how encoded color values should be interpreted.
- Color conversion APIs in `SixLabors.ImageSharp.ColorProfiles`, which convert color values between supported color spaces and profiles.

Preserving a profile is not the same thing as converting pixels. A profile can remain attached to an image as metadata without changing the decoded pixel values, and a conversion can change pixel values without preserving the original profile in the output file.

Most applications only need the first part: let ImageSharp decode the image and choose whether to preserve, compact, or convert embedded ICC profile data at the decode boundary. `ColorProfileConverter` is a lower-level API for code that is explicitly working with color values or custom color pipelines.

What ICC Profiles Do

An ICC profile describes the color space of encoded image data. The same numeric pixel value can represent different visible colors depending on the profile attached to the file. A pixel value that looks correct as sRGB may look too saturated, too dull, or shifted if it is interpreted as Adobe RGB, ProPhoto RGB, a display profile, or a printer profile without conversion.

That means an ICC profile is not just descriptive trivia. It tells color-managed software how to interpret the numbers in the file and, when needed, how to convert those numbers into another color space while preserving the intended appearance.

There are two common outcomes:

- Preserve the profile and pixel values so another color-managed tool can interpret the image later.
- Convert the pixels into a known working space, usually sRGB, so the rest of your pipeline can treat loaded images consistently.

What ImageSharp Does

By default, `DecoderOptions` preserves embedded ICC profiles in metadata and does not transform the decoded pixels.

Use `DecoderOptions.ColorProfileHandling` when your decode boundary needs a different policy:

- `Preserve` leaves embedded ICC profiles intact.
- `Compact` removes embedded standard sRGB ICC profiles without transforming pixels.
- `Convert` transforms pixels from the embedded ICC profile to the sRGB v4 profile and removes the original profile.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

DecoderOptions options = new()
{
    // Convert embedded ICC color data to sRGB during decode.
    ColorProfileHandling = ColorProfileHandling.Convert
};

using Image image = Image.Load(options, "input-with-icc.jpg");
```

`Convert` is useful when you want the rest of your pipeline to operate on normalized sRGB pixel values. `Preserve` is useful when the original profile should stay attached for round-tripping or later export. `Compact` is useful when you want to remove redundant standard sRGB profile data without changing pixel values.

Inspect Embedded Color Metadata

Embedded color metadata is exposed through `ImageMetadata`:

```
using SixLabors.ImageSharp;

using Image image = Image.Load("input.jpg");

// ICC and CICP are metadata profiles; reading them does not convert pixels.
if (image.Metadata.IccProfile is not null)
{
    Console.WriteLine(image.Metadata.IccProfile.Header.ProfileConnectionSpace);
    Console.WriteLine(image.Metadata.IccProfile.Header.RenderingIntent);
}

if (image.Metadata.CicpProfile is not null)
{
    Console.WriteLine(image.Metadata.CicpProfile.ColorPrimaries);
    Console.WriteLine(image.Metadata.CicpProfile.TransferCharacteristics);
    Console.WriteLine(image.Metadata.CicpProfile.MatrixCoefficients);
```

```
    Console.WriteLine(image.Metadata.CicpProfile.FullRange);
}
```

`IccProfile` is the richer general-purpose profile format used by many image workflows. `CicpProfile` stores coding-independent color signaling values such as primaries, transfer characteristics, matrix coefficients, and range.

Color Profile Types Are Not Pixel Formats

The value types used by `ColorProfileConverter` are not `Image<TPixel>` storage formats.

Types such as `Rgb`, `Cmyk`, `Hsl`, `YCbCr`, `CieLab`, and `CieXyz` model color values for conversion. They are not general-purpose pixel formats and do not implement `IPixel<TSelf>`.

That distinction matters because ImageSharp image processing is built around pixel formats that can move efficiently through RGBA-oriented `Vector4` conversion paths. Color profile types model color spaces and profile connection spaces instead.

Convert Color Values

Use `ColorProfileConverter` to convert individual color values or spans of values:

```
using SixLabors.ImageSharp.ColorProfiles;

ColorProfileConverter converter = new();

Rgb rgb = new(0.25F, 0.5F, 0.75F);
CieLab lab = converter.Convert<Rgb, CieLab>(rgb);
```

The converter works with color-profile value types, not whole images. This is appropriate when your own code is calculating, sampling, comparing, or exporting color values directly.

Convert Between RGB Working Spaces

RGB-to-RGB conversion can still change values when the source and destination working spaces are different. Set the working spaces through `ColorConversionOptions`:

```
using SixLabors.ImageSharp.ColorProfiles;

ColorProfileConverter converter = new(new ColorConversionOptions
{
```

```
    // The same Rgb value type can represent different RGB working spaces.
    SourceRgbWorkingSpace = KnownRgbWorkingSpaces.SRgb,
    TargetRgbWorkingSpace = KnownRgbWorkingSpaces.AdobeRgb1998
});

Rgb source = new(0.25F, 0.5F, 0.75F);
Rgb converted = converter.Convert<Rgb, Rgb>(source);
```

The source and target value types are both `Rgb`, but the working-space definitions are different. This is value-level color conversion, not a change to the in-memory `TPixel` type of an image.

`KnownRgbWorkingSpaces` includes common built-in working spaces such as:

- `SRgb`
- `Rec709`
- `Rec2020`
- `AdobeRgb1998`
- `ProPhotoRgb`
- `WideGamutRgb`

Choose the source and target working spaces explicitly when color values come from a known space outside the normal image decode path.

Convert Using ICC Profiles

When you have actual source and destination ICC profiles, set `SourceIccProfile` and `TargetIccProfile`:

```
using System.IO;
using SixLabors.ImageSharp.ColorProfiles;
using SixLabors.ImageSharp.Metadata.Profiles.Icc;

IccProfile sourceProfile = new(File.ReadAllBytes("source.icc"));
IccProfile targetProfile = new(File.ReadAllBytes("target.icc"));

ColorProfileConverter converter = new(new ColorConversionOptions
{
    // Supplying both ICC profiles selects the ICC-based conversion path.
    SourceIccProfile = sourceProfile,
    TargetIccProfile = targetProfile
});
```

```
Rgb source = new(0.25F, 0.5F, 0.75F);
Rgb converted = converter.Convert<Rgb, Rgb>(source);
```

When both ICC profiles are supplied, the converter uses the ICC conversion path for supported source and destination color value types. Use this path when device-specific or embedded profile data matters more than a generic named working space.

Advanced Conversion Options

`ColorConversionOptions` also exposes lower-level settings for specialized workflows:

- `SourceWhitePoint` and `TargetWhitePoint`
- `AdaptationMatrix`, which defaults to `KnownChromaticAdaptationMatrices.Bradford`
- `YCbCrTransform`, which defaults to `KnownYCbCrMatrices.BT601`

Most application code should leave these defaults alone. Change them only when your color pipeline has an explicit requirement for a different white point, chromatic adaptation matrix, or YCbCr transform.

Practical Guidance

- Preserve embedded ICC metadata when the original profile should remain attached to the image.
- Use decode-time `ColorProfileHandling.Convert` when you want loaded images normalized to sRGB pixel values.
- Use `ColorProfileHandling.Compact` when you want to remove redundant standard sRGB profile data without changing pixels.
- Use `ColorProfileConverter` when you are converting color values in your own code rather than changing file metadata.
- Keep pixel format decisions separate from color profile decisions. `Image<Rgba32>` describes memory layout; ICC and CICP data describe color interpretation.

Related Topics

- Working with Metadata
- Loading, Identifying, and Saving
- Convert Between Formats
- Pixel Formats

Pixel Formats

Pixel formats are one of the places where ImageSharp differs most clearly from older imaging APIs. The pixel type is not an afterthought or a hidden enum value; it is part of the image's actual type, which makes low-level code more explicit and more predictable once the model clicks.

`Image<TPixel>` is generic because the in-memory pixel type is part of the image contract. An `Image<Rgba32>` and an `Image<L8>` can represent the same picture, but they differ in channel layout, precision, memory usage, and what direct pixel access means for your code.

Image memory is usually treated as discontiguous, even though smaller images may fit in a single backing buffer. See Memory Management for more detail on how ImageSharp stores large images efficiently.

For multi-frame images, the individual bitmaps live in `image.Frames` as `ImageFrame<TPixel>` instances.

What Counts as a Pixel Format

A pixel format in ImageSharp is not just any color-related struct. To be used as `TPixel`, a type must implement `IPixel<TSelf>`.

That contract includes conversion members such as:

- `ToRgba32()`
- `ToScaledVector4()`
- `ToVector4()`
- `FromScaledVector4(...)`
- `FromVector4(...)`
- conversions to and from canonical pixel types such as `Rgba32`, `Rgb24`, `Bgra32`, `L8`, and `Rgba64`

This is what keeps the image processing pipeline practical. Many operations and batched conversion paths assume pixels can move efficiently through RGBA-oriented `Vector4` representations, and some optimized paths are specifically designed for `Rgba32`-compatible pixel types where `ToVector4()` and `FromVector4(...)` are not expensive.

Pixel Formats Are Not Color Profile Types

This is separate from the color-profile conversion APIs described in Color Profiles and Color Conversion.

Types such as `Rgb`, `Cmyk`, `Hsl`, `YCbCr`, `CieLab`, and `CieXyz` are color value types used by `ColorProfileConverter`. They are not `TPixel` implementations for `Image<TPixel>`.

That means ImageSharp can convert color values between spaces like RGB, CMYK, Lab, and XYZ without treating those color models as general-purpose in-memory image storage formats. ImageSharp pixel formats are intentionally limited to types that fit the RGBA-oriented processing pipeline without expensive per-pixel translation on every operation.

Choosing Pixel Formats

Choose a `TPixel` based on the kind of in-memory work you need to do:

- Use `Rgba32` as the general-purpose default.
- Use lower-memory formats such as `Rgb24` or `L8` when you know you do not need the extra channels or precision.
- Use higher-precision formats such as `Rgb48`, `Rgba64`, `RgbaHalf`, or `RgbaVector` when your pipeline benefits from more precision.

For application code, a good default rule is to decode into the format you plan to process in, not necessarily the format the file used on disk. A JPEG may decode naturally into RGB-like data, but `Image<Rgba32>` can still be the right working type if the next step composites with alpha, draws overlays, or passes pixels to an API that expects RGBA. Conversely, `Image<L8>` can be the right type for masks, analysis, and grayscale-only processing where carrying color channels would just waste memory.

Do not use a higher-precision format only because the source file is high quality. Use it when the operations you run benefit from the extra range or precision, such as repeated color transforms, scientific-style image data, high-bit-depth exports, or workflows where banding from 8-bit intermediate values would be visible.

If you want to inspect pixel characteristics before a full decode, `ImageInfo.PixelType` exposes `PixelTypeInfo`. See Read Image Info Without Decoding for more on that workflow.

Associated Alpha Pixel Formats

Most built-in pixel formats store straight alpha, where the color components are independent of the alpha value. ImageSharp also provides formats that store associated alpha, often called premultiplied alpha, where each color component is already multiplied by its alpha value.

The associated formats carry a `P` suffix:

- `Rgba32P`, `Bgra32P`, `Argb32P`, and `Abgr32P` mirror their straight-alpha counterparts with 8-bit components.
- `RgbaHalfP` and `HalfVector4P` store associated 16-bit floating-point components.
- `NormalizedByte4P` stores associated 8-bit signed normalized components.

Associated alpha is the working representation of most GPU pipelines and compositors. Use these formats when you exchange pixel memory with such an API, so the data crosses the boundary without a conversion pass. `RgbaHalf` and `RgbaHalfP` are binary-compatible with the `DXGI_FORMAT_R16G16B16A16_FLOAT` GPU format.

Pixel blending understands both alpha representations. When an operation blends pixels of an associated format, ImageSharp uses dedicated associated-alpha blend functions instead of converting through straight alpha.

Defining Custom Pixel Formats

You can define a custom pixel format by creating a struct that implements `IPixel<TSelf>` and using it as the generic argument for `Image<TPixel>`.

Baseline batched conversion primitives are provided by `PixelOperations<TPixel>`, and you can override those implementations if you have a more efficient specialization.

In practice, custom `TPixel` types should still fit the same RGBA-compatible conversion model as the built-in formats. Many of the packed and vector-style pixel types are deliberately in the same family as graphics-oriented packed color representations, and `IPackedVector<TPacked>` follows the same packed-value shape used by MonoGame and XNA types, which allows signature compatibility with them.

Single-Bit Monochrome Pixels

ImageSharp does not currently support sub-byte `TPixel` formats such as a true 1-bit pixel type. That trade-off keeps the processing model and API surface much simpler, and it avoids paying a heavy CPU cost across the rest of the pipeline for a niche storage optimization.

Choosing a Working Pixel Format

Use `Image<Rgba32>` as the default when you need predictable direct pixel access and no special memory or precision constraint pushes you elsewhere. It is a practical working format for composition, overlays, custom row processing, and interop with APIs that expect RGBA-like data.

Choose lower-memory formats only when the missing channels or precision are genuinely unnecessary. `L8` is a good fit for masks and grayscale analysis; `Rgb24` can be useful when alpha is not part of the workflow. Choose higher-precision formats because the processing pipeline benefits from them, such as repeated color transforms or high-bit-depth output, not simply because the source file is "high quality".

Pixel format and color profile are related but separate decisions. `Image<Rgba32>` tells you the in-memory channel layout and numeric representation; ICC and CICP handling tell you how color values should be interpreted or converted. A robust pipeline chooses both deliberately.

Image Formats

ImageSharp keeps the in-memory image model separate from the file format on disk. That means the same processing code can work across JPEG, PNG, WebP, TIFF, OpenEXR, GIF, and the other built-in codecs, while the encoder and metadata layers handle the format-specific details at the edges.

This page is the format map for the library: which built-in formats ship by default, what each one is good at, and where to go next for format-specific guidance.

Format, Codec, and Pixel Type

These terms refer to different parts of the imaging pipeline:

- A file format is the encoded representation on disk or in a stream, such as JPEG, PNG, WebP, or TIFF.
- A decoder reads encoded data from a file format and produces an `Image` or `Image<TPixel>`.
- An encoder writes an image back to a chosen file format.
- A pixel type, such as `Rgba32`, describes the in-memory representation used while the image is loaded and processed.
- Metadata describes information carried alongside the pixels, such as orientation, ICC profiles, frame timing, comments, and format-specific tags.

Changing the file format is not the same operation as changing the in-memory pixel type. Saving an `Image<Rgba32>` as JPEG writes JPEG-encoded data from RGBA pixels; loading a PNG as `Image<L8>` converts decoded image samples into an 8-bit luminance pixel buffer. Metadata handling is a separate concern again, controlled by decoder and encoder options.

What a Format Decides

An image file format is not only a filename extension. It defines which image information can be represented and how that information is stored. The important questions are:

- Is the encoded pixel data compressed, and is that compression lossy or lossless?
- Which color models, bit depths, and component precisions can the format represent?
- Can the format store alpha transparency, and if so is it full alpha or index-based transparency?
- Can the format store multiple frames, animation timing, blending, and disposal behavior?
- Which metadata can be represented, such as EXIF, ICC profiles, text chunks, frame metadata, or format-specific tags?

- Which applications, browsers, operating systems, and asset pipelines need to read the output?

These questions are why there is no universal "best" image format. JPEG, PNG, GIF, WebP, TIFF, and OpenEXR are not interchangeable containers with different extensions; they preserve and discard different parts of the image model.

Delivery, Interchange, and Working Formats

Many format decisions become clearer when you separate the job the file has to do:

- Delivery formats prioritize compatibility, size, and decode behavior for the consuming client. JPEG, PNG, GIF, and WebP are common examples.
- Interchange formats preserve information for another tool or workflow. TIFF, OpenEXR, TGA, BMP, QOI, and Netpbm-style formats can be useful here depending on the pipeline.
- Working formats are the files you keep before final export. They may be larger or richer than the public output because they need to preserve editability, metadata, precision, layers in another application, or a lossless source for later conversions.

ImageSharp works with raster images. Vector artwork, document formats, and application-native design files are outside the built-in codec set, although you can render or import them through other tools before handing raster pixels to ImageSharp.

Compression and Re-encoding

Lossless formats preserve the decoded pixel values exactly, subject to the color and pixel representation chosen by the encoder. PNG, QOI, lossless WebP, and many TIFF configurations fall into this category.

Lossy formats intentionally discard information to reduce file size. JPEG and lossy WebP are useful because the loss is often acceptable for photographs, but re-encoding lossy inputs can compound artifacts. Converting a JPEG to PNG does not restore detail that the JPEG encoder already removed; it only stores the current decoded pixels losslessly.

Some formats can be either lossy or lossless depending on encoder settings. WebP and TIFF are format families with multiple encoding modes, so the encoder configuration matters as much as the extension.

Built-In Formats

The source of truth for the built-in format list is `Configuration`: the default ImageSharp configuration preregisters encoder, decoder, and detector modules for the following public `IImageFormat` types:

Format	Public API type	Built in by default
ANI	`AniFormat`	Read and write
BMP	`BmpFormat`	Read and write
CUR	`CurFormat`	Read and write
EXR	`ExrFormat`	Read and write
GIF	`GifFormat`	Read and write
ICO	`IcoFormat`	Read and write
JPEG	`JpegFormat`	Read and write
PBM	`PbmFormat`	Read and write
PNG	`PngFormat`	Read and write
QOI	`QoiFormat`	Read and write
TGA	`TgaFormat`	Read and write
TIFF	`TiffFormat`	Read and write
WebP	`WebpFormat`	Read and write

ICO and CUR are distinct built-in formats even though detection is handled by a shared icon detector internally.

At a Glance

If you only need a quick rule of thumb:

- JPEG is the usual choice for photos when small files matter and transparency does not.
- PNG is the usual choice for lossless graphics, screenshots, and transparency.
- GIF is mainly useful for simple palette-based animation and legacy compatibility.
- WebP covers lossy, lossless, transparency, and animation in one format family.
- TIFF is primarily for archival, print, interchange, and imaging-pipeline workflows.

- OpenEXR is the format to consider for HDR and higher-precision imaging pipelines.

Another way to think about it:

- Lossy formats: JPEG, lossy WebP.
- Lossless formats: PNG, lossless WebP, TIFF, QOI, BMP.
- Higher-precision and HDR workflows: OpenEXR and TIFF.
- Transparency-friendly formats: PNG, WebP, TIFF, TGA, QOI.
- Animation-friendly formats: GIF, animated PNG workflows through `PngEncoder`, animated WebP, and ANI for Windows animated cursors.

No single format is best everywhere. The right choice depends on whether your priority is fidelity, file size, transparency, animation, compatibility, or workflow metadata.

When the output crosses a boundary you do not control, compatibility usually outranks theoretical capability. A format can support a feature and still be a poor choice if the receiving client, CDN, print tool, browser, or asset pipeline handles that feature inconsistently.

Load, Detect, and Preserve Formats

`Image<TPixel>` represents decoded pixel data. Once an image is loaded into memory, it is no longer tied to a specific file format unless you explicitly inspect or preserve that information.

ImageSharp can detect the encoded format of a source before loading it with `Image.DetectFormat()`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

IImageFormat format = Image.DetectFormat("input.bin");

Console.WriteLine(format.Name);
```

Decoded images also keep the original format in `ImageMetadata.DecodedImageFormat`.

That metadata is useful when you want to explicitly save back to the originally decoded format, especially when writing to a stream where there is no file extension to select an encoder for you:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");
```

```
image.Mutate(x => x.Resize(1200, 800));

if (image.Metadata.DecodedImageFormat is not null)
{
    using FileStream outputStream = File.Create("output.jpg");
    image.Save(outputStream, image.Metadata.DecodedImageFormat);
}
```

When you save by path, `image.Save("output.jpg")` or `image.Save("output.png")` selects the encoder from the destination file extension.

You can also choose a format explicitly by passing an encoder or by using the `SaveAs...()` helpers.

Save with Explicit Encoders

`ImageEncoder` implementations are lightweight configuration objects. Create one when you want to control how a format is written:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Formats.Png;

using Image image = Image.Load("input.png");

image.Save("output.jpg", new JpegEncoder { Quality = 85 });
image.Save("output.png", new PngEncoder());
```

ImageSharp also provides format-specific helpers:

- `image.SaveAsAni()` uses `AniEncoder`.
- `image.SaveAsBmp()` uses `BmpEncoder`.
- `image.SaveAsCur()` uses `CurEncoder`.
- `image.SaveAsExr()` uses `ExrEncoder`.
- `image.SaveAsGif()` uses `GifEncoder`.
- `image.SaveAsIco()` uses `IcoEncoder`.
- `image.SaveAsJpeg()` uses `JpegEncoder`.
- `image.SaveAsPbm()` uses `PbmEncoder`.
- `image.SaveAsPng()` uses `PngEncoder`.
- `image.SaveAsQoi()` uses `QoiEncoder`.
- `image.SaveAsTga()` uses `TgaEncoder`.
- `image.SaveAsTiff()` uses `TiffEncoder`.

- `image.SaveAsWebp()` uses `WebpEncoder`.

General Decoder Options

Use `DecoderOptions` with the general `Load()` APIs when you want to control metadata handling, frame limits, or decode-to-size behavior:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

DecoderOptions options = new()
{
    MaxFrames = 1,
    SkipMetadata = false,
    TargetSize = new Size(1600, 1600)
};

using Image image = Image.Load(options, "input.webp");
```

Format-specific decoder option types also exist for specialized scenarios such as JPEG and PNG.

Common Encoder Families

Several formats share useful option sets through common encoder base types:

- `ImageEncoder` exposes `SkipMetadata`.
- `AlphaAwareImageEncoder` adds `TransparentColorMode`.
- `QuantizingImageEncoder` adds `Quantizer` and `PixelSamplingStrategy`.
- `AnimatedImageEncoder` adds `RepeatCount`, `BackgroundColor`, and `AnimateRootFrame`.

Those inherited options are especially useful when working with GIF, APNG, and animated WebP. For a format-agnostic guide to palettes and dithered output, see Quantization, Palettes, and Dithering.

Format Guides

Use the format-specific guides for the common cases and specialized workflows:

- JPEG for photographic output and quality-focused lossy compression.
- PNG for lossless output, transparency, and APNG metadata.
- GIF for palette-based animation workflows.

- WebP for lossy, lossless, transparent, and animated WebP output.
- TIFF for workflows where compression mode, pixel layout, and TIFF metadata matter.
- OpenEXR for HDR and higher-precision imaging workflows.

The less commonly used built-in formats still have valid niches:

- BMP is simple and broadly understood, but usually much larger than modern alternatives.
- ICO stores Windows icon files, often with one or more embedded icon images.
- CUR stores Windows cursor files and hotspot metadata.
- ANI stores Windows animated cursors with per-step timing and hotspot metadata.
- PBM covers PBM/PGM/PPM-style Netpbm-family workflows and simple interchange scenarios.
- TGA appears most often in graphics and content-pipeline tooling.
- QOI is a fast, simple lossless format with a much smaller ecosystem than PNG or WebP.

Custom Format Registration

Format detectors, decoders, and encoders are registered through ImageSharp configuration. See Configuration if you need to customize the set of supported formats for your application.

Choosing the Right Encoder

The right encoder settings depend on the tradeoff you want to make between:

- Image file size
- Encoder speed
- Image quality

The format-specific pages below are the best place to start when you need to tune those tradeoffs.

Practical Guidance

- Use explicit encoders when output behavior matters; file extensions are convenient but hide important defaults.
- Inspect the source with `Identify(...)` before conversion when alpha, animation, bit depth, or metadata changes the output decision.
- Treat metadata as part of format conversion: orientation, ICC profiles, animation timing, and comments may or may not survive a target format.
- Register only the formats your application needs when you want a smaller or more controlled decoding surface.

JPEG

JPEG remains the workhorse format for photographs on the web and in many application pipelines. It is best when you care about keeping file sizes small and are willing to trade away some exact pixel fidelity to get there.

Format Characteristics

JPEG uses lossy compression. That means it reduces file size by permanently discarding some image information, which is usually acceptable for photos but much more noticeable on sharp edges, text, UI assets, or repeated save cycles.

JPEG is built around the assumption that photographic images can lose some high-frequency detail without the loss being obvious. It divides image data into blocks, transforms those blocks into frequency information, and quantizes that information according to the requested quality. This is why artifacts often appear as blockiness, ringing around edges, or smearing in areas that were originally detailed.

Most JPEG workflows also use chroma subsampling: color detail is stored at lower resolution than brightness detail because human vision is usually more sensitive to luminance than chroma. That is very effective for photos, but it can make saturated text, icons, and UI edges look soft or discolored. If a file contains sharp colored edges, compare JPEG output carefully against PNG or WebP.

JPEG has no alpha channel and no animation model. If the input contains transparency, the transparent pixels must be flattened onto a background before encoding. If the input is animated, save to a format that supports animation or choose a single frame deliberately.

A few practical implications:

- JPEG is usually excellent for photos and gradients.
- JPEG is usually poor for logos, diagrams, screenshots, and pixel-precise artwork.
- JPEG does not support alpha transparency.
- Re-encoding a JPEG repeatedly can compound visible artifacts over time.

Save as JPEG

Use `JpegEncoder` when you want to tune JPEG output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;

using Image image = Image.Load("input.png");

image.Save("output.jpg", new JpegEncoder
{
    Quality = 85,
    Progressive = true
});
```

Key JPEG Encoder Options

The most commonly used `JpegEncoder` options are:

- `Quality` controls the quality/compression tradeoff on a 1-100 scale.
- `Progressive` enables progressive JPEG output.
- `ProgressiveScans` controls how progressive data is split into scans.
- `Interleaved` controls interleaved versus non-interleaved output.
- `ColorType` lets you influence the encoded JPEG color model.

JPEG is a lossy format and does not preserve alpha transparency. If the source image includes transparency, composite it onto a background first.

`Quality` is not a percentage of original image quality. It controls quantization strength, and the visual difference between values is not linear. A move from 95 to 85 may save a lot of bytes with little visual change on many photos, while a move from 45 to 35 can be much more obvious. Pick values by testing representative images at the sizes you actually serve.

Progressive JPEG stores the image in multiple refinement passes. Browsers can show a rough version before the full file has arrived, which can improve perceived loading behavior for large images. Baseline JPEG is simpler and still broadly supported. Choose progressive output when public image delivery benefits from progressive rendering; choose baseline if a downstream system has strict compatibility requirements.

Read JPEG Metadata

You can inspect format-specific metadata through `GetJpegMetadata()`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
```

```
using Image image = Image.Load("input.jpg");

JpegMetadata jpegMetadata = image.Metadata.GetJpegMetadata();
```

General image metadata such as EXIF and ICC profiles remains available through Working with Metadata.

JPEG-Specific Decode Options

ImageSharp also exposes `JpegDecoderOptions` for specialized JPEG decoding scenarios, including decoder-specific resize behavior.

Decoder-specific resizing can be useful when you only need a smaller representation of a large JPEG. It can reduce work before a full ImageSharp resize pipeline runs, but it should be treated as a decode optimization rather than a replacement for layout-aware resizing with `ResizeOptions`.

When to Use JPEG

JPEG is usually a good fit when:

- The source is photographic rather than flat-color artwork.
- You do not need transparency.
- A smaller file size is more important than exact pixel preservation.

JPEG is usually a poor fit when:

- The image contains text, hard UI edges, or line art.
- You need pixel-perfect reproduction.
- You need an alpha channel.

If you need lossless output or alpha transparency, start with PNG or WebP instead.

Practical Guidance

Set `Quality` explicitly for public output. JPEG quality is a product decision that balances file size and visible artifacts, so it should be visible in code rather than inherited from whatever default is active. Test the value against representative photos, not only one sample image.

Flatten transparent sources before saving as JPEG because the format has no alpha channel. Choose the background color deliberately; white is common, but product photos, logos, and UI previews often need a different page or brand background.

Keep ICC metadata or convert to a known output profile when color consistency matters. Avoid repeated JPEG-to-JPEG saves in editing workflows because each lossy encode can discard additional detail. If users edit repeatedly, keep a higher-fidelity working source and encode JPEG only at the export boundary.

PNG

PNG is the format people usually reach for when they want the saved pixels to stay exactly as they were processed in memory. That makes it a natural fit for UI assets, screenshots, diagrams, icons, and any workflow where transparency and crisp edges matter more than aggressive compression.

ImageSharp also supports animated PNG metadata and encoding scenarios.

Format Characteristics

PNG is a lossless format. It preserves pixel data exactly, which makes it a strong fit for graphics where edges, text, and flat-color regions need to stay crisp.

PNG compresses image data without discarding pixel information. Before compression, scanline filters can transform rows into forms that compress better. The chosen filter does not change the decoded pixels, but it can change encoding speed and file size. Adaptive filtering lets the encoder choose filters per row and is usually a good default for mixed content.

PNG supports several pixel representations, including grayscale, grayscale with alpha, RGB, RGB with alpha, and palette-indexed color. That is why `ColorType` and `BitDepth` matter: they decide how pixels are represented in the file, not just how strongly the file is compressed. A screenshot with a small number of colors may be much smaller as a palette PNG, while a translucent UI asset usually needs RGBA-style output.

PNG can store ancillary information such as gamma, text chunks, and color-management data. Those chunks can be important for appearance or workflow, but they can also increase file size or carry information you do not want to publish. Treat metadata as part of the output decision.

A few practical implications:

- PNG is excellent for screenshots, icons, logos, diagrams, and UI assets.
- PNG supports alpha transparency.
- PNG is often much larger than JPEG for photographic content.
- PNG can also carry animated PNG data, though ecosystem support is not as universal as static PNG support.

Save as PNG

Use `PngEncoder` when you want to tune PNG output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Png;

using Image image = Image.Load("input.png");

image.Save("output.png", new PngEncoder
{
    CompressionLevel = PngCompressionLevel.BestCompression,
    FilterMethod = PngFilterMethod.Adaptive,
    ColorType = PngColorType.RgbWithAlpha
});
```

PNG encoding is lossless. The main tradeoffs are encoder speed, file size, and how the pixel data is represented.

Key PNG Encoder Options

The most commonly used `PngEncoder` options are:

- `CompressionLevel` controls deflate compression effort.
- `FilterMethod` controls how scanlines are filtered before compression.
- `ColorType` and `BitDepth` control how pixel data is represented.
- `InterlaceMethod` lets you write an Adam7 interlaced image.
- `ChunkFilter` and `TextCompressionThreshold` control which ancillary data is written and how text chunks are compressed.
- `Gamma` lets you write a gamma value into the output metadata.

Because `PngEncoder` inherits from `QuantizingAnimatedImageEncoder`, it also supports `Quantizer`, `PixelSamplingStrategy`, and `TransparentColorMode` when you are writing palette-based PNG data.

Compression level is a speed-versus-size choice. Higher compression can reduce output size, but it costs more CPU and does not improve image quality because PNG is already lossless. For high-volume services, benchmark realistic images before choosing the slowest compression level globally.

Adam7 interlacing allows a progressively refined display as bytes arrive. That can help in some delivery scenarios, but it can also increase file size. For small UI assets and cached application images, non-interlaced PNG is often simpler.

Quantization and Palette PNGs

PNG does not always quantize. Quantization is only part of the encode path when you target a palette PNG by setting `PngColorType.Palette`. For RGB, RGBA, grayscale, or grayscale-with-alpha PNG output, ImageSharp writes the image in those representations without first reducing it to a palette.

Palette PNGs can be a very good fit for icons, diagrams, pixel art, and other flat-color assets where a smaller indexed palette is acceptable. They are usually a poor fit for photos, gradients, and other images with subtle transitions.

When you choose palette PNG output, ImageSharp uses the same quantization building blocks as the GIF encoder:

- `Quantizer` selects the palette-generation algorithm.
- `PixelSamplingStrategy` controls how pixels are sampled when building the palette.
- `TransparentColorMode` controls how fully transparent pixels are normalized during encoding.

If you pass a quantizer with custom `QuantizerOptions`, palette matching is configured through `ColorMatchingMode`, which offers the `Coarse` and `Exact` choices.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Png;
using SixLabors.ImageSharp.Processing.Processors.Quantization;

using Image image = Image.Load("input.png");

image.Save("output-indexed.png", new PngEncoder
{
    ColorType = PngColorType.Palette,
    Quantizer = new WuQuantizer(new QuantizerOptions
    {
        MaxColors = 64,
        Dither = null,
        TransparentColorMode = TransparentColorMode.Preserve
    })
});
```

If you need a fixed output palette instead of an adaptive one, use `PaletteQuantizer`. If you keep `ColorType` as `Rgb`, `RgbWithAlpha`, `Grayscale`, or `GrayscaleWithAlpha`, the quantizer settings are not the main control surface because the encoder is not writing palette-indexed PNG data.

Read PNG and APNG Metadata

Use `GetPngMetadata()` to inspect PNG-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Png;

using Image image = Image.Load("input.png");

PngMetadata pngMetadata = image.Metadata.GetPngMetadata();
```

`PngMetadata` includes values such as:

- `ColorType`
- `BitDepth`
- `InterlaceMethod`
- `Gamma`
- `RepeatCount`
- `AnimateRootFrame`

For animated PNGs, frame-level metadata is available through `PngFrameMetadata`, including `FrameDelay`, `DisposalMode`, and `BlendMode`.

PNG-Specific Decode Options

`PngDecoderOptions` exposes `MaxUncompressedAncillaryChunkSizeBytes`, which can be useful when controlling how much memory decompressed ancillary chunks are allowed to occupy.

Apple CgBI PNGs

The decoder reads Apple CgBI PNG files, the iOS-optimized variant that `pngcrush -iphone` produces. These files store raw DEFLATE image data, swap the channel order, and premultiply color by alpha. ImageSharp detects the `CgBI` chunk and reverses those changes automatically, so no decoder option is required. This support is decode only: the encoder always writes standard PNG output.

When to Use PNG

PNG is usually a good fit when:

- You need lossless output.
- The image uses transparency.
- You are working with screenshots, logos, diagrams, or UI assets.

- You need APNG-style animation metadata and frame control.

PNG is usually a poor fit when:

- The content is photographic and file size is a major concern.
- You only need a web-first animated format and modern browser-oriented compression matters more than static PNG compatibility.

If you want a lossy photographic format, start with JPEG. If you want a modern alternative that supports both lossy and lossless output, see WebP.

Practical Guidance

Use PNG when lossless pixels, transparency, screenshots, diagrams, or UI assets matter more than the smallest possible file. It is a strong default for sharp graphics because it avoids lossy artifacts around text, icons, and hard edges.

When PNG size matters, consider palette output and quantization rather than switching formats immediately. A palette PNG can be much smaller for limited-color graphics, but that choice should be tested against gradients, shadows, and transparency because quantization can introduce visible banding or dithering texture.

Preserve or convert color profiles intentionally. PNG is often used in workflows where exact appearance matters, so silently dropping profile information can be a real output bug. For photographic delivery where smaller files matter more than lossless pixels, compare JPEG and WebP instead.

GIF

GIF is one of the oldest formats ImageSharp supports, and it comes with tradeoffs that matter more than many newcomers expect. It is still useful for simple animation and very broad compatibility, but because it is palette based, color reduction and frame metadata are part of the story from the start.

In ImageSharp, GIF encoding is built on a quantizing animated encoder, which means palette generation and frame metadata are both important parts of the workflow.

Format Characteristics

GIF is fundamentally a palette format. Each frame is limited to indexed colors rather than storing full true-color pixel data, which is why quantization and palette choice matter so much.

GIF stores each pixel as an index into a color table. A frame can use a global color table shared by the animation or a local color table for that frame. That design keeps the format simple and compatible, but it means full-color source images must be reduced to a limited palette before they can be written.

Transparency in GIF is also index based. A palette entry can be treated as transparent, but GIF does not have smooth per-pixel alpha like PNG or WebP. Soft edges, shadows, and semitransparent UI elements can therefore look jagged or require a matte color baked into the pixels.

Animation behavior is controlled by frame metadata. Frame delay, disposal mode, transparency index, repeat count, and color table mode all affect how the animation plays. When a converted GIF looks wrong, the issue is often frame metadata rather than the pixels alone.

A few practical implications:

- GIF is well known and widely compatible for simple animations.
- GIF is limited compared to modern formats for photographic or high-color imagery.
- GIF transparency is palette/index based rather than full alpha blending.
- GIF is usually chosen for compatibility or simplicity rather than compression efficiency.

Save as GIF

Use `GifEncoder` when you want to control GIF output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Gif;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> gif = new(120, 120, Color.Black.ToPixel<Rgba32>());

gif.Metadata.GetGifMetadata().RepeatCount = 0;
gif.Frames.RootFrame.Metadata.GetGifMetadata().FrameDelay = 10;

using Image<Rgba32> frame = new(120, 120, Color.Orange.ToPixel<Rgba32>());
frame.Frames.RootFrame.Metadata.GetGifMetadata().FrameDelay = 10;

gif.Frames.AddFrame(frame.Frames.RootFrame);

gif.Save("output.gif", new GifEncoder
{
    ColorTableMode = FrameColorTableMode.Global
});
```

Key GIF Encoder Options

The main `GifEncoder` option is `ColorTableMode`, which controls whether frames use a shared global palette or per-frame local palettes.

Because `GifEncoder` inherits from `QuantizingAnimatedImageEncoder`, it also supports:

- `RepeatCount`
- `BackgroundColor`
- `AnimateRootFrame`
- `Quantizer`
- `PixelSamplingStrategy`
- `TransparentColorMode`

A global palette can keep animation output more consistent and can reduce overhead when frames share a similar color set. Local palettes can improve quality when frames differ significantly, but they can increase file size and make palette behavior harder to reason about. Choose based on the animation, not as a fixed rule.

`RepeatCount` controls looping. A value of `0` represents infinite looping in normal GIF usage. Frame delays are stored on frame metadata, so set them on the frames whose timing matters rather than assuming the encoder will infer the intended animation speed.

Quantization and Palette Control

Every GIF encode in ImageSharp is a quantization step, because GIF stores indexed palette entries rather than full true-color pixels. If you do nothing, ImageSharp will still build a palette for you, but for gradients, photographic frames, UI art, or brand colors it is often worth controlling the quantizer explicitly.

The main knobs are:

- `Quantizer` to choose the palette-generation algorithm.
- `PixelSamplingStrategy` to control how source pixels are sampled while building the palette.
- `TransparentColorMode` to control how transparent pixels are treated during quantization.

Common choices include:

- `HexadecatreeQuantizer` for a solid general-purpose adaptive palette.
- `WuQuantizer` when you want a high-quality adaptive palette with configurable `QuantizerOptions`.
- `PaletteQuantizer` when you need to lock output to a known palette.

`QuantizerOptions` also exposes `ColorMatchingMode` with the simplified `Coarse` and `Exact` choices for palette matching.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Gif;
using SixLabors.ImageSharp.Processing.Processors.Quantization;

using Image image = Image.Load("input.gif");

image.Save("output.gif", new GifEncoder
{
    ColorTableMode = FrameColorTableMode.Global,
    Quantizer = new WuQuantizer(new QuantizerOptions
    {
        MaxColors = 128,
        Dither = null,
        TransparentColorMode = TransparentColorMode.Preserve
    })
});
```

Reducing `MaxColors` can shrink files, but it also makes banding and contouring more likely. Dithering can hide some of that, at the cost of more visible texture.

GIF Metadata

Use `GetGifMetadata()` to inspect or modify GIF-level metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Gif;

using Image image = Image.Load("input.gif");

GifMetadata gifMetadata = image.Metadata.GetGifMetadata();
```

`GifMetadata` includes values such as `RepeatCount`, `ColorTableMode`, and the global color table.

Per-frame metadata is available through `GifFrameMetadata`, including:

- `FrameDelay`
- `DisposalMode`
- `ColorTableMode`
- `HasTransparency`
- `TransparencyIndex`

GIF Tradeoffs

GIF is best suited to simple animation and palette-based content. It is usually not the best fit for photographic imagery because the format is palette-constrained and heavily depends on quantization.

GIF is usually a good fit when:

- You need a simple looping animation format with broad legacy support.
- The content uses relatively few colors.
- You are comfortable with palette-based tradeoffs.

GIF is usually a poor fit when:

- The animation contains gradients, photos, or subtle color transitions.
- You want efficient compression.
- You need modern transparency behavior.

For a step-by-step multi-frame workflow, see Working with Animations. For a more modern animated format, see WebP.

Practical Guidance

Use GIF for compatibility. It remains useful for simple looping animations and legacy-friendly workflows, but it is palette-constrained and rarely the most efficient modern animated format.

Control quantization deliberately because GIF quality depends heavily on palette choice. Gradients, photos, and subtle color changes can degrade quickly if the palette is poorly matched. Dithering can hide banding, but it can also add visible texture.

When converting existing animations, inspect frame delay, disposal mode, transparency, and repeat count. Those values define the animation behavior just as much as the pixels do. Prefer animated WebP or APNG when modern compression, alpha behavior, or color quality matters more than legacy support.

WebP

WebP is often the first format to consider when you want one codec that can cover several common web scenarios. It can handle photographs, transparency, and animation, which makes it a flexible alternative to juggling separate JPEG, PNG, and GIF outputs.

In ImageSharp, it is one of the most flexible general-purpose web output formats.

Format Characteristics

WebP is a modern format family rather than a single narrow use case. It can be used as:

- a lossy alternative to JPEG,
- a lossless alternative to PNG in many workflows,
- a transparency-capable web format,
- and an animation format.

A few practical implications:

- WebP is often the most flexible web-oriented output option.
- WebP supports both alpha transparency and animation.
- Lossy and lossless modes have different tuning behavior.
- Compatibility is generally strong in modern environments, but not identical to long-established formats like JPEG or PNG everywhere.

Lossy WebP is aimed at photographic and mixed web content, similar to JPEG, but with a different compression model and more tuning controls. It is often competitive for public delivery where byte size matters, especially after comparing quality settings against the JPEG baseline you would otherwise use.

Lossless WebP is closer to PNG in intent: preserve exact pixels while reducing file size. It can be attractive for transparent graphics and UI assets when client support is known. It should still be tested against PNG because the smaller file is not guaranteed for every image.

Animated WebP can replace GIF in modern delivery pipelines, but the animation behavior is more than the encoded frames. Frame delay, blend mode, disposal mode, repeat count, and background color all affect the final result. When converting an existing animation, inspect and set those values deliberately.

Save as WebP

Use `WebpEncoder` when you want to tune WebP output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Webp;

using Image image = Image.Load("input.png");

image.Save("output.webp", new WebpEncoder
{
    FileFormat = WebpFileFormatType.Lossless,
    Quality = 75,
    Method = WebpEncodingMethod.BestQuality,
    UseAlphaCompression = true
});
```

Set `FileFormat` to choose between lossy and lossless output.

Key WebP Encoder Options

The most commonly used `WebpEncoder` options are:

- `FileFormat` chooses lossy or lossless encoding.
- `Quality` controls quality or compression effort, depending on the mode.
- `Method` controls the speed/quality tradeoff.
- `UseAlphaCompression` controls how the alpha plane is compressed.
- `NearLossless` and `NearLosslessQuality` tune near-lossless workflows.
- `EntropyPasses`, `SpatialNoiseShaping`, and `FilterStrength` expose more advanced tuning.

Because `WebpEncoder` inherits from `AnimatedImageEncoder`, it also supports `RepeatCount`, `BackgroundColor`, and `AnimateRootFrame`.

In lossy mode, `Quality` controls the visual quality and compression tradeoff. In lossless mode, quality-style settings are better understood as compression-effort controls. `Method` also changes encoding effort: higher-quality methods can produce smaller or better-looking output but cost more CPU. That distinction matters on web servers where many variants may be generated on demand.

Alpha compression is a separate concern from RGB compression. If transparent edges are important, test images with soft shadows, icons, and antialiased cutouts; those are the places where alpha handling becomes visible.

Read WebP Metadata

Use `GetWebpMetadata()` to inspect WebP-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Webp;

using Image image = Image.Load("input.webp");

WebpMetadata webpMetadata = image.Metadata.GetWebpMetadata();
```

`WebpMetadata` includes values such as:

- `FileFormat`
- `ColorType`
- `BitsPerPixel`
- `RepeatCount`
- `BackgroundColor`

For animated WebP, frame-level metadata is available through `WebpFrameMetadata`, including `FrameDelay`, `BlendMode`, and `DisposalMode`.

When to Use WebP

WebP is a strong choice when you want:

- Lossy or lossless output from the same family of encoders.
- Transparency support.
- Animation support.
- More control over size/quality tradeoffs than a simple save-by-extension workflow provides.

WebP is often the best first alternative to compare against both JPEG and PNG when optimizing for delivery size.

If you need strict lossless preservation with a more traditional workflow, see PNG. If you specifically need TIFF-style metadata and pixel layout control, see TIFF.

Practical Guidance

Compare lossy WebP against the JPEG settings you would otherwise ship. WebP often produces smaller files at similar visual quality, but the right quality value depends on content and delivery expectations. Test photos, screenshots, graphics, and mixed-content images separately.

Use lossless WebP when transparency and smaller files matter but PNG compatibility is not required. That can be a good fit for controlled clients and modern web delivery, but it should not be the only public format unless your client and CDN support story is clear.

Animated WebP carries timing, blending, disposal, repeat count, and background behavior. When converting from GIF or APNG, inspect that metadata instead of assuming the animation will behave identically after re-encoding.

TIFF

TIFF is less about browser delivery and more about control. When a workflow cares about archival fidelity, scanning, publishing, print, or carrying richer metadata and pixel-layout choices forward, TIFF is often the format that gives you the room to do it.

ImageSharp exposes a range of TIFF-specific encoder and metadata options for those cases.

Format Characteristics

TIFF is best thought of as a flexible imaging container with multiple possible encodings and metadata conventions rather than a single narrow web format.

TIFF can describe images using different photometric interpretations, bit depths, compression schemes, byte orders, and predictors. That flexibility is why it remains useful in scanning, print, archival, and professional imaging workflows, but it also means "TIFF support" varies between tools. A pipeline should choose the specific TIFF shape the downstream system expects.

Compression is not one-size-fits-all. LZW and deflate-style compression are common lossless choices, and predictors can improve compression by making neighboring sample values easier to encode. Those settings affect file size and compatibility rather than visual quality when the output is lossless.

TIFF metadata can be part of the workflow contract. Some files carry scanner, camera, print, publishing, or application-specific metadata. Before stripping or rewriting metadata, decide whether another system relies on it.

A few practical implications:

- TIFF is common in archival, print, scanning, publishing, and professional imaging workflows.
- TIFF can represent different compression schemes and pixel layouts.
- TIFF can carry rich format-specific metadata.
- TIFF is usually not the best choice for browser-facing delivery.

Save as TIFF

Use `TiffEncoder` when you want to control how TIFF data is written:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Tiff;
using SixLabors.ImageSharp.Formats.Tiff.Constants;
```

```
using Image image = Image.Load("input.png");

image.Save("output.tiff", new TiffEncoder
{
    Compression = TiffCompression.Lzw,
    HorizontalPredictor = TiffPredictor.Horizontal,
    BitsPerPixel = TiffBitsPerPixel.Bit24,
    PhotometricInterpretation = TiffPhotometricInterpretation.Rgb
});
```

Key TIFF Encoder Options

The most commonly used `TiffEncoder` options are:

- `Compression` controls the TIFF compression algorithm.
- `CompressionLevel` controls deflate compression effort when deflate is used.
- `BitsPerPixel` controls the encoded pixel depth.
- `PhotometricInterpretation` controls how pixel data is interpreted.
- `HorizontalPredictor` can improve compression ratios for deflate or LZW output.

Some compression and photometric values are defined by the TIFF specification but are not currently supported by the encoder. In those cases, the encoder falls back rather than emitting unsupported output.

Because TIFF has many valid combinations, choose `BitsPerPixel`, `PhotometricInterpretation`, compression, and predictor settings together. For example, an RGB interchange file, a bilevel scanned document, and a higher-bit-depth imaging asset are all TIFF files, but they should not use the same encoder assumptions.

Read TIFF Metadata

Use `GetTiffMetadata()` to inspect TIFF-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Tiff;

using Image image = Image.Load("input.tiff");

TiffMetadata tiffMetadata = image.Metadata.GetTiffMetadata();
```

`TiffMetadata` includes values such as:

- `ByteOrder`
- `FormatType`
- `Compression`
- `BitsPerPixel`
- `PhotometricInterpretation`
- `Predictor`

When to Use TIFF

TIFF is usually worth considering when:

- You need TIFF-specific compression or pixel layout options.
- You care about byte order, predictor behavior, or TIFF format metadata.
- The workflow is archival, interchange, print, or imaging-pipeline oriented rather than browser-first.

TIFF is usually a poor fit when:

- The output is primarily intended for browser delivery.
- You just need a simple photo or web asset format.

For more typical application and web workloads, PNG, JPEG, and WebP are usually better starting points.

Practical Guidance

Choose compression, predictor, and pixel layout explicitly when TIFF is part of an interchange contract. TIFF is a container family with many valid combinations, and downstream tools often support only the subset they care about. A file that is valid TIFF is not automatically compatible with every TIFF-consuming application.

Treat metadata as workflow data. TIFF files often carry scanner, archival, print, or pipeline-specific metadata, so decide whether that information should be preserved, transformed, or stripped. Test with the consuming application, because compatibility matters more than theoretical format support.

For browser delivery, ordinary thumbnails, and most application assets, PNG, JPEG, or WebP are usually easier to operate and validate.

OpenEXR

OpenEXR is the format to reach for when dynamic range and channel precision matter more than browser compatibility. It is most at home in rendering, compositing, HDR capture, and other imaging pipelines where half-float or float data is part of the workflow.

ImageSharp supports OpenEXR read and write workflows and exposes EXR-specific metadata through `ExrMetadata`.

Format Characteristics

OpenEXR is best thought of as a high-precision interchange format rather than a delivery format.

OpenEXR is designed for scene-referred and high-dynamic-range image data. Values are often intended to survive rendering, compositing, lighting, or color-grading steps before they are tone mapped for display. That makes EXR very different from web formats, where pixels are usually already display-referred.

The pixel type matters. Half-float storage is common because it gives much more range than 8-bit formats while keeping file size lower than full 32-bit float data. Full float output is useful when the pipeline needs the extra precision. Unsigned integer storage exists for workflows that require it, but it is not the common "HDR image" choice.

Compression should be chosen with the consuming pipeline in mind. ZIP and ZIPS are both lossless options, but they organize compression differently. Renderer, compositor, and asset-pipeline expectations are often more important than theoretical compression ratios.

A few practical implications:

- OpenEXR is common in VFX, rendering, compositing, and HDR-oriented workflows.
- ImageSharp tracks EXR pixel storage through `ExrPixelType` and image layout through `ExrImageDataType`.
- The current decoder supports uncompressed, ZIP, ZIPS, RLE, and B44-compressed EXR files.
- The current encoder supports uncompressed, ZIP, and ZIPS output.
- OpenEXR is usually not the best choice for browser-facing assets.

Save as OpenEXR

Use `ExrEncoder` when you want to control how EXR data is written:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Exr;
using SixLabors.ImageSharp.Formats.Exr.Constants;

using Image image = Image.Load("input.png");

image.Save("output.exr", new ExrEncoder
{
    PixelType = ExrPixelType.Half,
    Compression = ExrCompression.Zip
});
```

Key OpenEXR Encoder Options

The most commonly used `ExrEncoder` options are:

- `PixelType` controls whether channels are written as `Half`, `Float`, or `UnsignedInt`.
- `Compression` controls the current EXR encoder compression mode. Use `None`, `Zip`, or `Zips`.

Do not use OpenEXR only because an image is "high quality." Use it when the numeric range and precision are needed by the pipeline. If the next step is ordinary display, web delivery, or a thumbnail, tone mapping or conversion to a delivery format is usually the next deliberate step.

Read OpenEXR Metadata

Use `GetExrMetadata()` to inspect EXR-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Exr;

using Image image = Image.Load("input.exr");

ExrMetadata exrMetadata = image.Metadata.GetExrMetadata();

Console.WriteLine(exrMetadata.PixelType);
Console.WriteLine(exrMetadata.ImageDataType);
Console.WriteLine(exrMetadata.Compression);
```

`ExrMetadata` includes values such as:

- `PixelType`
- `ImageDataType`

- `Compression`

When to Use OpenEXR

OpenEXR is usually worth considering when:

- You need HDR or higher-precision image data in a rendering or imaging pipeline.
- You want floating-point or half-float channel storage.
- You care about EXR-specific compression and channel-layout metadata.

OpenEXR is usually a poor fit when:

- The output is primarily for browsers or ordinary app delivery.
- You want the broadest ecosystem compatibility for day-to-day assets.

For everyday application and web output, PNG, JPEG, WebP, and TIFF are usually easier starting points.

Practical Guidance

- Use OpenEXR when high dynamic range, floating-point data, or rendering-pipeline interchange is the real requirement.
- Keep tone mapping and display conversion separate from storing scene-referred data.
- Test compression and channel layout with the downstream renderer or compositing tool.
- Prefer TIFF, PNG, or WebP when the workflow does not need EXR-specific precision or metadata.

BMP

BMP is the classic Windows bitmap format. It is simple, broadly recognized, and sometimes useful when you need predictable bit-depth control or interoperability with older Windows-oriented tools.

ImageSharp exposes BMP-specific APIs through `BmpEncoder`, `BmpDecoderOptions`, and `BmpMetadata`.

Format Characteristics

BMP is best thought of as a straightforward bitmap container rather than a delivery format optimized for file size.

BMP files are useful when another system expects simple Windows bitmap data, but they are rarely the best choice for public delivery. Depending on bit depth, the file may store direct color values or palette indexes. Lower bit depths require palette generation and therefore can change colors in the same way other indexed formats do.

The format's simplicity is also its main tradeoff. BMP output can be easy for older tools to consume, but it usually produces much larger files than PNG, WebP, or QOI for ordinary application assets. Use it because the receiving workflow expects BMP, not because it is generally efficient.

Transparency support is limited and workflow-dependent. In ImageSharp, `SupportTransparency` applies to 32-bit BMP output. If alpha preservation is the main requirement, PNG or WebP is usually a better default.

A few practical implications:

- ImageSharp can write BMP output at 1, 2, 4, 8, 16, 24, or 32 bits per pixel.
- Lower bit-depth BMP output is palette based, so encoding can reduce colors rather than preserving full true-color data.
- `SupportTransparency` only applies when writing 32-bit BMP output.
- BMP is easy to exchange with older tooling, but it is usually much larger than PNG, WebP, or QOI for the same image.

Save as BMP

Use `BmpEncoder` when you want explicit control over BMP bit depth:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Bmp;

using Image image = Image.Load("input.png");

image.Save("output.bmp", new BmpEncoder
{
    BitsPerPixel = BmpBitsPerPixel.Bit32,
    SupportTransparency = true
});
```

Key BMP Encoder Options

The most commonly used `BmpEncoder` options are:

- `BitsPerPixel` controls the encoded BMP bit depth.
- `SupportTransparency` enables BMP alpha support for 32-bit output.
- `Quantizer` and `PixelSamplingStrategy` matter when you target indexed BMP output such as 1, 4, or 8 bits per pixel.

When targeting 1, 2, 4, or 8 bits per pixel, the encoder must map source colors into a limited palette. That can be useful for compatibility with old systems, but it should be treated as a color-reduction step. For 24-bit or 32-bit BMP output, the file is larger but avoids indexed palette tradeoffs.

Read BMP Metadata

Use `GetBmpMetadata()` to inspect BMP-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Bmp;

using Image image = Image.Load("input.bmp");

BmpMetadata bmpMetadata = image.Metadata.GetBmpMetadata();

Console.WriteLine(bmpMetadata.BitsPerPixel);
Console.WriteLine(bmpMetadata.InfoHeaderType);
```

`BmpMetadata` includes values such as:

- `BitsPerPixel`
- `InfoHeaderType`
- `ColorTable`

BMP-Specific Decode Options

ImageSharp also exposes `BmpDecoderOptions` when you need to control how skipped pixels in RLE-compressed BMP data are interpreted:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Bmp;

BmpDecoderOptions options = new()
{
    RleSkippedPixelHandling = RleSkippedPixelHandling.Transparent
};

using Image image = Image.Load(options, "input.bmp");
```

When to Use BMP

BMP is usually worth considering when:

- You need explicit low-level BMP bit-depth control.
- You are interoperating with Windows-oriented tools or older software that expects BMP input.
- File size is a secondary concern.

BMP is usually a poor fit when:

- You need compact files for storage or delivery.
- You want a modern web-oriented format.

For most application and web output, PNG, WebP, or QOI are usually better starting points.

Practical Guidance

- Use BMP mainly for compatibility with software that specifically expects it.
- Expect large files compared with modern compressed formats.
- Choose bit depth deliberately when interoperating with older Windows-oriented tooling.
- Prefer PNG or WebP when the same image is intended for storage, delivery, or web use.

ICO

ICO is the Windows icon container format. It is designed to carry icon image data rather than act as a general-purpose picture format, and ImageSharp exposes both image-level and frame-level ICO metadata because individual embedded icon images can vary.

ImageSharp exposes ICO-specific APIs through `IcoEncoder`, `IcoMetadata`, and `IcoFrameMetadata`.

Format Characteristics

ICO is best thought of as a container for one or more icon images.

An ICO file can contain multiple frames for different icon sizes and bit depths. That lets Windows and other consumers choose the most appropriate embedded image for a particular scale or display context. A single-frame ICO can work, but multi-size icon assets are often more useful in real applications.

Frames can be stored using BMP-style data or PNG-compressed data. PNG-compressed icon frames are common for larger or more detailed icons because they can preserve alpha and reduce file size. BMP-style frames can still matter for older compatibility workflows.

The encoded width and height are part of the frame metadata, not just a consequence of the decoded ImageSharp frame size. When generating icons, keep frame metadata aligned with the asset sizes your target platform expects.

A few practical implications:

- Existing ICO files can contain one or more embedded icon images.
- ImageSharp exposes per-frame icon details such as `Compression`, `BmpBitsPerPixel`, `EncodingWidth`, and `EncodingHeight`.
- Frame compression is represented through `IconFrameCompression`.
- ICO is a Windows asset format, not a general interchange format for ordinary images.

Save as ICO

Use `IcoEncoder` when you want Windows icon output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Ico;
using SixLabors.ImageSharp.Formats.Icon;
using SixLabors.ImageSharp.PixelFormats;
```

```
using Image<Rgba32> image = Image.Load<Rgba32>("icon-source.png");

IcoFrameMetadata frameMetadata = image.Frames.RootFrame.Metadata.GetIcoMetadata();
frameMetadata.Compression = IconFrameCompression.Png;
frameMetadata.EncodingWidth = 64;
frameMetadata.EncodingHeight = 64;

image.Save("app.ico", new IcoEncoder());
```

ICO Frame Metadata

The most useful ICO-specific values live on `IcoFrameMetadata`:

- `Compression` controls whether the encoded frame uses BMP or PNG storage.
- `BmpBitsPerPixel` controls the BMP bit depth when the frame is stored as BMP.
- `EncodingWidth` and `EncodingHeight` describe the encoded icon dimensions for that frame.

`IcoMetadata` mirrors the root frame's compression, bit depth, and color-table information at the image level.

Treat each frame as an icon candidate, not just a page in an image sequence. If you create a multi-frame ICO, inspect every frame's dimensions, compression, and bit depth before saving.

Read ICO Metadata

Use `Image.Identify()` when you want to inspect the icon container without decoding every embedded image:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Ico;

ImageInfo info = Image.Identify("app.ico");

Console.WriteLine($"Embedded images: {info.FrameMetadataCollection.Count}");

IcoMetadata icoMetadata = info.Metadata.GetIcoMetadata();
IcoFrameMetadata firstFrame = info.FrameMetadataCollection[0].GetIcoMetadata();

Console.WriteLine(icoMetadata.Compression);
Console.WriteLine(firstFrame.EncodingWidth);
Console.WriteLine(firstFrame.EncodingHeight);
```

When to Use ICO

ICO is usually worth considering when:

- You need a Windows icon file.
- You care about icon-specific frame metadata such as encoded icon dimensions or frame compression.

ICO is usually a poor fit when:

- You are storing ordinary images rather than icons.
- You want a broadly portable web or application image format.

For ordinary image delivery or storage, PNG, WebP, and JPEG are usually better choices.

Practical Guidance

- Include the sizes your target platform expects instead of assuming one frame is enough.
- Inspect frame metadata when converting existing icons so dimensions and compression remain intentional.
- Use ICO for Windows icon assets, not general image storage.
- Keep source artwork separately; generated icon files are usually deployment artifacts.

CUR

CUR is the Windows cursor format. It is closely related to ICO, but it carries cursor-specific hotspot information so the runtime knows which pixel is the active click point.

ImageSharp exposes CUR-specific APIs through `CurEncoder`, `CurMetadata`, and `CurFrameMetadata`.

Format Characteristics

CUR is best thought of as a cursor container rather than a normal image file format.

CUR shares much of the icon-container shape with ICO, but cursor files add hotspot coordinates. The hotspot is the active point of the cursor: for an arrow it is normally the tip; for a crosshair it may be the center. If the hotspot is wrong, the image can look correct while clicking and hit testing feel wrong.

Like ICO, a CUR file can contain multiple embedded images for different sizes. Consumers can choose a frame based on display scale or cursor size. Hotspot metadata is per frame, so every embedded cursor image needs correct hotspot coordinates.

A few practical implications:

- Existing CUR files can contain one or more cursor images.
- Cursor-specific metadata lives primarily on `CurFrameMetadata`.
- `HotspotX` and `HotspotY` are the key extra values that distinguish cursor assets from icons.
- CUR is useful when you need Windows cursor output with hotspot metadata, not when you need a general-purpose image format.

Save as CUR

Use `CurEncoder` when you want Windows cursor output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Cur;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = Image.Load<Rgba32>("cursor-source.png");

CurFrameMetadata frameMetadata = image.Frames.RootFrame.Metadata.GetCurMetadata();
frameMetadata.HotspotX = 4;
frameMetadata.HotspotY = 4;
```

```
image.Save("pointer.cur", new CurEncoder());
```

CUR Frame Metadata

The most useful CUR-specific values live on `CurFrameMetadata`:

- `HotspotX` and `HotspotY` control the cursor hotspot coordinates.
- `EncodingWidth` and `EncodingHeight` describe the encoded cursor dimensions for that frame.
- `Compression` and `BmpBitsPerPixel` describe how the frame is stored.

`CurMetadata` mirrors the root frame's compression, bit depth, and color-table information at the image level.

Treat hotspot coordinates as part of the user interaction contract. They should be chosen from the cursor design, not copied blindly from another size unless the coordinate scales correctly.

Read CUR Metadata

Use `Image.Identify()` when you want cursor metadata without a full decode:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Cur;

ImageInfo info = Image.Identify("pointer.cur");

Console.WriteLine($"Embedded cursor images: {info.FrameMetadataCollection.Count}");

CurMetadata curMetadata = info.Metadata.GetCurMetadata();
CurFrameMetadata firstFrame = info.FrameMetadataCollection[0].GetCurMetadata();

Console.WriteLine(curMetadata.Compression);
Console.WriteLine(firstFrame.HotspotX);
Console.WriteLine(firstFrame.HotspotY);
```

When to Use CUR

CUR is usually worth considering when:

- You need a Windows cursor file.
- The hotspot position is part of the asset contract.

CUR is usually a poor fit when:

- You are storing a normal image rather than a cursor asset.
- You want broad compatibility outside Windows cursor workflows.

For Windows icon assets without cursor hotspots, see ICO. For animated cursors, see ANI.

Practical Guidance

- Treat hotspot coordinates as part of the cursor asset, not incidental metadata.
- Validate all embedded frames when generating multi-size cursor files.
- Use CUR only when the output is meant to behave as a cursor.
- Use ICO, PNG, or another ordinary image format when hotspot metadata is not required.

ANI

ANI is the Windows animated cursor format. It is a RIFF container that stores a sequence of embedded CUR, ICO, or BMP resources together with animation timing and ordering data.

ImageSharp exposes ANI-specific APIs through `AniEncoder`, `AniMetadata`, and `AniFrameMetadata`.

Format Characteristics

ANI is best thought of as an animation container over cursor resources rather than a normal image file format.

Each frame resource is a complete embedded file, most commonly a Windows cursor. A `seq` chunk can map animation steps to frame resources, so one resource can appear at several points in the animation. A `rate` chunk can give each step its own display time. Timing is measured in sixtieths of a second, sometimes called jiffies.

A few practical implications:

- One frame resource can contain multiple sizes, exactly like ICO and CUR files.
- Frame timing lives per step, with a file-level default display rate.
- Cursor hotspot coordinates apply per frame, exactly as in CUR.
- ANI is useful when you need Windows animated cursor output, not when you need a general-purpose animation format.

Decoding Behavior

When ImageSharp decodes an ANI file, each animation step becomes one or more frames in the decoded image:

- If the file contains a `seq` chunk, the decoder follows it to project steps onto frame resources. Without one, the resources play in storage order.
- Every image inside the referenced resource becomes a frame on a shared canvas sized to the largest resource. The encoded dimensions of each variant are preserved in the frame metadata.
- Frames decoded from the same animation step share a `SequenceNumber`, which groups them as resolution variants of one step.
- Each step's display time comes from the `rate` chunk when present, or from the header's default display rate.

Save as ANI

Use `AniEncoder` or `image.SaveAsAni(...)` when you want Windows animated cursor output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Ani;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = Image.Load<Rgba32>("frame-source.png");

AniFrameMetadata frameMetadata = image.Frames.RootFrame.Metadata.GetAniMetadata();
frameMetadata.FrameDelay = 10; // Sixtieths of a second.
frameMetadata.HotspotX = 4;
frameMetadata.HotspotY = 4;

image.SaveAsAni("pointer.ani");
```

The encoder derives one animation step per frame by default. Give adjacent frames the same positive `SequenceNumber` to group them as resolution variants of a single step. A non-positive value encodes the frame as its own animation step.

`AniEncoder` inherits the quantizer and pixel sampling options of `QuantizingImageEncoder`. These options are passed through to the embedded CUR, ICO, or BMP encoders.

ANI Metadata

Image-level values live on `AniMetadata`:

- `DisplayRate` is the default frame display time in sixtieths of a second.
- `Flags` records whether frame resources are ICO/CUR or BMP data, and whether the file carries a sequence chunk.
- `Name` and `Artist` carry the animation title and author when present.
- `Width`, `Height`, `BitCount`, and `Planes` mirror the ANI header. Icon-based files commonly store zero here because each embedded resource declares its own dimensions and pixel layout.

The most useful frame-level values live on `AniFrameMetadata`:

- `FrameDelay` is the display time for the frame's step in sixtieths of a second.
- `SequenceNumber` groups adjacent frames as resolution variants of one animation step.
- `FrameFormat` records whether the frame came from a CUR, ICO, or BMP resource.
- `HotspotX` and `HotspotY` control the cursor hotspot coordinates.

- `EncodingWidth`, `EncodingHeight`, `Compression`, `BmpBitsPerPixel`, and `ColorTable` describe how the frame is stored, exactly as in ICO and CUR.

Read ANI Metadata

Use `Image.Identify()` when you want animation metadata without a full decode:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Ani;

ImageInfo info = Image.Identify("pointer.ani");

Console.WriteLine($"Frames: {info.FrameMetadataCollection.Count}");

AniMetadata aniMetadata = info.Metadata.GetAniMetadata();
AniFrameMetadata firstFrame = info.FrameMetadataCollection[0].GetAniMetadata();

Console.WriteLine(aniMetadata.DisplayRate);
Console.WriteLine(firstFrame.FrameDelay);
Console.WriteLine(firstFrame.HotspotX);
```

When to Use ANI

ANI is usually worth considering when:

- You need a Windows animated cursor file.
- Hotspot position and per-step timing are part of the asset contract.

ANI is usually a poor fit when:

- You need a general-purpose animation format. Use GIF, animated PNG, or animated WebP instead.
- You want broad compatibility outside Windows cursor workflows.

For static Windows cursor assets, see CUR. For Windows icon assets, see ICO.

Practical Guidance

- Treat hotspot coordinates and step timing as part of the cursor asset, not incidental metadata.
- Use `SequenceNumber` to group resolution variants, and validate every embedded size.

- Use ANI only when the output is meant to behave as an animated cursor.
- Use GIF, animated PNG, or animated WebP for ordinary animation output.

PBM / PGM / PPM

In ImageSharp, `PbmFormat` covers the Netpbm PNM family: PBM for black-and-white images, PGM for grayscale images, and PPM for RGB images. These formats are intentionally simple and are often used for straightforward interchange or tooling pipelines.

ImageSharp exposes PNM-specific APIs through `PbmEncoder` and `PbmMetadata`.

Format Characteristics

The PNM family is best thought of as a simple interchange family rather than a compact delivery format.

The family covers three related subformats. PBM stores black-and-white images. PGM stores grayscale images. PPM stores RGB images. Each can be useful for tests, examples, and simple tooling because the structure is easy to generate and inspect.

Plain-text encoding is human-readable, which can be valuable for debugging small fixtures. Binary encoding is more compact and more appropriate for larger files, but it is still not a modern compressed delivery format. The formats do not carry alpha transparency or rich metadata.

A few practical implications:

- `PbmColorType.BlackAndWhite` maps to PBM output.
- `PbmColorType.Grayscale` maps to PGM output.
- `PbmColorType.Rgb` maps to PPM output.
- `PbmEncoding` lets you choose plain-text or binary pixel encoding.
- `PbmComponentType` lets you choose 1-bit black-and-white, 8-bit components, or 16-bit components depending on the target subformat.

Save as PBM / PGM / PPM

Use `PbmEncoder` when you want to choose the exact PNM subformat and encoding style:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Pbm;

using Image image = Image.Load("input.png");

image.Save("output.ppm", new PbmEncoder
{
```

```
    ColorType = PbmColorType.Rgb,
    ComponentType = PbmComponentType.Byte,
    Encoding = PbmEncoding.Binary
});
```

Key PNM Encoder Options

The most commonly used `PbmEncoder` options are:

- `ColorType` selects PBM, PGM, or PPM style output.
- `ComponentType` selects 1-bit, 8-bit, or 16-bit component storage where that subformat allows it.
- `Encoding` selects plain-text or binary pixel encoding.

Choose the subformat from the data model. A mask or thresholded image belongs in PBM, grayscale analysis output belongs in PGM, and ordinary RGB test data belongs in PPM. Choose plain text when inspection matters more than size.

Read PNM Metadata

Use `GetPbmMetadata()` to inspect PNM-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Pbm;

using Image image = Image.Load("input.ppm");

PbmMetadata pbmMetadata = image.Metadata.GetPbmMetadata();

Console.WriteLine(pbmMetadata.ColorType);
Console.WriteLine(pbmMetadata.ComponentType);
Console.WriteLine(pbmMetadata.Encoding);
```

`PbmMetadata` includes values such as:

- `ColorType`
- `ComponentType`
- `Encoding`

When to Use PBM / PGM / PPM

The Netpbm family is usually worth considering when:

- You need a very simple interchange format.
- Human-readable plain-text image data is useful for debugging or tooling.
- You are working with existing Netpbm-style workflows.

It is usually a poor fit when:

- File size matters.
- You need richer metadata, transparency, or modern delivery characteristics.

For more compact or full-featured output, start with PNG, WebP, or QOI.

Practical Guidance

- Use Netpbm formats for simple tooling, tests, and interchange workflows where readability matters.
- Avoid them for public delivery or storage where compression, metadata, or alpha support matters.
- Be explicit about plain versus binary encoding when files are consumed by external tools.
- Prefer PNG when you need a simple lossless format with a much broader ecosystem.

TGA

TGA, or Truevision TGA, is a straightforward raster format that still shows up in graphics tooling and content pipelines. It is less about delivery to browsers and more about simple pixel storage with familiar bit-depth and compression choices.

ImageSharp exposes TGA-specific APIs through `TgaEncoder` and `TgaMetadata`.

Format Characteristics

TGA is best thought of as a simple raster format for tooling and interchange.

TGA is common in graphics pipelines because it is straightforward and historically supported by many tools. It can store 8, 16, 24, or 32-bit output, and the alpha-channel bit count is part of the metadata story. That makes it useful when an asset pipeline expects a simple raster file with predictable channel layout.

Run-length encoding can reduce file size for images with repeated runs of pixels, such as flat-color artwork or masks. It is much less useful for noisy images or photographs. Choose compression based on the assets being exchanged and the expectations of the consuming tool.

A few practical implications:

- ImageSharp can write TGA output at 8, 16, 24, or 32 bits per pixel.
- ImageSharp supports uncompressed output or run-length encoded output through `TgaCompression`.
- `TgaMetadata` exposes encoded bit depth and alpha-channel bit information.
- TGA is often useful in asset pipelines, but it is rarely the best choice for browser-facing delivery.

Save as TGA

Use `TgaEncoder` when you want explicit TGA bit-depth and compression control:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Tga;

using Image image = Image.Load("input.png");

image.Save("output.tga", new TgaEncoder
{
```

```
    BitsPerPixel = TgaBitsPerPixel.Bit32,
    Compression = TgaCompression.RunLength
});
```

Key TGA Encoder Options

The most commonly used `TgaEncoder` options are:

- `BitsPerPixel` controls the encoded TGA bit depth.
- `Compression` switches between uncompressed and run-length encoded output.

Bit depth controls more than file size. A 32-bit TGA can carry alpha, while lower bit depths may not represent the same source data. Check `AlphaChannelBits` when moving assets through tools that care about alpha channels.

Read TGA Metadata

Use `GetTgaMetadata()` to inspect TGA-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Tga;

using Image image = Image.Load("input.tga");

TgaMetadata tgaMetadata = image.Metadata.GetTgaMetadata();

Console.WriteLine(tgaMetadata.BitsPerPixel);
Console.WriteLine(tgaMetadata.AlphaChannelBits);
```

`TgaMetadata` includes values such as:

- `BitsPerPixel`
- `AlphaChannelBits`

When to Use TGA

TGA is usually worth considering when:

- You are working with graphics or content-pipeline tooling that expects TGA.
- You want a simple raster format with predictable bit-depth choices.

TGA is usually a poor fit when:

- The output is primarily intended for browsers or compact delivery.
- You need richer metadata or broader ecosystem support.

For ordinary web or application output, PNG, JPEG, and WebP are usually better starting points.

Practical Guidance

- Use TGA when an asset pipeline or graphics toolchain explicitly expects it.
- Check alpha-channel bit depth when moving assets between tools.
- Keep an editable source format alongside generated TGA assets.
- Prefer PNG, JPEG, or WebP for application and web delivery.

QOI

QOI, the Quite OK Image Format, is a simple lossless image format designed around easy implementation and fast encode/decode loops. In ImageSharp, it is a compact option when you want lossless RGB or RGBA output without the broader feature surface of PNG.

ImageSharp exposes QOI-specific APIs through `QoiEncoder` and `QoiMetadata`.

Format Characteristics

QOI is best thought of as a small, focused lossless format.

QOI is intentionally simple: it stores RGB or RGBA pixel streams using a compact set of operations that are easy to encode and decode. That simplicity is the appeal. It can be fast and convenient in controlled pipelines where both sides agree to use the format.

The tradeoff is ecosystem breadth. QOI does not carry the same metadata surface as PNG, WebP, or TIFF, and it is not a browser delivery format. Use it when simple lossless interchange is valuable and the producer and consumer are both under your control.

A few practical implications:

- QOI is lossless.
- The format stores image channel count as RGB or RGBA.
- The format stores a simple color-space flag.
- In ImageSharp, `Channels` and `ColorSpace` are informative metadata. They do not change how the pixel chunks themselves are encoded.
- QOI has a much smaller ecosystem than PNG or WebP.

Save as QOI

Use `QoiEncoder` when you want QOI output:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Qoi;

using Image image = Image.Load("input.png");

image.Save("output.qoi", new QoiEncoder
{
    Channels = QoiChannels.Rgba,
```

```
    ColorSpace = QoiColorSpace.SrgbWithLinearAlpha
});
```

Key QOI Metadata

The most useful QOI-specific values are:

- `Channels`, which records whether the image is RGB or RGBA.
- `ColorSpace`, which records whether the image is tagged as sRGB with linear alpha or all-channels-linear.

Those values describe how consumers should interpret the stored pixel stream. They are not a substitute for full color-management metadata, so QOI is usually a poor choice when ICC workflows or rich metadata are part of the contract.

Read QOI Metadata

Use `GetQoiMetadata()` to inspect QOI-specific metadata:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Qoi;

using Image image = Image.Load("input.qoi");

QoiMetadata qoiMetadata = image.Metadata.GetQoiMetadata();

Console.WriteLine(qoiMetadata.Channels);
Console.WriteLine(qoiMetadata.ColorSpace);
```

When to Use QOI

QOI is usually worth considering when:

- You want a simple lossless format in a controlled pipeline.
- Fast, straightforward encoding and decoding matters more than ecosystem breadth.

QOI is usually a poor fit when:

- You need broad browser or tool compatibility.
- You need richer metadata or more mature ecosystem support.

For wider compatibility, PNG and WebP are usually better starting points.

Practical Guidance

- Use QOI in controlled pipelines where both producer and consumer agree on the format.
- Prefer PNG or WebP when files need to be opened broadly by browsers, design tools, or operating systems.
- Treat QOI as an interchange or internal-storage choice, not a general publishing format.
- Test size and speed against representative data; simple formats are not automatically smaller for every image.

Processing Images

Once an image is in memory, most work in ImageSharp happens through small ordered processing pipelines. That is one of the library's strengths: the code you write usually reads in the same order the pixels are transformed, which makes even longer pipelines approachable for newcomers.

The main entry points are `Mutate` and `Clone`:

- `Mutate()` applies processors to the current image.
- `Clone()` creates a deep copy and applies the processors to that copy.

Processors are deliberately composable. Each call in the pipeline receives the result of the previous call, so the code order is also the image-processing order. That makes pipelines easy to read, but it also means a misplaced operation can change the result significantly.

The Raster Processing Model

ImageSharp processes raster images: a rectangular grid of pixels with a width, height, pixel type, and optional frame collection. Processing changes that in-memory image model. It does not work on encoded JPEG blocks, PNG chunks, file extensions, or metadata records unless a processor is specifically about those concerns.

That distinction matters when designing a pipeline. A crop removes pixel regions from the decoded image. A resize resamples pixels into a new grid. A color effect changes pixel values. An encoder later decides how those processed pixels become JPEG, PNG, WebP, TIFF, or another file format. Metadata such as EXIF orientation and ICC profiles can influence the right processing choices, but they are not the same thing as pixel processing.

Mutate the Current Image

Use `Mutate()` when you want to transform the current image in place:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Resize(1200, 800)
```

```
    .Grayscale());

image.Save("output.jpg");
```

This is the most common choice for request processing, thumbnails, and one-way export workflows.

Use `Mutate()` when the loaded image is an intermediate value and there is no need to keep the original pixels. This keeps ownership simple and avoids a second full image allocation.

`Mutate()` does not mean "unsafe"; it means the current image instance is the output. That is exactly what you want for many pipelines: load a file, normalize it, resize it, adjust it, and save the result. The important ownership question is whether any later code still needs the original pixels. If not, mutating keeps memory use and code shape straightforward.

Clone When You Need to Preserve the Original

Use `Clone()` when the original image must remain unchanged:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");
using Image thumbnail = image.Clone(x => x
    .Resize(160, 160)
    .Sepia());

thumbnail.Save("thumbnail.jpg");
```

This is useful when you need multiple derived outputs from the same source image.

Use `Clone()` when the original image is a reusable source asset: for example, generating several thumbnail sizes, producing multiple export formats, or running a preview operation while keeping an editable original.

`Clone()` creates a separate image with its own pixel buffers. That makes it the right tool for fan-out workflows, but it is not free. If a service generates five output sizes from one upload, cloning for each output may be worth the clarity. If a pipeline only writes one result, cloning usually just allocates another full image for no benefit.

Build Ordered Pipelines

Processor order matters. For example, auto-orienting before resizing usually produces more predictable results than resizing first and correcting orientation later:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Crop(new Rectangle(200, 100, 1200, 800))
    .Resize(600, 400)
    .BackgroundColor(Color.White));
```

As a rule of thumb:

- Normalize orientation early.
- Crop before expensive down-stream work when the crop meaningfully reduces the pixel area.
- Apply output-specific effects near the end of the pipeline.
- Save with an explicit encoder when output quality, metadata, compression, or compatibility matters.

A useful way to think about processor order is to group the pipeline into stages:

1. Normalize the source into the coordinate system you intend to work in.
2. Remove pixels you no longer need.
3. Resize or otherwise change geometry.
4. Apply visual effects that depend on the final output.
5. Encode with explicit output settings.

That ordering is not mandatory, but it gives you a good default. For example, a blur before resize looks different from a blur after resize, and a crop before an expensive effect can reduce the amount of work dramatically.

Common Processing Topics

- Resizing Images covers `Resize()` and `ResizeOptions`.
- Crop, Pad, and Canvas covers `Crop()`, `Pad()`, `BackgroundColor()`, and `EntropyCrop()`.
- Rotate, Flip, and Auto-Orient covers `AutoOrient()`, `Rotate()`, `Flip()`, and `RotateFlip()`.
- Color and Effects covers `Grayscale()`, `Sepia()`, `Brightness()`, `Contrast()`, `Hue()`, `Saturate()`, and `Opacity()`.

- Quantization, Palettes, and Dithering covers `Quantize()`, palette selection, encoder quantizers, and dithering algorithms.
- Working with Animations covers multi-frame workflows for GIF, APNG, and WebP.

Related APIs

Most built-in processors live under the `SixLabors.ImageSharp.Processing` namespace. Import that namespace in files where you build processing pipelines.

Practical Guidance

Use `Mutate()` for one-way processing and `Clone()` when the original image remains a source. Order processors in the same order you would describe the visual transformation, and be especially deliberate around orientation, crop, resize, and effects. Save with explicit encoder options at the final boundary so the processed pixels are not undermined by accidental output defaults.

Resizing Images

Resizing looks simple on the surface, but it is also one of the easiest places to make an image look subtly wrong. Aspect ratio, resampler choice, fit mode, alpha handling, and decode-time downscaling all influence the result, so this page walks through the common paths in the order most people need them.

The simple `Resize()` overloads are good for direct width and height changes, while `ResizeOptions` gives you control over fit mode, anchor position, background padding, sampler choice, alpha handling, and manual target rectangles.

Start by choosing the layout promise. If the full image must remain visible, use a fit mode such as `Max` or `Pad`. If the output box must be completely filled, use `Crop` and choose an anchor or focal point. If exact aspect ratio is not important, `Stretch` is available, but it should be a deliberate visual choice.

Basic Resize

Use the basic overloads when you already know the destination size:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Resize(1200, 800));
image.Save("output.jpg");
```

ImageSharp defaults to `KnownResamplers.Bicubic`, which is a solid general-purpose choice for both downscaling and upscaling.

If either `width` or `height` is `0`, ImageSharp calculates the missing dimension to preserve aspect ratio:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Resize(600, 0));
```

Choose the Right Resampler

Resampling is the part of resizing that decides how source pixels contribute to destination pixels. When shrinking an image, many source pixels must be combined into fewer destination pixels. When enlarging an image, new destination pixels must be estimated from the surrounding source pixels. Different resamplers make different tradeoffs between sharpness, smoothness, ringing, aliasing, and speed.

`Bicubic` is ImageSharp's default because it is a balanced general-purpose choice. It produces smoother results than nearest-neighbor sampling and is less prone to visible ringing than more aggressive high-lobe filters. For many application thumbnails and ordinary web images, it is a good starting point.

`Lanczos3` is often a strong choice for high-quality downscaling because it preserves detail well. That extra sharpness can also produce halos or ringing around hard contrast edges, so it should be tested on screenshots, line art, product photos, and portraits before becoming a global default. Larger Lanczos variants such as `Lanczos5` and `Lanczos8` use wider kernels and can preserve even more detail, but they cost more work and can make ringing more visible.

`Spline`, `CatmullRom`, `MitchellNetravali`, `Robidoux`, and `RobidouxSharp` are useful when you are tuning a visual pipeline and want a different balance of softness and edge contrast. The right choice is content-dependent: UI screenshots, portraits, scanned documents, and generated artwork can prefer different filters.

`NearestNeighbor` does no smoothing. It is the right choice for pixel art, masks, indexed-style data, and any workflow where hard pixel boundaries must remain hard. It is usually the wrong choice for photos because it creates blocky stair-step artifacts.

Simple filters such as `Box`, `Triangle`, and `Hermite` can be useful when speed, softness, or predictable low-detail output matters more than maximum sharpness.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.png");

image.Mutate(x => x.Resize(320, 240, KnownResamplers.Lanczos3));
```

If you are building a reusable pipeline, choose a default sampler per content type rather than one sampler for everything. For example, product thumbnails, user avatars, pixel-art previews, and scanned documents often deserve different choices.

Use ResizeOptions for Real-World Layout Rules

`ResizeOptions` is the main API for fit-and-fill workflows. When you use it, set `Mode` explicitly; its default is `Crop`.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Resize(new ResizeOptions
{
    Size = new Size(300, 300),
    Mode = ResizeMode.Crop,
    Position = AnchorPositionMode.Center,
    Sampler = KnownResamplers.Lanczos3,
    Compand = true
}));
```

The resize modes are:

- `Crop`: fills the target box and crops overflow.
- `Pad`: fits within the target box and fills the remainder.
- `BoxPad`: pads without upscaling the original image; when downscaling it behaves like `Pad`.
- `Max`: fits within the box without cropping.
- `Min`: scales until the shortest side reaches the target, without upscaling.
- `Stretch`: ignores aspect ratio and forces the exact size.
- `Manual`: uses `TargetRectangle` to place the resized result explicitly.

For user-visible images, `Crop`, `Pad`, and `Max` cover most layouts. `Manual` is for composition systems where you already calculated the destination rectangle. `Stretch` is mostly for data, masks, or intentionally distorted visual effects.

Position, Padding, and Manual Placement

`ResizeOptions` also controls where the result lands inside the output canvas:

- `Position` sets the anchor for crop and pad operations.
- `CenterCoordinates` lets you bias crop focus more precisely.
- `PadColor` fills the background for padded results.

- `TargetRectangle` is required for `ResizeMode.Manual`.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.png");

image.Mutate(x => x.Resize(new ResizeOptions
{
    Size = new Size(1200, 1200),
    Mode = ResizeMode.Pad,
    Position = AnchorPositionMode.Center,
    PadColor = Color.White
}));
```

Companding and Alpha Handling

Resizing blends neighboring pixels. If those pixel values are blended directly in a gamma-encoded space, midtones can shift in ways that are visible on gradients, shadows, and high-contrast photographic content. `Compand` enables gamma-aware resizing so interpolation happens in a space that is often visually more appropriate. It costs extra work, so test it against your image set instead of enabling it blindly everywhere.

Alpha needs similar care. Transparent images usually look better when color channels are interpolated in premultiplied form, because transparent edge pixels then contribute color in proportion to their coverage. `PremultiplyAlpha` defaults to `true` and should normally stay enabled for logos, sprites, UI elements, and cutouts. Turning it off is an advanced choice for data images or pipelines that already handle alpha in a very specific way.

Decode Smaller When That Is Enough

If you only need a bounded preview or thumbnail, consider decoding directly to a smaller size with `DecoderOptions.TargetSize` instead of fully decoding and then resizing. ImageSharp treats that target as a fit-within box equivalent to `ResizeMode.Max`.

See Loading, Identifying, and Saving and Security Considerations for examples.

WrapMemory Caveat

`Resize()` changes image dimensions and therefore needs a new backing buffer. Images created with `WrapMemory(...)` are best suited to fixed-size interop workflows, so resize them only after

copying or cloning into a regular ImageSharp-owned image.

See Interop and Raw Memory for the full wrapped-memory guidance.

Practical Guidance

A resize should start from the product promise, not from the overload. If a user must see the whole image, choose a fit mode such as `Max` or `Pad`. If the layout must be filled edge-to-edge, choose `Crop` and decide how the crop should be anchored. If you are building a composition engine and already know the exact destination rectangle, use `Manual`. `Stretch` is available, but it should be reserved for cases where distortion is acceptable or meaningful.

For user-uploaded photos, call `AutoOrient()` before resizing unless preserving the raw encoded pixel orientation is intentional. Crop coordinates, anchors, and focal points are much easier to reason about after the pixels match the way a person sees the image. For very large inputs where only a bounded preview is needed, `DecoderOptions.TargetSize` can reduce decode cost before the resize pipeline runs.

Resampler choice should be tested against representative images. A sharper result is not always a better result: line art, screenshots, photos, and pixel art often want different tradeoffs. After resizing, save with an explicit encoder when final quality, compression, metadata, or color handling must be predictable.

Related Topics

- Processing Images
- Crop, Pad, and Canvas
- Generate Thumbnails

Crop, Pad, and Canvas

Cropping and canvas operations are closely related, but they solve different problems. Cropping decides which part of the current pixels you keep. Canvas operations decide how much room the image has and where those pixels sit inside it.

Thinking about those as separate questions makes the API much easier to navigate.

Coordinate choices matter here. Crop rectangles describe source pixels you want to keep. Padding and canvas-style operations describe the destination bounds you want after the operation. When a workflow feels confusing, write those two rectangles down separately: source region first, output canvas second.

Crop to an Explicit Rectangle

Use `Crop()` when you know the exact rectangle you want to keep:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Crop(new Rectangle(100, 80, 1200, 800)));
```

This removes everything outside the requested bounds.

The crop rectangle is expressed in the image's current coordinate space. If the source may contain EXIF orientation, call `AutoOrient()` before choosing crop coordinates that should match what a person sees.

Cropping changes the image size and shifts the remaining pixels so the cropped rectangle becomes the new image. Any coordinates you calculated before the crop no longer refer to the same positions afterward. In workflows that add overlays, annotations, or drawing after cropping, calculate those later positions against the post-crop image.

Crop by Width and Height

If the crop should start at the top-left corner, you can pass just width and height:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Crop(800, 600));
```

This overload is intentionally simple: it keeps the top-left region of the current image. Use the rectangle overload when the crop needs to be centered, anchored, or based on detected content.

Pad to a Larger Canvas

Use `Pad()` when you want to enlarge the canvas without scaling the image:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.png");

image.Mutate(x => x.Pad(1200, 1200, Color.White));
```

This is useful when generating square thumbnails, social cards, or export assets that require a fixed output size.

Padding does not scale the original image. If the image must fit inside a larger box with a background, resize to the intended content size first, then pad to the final canvas size.

Fill Transparent Areas or Flatten Onto a Background

Use `BackgroundColor()` to fill transparent pixels or composite the current image over a solid color:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.png");

image.Mutate(x => x.BackgroundColor(Color.White));
```

This is a common step before saving a transparent source image to a format that does not support transparency.

The background color becomes real pixel data. If you flatten before resizing, the background participates in interpolation at transparent edges. If you resize first and flatten later, transparent edge pixels are resized with alpha preserved and then composited onto the chosen background. For logos and cutouts, the difference can be visible around antialiased edges.

Crop Automatically Based on Content

Use `EntropyCrop()` when you want ImageSharp to trim low-information borders automatically:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.EntropyCrop());
```

This can be useful for removing large flat borders or whitespace-like areas before additional processing.

Automatic cropping is content-driven, so treat it as a convenience rather than a layout contract. It is useful for cleanup workflows, but explicit rectangles or resize anchors are better when output dimensions must be predictable.

Combine Crop and Resize

Cropping and resizing are often used together:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Crop(new Rectangle(200, 120, 1000, 1000))
    .Resize(400, 400));
```

Cropping first can reduce the amount of pixel data that later processors need to touch.

Related Topics

- Processing Images
- Resizing Images
- Rotate, Flip, and Auto-Orient

Practical Guidance

Normalize orientation before choosing crop rectangles that should match what users see. Keep source-region decisions separate from final canvas-size decisions: crop decides what pixels survive, resize decides how large they become, and padding decides how much output room surrounds them. Use automatic cropping for cleanup, but prefer explicit rectangles, anchors, or resize options when the output dimensions are part of a layout contract.

Rotate, Flip, and Auto-Orient

Orientation issues usually show up the first time a phone photo looks rotated even though the file came straight from a camera roll. This page covers the small set of operations you will use most often to normalize orientation metadata or apply explicit geometric transforms.

The most common entry points are `AutoOrient()`, `Rotate()`, `Flip()`, and `RotateFlip()`.

Correct Orientation from EXIF Metadata

Use `AutoOrient()` early in your pipeline to normalize images captured by cameras and phones:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.AutoOrient());
```

`AutoOrient()` uses embedded EXIF orientation metadata when it is present. This is often the right first processing step for user-uploaded photos.

Rotate by a Known Angle

Use `Rotate()` to rotate by a specific number of degrees or a known rotate mode:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Rotate(90));
```

ImageSharp also supports `RotateMode` when you want a predefined quarter-turn rotation.

Flip Horizontally or Vertically

Use `Flip()` to mirror the image:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Flip(FlipMode.Horizontal));
```

See `FlipMode` for the available options.

Combine Rotation and Flipping

Use `RotateFlip()` when you need both operations together:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.RotateFlip(RotateMode.Rotate90, FlipMode.Vertical));
```

This can make intent clearer than chaining separate calls when the final transformation is a single orientation step.

Normalize Orientation Before Resizing

In most workflows, orient the image before cropping or resizing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Resize(1200, 800));
```

That keeps downstream dimensions and crop coordinates aligned with the final visual orientation.

Related Topics

- Processing Images
- Crop, Pad, and Canvas
- Working with Metadata

Practical Guidance

- Call `AutoOrient()` early for user-uploaded photos unless preserving raw pixel orientation is intentional.
- Normalize orientation before crop and resize operations based on what a person sees.
- Strip or update orientation metadata only after the pixel data reflects the intended display orientation.
- Test orientation workflows with real phone images, not only images already stored upright.

Color and Effects

Color adjustments are where many small ImageSharp processors start to feel composable instead of isolated. You can reach for named helpers like `Grayscale()` and `Sepia()`, or drop down to `ColorMatrix` when you want to express the transformation yourself.

ImageSharp includes a wide range of processors for tonal adjustment, color transforms, and simple stylistic effects. Common entry points include `Grayscale()`, `Sepia()`, `Brightness()`, `Contrast()`, `Hue()`, `Saturate()`, and `Opacity()`. Under the hood, many of these effects are expressed as a `ColorMatrix` and applied with `Filter()`.

Convert to Grayscale

Use `Grayscale()` to remove color information:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Grayscale());
```

ImageSharp also supports `GrayscaleMode` when you need a specific conversion mode.

Grayscale conversion is not just averaging red, green, and blue. Different modes weight channels differently, and those choices affect perceived brightness. Use a specific `GrayscaleMode` when output must match a known visual or analytical expectation.

Apply a Sepia Tone

Use `Sepia()` for a classic warm-tone effect:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x.Sepia());
```

Adjust Brightness and Contrast

Use `Brightness()` and `Contrast()` to tune exposure and punch:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .Brightness(1.1F)
    .Contrast(1.2F));
```

Values greater than `1` increase the effect. Values less than `1` reduce it.

Brightness and contrast are simple global operations. They are useful for quick output tuning, but they do not replace tone mapping, exposure recovery, or color-managed workflows. Apply them after geometry changes when the effect is meant for the final exported image.

Shift Hue and Saturation

Use `Hue()` and `Saturate()` when you want to push color balance or intensity:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .Hue(30)
    .Saturate(1.25F));
```

Hue shifts rotate color relationships, while saturation changes color intensity. Both can create out-of-gamut or unnatural-looking results if pushed too far. Use small values for photographic correction and stronger values for intentional stylized output.

Adjust Opacity

Use `Opacity()` to reduce alpha values across the image:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.png");

image.Mutate(x => x.Opacity(0.5F));
```

This is most useful when working with images that already include transparency.

Opacity changes alpha values; it does not composite the image onto a background. If the final format cannot store alpha, use `BackgroundColor()` or another compositing step before saving.

Use ColorMatrix for Custom Filters

`ColorMatrix` is the low-level type for custom channel transforms. It is a 5×4 matrix over the color and alpha channels, and `Filter()` applies that matrix to the image.

That makes `ColorMatrix` the right tool when the built-in processors are close to what you need but not quite exact. The diagonal fields such as `M11`, `M22`, `M33`, and `M44` scale channels, while the last-row fields such as `M51`, `M52`, and `M53` add channel bias.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

ColorMatrix warmTone = new()
{
    M11 = 1.08F,
    M22 = 1.00F,
    M33 = 0.92F,
    M44 = 1F,
    M51 = 0.02F
};

image.Mutate(x => x.Filter(warmTone));
```

Reuse Known Filter Matrices

If you want matrix-based control without building every matrix by hand, use `KnownFilterMatrices`. The built-in brightness, contrast, grayscale, hue, saturation, opacity, sepia, and preset camera-

look filters all come from this API surface.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

ColorMatrix matrix =
    KnownFilterMatrices.CreateHueFilter(20F)
    * KnownFilterMatrices.CreateSaturateFilter(1.15F);

image.Mutate(x => x.Filter(matrix));
```

You can also use the predefined matrices directly for stylized looks such as `KodachromeFilter` and `PolaroidFilter`.

Chain Effects in a Single Pipeline

These processors are designed to compose cleanly:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Resize(800, 800)
    .Brightness(1.05F)
    .Contrast(1.1F)
    .Saturate(1.15F));
```

As with other processors, order matters when combining effects.

Related Topics

- Processing Images
- Rotate, Flip, and Auto-Orient
- Crop, Pad, and Canvas

Practical Guidance

Apply color effects after geometry changes when the effect is output-specific. Keep source images in a suitable working color space before judging color adjustments, and use explicit encoder settings afterward so compression does not hide the result you tuned. Test effects on representative images, because a setting that flatters one sample can damage skin tones, brand colors, gradients, or shadows elsewhere.

Quantization, Palettes, and Dithering

Quantization is the part of image processing where you stop thinking in terms of continuous color and start thinking in terms of a limited palette. Even if you never call `Quantize()` directly, it still matters because formats like GIF, indexed PNG, CUR, and ICO rely on the same ideas.

In ImageSharp, quantization matters both as an explicit processing step and as part of formats that write indexed or palette-constrained output.

Where Quantization Shows Up

Quantization is relevant in a few common places:

- `Quantize()` when you want to reduce colors as part of a processing pipeline.
- `GifEncoder`, because GIF output is palette based.
- `PngEncoder` when you target `PngColorType.Palette`.
- `IcoEncoder` and `CurEncoder` for icon and cursor workflows.

Use quantization when you want smaller palette-based outputs, fixed-color branding palettes, retro or posterized looks, or more control over indexed formats.

Quantize as a Processing Step

The default `Quantize()` overload uses `KnownQuantizers.Hexadecatree`, which is a fast, good general-purpose adaptive quantizer.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;
using SixLabors.ImageSharp.Processing.Processors.Quantization;

using Image image = Image.Load("input.png");

image.Mutate(x => x.Quantize(new WuQuantizer(new QuantizerOptions
{
    MaxColors = 64
})));

image.Save("output.png");
```

This remaps the image content to a smaller palette before you save it. That can be useful when you want the palette reduction to be part of the visible processing result rather than only an encoder detail.

Choose the Quantizer

`KnownQuantizers` exposes reusable built-in choices:

- `KnownQuantizers.Hexadecatree` for a fast adaptive quantizer with solid general results.
- `KnownQuantizers.Wu` for high-quality adaptive palette generation.
- `KnownQuantizers.WebSafe` for the fixed web-safe palette.
- `KnownQuantizers.Werner` for the fixed Werner palette.

When you need more control, create a quantizer directly:

- `WuQuantizer` for adaptive palette generation with configurable `QuantizerOptions`.
- `HexadecatreeQuantizer` for fast adaptive quantization.
- `PaletteQuantizer` when you want to force output to a known palette.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;
using SixLabors.ImageSharp.Processing.Processors.Quantization;

using Image image = Image.Load("input.png");

Color[] brandPalette =
{
    Color.Black,
    Color.White,
    Color.ParseHex("0057B8"),
    Color.ParseHex("FFD100")
};

image.Mutate(x => x.Quantize(new PaletteQuantizer(brandPalette)));
```

Dithering Choices

`QuantizerOptions` controls the main quantization tradeoffs:

- `MaxColors` limits the palette size.
- `Dither` selects the dithering algorithm.
- `DitherScale` adjusts how strongly dithering is applied.

- `ColorMatchingMode` chooses how pixels are matched back to palette entries after the palette has been built.
- `TransparencyThreshold` and `TransparentColorMode` affect how transparent pixels are reduced into the palette.

`ColorMatchingMode` has two built-in choices:

- `Coarse` is the default and favors speed.
- `Exact` uses more precise palette matching for cases where the extra accuracy matters more than throughput.

The API surface is intentionally small here: pick the quantizer that builds the palette you want, then choose either `Coarse` or `Exact` for the palette-matching pass.

`KnownDitherings` exposes the built-in dithering algorithms, including ordered Bayer variants and error-diffusion algorithms such as Floyd-Steinberg, Atkinson, Burks, Jarvis-Judice-Ninke, and Stucki.

Set `Dither = null` when you want flatter output with no dithering pattern. Keep dithering enabled when you want to hide banding in gradients or other smooth transitions.

ImageSharp also has a separate `Dither()` processing extension. Its default overload reduces the image to the web-safe palette using `KnownDitherings.Bayer8x8`, and other overloads let you dither against a palette you provide.

Encoder-Time Quantization

Many palette-sensitive exports are better controlled at save time by configuring the encoder directly:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Png;
using SixLabors.ImageSharp.Processing;
using SixLabors.ImageSharp.Processing.Processors.Quantization;

using Image image = Image.Load("input.png");

image.Save("output-indexed.png", new PngEncoder
{
    ColorType = PngColorType.Palette,
    Quantizer = new WuQuantizer(new QuantizerOptions
    {
        MaxColors = 128,
```

```
        ColorMatchingMode = ColorMatchingMode.Exact,
        Dither = KnownDitherings.FloydSteinberg,
        DitherScale = 0.75F,
        TransparentColorMode = TransparentColorMode.Preserve
    }),
    PixelSamplingStrategy = new ExtensivePixelSamplingStrategy()
});
```

This approach is usually the right choice when you want format-specific palette output without permanently changing the in-memory image first.

Sampling and Transparency

Encoders that implement quantizing behavior also expose a pixel-sampling strategy. The default strategy samples a subset of pixels on large inputs to keep palette generation practical. `ExtensivePixelSamplingStrategy` scans all pixels instead, which can improve results when rare colors matter, at the cost of more work.

Transparency handling matters most for GIF, palette PNG, ICO, and CUR output. `TransparentColorMode` controls how transparency is represented in the reduced palette, while `TransparencyThreshold` controls when partially transparent pixels are treated as transparent during quantization.

Related Topics

- GIF
- PNG
- Convert Between Formats
- Read Image Info Without Decoding

Practical Guidance

- Choose quantization when palette output is a requirement, not as a default quality improvement.
- Pick dithering based on the content; it can hide banding but add visible texture.
- Use extensive sampling only when rare colors matter enough to justify the extra work.
- Pay special attention to transparency for GIF, palette PNG, ICO, and CUR output.

Working with Animations

ImageSharp treats animation as a multi-frame `Image<TPixel>`. The authoring model is the same whether you save as GIF, animated PNG (APNG), or animated WebP: build the frame collection, set image-level animation metadata, set per-frame metadata, then save with the encoder for the format you want.

You still work with full-size frames in memory, but ImageSharp's animated encoders optimize the output by de-duplicating unchanged pixels between frames and writing only the differing region for later frames where the format supports it.

For format-specific background, palette, and compression tradeoffs, see GIF, PNG, and WebP. Windows animated cursors use their own container and metadata model; see ANI. This page focuses on the shared multi-frame workflow.

Build a Multi-Frame Animation

The root frame is the first animation frame. Additional frames are appended through `ImageFrameCollection.AddFrame()`, which clones the source frame. In ImageSharp, animation frames must always match the image dimensions.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

Color[] colors =
{
    Color.Orange,
    Color.DeepSkyBlue,
    Color.MediumSeaGreen
};

using Image<Rgba32> animation = new(120, 120, colors[0].ToPixel<Rgba32>());

for (int i = 1; i < colors.Length; i++)
{
    using Image<Rgba32> frameImage = new(120, 120, colors[i].ToPixel<Rgba32>());
    animation.Frames.AddFrame(frameImage.Frames.RootFrame);
}
```

When you start from `Color` values, convert them to the target pixel type with `ToPixel<TPixel>()` before passing them to generic image constructors.

ImageSharp Optimizes Later Frames

When encoding GIF, APNG, or animated WebP, ImageSharp compares later frames with the previous composited result and trims unchanged pixels from the encoded output. In practice, that means you usually author full-canvas frames, but the encoder writes only the changed bounds for later frames when that produces an equivalent animation.

This is especially helpful for sprite, UI, and cursor-style animations where only a small region changes from one frame to the next.

Configure GIF Metadata

Use `GifMetadata` and `GifFrameMetadata` when you are saving palette-based animation:

- `RepeatCount` controls looping. `0` means loop indefinitely.
- `FrameDelay` is measured in hundredths of a second.
- `DisposalMode` controls how the previous composited frame is treated before the next frame is shown.

Starting from an existing `Image<Rgba32> animation`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Gif;
using SixLabors.ImageSharp.PixelFormats;

GifMetadata gifMetadata = animation.Metadata.GetGifMetadata();
gifMetadata.RepeatCount = 0;

foreach (ImageFrame<Rgba32> frame in animation.Frames)
{
    GifFrameMetadata frameMetadata = frame.Metadata.GetGifMetadata();
    frameMetadata.FrameDelay = 10;
    frameMetadata.DisposalMode = FrameDisposalMode.RestoreToBackground;
}

animation.Save("output.gif", new GifEncoder
{
    ColorTableMode = FrameColorTableMode.Global
});
```

GIF is always palette-based, so palette selection still matters. See GIF and Quantization, Palettes, and Dithering for the full quantization story.

Configure APNG Metadata

Use `PngMetadata` and `PngFrameMetadata` when you want animated PNG output:

- `RepeatCount` controls looping.
- `AnimateRootFrame` controls whether the root frame participates in the animation.
- `FrameDelay` is stored as a `Rational`.
- `DisposalMode` and `BlendMode` control how frames compose.

Continuing from the same `Image<Rgba32> animation`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Png;
using SixLabors.ImageSharp.PixelFormats;

PngMetadata pngMetadata = animation.Metadata.GetPngMetadata();
pngMetadata.RepeatCount = 0;
pngMetadata.AnimateRootFrame = true;

foreach (ImageFrame<Rgba32> frame in animation.Frames)
{
    PngFrameMetadata frameMetadata = frame.Metadata.GetPngMetadata();
    frameMetadata.FrameDelay = new Rational(1, 10);
    frameMetadata.DisposalMode = FrameDisposalMode.DoNotDispose;
    frameMetadata.BlendMode = FrameBlendMode.Over;
}

animation.Save("output.png", new PngEncoder());
```

Configure Animated WebP Metadata

Use `WebpMetadata` and `WebpFrameMetadata` when you want animated WebP output:

- `RepeatCount` controls looping.
- `BackgroundColor` is used by the format and matters when a frame uses `RestoreToBackground`.
- `FrameDelay` is measured in milliseconds.
- `DisposalMode` and `BlendMode` control how frames compose.

Continuing from the same `Image<Rgba32> animation`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Webp;
using SixLabors.ImageSharp.PixelFormats;

WebpMetadata webpMetadata = animation.Metadata.GetWebpMetadata();
webpMetadata.RepeatCount = 0;
webpMetadata.BackgroundColor = Color.Transparent;

foreach (ImageFrame<Rgba32> frame in animation.Frames)
{
    WebpFrameMetadata frameMetadata = frame.Metadata.GetWebpMetadata();
    frameMetadata.FrameDelay = 100;
    frameMetadata.DisposalMode = FrameDisposalMode.DoNotDispose;
    frameMetadata.BlendMode = FrameBlendMode.Over;
}

animation.Save("output.webp", new WebpEncoder());
```

Practical Guidance

- ImageSharp animation frames must always be the same size as the animation canvas.
- Set timing and disposal or blend metadata on every frame you care about rather than relying on defaults.
- Choose GIF when broad legacy compatibility matters more than palette and compression tradeoffs.
- Choose APNG when you want PNG-style lossless color and alpha with explicit frame blending and disposal.
- Choose WebP when you want a modern animated format with flexible compression behavior.

Working with Pixel Buffers

When you first start with ImageSharp, the indexer is often enough. As soon as performance, reuse across pixel formats, or interop enter the picture, it helps to know the other buffer-access patterns the library offers and why they exist.

This page is the map for that lower-level work.

Pixel Buffers Are Decoded Image Data

Pixel-buffer APIs expose the decoded raster grid in memory. They do not expose the original file bytes and they do not preserve format-specific packing such as JPEG blocks, PNG filters, GIF color-table indexes, or TIFF strip layout. By the time you are working with row spans, ImageSharp has decoded the source into the chosen `TPixel` representation.

Rows are addressed by image coordinates: `y` selects a row and `x` selects a pixel within that row. The APIs are row-oriented because image memory is optimized for scanning contiguous rows, even when a large image is backed by several internal buffers instead of one single allocation.

Choose the Right Access Pattern

Use:

- indexers for occasional pixel reads or writes;
- `ProcessPixelRows(...)` for fast row-by-row work in a known `TPixel`;
- `ProcessPixelRowsAsVector4(...)` for reusable pixel-format-agnostic processing;
- `CopyPixelDataTo(...)`, `LoadPixelData(...)`, and `WrapMemory(...)` when exchanging raw data with other systems.

Use Indexers for Simple Cases

If you only need to touch a few pixels, the indexer is the simplest option:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = new(400, 400);
image[200, 200] = Rgba32.White;
```

That is fine for small amounts of work, but repeated random pixel access has more overhead than processing full rows.

Use `ProcessPixelRows(...)` for Fast Known-Format Access

`Image<TPixel>` and `ImageFrame<TPixel>` expose `ProcessPixelRows(...)` so you can work with row spans directly:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = Image.Load<Rgba32>("input.png");

image.ProcessPixelRows(accessor =>
{
    for (int y = 0; y < accessor.Height; y++)
    {
        Span<Rgba32> row = accessor.GetRowSpan(y);

        for (int x = 0; x < row.Length; x++)
        {
            ref Rgba32 pixel = ref row[x];
            if (pixel.A == 0)
            {
                pixel = Rgba32.Transparent;
            }
        }
    }
});
```

This is the usual replacement for `LockBits`-style workflows when your algorithm already knows the working pixel format.

Process Multiple Images Row by Row

There are overloads for processing multiple images together:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> source = Image.Load<Rgba32>("source.png");
```

```
using Image<Rgba32> target = new(source.Width, source.Height);

source.ProcessPixelRows(target, (sourceAccessor, targetAccessor) =>
{
    for (int y = 0; y < sourceAccessor.Height; y++)
    {
        Span<Rgba32> sourceRow = sourceAccessor.GetRowSpan(y);
        Span<Rgba32> targetRow = targetAccessor.GetRowSpan(y);
        sourceRow.CopyTo(targetRow);
    }
});
```

This is a good fit for compositing, comparisons, or custom copy/merge logic.

Use `ProcessPixelRowsAsVector4(...)` for Pixel-Format-Agnostic Logic

If you want one processor that can run on many `TPixel` formats, use `ProcessPixelRowsAsVector4(...)`:

```
using System.Numerics;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.ProcessPixelRowsAsVector4(row =>
{
    for (int x = 0; x < row.Length; x++)
    {
        row[x] = Vector4.SquareRoot(row[x]);
    }
}));
```

This is extremely useful for reusable processing logic, but remember that it introduces conversion work to and from `Vector4`. It is often a great tradeoff for flexibility, but it is not always the fastest possible path for a hot server-side workload.

Convert to a Working Pixel Format

Sometimes the cleanest approach is to convert the image into a known working format first:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image source = Image.Load("input.tiff");
using Image<Rgba32> working = source.CloneAs<Rgba32>();
```

`CloneAs<TPixel>()` is especially useful when you want to standardize a pipeline on `Rgba32`, `Bgra32`, or another specific working format.

Copy Raw Pixels In and Out

Use `CopyPixelDataTo(...)` when you need a flattened copy of the root frame pixel buffer:

```
using SixLabors.ImageSharp.PixelFormats;

Rgba32[] pixels = new Rgba32[image.Width * image.Height];
image.CopyPixelDataTo(pixels);
```

Use `LoadPixelData(...)` when you want ImageSharp to create an owned image from raw input:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

byte[] rgba = GetRgbaBytes();
using Image<Rgba32> image = Image.LoadPixelData<Rgba32>(rgba, width, height);
```

There are stride-aware overloads for both pixel and byte input. For zero-copy interop, see Interop and Raw Memory.

`Span<T>` Rules Still Apply

The row spans you get from pixel accessors are `Span<T>` values. That means they are stack-only:

- They cannot be stored on the heap.
- They cannot cross `await` boundaries.
- They cannot be captured and used after the callback returns.

Keep all row work inside the callback that received the accessor.

Related Topics

- Pixel Formats
- Interop and Raw Memory
- Memory Management
- Migrating from System.Drawing

Practical Guidance

- Prefer row access over per-pixel indexers for non-trivial work.
- Keep span usage inside the callback that supplied the row accessor.
- Use `ProcessPixelRowsAsVector4(...)` when logic should be pixel-format agnostic.
- Convert to a known working pixel format when the algorithm benefits from simpler direct access.

Interop and Raw Memory

Most applications can stay inside ImageSharp's managed image model. When you need to exchange raw buffers with another library, device API, or native component, the important question becomes who owns the memory and whether you want a copy or a view.

That is the lens this page uses for the interop APIs.

Choose the Right API

Need	API	Copies pixel data?	Who owns the memory?
Import raw pixels into a normal ImageSharp-owned image	`Image.LoadPixelData(...)`	Yes	ImageSharp
Export pixels from an image	`CopyPixelDataTo(...)`	Yes	Caller
Wrap existing managed memory	`Image.WrapMemory(...)` with `Memory<T>` or `Memory<byte>`	No	Caller
Wrap an owned buffer and transfer disposal to ImageSharp	`Image.WrapMemory(...)` with `IMemoryOwner<T>` or `IMemoryOwner<byte>`	No	ImageSharp
Wrap unmanaged or pinned memory	`Image.WrapMemory(...)` pointer overloads	No	Caller

`WrapMemory(...)` Creates a View, Not a Copy

`Image.WrapMemory(...)` does not decode, convert, or clone the source pixels. It creates an `Image<TPixel>` view over memory you already have.

That makes it ideal for zero-copy interop, but it also means:

- the wrapped memory must already match the chosen `TPixel` layout;

- the source buffer lifetime rules still matter;
- the image is tied to the shape and stride of that existing buffer.

Import Raw Pixels with `LoadPixelData(...)`

Use `LoadPixelData(...)` when you want ImageSharp to create a normal owned image from existing pixels:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

byte[] rgba = GetRgbaBytes();
using Image<Rgba32> image = Image.LoadPixelData<Rgba32>(rgba, width, height);
```

There are overloads for:

- `ReadOnlySpan<TPixel>`
- `ReadOnlySpan<byte>`
- stride-aware pixel input
- stride-aware byte input

This is the safest choice when you do not need zero-copy behavior.

Export Raw Pixels with `CopyPixelDataTo(...)`

Use `CopyPixelDataTo(...)` when you want a flattened copy of the root frame pixels:

```
using SixLabors.ImageSharp.PixelFormats;

Rgba32[] pixels = new Rgba32[image.Width * image.Height];
image.CopyPixelDataTo(pixels);
```

There is also a `Span<byte>` overload if you need raw bytes instead of `TPixel` values.

If you need frame-specific access, the same API is available on `ImageFrame<TPixel>`.

Wrap Existing Managed Memory Without Copying

Use `Image.WrapMemory(...)` when you already have raw memory and want ImageSharp to view it in place:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

byte[] bgra = GetBgraBytes();
using Image<Bgra32> image = Image.WrapMemory<Bgra32>(bgra, width, height, rowStrideInBytes);
```

Important ownership rule:

- If you pass `Memory<T>` or `Memory<byte>`, ownership stays with you.
- The underlying buffer must remain valid for the entire lifetime of the image.

That makes `WrapMemory(...)` a good fit for shared buffers, pinned arrays, and memory you already control.

All `WrapMemory(...)` families also have overloads that accept `Configuration` and `ImageMetadata`, so you can attach metadata or use a non-default configuration while still keeping the zero-copy behavior.

Choose the Right `WrapMemory(...)` Overload

Within the `WrapMemory(...)` family, the main choice is what kind of source memory you have:

- use `Memory<TPixel>` when you already have typed pixel data;
- use `Memory<byte>` when the source buffer is raw bytes in a known `TPixel` layout;
- use `IMemoryOwner<TPixel>` or `IMemoryOwner<byte>` when you want the wrapped image to take ownership and dispose the backing owner;
- use pointer overloads only for unmanaged or pinned memory that cannot be expressed more safely as `Memory<T>` or `Memory<byte>`.

If the source buffer has row padding, use the stride-aware overload:

- `rowStride` for typed pixel memory;
- `rowStrideInBytes` for byte or pointer memory.

Transfer Ownership with `IMemoryOwner<T>`

If you want ImageSharp to dispose the wrapped buffer together with the image, use an `IMemoryOwner<T>` overload:

```
using System.Buffers;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

IMemoryOwner<byte> owner = MemoryPool<byte>.Shared.Rent(bufferSize);
using Image<Bgra32> image = Image.WrapMemory<Bgra32>(owner, width,
height, rowStrideInBytes);
```

In that form, the ownership of `owner` is transferred to the image. Do not dispose it yourself after wrapping.

Packed vs Strided Wrapped Buffers

Wrapped buffers can be either tightly packed or strided.

A packed wrapper uses one logical row immediately after the previous one. A strided wrapper uses extra elements or bytes between row starts, which is common when working with foreign image APIs.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

Rgba32[] source = new Rgba32[8];

using Image<Rgba32> image = Image.WrapMemory(
    source.AsMemory(),
    width: 3,
    height: 2,
    rowStride: 4);

bool contiguous = image.DangerousTryGetSinglePixelMemory(out _); // false
```

Important consequences of a strided wrapper:

- `DangerousTryGetSinglePixelMemory(...)` returns `false`, because it only succeeds when the image's logical pixels can be exposed as one tightly packed `width * height` block;
- `CopyPixelDataTo(...)` uses the backing row layout, so destination length must account for stride, not only `width * height`;
- row padding belongs to the wrapped view contract, so make sure the caller and callee agree on it.

Work with Native or Pinned Memory

`Image.WrapMemory(...)` also has pointer overloads for unmanaged or manually pinned buffers. Those overloads are intended for advanced interop scenarios where you already have a stable pointer and buffer length.

Use them carefully:

- The pointer must remain valid for the full lifetime of the wrapped image.
- The buffer size and row stride must match the image dimensions.
- If you have `Memory<T>` or `Memory<byte>`, prefer those overloads instead because they are much easier to reason about safely.

Wrapped Images Are Best for Fixed-Size Work

`WrapMemory(...)` is best when you want ImageSharp to operate on an existing fixed-size buffer.

That means in-place pixel work, analysis, format conversion, and encode/decode bridges are good fits. Operations that need to replace the backing buffer, especially dimension-changing processors like `Resize()`, are not a good fit for a wrapped image and may throw.

If you need to resize, crop into a new image, pad, or otherwise move into a normal ImageSharp-owned lifecycle, clone first:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Bgra32> wrapped = Image.WrapMemory<Bgra32>(bgra, width,
height, rowStrideInBytes);
using Image<Rgba32> owned = wrapped.CloneAs<Rgba32>();

owned.Mutate(x => x.Resize(width / 2, height / 2));
```

Get a Contiguous Buffer from an ImageSharp Image

If you need to hand ImageSharp-owned pixels to native code, ask for contiguous allocation up front and then call `DangerousTryGetSinglePixelMemory(...)`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

Configuration config = Configuration.Default.Clone();
config.PreferContiguousImageBuffers = true;

using Image<Rgba32> image = new(config, width, height);

if (!image.DangerousTryGetSinglePixelMemory(out Memory<Rgba32> pixels))
{
    throw new InvalidOperationException("The image is not backed by one
contiguous buffer.");
}
```

From there, you can pin the returned `Memory<T>` if your native API requires an address. Keep the image alive for the full duration of that native access.

Stride Matters

Several interop APIs take a row stride:

- `rowStride` for pixel-count-based overloads
- `rowStrideInBytes` for byte-count-based overloads

Use the stride-aware overloads whenever your source buffer contains padding between rows. Do not assume every foreign buffer is tightly packed.

Make a Normal Owned Copy When Needed

If you wrapped foreign memory only as a temporary bridge, you can switch back to a normal ImageSharp-owned image with `Clone()` or `CloneAs<TPixel>()`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Bgra32> wrapped = Image.WrapMemory<Bgra32>(bgra, width,
height, rowStrideInBytes);
using Image<Rgba32> owned = wrapped.CloneAs<Rgba32>();
```

That is often the right move if the wrapped buffer has awkward lifetime rules, if you want a different working pixel format, or if the next processing steps may need a different backing buffer

shape.

Related Topics

- Working with Pixel Buffers
- Memory Management
- Troubleshooting
- Migrating from System.Drawing

Practical Guidance

- Use `LoadPixelData(...)` when ImageSharp should own a copy of the pixels.
- Use `WrapMemory(...)` only when the external buffer lifetime is clearly controlled.
- Respect stride when importing or exporting foreign buffers.
- Clone wrapped images before operations that may require a different buffer shape or ownership model.

Configuration

Most applications can use ImageSharp exactly as it comes out of the box. `Configuration` only becomes interesting when you need to change what formats are available, how memory is allocated, how streams are read, or how aggressively work is parallelized.

That is why this page treats configuration as an opt-in advanced topic. Start with `Configuration.Default`, and customize only the parts your workload truly needs.

Use a Local Configuration for Targeted Overrides

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

Configuration config = Configuration.Default.Clone();
config.MaxDegreeOfParallelism = 2;
config.PreferContiguousImageBuffers = true;

DecoderOptions options = new()
{
    Configuration = config
};

using Image image = Image.Load(options, stream);
```

This pattern is usually better than mutating `Configuration.Default` when the override only matters for one pipeline.

What Configuration Controls

The main knobs on `Configuration` are:

- `ImageFormatsManager` for format registration, encoders, decoders, and detectors.
- `MemoryAllocator` for pooled buffer allocation and custom allocator limits.
- `MaxDegreeOfParallelism` for row and processor parallelism.
- `PreferContiguousImageBuffers` for interop-oriented contiguous buffers.
- `StreamProcessingBufferSize` for stream copy buffer size.

- `ReadOrigin` for whether decode operations read from the current stream position or from the beginning.
- `Properties` for processor-specific defaults and shared settings.

Register a Specific Format Set

`Configuration` can be created with an explicit set of `IImageFormatConfigurationModule` registrations:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Gif;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Formats.Png;

Configuration config = new(
    new PngConfigurationModule(),
    new JpegConfigurationModule(),
    new GifConfigurationModule());
```

This is useful when you want a deliberately restricted format set for a service or plugin boundary. For more advanced scenarios, `ImageFormatManager` also exposes methods such as `SetEncoder(...)`, `SetDecoder(...)`, and `AddImageFormatDetector(...)`.

Tune Processor Defaults

ImageSharp stores some processor defaults through `Configuration.Properties`. A common example is `GraphicsOptions`:

```
using SixLabors.ImageSharp;

Configuration config = Configuration.Default.Clone();
config.SetGraphicsOptions(options =>
{
    options.Antialias = false;
    options.BlendPercentage = 0.75F;
});
```

Those defaults then flow into processing APIs that read graphics options from the current configuration or processing context.

Parallelism and Throughput

`MaxDegreeOfParallelism` defaults to the machine processor count. That is often a good default for desktop and batch workloads.

For server-side applications running many requests at once, lowering this value on a local configuration can improve overall throughput by avoiding excessive per-image parallel work.

Stream Behavior

`ReadOrigin` controls whether decoding starts at the current stream position or the beginning of a seekable stream.

`StreamProcessingBufferSize` controls the buffer size used when ImageSharp copies stream data internally. Most applications should leave it alone unless profiling shows a reason to change it.

When to Customize Configuration

Use a custom or cloned configuration when:

- You want a restricted set of supported formats.
- You need a custom `MemoryAllocator`.
- You want different parallelism settings for a specific workload.
- You need contiguous buffers for interop.
- You need different stream-origin behavior for a pipeline that reads partially consumed streams.

Related Topics

- Image Formats
- Memory Management
- Interop and Raw Memory
- Troubleshooting

Practical Guidance

- Use the default configuration unless you have a specific format, allocator, parallelism, or stream behavior to change.
- Clone configuration for targeted overrides instead of mutating global defaults.
- Restrict formats at trust boundaries when your workload only supports a known subset.
- Profile before changing allocator, buffer, or parallelism settings.

Memory Management

ImageSharp is designed so large images are practical to process without forcing every workload into one giant contiguous allocation. That is a big part of why the library scales well, but it also means memory behavior is worth understanding once you move beyond simple load-process-save samples.

This page explains the parts most developers eventually need: the default pooled allocator, when to customize it, and how those choices affect lower-level interop code.

Compressed Size Is Not Memory Size

An encoded file can be much smaller than the image memory needed to process it. A 20 MB JPEG may decode into hundreds of megabytes of pixel data, and a multi-frame image can require far more when all frames are loaded. Memory planning should therefore start from decoded dimensions, pixel format, and frame count rather than the source file size alone.

Use `Image.Identify()` and `ImageInfo.GetPixelMemorySize()` when you need to estimate decoded memory before loading pixels. Use `DecoderOptions.MaxFrames` and `DecoderOptions.TargetSize` when the workload can deliberately load less data.

Default Behavior

`Configuration.MemoryAllocator` defaults to `MemoryAllocator.Default`. For most applications, that default allocator is the right choice.

The ImageSharp source explicitly recommends using a single busy allocator per process. If you customize allocation behavior, prefer doing so by replacing the allocator on a shared configuration rather than creating many short-lived allocators.

Customize the Allocator

If you need tighter control over pool size or allocation limits, create a custom allocator with `MemoryAllocator.Create(...)` and `MemoryAllocatorOptions`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Memory;

Configuration config = Configuration.Default.Clone();
```

```
config.MemoryAllocator = MemoryAllocator.Create(new MemoryAllocatorOptions
{
    MaximumPoolSizeMegabytes = 128,
    AllocationLimitMegabytes = 1024,
    AccumulativeAllocationLimitMegabytes = 2048,
    SingleBufferAllocationLimitMegabytes = 512
});
```

`MaximumPoolSizeMegabytes` controls retained pool size. `AllocationLimitMegabytes` controls the maximum discontiguous buffer size the allocator will allow for one live allocation group. When it is unset, the platform default is 1 GB on 32-bit processes and 4 GB on 64-bit processes. `AccumulativeAllocationLimitMegabytes` controls the maximum combined size of all active allocations made through that allocator instance. It is unset by default, so the allocator does not impose an accumulative cap unless you configure one.

`SingleBufferAllocationLimitMegabytes` controls the maximum size of one contiguous buffer, with a default of 1 GB when unset. The limit applies to contiguous image buffers requested through `PreferContiguousImageBuffers`. The largest accepted value is 2047, because a single buffer can never exceed `int.MaxValue` bytes. The applied value is also capped to `AllocationLimitMegabytes`, because one buffer can never be larger than its allocation group.

Custom allocator implementations can read the resolved limits through `MemoryAllocator.MemoryGroupAllocationLimitBytes`, `AccumulativeAllocationLimitBytes`, and `SingleBufferAllocationLimitBytes`, and apply an options value with the protected `ApplyOptions(...)` method.

Prefer Contiguous Buffers Only When You Need Them

`PreferContiguousImageBuffers` asks ImageSharp to use contiguous image buffers whenever possible:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

Configuration config = Configuration.Default.Clone();
config.PreferContiguousImageBuffers = true;

using Image<Rgba32> image = new(config, 2048, 2048);

bool contiguous = image.DangerousTryGetSinglePixelMemory(out Memory<Rgba32> pixels);
```

This is primarily for interop scenarios. It is not something to enable globally without a reason, because it reduces ImageSharp's flexibility around large pooled allocations and can hurt throughput.

For more on that workflow, see Interop and Raw Memory.

Dispose Images Promptly

`Image` and `Image<TPixel>` own unmanaged resources. Always dispose them with `using` or `await using` patterns where appropriate.

ImageSharp includes finalizer-based safety nets, but relying on finalization instead of deterministic disposal can still create avoidable memory pressure and latency.

Track Undisposed Allocations

`MemoryDiagnostics` exposes two useful diagnostics:

- `TotalUndisposedAllocationCount`
- `UndisposedAllocation`

```
using SixLabors.ImageSharp.Diagnostics;

Console.WriteLine(MemoryDiagnostics.TotalUndisposedAllocationCount);
```

For troubleshooting, you can subscribe to `UndisposedAllocation` to capture allocation stack traces for resources that leaked to the finalizer. That event is intended for diagnostics and carries significant overhead, so it should not stay enabled in normal production traffic.

Releasing Retained Resources

If you create a custom allocator and later retire it, dispose all associated images first and then call `ReleaseRetainedResources()`:

```
using SixLabors.ImageSharp.Memory;

MemoryAllocator allocator = MemoryAllocator.Create(new MemoryAllocatorOptions
{
    MaximumPoolSizeMegabytes = 64
});
```

```
allocator.ReleaseRetainedResources();
```

That tells the allocator to drop retained pooled buffers that are no longer needed.

Practical Guidance

- Keep `MemoryAllocator.Default` unless profiling shows a real need to customize it.
- Use one shared allocator per process rather than many temporary allocators.
- Use `AccumulativeAllocationLimitMegabytes` when the total amount of live ImageSharp allocation should be capped, not only the size of one image allocation group.
- Avoid forcing contiguous buffers unless you truly need a single `Memory<T>` or pointer.
- Use `DecoderOptions.TargetSize` and `DecoderOptions.MaxFrames` when you want to limit decode cost up front.
- Track leaked images with `MemoryDiagnostics` if disposal bugs are suspected.

Related Topics

- Configuration
- Working with Pixel Buffers
- Interop and Raw Memory
- Troubleshooting

Security Considerations

Image processing is powerful, but it is also one of the easier places for an application to burn CPU, memory, and time on untrusted input. This page is written as a practical hardening guide: what to check early, what to limit, and which ImageSharp hooks help you keep risky inputs under control.

Preflight with Identify When Possible

If you only need to validate dimensions, frame count, metadata presence, or pixel information, use `Image.Identify()` instead of a full decode.

```
using SixLabors.ImageSharp;

ImageInfo info = Image.Identify("upload.bin");

Console.WriteLine($"{info.Width}x{info.Height}");
Console.WriteLine($"Frames: {info.FrameCount}");
Console.WriteLine($"Bits per pixel: {info.PixelType.BitsPerPixel}");
Console.WriteLine($"Estimated pixel memory: {info.GetPixelMemorySize():N0} bytes");
```

This lets you reject obviously unsuitable files before allocating the full decoded image buffers.

`ImageInfo.GetPixelMemorySize()` is particularly useful here. It gives you a decoded pixel-memory estimate up front, which helps protect services against inputs that are cheap to upload but expensive to expand into memory, especially when many frames are involved.

Reduce Decode Cost with DecoderOptions

`DecoderOptions` is the main place to constrain decode behavior:

- `TargetSize` decodes to a bounded fit-within size equivalent to `ResizeMode.Max`.
- `MaxFrames` limits how many frames are decoded from animated formats.
- `SkipMetadata` avoids loading encoded metadata when you do not need it.
- `SegmentIntegrityHandling` controls how tolerant decoding is of damaged segments.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

DecoderOptions options = new()
```

```
{
    MaxFrames = 1,
    SkipMetadata = true,
    TargetSize = new Size(1600, 1600),
    SegmentIntegrityHandling = SegmentIntegrityHandling.Strict
};

using Image image = Image.Load(options, stream);
```

For public upload endpoints, `MaxFrames = 1` is often appropriate when you only need a poster frame or preview. Likewise, `SkipMetadata = true` is a straightforward win when EXIF, ICC, IPTC, and XMP data are irrelevant to the workflow.

Be Deliberate About Error Tolerance

`SegmentIntegrityHandling` is a tradeoff between strictness and recovery:

- `Strict` rejects files on any recoverable segment validation error.
- `IgnoreAncillary` is the library default and ignores recoverable errors in optional metadata or other ancillary segments.
- `IgnoreImageData` also ignores recoverable image data segment errors and is better suited to recovery tools than to public-facing ingest paths.

That recommendation is an inference from the enum semantics: the more errors you ignore, the more "best effort" your decode path becomes.

Restrict the Supported Format Set

If your service only needs a small number of formats, build a dedicated `Configuration` instead of exposing every registered codec:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Formats.Png;

Configuration config = new(
    new PngConfigurationModule(),
    new JpegConfigurationModule());

DecoderOptions options = new()
{
```

```
    Configuration = config
};
```

This reduces the amount of format detection and decoder surface area involved in that pipeline.

Limit Memory Use

You can impose conservative allocation limits with a custom `MemoryAllocator`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Memory;

Configuration config = Configuration.Default.Clone();
config.MemoryAllocator = MemoryAllocator.Create(new MemoryAllocatorOptions
{
    // Roughly limits the workload to about 64 megapixels of Rgba32 data.
    AllocationLimitMegabytes = 256,

    // Limits the combined size of all active allocations made through this allocator.
    AccumulativeAllocationLimitMegabytes = 512
});
```

`AllocationLimitMegabytes` limits the size of any one live allocation group. When it is unset, ImageSharp uses the platform default: 1 GB on 32-bit processes and 4 GB on 64-bit processes. `AccumulativeAllocationLimitMegabytes` limits the combined size of all active allocations made through the allocator instance. It is unset by default, which means there is no accumulative cap unless you configure one. Use both limits together when a service can process several images or several intermediate buffers at the same time.

This is one of the most important safeguards for services that handle arbitrary uploads. For broader guidance on allocator behavior and tradeoffs, see Memory Management.

Put Outer Limits Around Streams and Requests

ImageSharp requires a readable stream, and for non-seekable streams it copies the input into an internal seekable memory stream before decoding. In practice that means request-body limits, upload-size limits, and outer buffering rules still matter even before pixel buffers are allocated.

Use your hosting layer to enforce:

- maximum request body size;
- authentication or signed commands when appropriate;

- rate limiting;
- reverse proxy limits; and
- service or container isolation for expensive workloads.

For ImageSharp.Web command signing, see Securing Requests in ImageSharp.Web.

Practical Security Defaults

- Use `Identify()` first whenever a full decode is not necessary.
- Use `GetPixelMemorySize()` during identify-based preflight when you need a decoded memory budget check.
- Use `TargetSize`, `MaxFrames`, and `SkipMetadata` to shrink decode cost up front.
- Prefer `Strict` or the default `IgnoreAncillary` over broader error ignoring on untrusted inputs.
- Restrict the enabled format modules when your workload only needs a few codecs.
- Use per-allocation, accumulative allocator, and host-level request limits together rather than relying on only one layer.

Troubleshooting

Most ImageSharp issues turn out to be understandable once you know which layer is complaining: format detection, decoding, memory, streams, or disposal. This page groups the common failures that way so it is easier to move from symptom to likely cause.

"Image format is unknown"

`UnknownImageFormatException` means ImageSharp could not match the input to a registered format detector or decoder.

Common causes:

- the input is empty;
- the stream is positioned incorrectly;
- the format is not registered in the current `Configuration`;
- the input is not an image at all.

Useful first checks:

```
using SixLabors.ImageSharp;

var format = Image.DetectFormat(bytes);
ImageInfo? info = Image.Identify(bytes);
```

If `DetectFormat(...)` fails, focus on the source bytes or the active configuration before debugging anything else.

"The format is known, but loading still fails"

`InvalidImageContentException` means the decoder recognized the format but the encoded data was invalid, truncated, or unsupported in some way.

That usually points to corrupted input, partial downloads, damaged metadata blocks, or malformed animation/frame data rather than a registration issue.

`Identify(...)` is often useful here because it lets you confirm whether basic header parsing works before you commit to a full decode.

Stream Position Problems

By default, ImageSharp respects the current position of a seekable stream. If your stream has already been read from, loading may fail even though the underlying data is valid.

You can fix that either by resetting the stream manually or by using `ReadOrigin.Begin` on a custom configuration:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

Configuration config = Configuration.Default.Clone();
config.ReadOrigin = ReadOrigin.Begin;

DecoderOptions options = new()
{
    Configuration = config
};

using Image image = Image.Load(options, stream);
```

Large Images or Animations Use Too Much Memory

If decoding fails with an `InvalidImageContentException` that wraps an `InvalidMemoryOperationException`, the requested image size or frame set may be beyond the allocator limits or practical memory budget.

Before loading, run `Identify(...)` and inspect `ImageInfo.GetPixelMemorySize()`. That gives you a decoded pixel-memory estimate up front and is often the fastest way to spot small encoded files that expand into very large multi-frame allocations.

Ways to reduce decode cost:

- use `DecoderOptions.TargetSize` when a smaller decode is acceptable;
- use `DecoderOptions.MaxFrames` to cap animated formats;
- use `DecoderOptions.SkipMetadata` when metadata is not needed;
- adjust `MemoryAllocatorOptions.AllocationLimitMegabytes` for a larger per-allocation budget.

Also avoid turning on `PreferContiguousImageBuffers` unless you explicitly need contiguous memory for interop.

`DangerousTryGetSinglePixelMemory(...)` Returns `false`

That means the image is not backed by one contiguous buffer. This is normal for ImageSharp.

If you truly need a single `Memory<T>`, create or load the image with a local configuration that has `PreferContiguousImageBuffers` set to `true`. Even then, very large images may still be unable to satisfy a contiguous allocation request.

Raw Memory Import Fails

`LoadPixelData(...)` and `WrapMemory(...)` validate:

- width and height;
- row stride;
- byte-stride divisibility by pixel size;
- required buffer length.

If you get `ArgumentException` or `ArgumentOutOfRangeException`, double-check:

- whether the buffer is tightly packed or padded;
- whether you passed pixel stride or byte stride to the correct overload;
- whether the `TPixel` matches the actual input layout.

Transform Operations Throw `DegenerateTransformException`

`DegenerateTransformException` means a transform matrix or builder input collapsed into an invalid transform.

This usually happens when a perspective or affine transform is built from duplicate points, zero-area geometry, or other mathematically degenerate inputs.

When that happens, validate the source geometry before building the transform rather than treating it as a decoder or encoder problem.

Memory Keeps Growing

The first question to ask is whether images are being disposed promptly.

Use:

- `MemoryDiagnostics.TotalUndisposedAllocationCount` for a low-overhead signal;
- `MemoryDiagnostics.UndisposedAllocation` when you need stack traces for leaked allocations.

```
using SixLabors.ImageSharp.Diagnostics;

Console.WriteLine(MemoryDiagnostics.TotalUndisposedAllocationCount);
```

If that number trends upward in a steady-state workload, start looking for missing `Dispose()` or `using` blocks.

A Good Debugging Order

When an image pipeline misbehaves, this order is usually productive:

1. Run `DetectFormat(...)` or `Identify(...)`.
2. Confirm the stream position and active configuration.
3. Check whether the problem is a format-registration issue or an invalid-content issue.
4. Reduce decode cost with `TargetSize`, `MaxFrames`, or `SkipMetadata` if memory is the problem.
5. Only then investigate deeper processing or interop assumptions.

Related Topics

- Loading, Identifying, and Saving
- Configuration
- Memory Management
- Interop and Raw Memory

Practical Guidance

- Start by separating encoded-format detection, metadata inspection, full decode, and processing.
- Check stream position and configuration before assuming a codec bug.
- Use identify-based memory estimates before decoding large or untrusted images.
- Reduce the failing pipeline to the first processor or interop boundary that changes behavior.

Migrating from System.Drawing

If you are coming from `System.Drawing`, the biggest adjustment is not learning a brand-new set of image concepts. It is mostly learning that ImageSharp makes a few things explicit that GDI+ used to hide: pixel type, processing pipelines, and encoder choices.

Once that shift lands, most everyday workflows map over cleanly.

Core Type Mapping

`System.Drawing` concept	ImageSharp equivalent
`Image` / `Bitmap`	`Image` or `Image<TPixel>`
`Color`	`Color` or a specific pixel type such as `Rgba32`
`PixelFormat`	generic `TPixel` plus `PixelTypeInfo`
`GetPixel` / `SetPixel`	indexers or `ProcessPixelRows(...)`
`LockBits` / `UnlockBits`	`ProcessPixelRows(...)`, `CopyPixelDataTo(...)`, `LoadPixelData(...)`, `WrapMemory(...)`, `DangerousTryGetSinglePixelMemory(...)`
`Image.Save(...)` with codec choices	`Save(...)`, `SaveAsJpeg(...)`, `SaveAsPng(...)`, or explicit encoder types
`Graphics.DrawImage(...)`	`Mutate(...)` with `DrawImage(...)`

Loading, Processing, and Saving

A typical `System.Drawing` workflow translates to:

System.Drawing:

```
using System.Drawing;
using System.Drawing.Imaging;

using Bitmap source = new("input.jpg");
using Bitmap output = new(400, 300);
using Graphics graphics = Graphics.FromImage(output);
```

```
graphics.DrawImage(source, 0, 0, 400, 300);
output.Save("output.png", ImageFormat.Png);
```

ImageSharp:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(context => context.Resize(400, 300));

image.SaveAsPng("output.png");
```

Instead of mutating through a separate `Graphics` object, ImageSharp uses processing pipelines built with `Mutate(...)` or `Clone(...)`.

Pixels: Prefer Row Access Over Per-Pixel APIs

If you used `Bitmap.GetPixel()` or `Bitmap.SetPixel()` heavily, the closest ImageSharp equivalent is the indexer:

System.Drawing:

```
using System.Drawing;

Color pixel = bitmap.GetPixel(10, 20);
bitmap.SetPixel(10, 20, Color.White);
```

ImageSharp:

```
using SixLabors.ImageSharp.PixelFormats;

Rgba32 pixel = image[10, 20];
image[10, 20] = Rgba32.White;
```

For real throughput, move to `ProcessPixelRows(...)` instead. That is the ImageSharp replacement for most `LockBits`-driven loops:

System.Drawing:

```
using System.Drawing;
using System.Drawing.Imaging;
using System.Runtime.InteropServices;

BitmapData data = bitmap.LockBits(
    new Rectangle(0, 0, bitmap.Width, bitmap.Height),
    ImageLockMode.ReadWrite,
    PixelFormat.Format32bppArgb);

try
{
    int stride = Math.Abs(data.Stride);
    int byteCount = stride * data.Height;
    byte[] pixels = new byte[byteCount];

    Marshal.Copy(data.Scan0, pixels, 0, pixels.Length);

    for (int y = 0; y < data.Height; y++)
    {
        int rowStart = y * stride;

        // Format32bppArgb stores one pixel in four bytes, so reverse pixels rather than
individual bytes.
        for (int x = 0; x < data.Width / 2; x++)
        {
            int left = rowStart + (x * 4);
            int right = rowStart + ((data.Width - x - 1) * 4);

            for (int b = 0; b < 4; b++)
            {
                (pixels[left + b], pixels[right + b]) = (pixels[right + b], pixels[left
+ b]);
            }
        }
    }

    Marshal.Copy(pixels, 0, data.Scan0, pixels.Length);
}
finally
{
    bitmap.UnlockBits(data);
}
```

ImageSharp:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = Image.Load<Rgba32>("input.png");

image.ProcessPixelRows(accessor =>
{
    for (int y = 0; y < accessor.Height; y++)
    {
        Span<Rgba32> row = accessor.GetRowSpan(y);
        row.Reverse();
    }
});
```

`Color` and `TPixel` Are Different

This is one of the biggest mental shifts.

`Color` in ImageSharp is a general color value that can convert to any `IPixel<TSelf>` type. It is not the same thing as the `TPixel` storage type used by `Image<TPixel>`.

That means:

- use `Color` when you want a pixel-agnostic color value;
- use `Rgba32`, `Bgra32`, `L8`, and similar types when you care about actual in-memory layout.

Replace `PixelFormat` with `TPixel`

Instead of storing a runtime `PixelFormat` enum and branching on it later, ImageSharp encourages you to choose a generic working type:

System.Drawing:

```
using System.Drawing;
using System.Drawing.Imaging;

using Bitmap bitmap = new("input.tiff");

bool isArgb = bitmap.PixelFormat == PixelFormat.Format32bppArgb;
```

ImageSharp:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> rgba = Image.Load<Rgba32>("input.tiff");
using Image<Bgra32> bgra = rgba.CloneAs<Bgra32>();
```

If you only need metadata about the decoded source layout, `ImageInfo.PixelType` exposes `PixelTypeInfo`.

Replace `LockBits` with the Right Raw-Pixel API

If your old code used `LockBits`, the best ImageSharp replacement depends on what the code was really trying to do:

- use `ProcessPixelRows(...)` for most in-place managed algorithms;
- use `CopyPixelDataTo(...)` when you need a copied export buffer;
- use `LoadPixelData(...)` when you want to import raw bytes or pixels into a normal owned image;
- use `WrapMemory(...)` when you need a zero-copy bridge to existing memory;
- use `DangerousTryGetSinglePixelMemory(...)` only when you truly need one contiguous ImageSharp-owned buffer.

Compositing vs Drawing APIs

If your `System.Drawing` code mainly used `Graphics.DrawImage(...)`, the closest ImageSharp equivalent is `DrawImage(...)` inside a processing pipeline.

If the old code also draws shapes, paths, or text, you will usually want the companion packages documented elsewhere in this repo:

- `SixLabors.ImageSharp.Drawing`
- `SixLabors.Fonts`

Practical Migration Strategy

For most migrations, the least painful path is:

1. Keep the old high-level workflow the same.
2. Replace `Bitmap` with `Image<TPixel>`.
3. Replace `Graphics` operations with `Mutate(...)` or `Clone(...)`.

4. Replace `LockBits` loops with `ProcessPixelRows(...)`.
5. Standardize on a working pixel format such as `Rgba32` unless you have a reason not to.

Related Topics

- Getting Started
- Working with Pixel Buffers
- Interop and Raw Memory
- Pixel Formats

Practical Guidance

- Replace `Bitmap` with the `Image<TPixel>` type that matches your working pixel model.
- Replace `LockBits` loops with row-based processing.
- Keep rendering concerns in ImageSharp.Drawing when the old code used `Graphics`.
- Validate behavior on non-Windows systems when the migration goal is cross-platform support.

Migrating from SkiaSharp

If you are coming from SkiaSharp, the biggest adjustment is separating core image work from drawing work. ImageSharp owns loading, saving, metadata, pixel buffers, color conversion, and processing pipelines. ImageSharp.Drawing owns vector drawing, text, paths, and canvas composition.

This page focuses on core ImageSharp workflows. For canvas, brush, pen, path, transform, and text migration examples, see the ImageSharp.Drawing Migrating from SkiaSharp guide.

Core Type Mapping

SkiaSharp concept	ImageSharp equivalent
`SKBitmap` / `SKPixmap`	`Image<TPixel>`, pixel buffers, or row access APIs
`SKImage`	`Image` or `Image<TPixel>`
`SKColor`	`Color`, or a concrete pixel type such as `Rgba32`
`SKColorType` / `SKAlphaType`	generic `TPixel` plus `PixelTypeInfo`
`SKBitmap.Decode(...)`	`Image.Load(...)` or `Image.Load<TPixel>(...)`
`SKImage.Encode(...)`	`Save(...)`, `SaveAsJpeg(...)`, `SaveAsPng(...)`, or explicit encoder types
`SKPixmap.GetPixelColor(...)`	indexers or `ProcessPixelRows(...)`
`SKPixmap` / `InstallPixels(...)`	`LoadPixelData(...)`, `WrapMemory(...)`, `CopyPixelDataTo(...)`, or `ProcessPixelRows(...)`

Loading, Processing, and Saving

A typical SkiaSharp decode, resize, and encode flow maps to ImageSharp loading, mutation, and saving.

SkiaSharp:

```
using SkiaSharp;

using SKBitmap source = SKBitmap.Decode("input.jpg");
using SKBitmap output = source.Resize(new SKImageInfo(400, 300), SKFilterQuality.High);
using SKImage image = SKImage.FromBitmap(output);
using SKData data = image.Encode(SKEncodedImageFormat.Png, 100);

using FileStream stream = File.OpenWrite("output.png");
data.SaveTo(stream);
```

ImageSharp:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(context => context.Resize(400, 300));
image.SaveAsPng("output.png");
```

ImageSharp processors run through `Mutate(...)` for in-place updates or `Clone(...)` when you want a separate output image.

Pixels: Prefer Row Access Over Per-Pixel APIs

If your SkiaSharp code reads or writes individual pixels, the closest ImageSharp equivalent is the image indexer.

SkiaSharp:

```
using SkiaSharp;

SKColor pixel = bitmap.GetPixel(10, 20);
bitmap.SetPixel(10, 20, SKColors.White);
```

ImageSharp:

```
using SixLabors.ImageSharp.PixelFormats;

Rgba32 pixel = image[10, 20];
image[10, 20] = Rgba32.White;
```

For real throughput, move to row access instead of per-pixel calls.

SkiaSharp:

```
using SkiaSharp;

for (int y = 0; y < bitmap.Height; y++)
{
    for (int x = 0; x < bitmap.Width / 2; x++)
    {
        SKColor left = bitmap.GetPixel(x, y);
        SKColor right = bitmap.GetPixel(bitmap.Width - x - 1, y);

        bitmap.SetPixel(x, y, right);
        bitmap.SetPixel(bitmap.Width - x - 1, y, left);
    }
}
```

ImageSharp:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = Image.Load<Rgba32>("input.png");

image.ProcessPixelRows(accessor =>
{
    for (int y = 0; y < accessor.Height; y++)
    {
        Span<Rgba32> row = accessor.GetRowSpan(y);
        row.Reverse();
    }
});
```

Color and Pixel Format

SkiaSharp often carries pixel layout through `SKImageInfo`, `SKColorType`, and `SKAlphaType`. ImageSharp makes the working pixel format explicit through `Image<TPixel>`.

SkiaSharp:

```
using SkiaSharp;
```

```
SKImageInfo info = new(640, 360, SKColorType.Rgba8888, SKAlphaType.Unpremul);
using SKBitmap bitmap = new(info);
```

ImageSharp:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = new(640, 360);
```

Use `Color` when you want a pixel-agnostic color value, and use a concrete pixel type such as `Rgba32` when the memory layout matters.

Raw Pixel Buffers

SkiaSharp code that installs or peeks pixel memory usually maps to one of ImageSharp's explicit raw-memory APIs.

SkiaSharp:

```
using SkiaSharp;

SKImageInfo info = new(320, 200, SKColorType.Rgba8888, SKAlphaType.Unpremul);
using SKBitmap bitmap = new();

bitmap.InstallPixels(info, pixels, info.RowBytes);
```

ImageSharp:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = Image.LoadPixelData<Rgba32>(pixels, 320, 200);
```

Use `LoadPixelData(...)` when ImageSharp should own a normal image copy. Use `WrapMemory(...)` when you need ImageSharp to operate over existing memory without copying. Use `CopyPixelDataTo(...)` when you need to export pixels.

Drawing APIs

If your SkiaSharp code mainly uses `SKCanvas`, `SKPaint`, `SKPath`, or text drawing, use the ImageSharp.Drawing migration guide:

- Migrating from SkiaSharp in ImageSharp.Drawing

Practical Migration Strategy

For most SkiaSharp image migrations:

1. Replace decode and encode code with `Image.Load(...)` and `Save(...)` or format-specific save methods.
2. Replace `SKBitmap` pixel storage with `Image<TPixel>`.
3. Replace `SKColorType` and `SKAlphaType` branching with explicit `TPixel` choices.
4. Replace per-pixel loops with `ProcessPixelRows(...)`.
5. Replace raw memory interop with `LoadPixelData(...)`, `WrapMemory(...)`, or `CopyPixelDataTo(...)`.
6. Move canvas drawing code to ImageSharp.Drawing rather than mixing it into the core image migration.

Related Topics

- Getting Started
- Processing Images
- Working with Pixel Buffers
- Interop and Raw Memory
- Migrating from SkiaSharp in ImageSharp.Drawing

Practical Guidance

- Keep image load/save behavior equivalent before changing processing behavior.
- Replace pixel storage decisions with explicit `Image<TPixel>` choices.
- Use row processing instead of per-pixel object-style APIs.
- Move canvas drawing concerns to ImageSharp.Drawing rather than forcing them into core ImageSharp processors.

Recipes

These pages are the fast path through the ImageSharp docs. They skip most of the background explanation and focus on the handful of workflows people reach for over and over again, while linking back to the deeper guides when you need more context.

Use recipes when you already know the outcome you want: "make thumbnails", "convert this upload", "remove metadata", or "inspect before loading". Use the conceptual pages when you need to choose architecture, tune memory, handle untrusted input, or understand why a format behaves differently from another format.

Common Tasks

- Generate Thumbnails for fit-within-box and square-crop thumbnail workflows.
- Convert Between Formats for common re-encode scenarios such as PNG to JPEG or JPEG to WebP.
- Strip Metadata for removing EXIF, ICC, IPTC, XMP, and related metadata before export.
- Read Image Info Without Decoding for dimensions, frame count, pixel info, and format detection without a full decode.

How to Adapt a Recipe

- Use `Identify(...)` before decoding when routing decisions only need dimensions, metadata, or format detection.
- Use `Mutate(...)` when changing an existing image and `Clone(...)` when the original image must be preserved.
- Choose encoder options deliberately when file size, quality, metadata retention, or color profile behavior matters.
- Keep stream ownership clear: load from streams that stay readable for the load call, then save to streams you control.

Practical Guidance

The recipe examples show the core workflow, but production image pipelines need policy around the workflow. For untrusted images, put limits around request size, decoded pixel budget, and frame count before you decode the full image. Use `Identify(...)` to make those decisions cheaply when possible, then load only the images your application is willing to process.

Normalize orientation before generating user-visible derivatives such as thumbnails, crops, or social cards. Decide what happens to metadata and color profiles before export: some workflows need privacy-focused stripping, while others need ICC conversion or preservation. Public output should usually use explicit encoders so format, quality, compression, and metadata behavior do not drift because of a file extension or default setting.

Related Topics

- Loading, Identifying, and Saving
- Working with Metadata
- Image Formats
- Processing Images

Generate Thumbnails

Thumbnail generation is one of those jobs that sounds trivial until you have to decide what "good enough" means. Do you keep the whole image visible? Do you crop to fill? Do you normalize orientation first? This page covers the two thumbnail patterns people use most often.

The usual patterns are:

- fit the image within a bounding box while preserving aspect ratio, and
- create a fixed-size thumbnail that fills the target area by cropping.

Before choosing between them, decide what the thumbnail promises to users. A catalog image often needs to show the whole object, so fit-within-box is safer. Avatars, cards, and masonry layouts usually need consistent dimensions, so crop-to-fill is a better match. That product decision should drive the resize mode rather than the other way around.

Fit Within a Bounding Box

Use `ResizeOptions` with `ResizeMode.Max` when you want the full image to fit inside a target box:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Resize(new ResizeOptions
    {
        Size = new Size(300, 300),
        Mode = ResizeMode.Max
    }));

image.Save("thumbnail.jpg", new JpegEncoder { Quality = 85 });
```

This keeps the whole image visible and preserves aspect ratio.

The output may be smaller than the requested box in one dimension. That is the point of `ResizeMode.Max`: it respects both the maximum bounds and the source aspect ratio. If your

downstream layout requires an exact canvas size, resize first and then pad onto a fixed background.

Create a Square Center-Crop Thumbnail

Use `ResizeMode.Crop` to fill the target bounds and crop the overflow:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.jpg");

image.Mutate(x => x
    .AutoOrient()
    .Resize(new ResizeOptions
    {
        Size = new Size(256, 256),
        Mode = ResizeMode.Crop,
        Position = AnchorPositionMode.Center
    }));

image.Save("avatar.jpg");
```

This is the usual pattern for avatars, cards, and tile-based UI.

For user-generated photos, consider exposing a focal point or crop anchor instead of always using the center. Faces, products, and text are not always centered in the source image.

Keep Transparency in Thumbnails

If the source image uses transparency and you want to preserve it, save the thumbnail to a format that supports alpha, such as PNG or WebP:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Png;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.png");

image.Mutate(x => x.Resize(new ResizeOptions
{
    Size = new Size(256, 256),
```

```
    Mode = ResizeMode.Max
}));

image.Save("thumbnail.png", new PngEncoder());
```

Notes

- `AutoOrient()` is usually the right first step for user-uploaded photos.
- `ResizeMode.Max` is for fit-within-box results.
- `ResizeMode.Crop` is for fixed output dimensions that must be fully filled.
- Use explicit encoders when thumbnail quality, metadata, color profile behavior, or file size needs to be predictable.

For more detail on resizing behavior, see Resizing Images.

Practical Guidance

- Choose `Max` when the whole source must remain visible.
- Choose `Crop` when the output box must be fully filled.
- Normalize orientation before generating thumbnails from user-uploaded photos.
- Use explicit encoders so thumbnail quality, metadata, and file size are predictable.

Convert Between Formats

Format conversion is one of the most common reasons people adopt ImageSharp. In ImageSharp, conversion is a decode-and-encode workflow: load into the common image model, make any changes the output requires, then save to the destination path, stream, or encoder.

That decode-and-re-encode flow is not a blind one. Once an image is loaded, the processing pipeline works with format-agnostic pixel data, while the metadata layer still carries enough information for the destination format to choose the best representation it supports.

For common conversions, saving to a destination path or format is intentionally useful. ImageSharp combines the decoded image, bridged metadata, pixel information, and registered encoder defaults to produce strong automated output. Use explicit encoders when your application has a specific output policy to express, not because the default conversion path is something to avoid.

What Conversion Changes

Format conversion has three separate parts:

- Decode the source format into ImageSharp's image model.
- Optionally process or transform the image.
- Encode the result into the destination format.

The destination encoder decides what can be represented in the output file. If the target format cannot store something from the source, that information must be transformed, approximated, or dropped. Common examples are alpha transparency when writing JPEG, high bit depth when writing an 8-bit output, rich TIFF metadata when writing a simpler web format, or animation timing when writing a single-frame format.

Changing the file format does not automatically choose a better working pixel type, repair lossy compression damage, apply EXIF orientation, flatten transparency, or convert color profiles into a preferred output space. Those are separate policy decisions. ImageSharp gives you APIs for each step, but the conversion boundary is where your application decides what the destination file is allowed to mean.

How ImageSharp Bridges Formats

ImageSharp's built-in codec metadata translates through `FormatConnectingMetadata` and `FormatConnectingFrameMetadata`. Those bridge types carry the common image and frame semantics

that can be shared across formats, including:

- Encoded pixel information through `PixelTypeInfo`, such as color model, alpha behavior, bit depth, and component precision.
- Encoding intent such as lossy versus lossless output and quality.
- Indexed-color settings such as shared color table mode.
- Animation settings such as background color, repeat count, frame duration, blend mode, and disposal mode.

That is why ImageSharp's conversion model carries more information than a raw pixel buffer alone. For example, PNG metadata can derive palette, grayscale, RGB, or RGBA output and choose 1, 2, 4, 8, or 16-bit encoding from bridged pixel information, while GIF metadata can carry indexed color-table mode and repeat-count behavior forward when the target format supports them. These bridges make the automatic conversion APIs useful for real application workflows where pixel data, metadata, and target format capabilities all matter.

Use Identify to Plan the Conversion

You do not need to preflight every conversion. Use `Image.Identify()` when routing depends on how the source is encoded, or when you want to choose a different destination format before paying the cost of a full decode. `ImageInfo.PixelType` exposes `PixelTypeInfo`, including:

- `BitsPerPixel`
- `ColorType`
- `AlphaRepresentation`
- `ComponentInfo` for component count and precision

This is useful when you need to decide whether to flatten transparency for JPEG, keep higher-precision data in PNG, TIFF, or OpenEXR, preserve indexed workflows where the target format supports them, or select between several acceptable delivery formats.

Convert PNG to JPEG

JPEG does not support alpha transparency, so transparent sources usually need to be flattened onto a background color first:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Processing;

using Image image = Image.Load("input.png");
```

```
image.Mutate(x => x.BackgroundColor(Color.White));

image.Save("output.jpg", new JpegEncoder
{
    Quality = 85
});
```

Choose the flattening color deliberately. White is common for documents and many web layouts, but logos, UI assets, and product imagery may need a brand color, a page background color, or a checkerboard-style review workflow before final export.

Convert JPEG to WebP

Save with a WebP extension for the default WebP output, or pass a WebP encoder when you want to set a delivery policy such as lossy output and a specific quality value:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Webp;

using Image image = Image.Load("input.jpg");

image.Save("output.webp", new WebpEncoder
{
    FileFormat = WebpFileFormatType.Lossy,
    Quality = 80
});
```

For web delivery, compare both file size and visual quality against your JPEG baseline. WebP often wins for photographic content, but the right quality value is product-specific and should be chosen against representative images rather than one sample.

Convert Any Input to PNG

PNG is a good target when you want lossless output or transparency support:

```
using SixLabors.ImageSharp;

using Image image = Image.Load("input.bin");

image.Save("output.png");
```

PNG is not automatically the best "safe" target for every input. It preserves sharp graphics and transparency well, but photographic sources can become much larger than JPEG or WebP. Use PNG when lossless output, alpha, indexed color, or broad compatibility matter more than smallest file size.

Choose the Output Based on Pixel Info

When you need to implement a routing policy, inspect the encoded pixel type first and then choose the destination accordingly:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Formats.Png;
using SixLabors.ImageSharp.PixelFormats;

ImageInfo info = Image.Identify("input.tiff");

bool hasAlpha = info.PixelType.ColorType.HasFlag(PixelColorType.Alpha);
int precision = info.PixelType.ComponentInfo?.GetMaximumComponentPrecision() ?? 8;

using Image image = Image.Load("input.tiff");

if (hasAlpha)
{
    image.Save("output.png", new PngEncoder
    {
        BitDepth = precision > 8 ? PngBitDepth.Bit16 : PngBitDepth.Bit8
    });
}
else
{
    image.Save("output.jpg", new JpegEncoder
    {
        Quality = 85
    });
}
```

Notes

- Converting from a lossy format to a lossless format does not restore discarded detail.
- Converting a transparent image to JPEG requires flattening or compositing first.
- Converting to a palette or lower-bit-depth format is a color-reduction step.

- Converting an animated image to a single-frame format requires choosing which frame or composited result you want.
- ImageSharp uses bridged metadata, pixel-type information, and encoder defaults to pick good destination settings when the target format can represent them.
- Save-by-extension is the simplest and recommended path for ordinary conversions. Pass an explicit encoder when you want to override defaults for quality, compression, bit depth, palette behavior, metadata handling, or another application policy.
- Format conversion is also a metadata decision. Decide whether orientation, color profiles, animation timing, and authoring metadata should be preserved, transformed, or stripped.

For more on format behavior and encoder options, see Image Formats. For more on inspecting pixel types before a conversion, see Read Image Info Without Decoding and Pixel Formats.

Practical Guidance

For everyday conversion, let ImageSharp do the normal thing: load the source, apply any processing you need, and save to the destination path or format. The conversion layer carries format-agnostic metadata and pixel information forward so encoders can choose strong defaults. This is a real feature of the library, especially for automated services that accept multiple input formats and produce a consistent output type.

Add policy only where policy is genuinely needed. A transparent PNG converted to JPEG still needs an explicit background color because JPEG cannot represent alpha. An animated input needs a target format that can represent frame timing and disposal behavior if animation must survive. A public API, cache, or asset pipeline may want fixed quality, compression, bit depth, palette behavior, or metadata handling. Those are reasons to pass an explicit encoder, but they are refinements on top of a capable automated conversion model rather than a workaround for it.

Strip Metadata

Removing metadata is usually about one of three goals: smaller files, less personal information, or a cleaner normalized export. ImageSharp makes that straightforward, but it helps to be clear about whether you want the encoder to skip metadata on write or whether you want to clear profiles in memory first.

The choice matters because metadata is not one thing. EXIF can contain camera settings, timestamps, thumbnails, orientation, and GPS data. ICC and CICP data affect color interpretation. XMP and IPTC often contain authoring, rights, caption, and workflow information. Stripping everything is correct for privacy-sensitive exports, but not always correct for archival, print, or color-managed workflows.

Strip Metadata with the Encoder

The simplest approach, when you control the output encoder, is to set `ImageEncoder.SkipMetadata` to `true`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats.Jpeg;

using Image image = Image.Load("input.jpg");

image.Save("output.jpg", new JpegEncoder
{
    Quality = 85,
    SkipMetadata = true
});
```

The same pattern works with other encoders that derive from `ImageEncoder`, such as `PngEncoder` and `WebpEncoder`.

Clear Common Metadata Profiles Manually

If you want to clear metadata directly on the decoded image before saving, remove the common profiles from `image.Metadata`:

```
using SixLabors.ImageSharp;

using Image image = Image.Load("input.jpg");
```

```
image.Metadata.ExifProfile = null;
image.Metadata.IccProfile = null;
image.Metadata.IptcProfile = null;
image.Metadata.XmpProfile = null;
image.Metadata.CicpProfile = null;

image.Save("output.jpg");
```

This approach is useful when you want to inspect or edit metadata before deciding what to keep.

Notes

- `SkipMetadata = true` is usually the easiest option when you are already choosing an explicit encoder.
- Manual profile clearing gives you more control over which metadata survives.
- Saving to a different format can also change which metadata can be represented in the output.
- If color fidelity matters, think carefully before removing ICC or CICP metadata. Convert to an intended working/output color space first when the source profile is meaningful.
- If orientation matters, call `AutoOrient()` before stripping EXIF orientation metadata so the pixels are physically normalized.

For more detail, see Working with Metadata.

Practical Guidance

- Use encoder-level `SkipMetadata` when the output should simply omit metadata.
- Clear profiles manually when you need to inspect, keep, or remove specific metadata groups.
- Convert or preserve color profiles intentionally before stripping ICC or CICP data.
- Apply `AutoOrient()` before stripping EXIF orientation metadata when display orientation matters.

Read Image Info Without Decoding

When you are working with uploads, queues, or validation rules, fully decoding every image is often unnecessary work. `Image.Identify()` and `Image.DetectFormat()` let you answer the early questions first: what is this file, how large is it, how many frames does it have, what kind of pixel data does it claim to contain, and how much pixel memory might a full decode require?

Read Dimensions, Frame Count, and Pixel Info

`Image.Identify()` returns an `ImageInfo` with dimensions, frame count, pixel type, and metadata:

```
using SixLabors.ImageSharp;

ImageInfo imageInfo = Image.Identify("input.webp");

Console.WriteLine($"{imageInfo.Width}x{imageInfo.Height}");
Console.WriteLine($"Frames: {imageInfo.FrameCount}");
Console.WriteLine($"Bits per pixel: {imageInfo.PixelType.BitsPerPixel}");
Console.WriteLine($"Estimated pixel memory: {imageInfo.GetPixelMemorySize():N0} bytes");
```

Estimate Pixel Memory Before Decoding

`ImageInfo.GetPixelMemorySize()` reports the estimated in-memory size of the decoded pixel data represented by the identified image.

```
using SixLabors.ImageSharp;

ImageInfo imageInfo = Image.Identify("input.gif");
long pixelBytes = imageInfo.GetPixelMemorySize();

if (pixelBytes > 256L * 1024 * 1024)
{
    throw new InvalidOperationException("Image is too large to decode safely.");
}
```

This is especially useful for upload validation and other untrusted-input workflows. A file can be small on disk but still expand into a very large decoded pixel budget, especially for multi-frame formats such as GIF, animated WebP, or TIFF. If frame metadata is available, the reported size includes all frames.

Inspect the Encoded Pixel Type

`ImageInfo.PixelType` gives you the encoded pixel characteristics reported by the format metadata. This is more than a single bit-depth number. `PixelTypeInfo` can tell you whether the source is indexed, grayscale, RGB, alpha-bearing, or higher precision:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

ImageInfo imageInfo = Image.Identify("input.tiff");
PixelTypeInfo pixelType = imageInfo.PixelType;

Console.WriteLine($"Bits per pixel: {pixelType.BitsPerPixel}");
Console.WriteLine($"Color type: {pixelType.ColorType}");
Console.WriteLine($"Alpha: {pixelType.AlphaRepresentation}");

if (pixelType.ComponentInfo is { } componentInfo)
{
    Console.WriteLine($"Components: {componentInfo.ComponentCount}");
    Console.WriteLine($"Max component precision:
{componentInfo.GetMaximumComponentPrecision()}");
}
```

This is especially useful before format conversion, because the same pixel-type information is used by ImageSharp's format-bridging metadata to choose the best destination encoding options the target format can support.

Detect the Encoded Format

If you specifically want to know what encoded format a file contains, use `Image.DetectFormat()`:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;

IImageFormat format = Image.DetectFormat("input.bin");

Console.WriteLine(format.Name);
```

This is useful when file extensions are missing or untrustworthy.

Use `DetectFormat()` when routing depends only on the encoded format. Use `Identify()` when you need dimensions, frame count, pixel type, or metadata-driven decisions. `DetectFormat()` answers

a narrower question and does less work.

Use Async APIs

For asynchronous workflows, use `IdentifyAsync()`:

```
using SixLabors.ImageSharp;

ImageInfo imageInfo = await Image.IdentifyAsync("input.webp");

Console.WriteLine(imageInfo.Width);
Console.WriteLine(imageInfo.Height);
```

Notes

- `Image.Identify()` is usually much cheaper than `Image.Load()` for inspection-only workflows.
- `ImageInfo.Metadata` still gives you access to metadata without allocating a full pixel buffer.
- `ImageInfo.PixelType` includes color model, alpha behavior, bit depth, and component precision without decoding the full image.
- `ImageInfo.GetPixelMemorySize()` estimates decoded pixel memory before you commit to a full load.
- `Image.DetectFormat()` is focused on encoded format detection, while `Image.Identify()` returns the broader inspection result.
- Identification is not a replacement for decode-time error handling. It is a cheap preflight step; malformed input can still fail later when pixels are decoded.

For more detail, see Loading, Identifying, and Saving, Working with Metadata, Convert Between Formats, and Pixel Formats.

Practical Guidance

- Use `DetectFormat(...)` for routing by encoded format only.
- Use `Identify(...)` when dimensions, frame count, pixel type, or metadata affect the decision.
- Use `GetPixelMemorySize()` before decoding untrusted or very large inputs.
- Still handle decode failures; identification is preflight, not a guarantee that the full image is valid.

Introduction

What is ImageSharp.Drawing?

ImageSharp.Drawing is the high-performance 2D drawing layer for ImageSharp. It adds vector geometry, strokes, fills, text rendering, image composition, clipping, layers, and optional WebGPU-backed rendering while keeping the same cross-platform, managed-code deployment model as ImageSharp.

The core model is deliberately small: geometry describes coverage, brushes and pens describe how pixels are produced, drawing options describe state, and `DrawingCanvas` records ordered drawing work into a replay timeline. That makes the same drawing code useful for one-off image generation, templated graphics, server-side rendering, retained backend scenes, and GPU-backed output.

Read the articles as a progression. Start with the canvas workflow because replay, state, and lifetime explain the rest of the API. Then learn geometry, brushes, pens, clipping, text, image composition, transforms, and WebGPU as separate pieces that combine into one drawing pipeline.

Start Here

- Getting Started introduces the `Paint(...)` and `DrawingCanvas` workflow.
- Canvas Drawing covers canvas state, clipping, regions, and applying ImageSharp processors to drawn regions.
- Primitive Drawing Helpers covers rectangles, ellipses, arcs, pies, lines, and Bezier helpers.
- Paths and Shapes covers built-in shapes, custom paths, and fill rules.
- Brushes and Pens covers solid, pattern, and gradient fills plus stroke options.
- Clipping, Regions, and Layers covers clip paths, region canvases, save/restore state, and isolated layer composition.
- Images, Masks, and Processing covers `DrawImage(...)`, image brushes, clipping masks, and `Apply(...)`.
- Transforms and Composition covers transforms, blending, alpha composition, and antialiasing.
- Drawing Text covers `RichTextOptions`, measuring, and text along paths.
- WebGPU introduces GPU-backed drawing targets and links to the focused WebGPU pages.
- WebGPU Environment and Support covers startup configuration, availability probes, compute-pipeline checks, and native error logging.
- WebGPU Window Rendering covers `WebGPUWindow`, frame loops, window state, framebuffer sizing, and presentation.

- WebGPU External Surfaces covers `WebGPUExternalSurface`, native surface hosts, host-owned resize, and frame acquisition.
- WebGPU Offscreen Render Targets covers `WebGPURenderTarget`, offscreen canvases, texture formats, and readback.
- Migrating from System.Drawing maps common GDI+ drawing concepts to ImageSharp.Drawing.
- Migrating from SkiaSharp maps common SkiaSharp drawing concepts to ImageSharp.Drawing.
- Recipes provides copy-pasteable solutions for common drawing tasks.
- Troubleshooting covers common canvas, clipping, text, image, and WebGPU issues.

Built against .NET 8, ImageSharp.Drawing can be used in device, cloud, and embedded/IoT scenarios.

License

ImageSharp.Drawing is licensed under the terms of the Six Labors Split License, Version 1.0. See https://sixlabors.com/pricing for commercial licensing details.

> **IMPORTANT**
>
> Projects that directly reference ImageSharp.Drawing require a valid Six Labors license key at build time from version 2.2.0 in the previous major and 3.0.0 in the current major. Enforcement applies to direct dependencies only. This is enforcement of the existing license terms, not a license change.

Installation

ImageSharp.Drawing is installed via NuGet with nightly builds available on Feedz.

Package Manager | .NET CLI | PackageReference | Paket CLI

```
PM > Install-Package SixLabors.ImageSharp.Drawing -Version VERSION_NUMBER
```

Prerelease versions installed via the Visual Studio NuGet Package Manager require the "include prerelease" checkbox to be checked.

How to use the license file

By default, the build searches from each project directory for `sixlabors.lic`. Place the supplied file in the directory that contains the project file, or in a subdirectory below it. Use the file as supplied; it contains the complete license string required by the build.

> **IMPORTANT**
>
> Assembly-scoped community keys can be committed to a public repository. Contributors can use the included key to build the named assemblies without obtaining their own keys. Unrestricted community keys and commercial keys must remain private. Use environment variables or repository secrets for those keys.

To keep the file elsewhere, including a repository root above the project directory, set `SixLaborsLicenseFile` in your project file or a shared props file:

```
<PropertyGroup>
  <!-- Set the location of the supplied license file. -->
  <SixLaborsLicenseFile>path/to/sixlabors.lic</SixLaborsLicenseFile>
</PropertyGroup>
```

To use an environment variable instead, set this project property:

```
<PropertyGroup>
  <!-- Read the complete license string from the environment. -->
  <SixLaborsLicenseKey>$(SIXLABORS_LICENSE_KEY)</SixLaborsLicenseKey>
</PropertyGroup>
```

Set `SIXLABORS_LICENSE_KEY` to the complete contents of your issued license, not only its `Key` field. Use the original value without escaping. Replace the entire example value below with your issued license contents.

PowerShell:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
$env:SIXLABORS_LICENSE_KEY = 'Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

Bash:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
export SIXLABORS_LICENSE_KEY='Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

If you pass the license directly through `-p:SixLaborsLicenseKey`, escape every semicolon as `%253B`. MSBuild treats semicolons as property separators even inside a quoted argument. The pipe between assembly signatures stays unchanged.

PowerShell:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
dotnet publish "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
```

Bash:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
dotnet publish "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
```

You do not need to reference the licensing package directly. Projects that directly reference a Six Labors package carry its build-time validation.

Community keys for public projects

For an open-source or source-available application, provide the exact `AssemblyName` of every project that directly references a Six Labors package, including test projects. Names are case-sensitive.

For example, if `MyProject.csproj` sets `<AssemblyName>MyProject.Core</AssemblyName>`, enter `MyProject.Core`. If it does not set `AssemblyName`, enter `MyProject`, the project filename without `.csproj`. If a shared props file sets the name, use that value. Do not include `.dll` or `.csproj`.

Existing community license holders can request one early replacement with an assembly-scoped key. These upgrade requests are approved automatically, without another eligibility review. You do not need to wait for your current key to enter its renewal window.

Apply at licensing.sixlabors.com. Select open-source or source-available, use the same email address, and provide the assembly names. Later requests follow the normal renewal window: the

final 90 days before the expiry date in your license. Commercial licenses are not eligible for this early replacement.

Security support

The latest major version of each library is eligible for security updates. Each major remains eligible for 12 months after the first stable release of its successor. A later major release does not shorten an existing support window.

Install the latest available patch or minor release within a supported major to receive security fixes. Older releases within that major are not maintained separately. This policy covers security fixes only. It does not include feature backports or extend Microsoft's support for the underlying .NET version. Security fixes remain at Six Labors' discretion.

How to Use These Docs

- Start with the canvas model, because replay, state, and lifetime explain the rest of the API.
- Use paths and brushes pages when geometry and styling decisions are still unclear.
- Use text and image-processing pages when drawing must combine rich text, source images, clipping, and effects.
- Use WebGPU pages only when the output target genuinely benefits from GPU-backed rendering.

Getting Started

ImageSharp.Drawing adds high-performance vector drawing, brush and pen styling, image composition, and text rendering to ImageSharp. It is designed for generated graphics where the image pipeline and drawing pipeline need to work together: badges, charts, thumbnails, watermarks, annotations, documents, server-side render output, and GPU-backed drawing targets.

The main workflow is:

1. Create or load an `Image`.
2. Call `Mutate(...)`.
3. Use `Paint(...)` to receive a `DrawingCanvas`.
4. Draw onto the canvas with brushes, pens, paths, shapes, images, or text.

The same canvas can mix all of those operations. The important idea is that drawing is recorded through `DrawingCanvas` in the order you call it, then replayed into the current frame. That replay model lets the library share the same public drawing code across CPU images, retained backend scenes, and WebGPU targets.

Vector Geometry, Raster Output

ImageSharp.Drawing lets you describe vector geometry, but the final target is still an ImageSharp raster image unless you are using a retained or GPU backend explicitly. Paths, shapes, strokes, text glyphs, clips, and brushes are converted into pixel coverage during replay. Antialiasing, transforms, blend modes, alpha composition, and layer boundaries all affect that rasterization step.

Keep geometry, styling, and canvas state separate in your code. Geometry answers where drawing can occur. Brushes and pens answer how covered pixels are shaded. Canvas state answers how later commands are transformed, clipped, blended, grouped, or processed.

Draw a Shape

Start with geometry, then choose how it is painted. Built-in shapes such as `StarPolygon`, `RectanglePolygon`, and `EllipsePolygon` are reusable geometry objects. A brush fills the area covered by the shape, and a pen generates and fills the stroke outline.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
```

```
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(320, 200, Color.White.ToPixel<Rgba32>());

StarPolygon star = new(x: 160, y: 100, prongs: 5, innerRadii: 42, outerRadii: 86);
Pen outline = Pens.DashDot(Color.MidnightBlue, 4);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.Gold), star);
    canvas.Draw(outline, star);
}));

image.Save("star.png");
```

`Paint(...)` creates a canvas for each frame being processed. Drawing is recorded through that canvas, and the canvas is disposed by the paint processor after your callback returns. That disposal step replays the recorded timeline into the frame.

Combine Drawing Operations

Most real compositions combine background fills, path drawing, text, image drawing, clipping, and image processors. Keep those concerns separate in the code: geometry decides where drawing can happen, brushes and pens decide how pixels are produced, text options decide layout, and canvas state decides which later commands are clipped, transformed, blended, or processed.

Keep source images, image brushes, fonts, and reusable paths alive until the `Paint(...)` call has completed because the canvas records commands first and replays them later. The `Apply(...)` call in this example is also a replay barrier: it processes the pixels produced by earlier commands and does not include drawing that happens later.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> image = new(640, 360, Color.White.ToPixel<Rgba32>());
```

```
Font font = SystemFonts.CreateFont("Arial", 34);
RichTextOptions titleOptions = new(font)
{
    Origin = new(40, 42),
    WrappingLength = 560,
    HorizontalAlignment = HorizontalAlignment.Center
};

EllipsePolygon focus = new(320, 195, 360, 190);
RectangleF photoArea = new(80, 92, 480, 230);
DrawingOptions clipToFocus = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.AliceBlue));
    canvas.DrawText(titleOptions, "Clipped photo with local processing",
Brushes.Solid(Color.MidnightBlue), pen: null);

    _ = canvas.Save(clipToFocus, focus);

    // DrawImage scales the selected source rectangle into the destination rectangle.
    canvas.DrawImage(source, source.Bounds, photoArea, KnownResamplers.Bicubic);
    canvas.Apply(focus, region => region.GaussianBlur(3));
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.DarkSlateBlue, 3), focus);
}));

image.Save("composition.png");
```

Use Drawing Options

`DrawingOptions` controls the shared drawing state used by the canvas. It is not a brush, pen, or shape; it is the context used to interpret later commands. `GraphicsOptions` controls edge coverage and pixel composition, `ShapeOptions` controls fill and clip behavior, and `Transform` moves vector output from local coordinates into final canvas coordinates.

Pass options to `Paint(...)` when the whole callback should use that state. Use `Save(options)` and `Restore()` when only part of the drawing should use it.

```
using System.Numerics;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(320, 200, Color.White.ToPixel<Rgba32>());

DrawingOptions options = new()
{
    GraphicsOptions = new()
    {
        Antialias = true,
        BlendPercentage = 0.85F
    },

    // Transform is applied to vector output before rasterization.
    Transform = new(Matrix3x2.CreateRotation(-0.18F, new(160, 100)))
};

Brush brush = Brushes.Horizontal(Color.DeepSkyBlue, Color.Navy);

image.Mutate(ctx => ctx.Paint(options, canvas =>
{
    canvas.FillEllipse(brush, new(160, 100), new(210, 96));
    canvas.DrawEllipse(Pens.Solid(Color.Black, 3), new(160, 100), new(210, 96));
}));
```

Draw Text

Text drawing uses SixLabors.Fonts for font discovery, shaping, measurement, and layout. Use `RichTextOptions` when you draw directly to a canvas. The options are the text layout contract: font, origin, wrapping, alignment, fallback, culture, and rich runs should be the same when measuring and drawing.

Prefer layout options over manual width subtraction. Wrapping and alignment let the text engine account for line height, glyph metrics, shaping, and fallback fonts, which manual coordinate guesses cannot do reliably.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(640, 240, Color.White.ToPixel<Rgba32>());

Font font = SystemFonts.CreateFont("Arial", 42);
RichTextOptions textOptions = new(font)
{
    Origin = new(48, 70),
    WrappingLength = 540,
    HorizontalAlignment = HorizontalAlignment.Center
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.DrawText(textOptions, "Drawing text with ImageSharp", Brushes.Solid(Color.Black),
pen: null);
}));
```

For deeper text guidance, see the Fonts docs.

Next Steps

- Canvas Drawing
- Paths and Shapes
- Brushes and Pens
- Drawing Text

Practical Guidance

- Keep reusable geometry, pens, brushes, fonts, and source images alive until `Paint(...)` completes.
- Create drawing options for the state you want to scope, then use `Save(...)` and `Restore()` around that scope.
- Use `Apply(...)` after the commands whose pixels should be processed.
- Move from primitive helpers to reusable paths when the same geometry drives more than one command.

Canvas Drawing

`DrawingCanvas` is the central drawing surface in ImageSharp.Drawing. You normally use it through `Paint(...)` inside an ImageSharp processing pipeline:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(400, 240, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Clear(Brushes.Solid(Color.White));
    Rectangle panel = new(24, 24, 160, 96);
    canvas.Fill(Brushes.Solid(Color.CornflowerBlue), panel);
    canvas.Draw(Pens.Solid(Color.Black, 3), panel);
}));
```

The callback receives a canvas for the current frame. Use the canvas for all drawing work that should happen together.

Immediate Mode, Retained Mode, and Canvas

Graphics APIs are often described as either immediate mode or retained mode.

In an immediate-mode API, the application issues the drawing commands needed for the current output. The graphics library does not own an editable scene graph for the application. If the output needs to be drawn again, the application normally issues those commands again.

In a retained-mode API, the graphics library owns a persistent object model of the scene. Application calls usually update that model rather than directly describing the drawing for one output pass. The library can then decide when and how to render the retained scene.

`DrawingCanvas` is closest to immediate-mode drawing at the public API level: application code calls `Fill(...)`, `Draw(...)`, `DrawText(...)`, `DrawImage(...)`, `Save(...)`, and `Restore()` in the order the output should be produced. The canvas is not a retained scene graph. You do not create editable rectangle, path, text, or image nodes and then change their properties later.

The important difference from a strictly immediate pixel-writing API is replay. Canvas calls record ordered drawing intent into a timeline. That timeline is replayed into the active backend when the canvas is disposed, or sealed into a reusable backend scene when you call `CreateScene()`. This lets ImageSharp.Drawing keep an immediate-style authoring model while still batching work, inserting barriers, supporting layers, and reusing backend-prepared scenes where that is useful.

Ordered Calls and Replay

`DrawingCanvas` is an ordered drawing API backed by a replay timeline. The calls look familiar if you have used immediate-mode drawing APIs: `Fill(...)`, `Draw(...)`, `DrawText(...)`, `Save(...)`, and `Restore()` are made in the order you want drawing to happen. The canvas does not, however, promise that each call immediately writes pixels to the destination.

The timeline is the core of the model. The canvas records drawing intent, seals that intent into timeline entries, and replays the timeline into the active backend.

Most drawing calls append drawing intent to a command buffer. Calls that must happen at a specific point, such as `Apply(...)` and `RenderScene(...)` are stored as entries in the canvas replay timeline. `SaveLayer(...)` is also timeline-sensitive: it records an isolated group that is later composited back into the parent.

The root canvas replays the timeline when it is disposed. During replay, command ranges are prepared into backend command batches, and the backend creates and renders scenes for those ranges. This is why a manually-created canvas must be disposed: disposal is the point where recorded work is actually rendered into the target.

The replay timeline can contain three kinds of entry:

- command ranges for normal drawing commands
- apply barriers for `Apply(...)` operations
- retained backend scene references inserted by `RenderScene(...)`

This model keeps drawing code straightforward while still allowing ImageSharp.Drawing to prepare command batches, insert replay barriers, reuse retained backend scenes, and target CPU images or WebGPU surfaces through the same public canvas API.

`Flush()` seals the commands recorded so far into a command-range timeline entry. It does not render immediately by itself. Most code does not need it; replay barriers such as `Apply(...)` already seal earlier commands before they run.

```
image.Mutate(ctx => ctx.Paint(canvas =>
{
```

```
    canvas.Fill(Brushes.Solid(Color.LightGray));

    // Apply is a replay barrier, so the blur sees the earlier fill.
    canvas.Apply(new Rectangle(40, 40, 180, 120), region => region.GaussianBlur(6));

    canvas.Draw(Pens.Solid(Color.Black, 3), new Rectangle(40, 40, 180, 120));
}));
```

Inside `Paint(...)`, ImageSharp.Drawing owns the canvas lifetime. When you call `CreateCanvas(...)` yourself, your `using` statement is what triggers replay.

Paint Versus CreateCanvas

Use `Paint(...)` for normal `Mutate(...)` and `Clone(...)` pipelines. It follows ImageSharp's processor model and handles each frame for you.

Use `CreateCanvas(...)` when you already have an image frame and want to manage the canvas lifetime yourself. Disposing the canvas replays the recorded work into the target frame.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = new(320, 180, Color.White.ToPixel<Rgba32>());
using DrawingCanvas canvas = image.Frames.RootFrame.CreateCanvas(image.Configuration,
new());

canvas.Fill(Brushes.Solid(Color.LightSteelBlue));
canvas.Draw(Pens.Dash(Color.Navy, 3), new Rectangle(18, 18, 284, 144));
```

Clear and Fill

Use `Fill(...)` when you want normal brush compositing. Use `Clear(...)` when you want to replace pixels in the covered area, including replacing them with transparent pixels.

`Clear(...)` can target the full canvas, a rectangle, or any `IPath`. It also honors the active clip state created by `Save(...)`, so clears can be scoped by both the supplied clear shape and the current canvas state.

The difference is compositing intent. `Fill(...)` draws a brush through the active `GraphicsOptions`, so source alpha and blend modes affect the destination. `Clear(...)` uses clear-style composition for the covered region, so it is the right API when the drawing command should replace or erase

what was there before. Use transparent clear for cutouts, masks, and punched holes. Use an opaque brush with `Clear(...)` when the area should be reset to a known color regardless of the pixels already underneath.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(320, 200, Color.Transparent.ToPixel<Rgba32>());
DrawingOptions clipToEllipse = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.MidnightBlue.WithAlpha(0.95F)));
    canvas.Fill(Brushes.Solid(Color.Crimson.WithAlpha(0.8F)), new Rectangle(26, 18,
268, 164));

    EllipsePolygon clip = new(160, 100, 214, 126);
    _ = canvas.Save(clipToEllipse, clip);

    canvas.Clear(Brushes.Solid(Color.LightYellow.WithAlpha(0.85F)));

    // Transparent clear removes content inside the supplied path and active clip.
    EllipsePolygon cutout = new(164, 98, 74, 48);
    canvas.Clear(Brushes.Solid(Color.Transparent), cutout);
    canvas.Restore();

    canvas.Draw(Pens.DashDot(Color.Black, 3), clip);
}));
```

State and Storage

`Save()` stores the current drawing state on a stack and `Restore()` returns to the previous state. The state includes drawing options, clip paths, target bounds, and layer information for later commands.

The overload `Save(DrawingOptions, params IPath[])` stores the supplied `DrawingOptions` instance by reference. Treat options passed to `Save(...)` as owned by the active canvas state until that state has been restored.

The active state reference is captured when each command is recorded. Later `Save(...)` or `Restore()` calls do not replace the state for commands already in the command buffer, but mutating a referenced `DrawingOptions` instance can still affect commands that captured that same instance.

In normal application code, create the options you want before saving them and avoid mutating the same instance while it is active. That keeps the recorded timeline easy to reason about: save a state, record commands under that state, then restore it. If different groups need different transforms, clips, or blending, use separate `DrawingOptions` instances.

The state captured for drawing includes:

- `DrawingOptions`, including graphics options, shape options, and transform
- clip paths supplied to `Save(DrawingOptions, params IPath[])`
- target bounds for the active canvas or region
- destination offset for region canvases
- whether the command is being recorded inside a layer

`Save()` pushes a normal state frame. `SaveLayer(...)` pushes a layer state frame. Only layer state frames create layer boundary commands when restored.

Save and Restore State

`Save(...)` pushes the current drawing state. The overload that accepts `DrawingOptions` and clip paths replaces the active state until you call `Restore()`.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());
DrawingOptions clipInside = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
```

```
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    EllipsePolygon clipPath = new(180, 110, 260, 140);

    _ = canvas.Save(clipInside, clipPath);
    canvas.Fill(Brushes.Solid(Color.MidnightBlue), new Rectangle(0, 0, 360, 220));
    canvas.Fill(Brushes.Solid(Color.Gold.WithAlpha(0.72F)), new Rectangle(56, 38,
248, 144));
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.Black, 3), clipPath);
}));
```

Use `SaveLayer(...)` when you need an isolated compositing layer that is later blended back onto the parent canvas.

Region Canvases

`CreateRegion(...)` creates a child canvas over a clipped subregion of the parent target. The child canvas has a local origin at `(0, 0)` for drawing commands, but it shares the parent replay timeline. The root canvas still owns final replay.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.LightGray));

    using DrawingCanvas region = canvas.CreateRegion(new Rectangle(80, 48, 180, 112));
    region.Fill(Brushes.Solid(Color.CornflowerBlue));

    // Region-local coordinates start at the region origin.
    region.Draw(Pens.Solid(Color.White, 5), new Rectangle(12, 12, 156, 88));

    canvas.Draw(Pens.Solid(Color.Black, 2), new Rectangle(80, 48, 180, 112));
}));
```

Use a region when you want a smaller local coordinate system. Use `Save(...)` with clip paths when you want to keep the parent coordinate system but clip later commands.

Layers

`SaveLayer(...)` starts an isolated composition scope. Commands drawn inside the layer are recorded into that scope, and `Restore()` closes the layer. The closed layer is composited back into the parent using the `GraphicsOptions` supplied to `SaveLayer(...)`.

Layer bounds limit the isolated target and final composition area. They do not move the canvas origin, so commands inside a bounded layer still use the same local coordinates as the parent canvas.

A layer is useful when a group of commands must be blended as one result. Without a layer, each command is blended into the parent independently. With a layer, commands first render into an isolated target, then that whole target is composited back once.

`SaveLayer(...)` overloads also accept a `LayerEffect`, such as blur, drop shadow, glow, or a backdrop filter, that transforms the layer when it is restored. See Layer and Backdrop Effects for the effect model.

The layer lifecycle is:

1. `SaveLayer(...)` records a begin-layer command and pushes a layer state.
2. Drawing commands inside the layer are recorded with the layer state.
3. `Restore()` or `RestoreTo(...)` records an end-layer command.
4. Disposal replay asks the backend to lower that layer scope for the target.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.SteelBlue), new Rectangle(24, 24, 312, 172));

    GraphicsOptions layerOptions = new()
    {
        BlendPercentage = 0.5F
```

```
    };

    _ = canvas.SaveLayer(layerOptions, new Rectangle(70, 46, 220, 128));

    // The layer bounds isolate composition; these coordinates are still parent-
canvas coordinates.
    canvas.FillEllipse(Brushes.Solid(Color.OrangeRed), new(180, 110), new(170, 96));
    canvas.Draw(Pens.Solid(Color.White, 8), new Rectangle(96, 74, 168, 72));
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.Black, 2), new Rectangle(70, 46, 220, 128));
}));
```

If a canvas is disposed while a layer is still active, disposal unwinds the layer using the same path as `Restore()`.

Use bounded layers deliberately. A smaller layer bounds can reduce the isolated composition area, but anything outside those bounds is not part of that layer's final composition.

Draw Images

`DrawImage(...)` records image drawing through the same canvas timeline as shape and text commands. Pass the source image, a source rectangle, a destination rectangle, and an optional resampler.

The source rectangle is sampled from the source image and scaled into the destination rectangle. The current transform and clip state apply to the destination drawing. Source rectangles that extend outside the source image are clipped to the available pixels.

Because canvas drawing is replayed later, the source image must remain alive until the canvas has replayed the command. With `Paint(...)`, that means keeping the source image alive for the duration of the `Mutate(...)` call. With a manually-created canvas, keep it alive until the canvas is disposed.

Treat source and destination rectangles as two different coordinate systems. The source rectangle selects pixels from the input image. The destination rectangle places those selected pixels on the canvas. That separation lets you crop, zoom, or letterbox an image without changing the source file.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
```

```
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> image = new(420, 260, Color.White.ToPixel<Rgba32>());
DrawingOptions clipInside = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
};

EllipsePolygon clip = new(210, 130, 300, 170);
Rectangle sourceRect = new(20, 12, 240, 180);
RectangleF destination = new(60, 45, 300, 170);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    _ = canvas.Save(clipInside, clip);

    // Bicubic resampling is a good default for scaled photographic content.
    canvas.DrawImage(source, sourceRect, destination, KnownResamplers.Bicubic);
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.Black, 3), clip);
}));
```

Strokes and Command Preparation

Stroke drawing is prepared during replay. A `Draw(...)` command records the original path, pen, stroke width, dash pattern, caps, joins, and active state. When the canvas prepares the command batch, it normalizes strokes for backend execution.

Simple solid line segments can stay as line commands. Dashed strokes, paths, joins, caps, and other complex strokes are prepared as stroke path commands or expanded into fillable geometry before backend handoff. Clip paths are applied during preparation so backends receive commands with consistent clipping semantics.

That means `Draw(...)` and `Fill(...)` share the same backend handoff model even though the public calls describe different drawing intent. Backends receive prepared commands and can focus on rendering them for their target.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 220, Color.White.ToPixel<Rgba32>());

PathBuilder builder = new();
builder.AddCubicBezier(new(36, 150), new(116, 32), new(292, 44), new(384, 158));

IPath path = builder.Build();
Pen pen = Pens.DashDot(Color.DarkSlateBlue, 10);
pen.StrokeOptions.LineCap = LineCap.Round;
pen.StrokeOptions.LineJoin = LineJoin.Round;

image.Mutate(ctx => ctx.Paint(canvas =>
{

    // Dash, cap, and join settings are part of the recorded stroke intent.
    canvas.Draw(pen, path);
}));
```

Use the pen's `StrokeOptions` for stroke shape:

- `LineCap` controls open path ends.
- `LineJoin` controls corners.
- `MiterLimit` controls how far miter joins can extend.
- dash pens such as `Pens.Dash(...)` and `Pens.DashDot(...)` record a stroke pattern.

Retained Scene Replay

Use `CreateScene()` when the same recorded drawing should be replayed into more than one canvas target. It seals and prepares the recorded drawing commands into a retained backend scene. `RenderScene(...)` inserts that retained backend scene into the receiving canvas timeline at the point where it is called.

The scene is backend-owned state, so keep it alive until every canvas that records it has been disposed. A canvas that receives `RenderScene(...)` still replays on disposal like any other canvas.

Retained backend scenes are useful when preparation is the repeated cost: logos, icons, map overlays, decorative vector art, or other drawing that is reused across many targets. They are not

editable scene graphs. If the geometry, brushes, text, or image resources need to change, record a new scene.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Drawing.Processing.Backends;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> source = new(160, 120, Color.Transparent.ToPixel<Rgba32>());
using DrawingCanvas sourceCanvas =
source.Frames.RootFrame.CreateCanvas(source.Configuration, new());

sourceCanvas.FillEllipse(Brushes.Solid(Color.Gold), new(80, 60), new(116, 72));
sourceCanvas.Draw(Pens.Solid(Color.Black, 3), new Rectangle(12, 12, 136, 96));

using DrawingBackendScene scene = sourceCanvas.CreateScene();

using Image<Rgba32> first = new(160, 120, Color.White.ToPixel<Rgba32>());
using DrawingCanvas firstCanvas = first.Frames.RootFrame.CreateCanvas(first.Configuration,
new());
firstCanvas.RenderScene(scene);
firstCanvas.Dispose();

using Image<Rgba32> second = new(160, 120, Color.LightGray.ToPixel<Rgba32>());
using DrawingCanvas secondCanvas =
second.Frames.RootFrame.CreateCanvas(second.Configuration, new());
secondCanvas.RenderScene(scene);
secondCanvas.Dispose();
```

`RenderScene(...)` preserves timeline order. Commands recorded before it replay before the retained backend scene; commands recorded after it replay after the retained backend scene.

Apply Image Processing to a Region

`Apply(...)` runs ImageSharp processors inside a rectangle, path, or path builder region. It is a replay barrier because the processor needs real pixels, not just recorded drawing commands.

During replay, ImageSharp.Drawing reads the covered target pixels into a temporary image, runs the processor operation on that temporary image, then writes the processed result back through the canvas pipeline using the recorded path and state.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.LightGray));
    canvas.Draw(Pens.Solid(Color.Black, 4), new Rectangle(24, 24, 312, 172));

    EllipsePolygon blurPath = new(180, 110, 220, 120);

    // The blur is clipped to the supplied path region.
    canvas.Apply(blurPath, region => region.GaussianBlur(8));
}));
```

Because `Apply(...)` reads pixels at its replay point, commands before the barrier affect the processed image, and commands after the barrier do not.

Practical Guidance

Use `Paint(...)` for ordinary ImageSharp processing pipelines. It gives you a canvas at the right point in `Mutate(...)` or `Clone(...)` and owns the replay lifetime for you. Use `CreateCanvas(...)` when you already have an image frame or backend target and need explicit lifetime control. In that case disposal is part of correctness: it is the point where the recorded work is replayed.

Because canvas drawing is replayed later, anything referenced by recorded commands must stay alive until replay has completed. That includes source images for `DrawImage(...)`, image brushes, fonts, paths, and retained backend scenes. This is the important difference from strictly immediate pixel-writing APIs: the call records drawing intent, but the referenced objects may still be needed later.

Scope state narrowly. `Save(...)` and `Restore()` are the right model for transforms, clipping, and graphics options that affect a limited part of the drawing. Use `SaveLayer(...)` when several commands should first render into an isolated group and then composite back as one result. Use `Apply(...)` when ImageSharp processors need to observe the timeline at a specific point, and keep those regions tight so CPU work and GPU readback stay bounded.

Primitive Drawing Helpers

Primitive helpers are convenience methods on `DrawingCanvas` for common one-off geometry. They let you draw rectangles, ellipses, lines, Beziers, arcs, and pies without first creating a reusable `IPath` object.

The helpers still follow the same rules as path drawing: fills use brushes, strokes use pens, `DrawingOptions` controls antialiasing and transforms, and active canvas state applies to the recorded command.

Primitive calls append drawing intent to the canvas as soon as you call them. They are a good fit for marks, guides, simple badges, outlines, and other geometry that is only used once. If the same geometry must be filled, stroked, clipped, transformed, measured, passed to text layout, or shared between commands, create a path or polygon object instead so the geometry becomes explicit.

Rectangles, Ellipses, Lines, and Beziers

Rectangle drawing is handled by rectangle-specific overloads: `Fill(brush, Rectangle)`, `Draw(pen, Rectangle)`, and `Clear(brush, Rectangle)`. There are no `FillRectangle(...)`, `DrawRectangle(...)`, or `ClearRectangle(...)` methods on `DrawingCanvas`. Ellipse helpers use `FillEllipse(...)` and `DrawEllipse(...)` with a center point plus size, matching `EllipsePolygon`. Lines and Beziers use explicit points in canvas coordinates, so they are easy to combine with image-space measurements.

Those coordinate conventions matter when translating from other libraries. Rectangle APIs usually describe a box from its top-left corner; ellipse, arc, and pie helpers describe an ellipse frame from its center. If the values look visually shifted, check whether the source API used top-left ellipse bounds while the Drawing helper expects a center.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 240, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Draw(Pens.Solid(Color.DimGray, 3), new Rectangle(16, 16, 328, 208));
    canvas.DrawEllipse(Pens.Solid(Color.CornflowerBlue, 6), new PointF(180, 120), new
```

```
SizeF(170, 100));

    // DrawLine accepts a polyline, so each point after the first extends the same stroke.
    canvas.DrawLine(
        Pens.Solid(Color.OrangeRed, 5),
        new PointF(28, 206),
        new PointF(110, 46),
        new PointF(248, 188),
        new PointF(332, 34));

    // DrawBezier is useful for one cubic curve; use PathBuilder for longer paths.
    canvas.DrawBezier(
        Pens.Solid(Color.MediumVioletRed, 4),
        new PointF(32, 126),
        new PointF(88, 30),
        new PointF(258, 210),
        new PointF(326, 118));
}));
```

Use the rectangle and ellipse helpers when the geometry exists only for that command. For example, `DrawEllipse(...)` is concise for a one-off ring, while `new EllipsePolygon(...)` is better when the same ellipse must be clipped, filled, and outlined.

Arcs and Pies

Arc and pie helpers take a center point, a size, a rotation angle, a start angle, and a sweep angle. Positive and negative sweeps are both valid, which makes clockwise and counter-clockwise segments easy to express.

Arc helpers describe the curved segment of an ellipse. Pie helpers close the segment back to the center, creating a wedge. Use arcs for gauges, rings, callouts, and curved marks. Use pies for chart slices, radial badges, and wedge-shaped fills. Use `PiePolygon` when the wedge is part of reusable geometry.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 240, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
```

```
    canvas.FillArc(
        Brushes.Solid(Color.CornflowerBlue),
        new PointF(112, 92),
        new SizeF(84, 58),
        rotation: 15,
        startAngle: -30,
        sweepAngle: 240);

    canvas.DrawArc(
        Pens.Solid(Color.ForestGreen, 4),
        new PointF(224, 92),
        new SizeF(116, 62),
        rotation: 15,
        startAngle: -25,
        sweepAngle: 220);

    // Pie helpers connect the arc back to the center, creating a wedge.
    canvas.FillPie(Brushes.Solid(Color.Goldenrod), new PointF(118, 172), new SizeF(58, 58),
startAngle: 20, sweepAngle: 240);
    canvas.DrawPie(Pens.Solid(Color.DarkSlateBlue, 6), new PointF(236, 170), new SizeF(62,
48), startAngle: 35, sweepAngle: -210);
}));
```

When to Use Paths Instead

Use `PathBuilder`, `Polygon`, `ComplexPolygon`, or a built-in shape type when you need to:

- reuse or transform the same geometry;
- combine multiple figures into one shape;
- choose fill rules for overlapping contours;
- stroke the generated outline with `Pen.GeneratePath(...)`;
- measure bounds, length, or area before drawing.

The primitive helpers are best for direct one-off drawing. Paths are better when the geometry is part of the model.

Practical Guidance

- Use primitive helpers for direct marks, guides, and simple one-off geometry.
- Switch to paths or polygons when the same geometry is filled, stroked, clipped, measured, or transformed.
- Remember that rectangle helpers use top-left coordinates while ellipse, arc, and pie helpers use center and size.

- Use built-in shape types when the geometry becomes part of your application model.

Paths and Shapes

ImageSharp.Drawing separates geometry from painting. Shapes and paths describe where drawing happens; brushes and pens describe how pixels are shaded. Keeping that split clear makes drawing code easier to reuse: the same path can be filled, stroked, clipped, measured, transformed, used as a text baseline, or combined with other paths without duplicating the styling code.

The core geometry types are:

- `IPath` for any path-like shape that can be filled or stroked.
- `Path` for an open path made from line segments, arcs, and curves.
- `Polygon` for a closed path.
- `ComplexPolygon` for a shape made from multiple paths, such as an outer contour with holes.
- `Polygon`, `RectanglePolygon`, `EllipsePolygon`, `RegularPolygon`, `StarPolygon`, and `PiePolygon` for common shapes.
- `PathBuilder` when you want to construct a custom path from line and curve commands.
- `PathCollection` when one operation should cover several paths.

`IPath.PathType` tells you whether a path is open, closed, or mixed. A mixed path is a composite path containing both open and closed figures.

Built-In Shapes

Built-in shape types are closed paths with a clear geometric meaning. Use them when the shape is part of the drawing model, not just a one-off primitive call. For example, an ellipse object can be reused for fill, stroke, clipping, hit testing, and layout bounds, while a primitive `DrawEllipse(...)` call only records that one drawing command.

The shape constructors use the coordinate model of the shape itself. Rectangle-like shapes use a position and size. Ellipses, regular polygons, stars, and pies are normally expressed from a center point plus radii or size. If a translated example looks offset, check whether the source API used top-left bounds while the ImageSharp.Drawing shape expects a center.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 260, Color.White.ToPixel<Rgba32>());
```

```
EllipsePolygon ellipse = new(120, 110, 160, 96);
StarPolygon star = new(x: 292, y: 128, prongs: 7, innerRadii: 34, outerRadii: 72);
PiePolygon pie = new(120, 202, radiusX: 120, radiusY: 86, startAngle: -30, sweepAngle: 245);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.SkyBlue), ellipse);
    canvas.Draw(Pens.Solid(Color.Navy, 3), ellipse);

    canvas.Fill(Brushes.Solid(Color.Orange), star);
    canvas.Draw(Pens.Solid(Color.DarkRed, 3), star);

    canvas.Fill(Brushes.Solid(Color.MediumSeaGreen), pie);
    canvas.Draw(Pens.Solid(Color.DarkGreen, 3), pie);
}));
```

Open and Closed Paths

Open paths are useful for strokes, polylines, and curved baselines. Closed paths enclose an area and are the normal input for fills. The distinction affects both fill behavior and stroke joins: a closed figure has a final join between the last and first segment, while an open figure has start and end caps.

`Path` is open by default. `Polygon` is closed. `PathBuilder.CloseFigure()` closes the current figure before starting the next one.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(440, 220, Color.White.ToPixel<Rgba32>());

PathBuilder openBuilder = new();
openBuilder.AddCubicBezier(
    new(36, 152),
    new(116, 34),
    new(252, 38),
    new(396, 154));

IPath openPath = openBuilder.Build();
```

```
PathBuilder closedBuilder = new();
closedBuilder.AddLines(new(64, 174), new(154, 54), new(244, 174));
closedBuilder.CloseFigure();

IPath closedPath = closedBuilder.Build();

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Draw(Pens.Solid(Color.MidnightBlue, 8), openPath);

    // Closed figures can be filled because they define an inside area.
    canvas.Fill(Brushes.Solid(Color.Gold.WithAlpha(0.6F)), closedPath);
    canvas.Draw(Pens.Solid(Color.DarkGoldenrod, 4), closedPath);
}));
```

When you fill an open path, ImageSharp.Drawing closes it for fill processing. Prefer building the figure as closed when the intended geometry is a filled area; that keeps the model clear and also gives stroke joins closed-contour behavior.

Custom Paths and Figures

Use `PathBuilder` for custom geometry. Build the path once, then reuse it for fill and stroke operations.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 220, Color.White.ToPixel<Rgba32>());

PathBuilder builder = new();
builder.AddLines(new(42, 176), new(112, 36), new(210, 154));
builder.AddCubicBezier(
    new(210, 154),
    new(268, 46),
    new(336, 50),
    new(376, 164));

IPath path = builder.Build();

image.Mutate(ctx => ctx.Paint(canvas =>
{
```

```
    canvas.Draw(Pens.Solid(Color.MidnightBlue, 8), path);
    canvas.Draw(Pens.Dot(Color.White, 3), path);
}));
```

`PathBuilder` supports multiple figures. If the builder contains more than one figure, `Build()` returns a `ComplexPolygon`. Each figure keeps its own open or closed state.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

PathBuilder builder = new();
builder.AddLines(new(52, 190), new(122, 54), new(196, 190));
builder.CloseFigure();

builder.AddCubicBezier(
    new(236, 178),
    new(268, 38),
    new(336, 48),
    new(374, 178));

IPath mixedPath = builder.Build();

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.LightSkyBlue), mixedPath);
    canvas.Draw(Pens.Solid(Color.Navy, 5), mixedPath);
}));
```

Use `PathBuilder.StartFigure()` when you want to begin a new figure without closing the previous one. Use `CloseAllFigures()` when every current figure should be closed.

Complex Polygons and Holes

`ComplexPolygon` represents multiple paths as one path. It is useful when a shape has multiple contours, or when you want to model an outer contour and one or more holes.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
```

```
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

Polygon outer = new(
[
    new PointF(60, 36),
    new PointF(360, 36),
    new PointF(360, 204),
    new PointF(60, 204)
]);

EllipsePolygon hole = new(210, 120, 178, 96);
ComplexPolygon complex = new(outer, hole);

DrawingOptions options = new()
{
    ShapeOptions = new()
    {
        IntersectionRule = IntersectionRule.EvenOdd
    }
};

image.Mutate(ctx => ctx.Paint(options, canvas =>
{
    canvas.Fill(Brushes.Solid(Color.MediumPurple), complex);
    canvas.Draw(Pens.Solid(Color.Black, 3), complex);
}));
```

The fill rule decides how overlapping contours inside a complex polygon are interpreted. `NonZero` is the default and matches the usual SVG and web canvas behavior: contour winding is meaningful, so holes are normally expressed by winding the inner contour in the opposite direction to its parent. Use `EvenOdd` when you want parity-based holes where contour direction is not significant.

Path Collections

`PathCollection` groups paths so one draw or fill call can apply the same brush, pen, and drawing state to all of them.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
```

```
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 220, Color.White.ToPixel<Rgba32>());

PathCollection bubbles = new(
    new EllipsePolygon(new PointF(104, 112), new SizeF(96, 72)),
    new EllipsePolygon(new PointF(210, 92), new SizeF(126, 86)),
    new EllipsePolygon(new PointF(316, 126), new SizeF(104, 78)));

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.LightCyan), bubbles);
    canvas.Draw(Pens.Solid(Color.DarkSlateBlue, 3), bubbles);
}));
```

Use a `PathCollection` when paths remain independent. Use `ComplexPolygon` or `Clip(...)` when the contours need to be interpreted together as one shape.

Clipping and Boolean Operations

`Clip(...)` creates a new path from a subject path and one or more clipping paths. The operation comes from `ShapeOptions.BooleanOperation`. The default boolean operation is `Difference`, which subtracts the clipping paths from the subject.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

EllipsePolygon subject = new(190, 120, 260, 154);
StarPolygon cutout = new(x: 226, y: 120, prongs: 6, innerRadii: 38, outerRadii: 82);

ShapeOptions clipOptions = new()
{
    BooleanOperation = BooleanOperation.Difference
};

IPath clipped = subject.Clip(clipOptions, cutout);
```

```
image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.Orange), clipped);
    canvas.Draw(Pens.Solid(Color.DarkRed, 3), clipped);
}));
```

Use `BooleanOperation.Intersection` when you want only the overlap, `Union` when you want to merge shapes, and `Xor` when you want areas covered by exactly one side.

Inspecting and Reusing Geometry

Paths are reusable geometry objects. You can measure them, transform them, convert open paths to closed paths, and use the same geometry for fills, strokes, clipping, and text paths.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

PathBuilder builder = new();
builder.AddLines(new(70, 178), new(132, 54), new(226, 172), new(318, 66), new(368, 178));

IPath openPath = builder.Build();
IPath closedPath = openPath.AsClosedPath();
IPath shiftedPath = closedPath.Translate(0, 24);

float length = openPath.ComputeLength();
float area = closedPath.ComputeArea();
RectangleF bounds = shiftedPath.Bounds;
RectanglePolygon boundsPath = new(bounds.X, bounds.Y, bounds.Width, bounds.Height);

image.Mutate(ctx => ctx.Paint(canvas =>
{

    // Use measurements to draw simple diagnostics around the transformed geometry.
    canvas.Fill(Brushes.Solid(Color.LightGoldenrodYellow), shiftedPath);
    canvas.Draw(Pens.Solid(Color.DarkGoldenrod, 4), shiftedPath);
    canvas.Draw(Pens.Dash(Color.Gray, 2), boundsPath);
}));
```

`ComputeLength()` follows open and closed contours. `ComputeArea()` is meaningful for closed shapes. `Transform(...)` applies an arbitrary matrix, while helpers such as `Translate(...)`, `Scale(...)`, and `RotateDegree(...)` cover common transforms.

Fill Rules

`ShapeOptions.IntersectionRule` controls how overlapping contours are interpreted during fill operations. `NonZero` is the default, matching the normal SVG and web canvas fill-rule default. Use `EvenOdd` when you explicitly want alternating inside/outside behavior for nested or overlapping contours.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());

EllipsePolygon outer = new(180, 110, 260, 150);
EllipsePolygon inner = new(180, 110, 126, 76);
PathCollection shape = new(outer, inner);

DrawingOptions options = new()
{
    ShapeOptions = new()
    {
        IntersectionRule = IntersectionRule.EvenOdd
    }
};

image.Mutate(ctx => ctx.Paint(options, canvas =>
{
    canvas.Fill(Brushes.Solid(Color.HotPink), shape);
    canvas.Draw(Pens.Solid(Color.Black, 3), shape);
}));
```

For lower-level polygon boolean operations, see PolygonClipper.

Practical Guidance

Use primitive helpers when geometry exists only for one command. Move to path and polygon objects when geometry becomes part of the model: the same shape is filled, stroked, clipped, transformed, measured, or shared between commands. That makes the relationship between layout and painting explicit.

Build closed paths deliberately when the shape represents an area. Filling an open path can work because the path is closed for fill processing, but a deliberately closed figure communicates intent and gives stroke joins closed-contour behavior. Use `ComplexPolygon` when multiple contours should be interpreted together as one region, especially when holes are involved.

The fill rule is part of the geometry contract. `NonZero` is the default and matches normal SVG and web canvas expectations, where winding direction is meaningful. Use `EvenOdd` when contour direction should not matter and nested contours should alternate inside/outside status.

Brushes and Pens

Brushes and pens separate *coverage* from *style*. A shape, path, text glyph, or generated stroke decides which pixels are covered. The brush then shades those covered pixels using a solid color, gradient, repeated pattern, image tile, or other brush source.

Pens are built on top of brushes. A pen does not directly paint a centerline; it expands the source line, path, or shape into stroke geometry using the pen width, caps, joins, miter limit, and dash pattern. That generated outline is then filled with the pen's brush. This matters when you debug output: stroke shape problems belong to `StrokeOptions`, while color, gradient, hatch, and image-fill problems belong to the brush.

Brushes and pens are recorded as part of canvas drawing intent, so keep any referenced resources alive until the canvas has replayed. This is especially important for `ImageBrush<TPixel>`, which references the source image rather than taking ownership of it.

Solid Brushes and Pens

Solid brushes and solid pens are the simplest styling objects. Use them for flat fills, outlines, guides, and most annotation work. The same brush can be used directly in `Fill(...)` or as the fill used by a pen stroke.

The pen width is expressed in the path's local coordinate space before the active drawing transform is applied. If you save a scaled transform on the canvas, the stroke geometry is prepared with that state during replay, so a scaled drawing state can scale the visible stroke as well as the path.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(320, 200, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    Rectangle panel = new(30, 28, 140, 92);
    canvas.Fill(Brushes.Solid(Color.LightSkyBlue), panel);
    canvas.Draw(Pens.Solid(Color.Navy, 4), panel);
```

```
    canvas.FillEllipse(Brushes.Solid(Color.Gold), new(230, 118), new(118, 72));
    canvas.DrawEllipse(Pens.Solid(Color.DarkOrange, 5), new(230, 118), new(118, 72));
}));
```

Pattern Brushes and Pattern Pens

Pattern brushes are small repeating color matrices. The built-in hatch helpers on `Brushes` create common foreground/background matrices such as horizontal, vertical, diagonal, and percentage patterns. Use a transparent background when the pattern should sit over existing pixels, or pass an opaque background color when the pattern should fully cover the area.

Pattern pens combine the same stroke-generation model as other pens with a dash pattern. `Pens.Dash(...)`, `Pens.Dot(...)`, `Pens.DashDot(...)`, and `Pens.DashDotDot(...)` are convenience factories for the common sequences. Pass a brush instead of a color when the stroke itself should be filled with a gradient, hatch, or image pattern.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

Brush hatchBrush = Brushes.ForwardDiagonal(Color.DarkSlateGray.WithAlpha(0.72F),
Color.Transparent);
Pen dashPen = Pens.Dash(Color.MidnightBlue, 5);
Pen dotPen = Pens.Dot(Color.Crimson, 4);
Pen dashDotPen = Pens.DashDot(Color.Black, 3);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    Rectangle hatchArea = new(28, 28, 160, 138);
    canvas.Fill(hatchBrush, hatchArea);
    canvas.Draw(dashPen, hatchArea);

    canvas.DrawEllipse(dotPen, new(292, 96), new(170, 92));
    canvas.DrawLine(dashDotPen, new(38, 206), new(150, 178), new(264, 210), new(382, 172));
}));
```

Pattern pens can also use a brush as their stroke fill, which is useful for gradient or hatch-pattern outlines.

Dash patterns are expressed as multiples of the pen width. The pattern `[3F, 1F]` means draw for three stroke widths, skip for one stroke width, then repeat.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 180, Color.White.ToPixel<Rgba32>());

PenOptions customDashOptions = new(Brushes.Solid(Color.DarkSlateBlue), 8, [4F, 1F, 1F, 1F])
{
    StrokeOptions = new()
    {
        LineCap = LineCap.Round,
        LineJoin = LineJoin.Round
    }
};

PatternPen customDash = new(customDashOptions);

PathBuilder builder = new();
builder.AddCubicBezier(new(32, 120), new(118, 18), new(286, 24), new(388, 132));

image.Mutate(ctx => ctx.Paint(canvas =>
{

    // The dash array is measured relative to the pen width.
    canvas.Draw(customDash, builder.Build());
}));
```

Gradient Brushes

Gradient brushes shade covered pixels from positions in canvas space. The color stops describe the ramp, and the brush geometry describes how that ramp is mapped into the drawn area. A linear gradient moves along a line between two points. A radial gradient expands from a center point and radius. Repetition mode controls what happens outside the primary gradient span: clamp to the edge colors, repeat the ramp, or reflect it.

Because the brush is evaluated over the covered pixels, the same gradient can be reused across several shapes to make them appear lit by one continuous source. If each shape needs its own

independent gradient, create a brush whose points and radius match that shape instead of sharing one global brush.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

LinearGradientBrush linear = new(
    new(24, 24),
    new(220, 150),
    GradientRepetitionMode.None,
    new(0F, Color.LightYellow),
    new(0.5F, Color.DeepSkyBlue),
    new(1F, Color.MediumBlue));

RadialGradientBrush radial = new(
    new(306, 116),
    82F,
    GradientRepetitionMode.Reflect,
    new(0F, Color.Orange),
    new(1F, Color.MediumVioletRed.WithAlpha(0.25F)));

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(linear, new Rectangle(24, 24, 190, 132));
    canvas.FillEllipse(radial, new(306, 116), new(156, 112));
}));
```

Image and Matrix Pattern Brushes

`PatternBrush` repeats a matrix of foreground/background values across the target. Use the `Brushes` helpers for common hatch styles, or construct a `PatternBrush` when you need a custom repeating matrix.

`ImageBrush<TPixel>` uses an image as the brush source. The source image is not disposed by the brush, so keep it alive for as long as the canvas might replay commands that reference it.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
```

```
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> tile = new(24, 24, Color.Transparent.ToPixel<Rgba32>());
tile.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.LightYellow));
    canvas.DrawLine(Pens.Solid(Color.DarkGoldenrod, 3), new PointF(0, 24), new
PointF(24, 0));
}));

using Image<Rgba32> image = new(420, 220, Color.White.ToPixel<Rgba32>());

ImageBrush<Rgba32> imageBrush = new(tile, new RectangleF(0, 0, 24, 24), new Point(0, 0));
PatternBrush matrixBrush = new(
    Color.DarkSlateGray.WithAlpha(0.75F),
    Color.Transparent,
    new bool[,]
    {
        { true, false, false, false },
        { false, true, false, false },
        { false, false, true, false },
        { false, false, false, true }
    });

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(imageBrush, new Rectangle(28, 28, 160, 132));
    canvas.Fill(matrixBrush, new Rectangle(232, 28, 160, 132));
}));
```

Stroke Shape Options

`StrokeOptions` controls the geometry produced before the pen's brush is applied. `LineCap` affects the ends of open paths and line segments. `LineJoin` affects corners where segments meet. `MiterLimit` limits how far sharp miter joins can extend before the join falls back to a bevel-style shape. `ArcDetailScale` controls the detail used when rounded joins and caps are converted into geometry.

Use stroke options when the outline itself is wrong: squared ends, overly sharp corners, clipped-looking miters, or rounded joins that need more detail. Use a different brush when the outline shape is correct but the stroke color, gradient, pattern, or image fill is wrong.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 220, Color.White.ToPixel<Rgba32>());

StrokeOptions strokeOptions = new()
{
    LineJoin = LineJoin.Round,
    LineCap = LineCap.Round,
    MiterLimit = 4,
    ArcDetailScale = 1
};

PenOptions penOptions = new(Brushes.Solid(Color.MidnightBlue), 12, strokePattern: null)
{
    StrokeOptions = strokeOptions
};

SolidPen pen = new(penOptions);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.DrawLine(pen, new(40, 170), new(130, 42), new(250, 166), new(374, 54));
}));
```

Pens do not paint centered pixels directly. A pen describes an outline generated from the source path, line, or shape. The generated outline is then filled with the pen's brush. This is why caps, joins, miter limits, dashes, and stroke width belong to the pen.

Use `Pen.GeneratePath(...)` when you need to inspect or reuse the stroked outline as a shape.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 220, Color.White.ToPixel<Rgba32>());

PathBuilder builder = new();
builder.AddLine(new(52, 164), new(178, 44));
```

```
builder.AddLine(new(178, 44), new(328, 166));

IPath centerLine = builder.Build();
Pen outlinePen = Pens.Solid(Color.MediumVioletRed, 18);
IPath outline = outlinePen.GeneratePath(centerLine);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.Pink.WithAlpha(0.45F)), outline);
    canvas.Draw(Pens.Solid(Color.DarkRed, 2), outline);
    canvas.Draw(Pens.Dash(Color.Gray, 1.5F), centerLine);
}));
```

Clipping Brushes and Pens

Clipping is canvas state, not a brush or pen property. Use `Save(DrawingOptions, params IPath[])` to apply one or more clip paths to later brush and pen commands, then `Restore()` when the clipped work is complete.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

EllipsePolygon clip = new(210, 120, 300, 150);
LinearGradientBrush brush = new(
    new PointF(40, 40),
    new PointF(380, 200),
    GradientRepetitionMode.None,
    new ColorStop(0F, Color.Gold),
    new ColorStop(1F, Color.MediumPurple));
DrawingOptions clipInside = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
```

```
    _ = canvas.Save(clipInside, clip);

    // Both the fill and the dashed outline are clipped by the saved canvas state.
    canvas.Fill(brush, new Rectangle(32, 34, 356, 172));
    canvas.Draw(Pens.Dash(Color.Black, 5), new Rectangle(32, 34, 356, 172));
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.DarkSlateGray, 2), clip);
}));
```

Practical Guidance

Brushes and pens answer different questions. A brush shades covered pixels. A pen describes how a stroke outline is generated and how that outline is filled. Keeping that distinction clear prevents a lot of awkward geometry code: cap, join, miter, dash, and stroke-width decisions belong on the pen, not in hand-built outline paths.

Create reusable pens and brushes when the same style appears across many commands. That keeps examples readable and production drawing code easier to audit. Use canvas clipping state when a style should be constrained to a region; clipping is part of drawing state, not something each brush or pen needs to know about.

Clipping, Regions, and Layers

Canvas state controls where later commands can draw and how grouped commands are composed. The three main tools are `Save(...)` with clip paths, `CreateRegion(...)`, and `SaveLayer(...)`.

These APIs solve different problems and should not be treated as interchangeable:

- Use a clip when later commands should be constrained by vector geometry while staying in the current coordinate system.
- Use a region when you want a rectangular child canvas where `(0, 0)` means the region origin.
- Use a layer when several commands should render together into an isolated target and then composite back as one result.

Clip Later Commands

`Save(DrawingOptions, params IPath[])` pushes a new state with the supplied options and clip paths. The clip paths are combined with each later command by `ShapeOptions.BooleanOperation`; they are not applied retroactively to commands already recorded.

The default boolean operation is `Difference`, which subtracts the clip path. For ordinary "draw inside this shape" clipping, set `BooleanOperation.Intersection`.

Think of clipping as a state scope. Save the clipped state immediately before the work that needs it, then restore as soon as that work is complete. That keeps borders, labels, shadows, and diagnostics from being clipped accidentally.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 260, Color.White.ToPixel<Rgba32>());

EllipsePolygon spotlight = new(210, 130, 300, 160);
DrawingOptions clipInside = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
```

```
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.MidnightBlue));

    _ = canvas.Save(clipInside, spotlight);

    // The rectangle is larger than the ellipse; the saved state keeps only
the intersection.
    canvas.Fill(Brushes.Horizontal(Color.Gold, Color.OrangeRed), new Rectangle(20, 40,
380, 180));
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.White, 3), spotlight);
}));
```

Use `Restore()` to pop the latest state, or `RestoreTo(saveCount)` when nested states must be unwound together.

Region Canvases

`CreateRegion(...)` creates a child canvas with local coordinates inside a rectangular area. It is useful for controls, panels, tiles, thumbnails, and other sub-layouts where `(0, 0)` should mean the region origin.

A region is a coordinate convenience, not an independent render target. The child canvas shares the parent replay timeline, and the root canvas still owns final replay. Disposing the child region closes that local drawing scope; it does not render the whole image immediately.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    using DrawingCanvas region = canvas.CreateRegion(new Rectangle(70, 48, 220, 124));

    region.Fill(Brushes.Solid(Color.LightSeaGreen.WithAlpha(0.8F)), new Rectangle(10, 10,
120, 68));
```

```
    // Region-local coordinates are relative to the region, not the parent canvas.
    region.Draw(Pens.Solid(Color.DarkBlue, 5), new Rectangle(0, 0, 220, 124));
    region.DrawLine(Pens.Solid(Color.OrangeRed, 4), new PointF(0, 123), new PointF(219, 0));
}));
```

Nested regions can also have their own saved state. Use them when nested layout is clearer than constantly adding offsets to parent-canvas coordinates.

Layers

`SaveLayer(...)` starts an isolated compositing scope. Commands drawn inside the layer render into that layer, then `Restore()` composites the layer back to the parent with the supplied `GraphicsOptions`.

Layer bounds limit the isolated target and final composition area. They do not shift the coordinate system. Commands inside a bounded layer still use the same local coordinates as the parent canvas, so the layer bounds should describe the affected area, not a new origin.

Use a layer when group behavior matters. Group opacity, group blending, and grouped masking are different from applying the same `GraphicsOptions` to each command independently. Without a layer, two semi-transparent shapes can blend with each other and the background one command at a time; with a layer, they first form one isolated result and then that result blends back once.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.SteelBlue), new Rectangle(24, 24, 312, 172));

    _ = canvas.SaveLayer(new GraphicsOptions { BlendPercentage = 0.55F }, new Rectangle(70,
46, 220, 128));

    // Layer bounds constrain compositing; these coordinates are still parent coordinates.
    canvas.FillEllipse(Brushes.Solid(Color.OrangeRed), new(180, 110), new(170, 96));
    canvas.Draw(Pens.Solid(Color.White, 8), new Rectangle(96, 74, 168, 72));
    canvas.Restore();
```

```
    canvas.Draw(Pens.Solid(Color.Black, 2), new Rectangle(70, 46, 220, 128));
}));
```

Use layers when a group of commands should blend back as one result. Without a layer, each command blends into the parent independently.

Layer Effects

`SaveLayer(...)` overloads accept effects that transform the layer when it is restored: blur, drop shadow, glow, inner shadow, color matrix, and CSS-style backdrop filters. The built-in set, the GPU shader implementations, and custom effect authoring have a dedicated page. See Layer and Backdrop Effects.

Practical Guidance

- Use clips when geometry should constrain later commands in the current coordinate space.
- Use regions when a child layout should have its own local origin.
- Use layers when several commands should blend back as one grouped result.
- Use a `LayerEffect` when the layer content needs blur, shadow, glow, or color treatment as a group. See Layer and Backdrop Effects.
- Use a `BackdropLayerEffect` when the pixels behind a region need treatment while the content on top stays sharp.
- Remember that layer bounds constrain composition but do not move the coordinate system.
- Restore saved state as soon as the scoped work is complete.

Layer and Backdrop Effects

`SaveLayer(...)` isolates a group of drawing commands into a compositing layer. An effect transforms that layer as part of the same operation: blur it, shadow it, recolor it, or filter the pixels behind it. This page covers the built-in effects, the GPU shader implementations, and how to build your own. For the layer model itself, see Clipping, Regions, and Layers.

Layer Effects

`SaveLayer(...)` overloads accept a `LayerEffect` that transforms the layer content when the layer is restored. The layer isolates the content, so the effect operates on exactly what was drawn between `SaveLayer(...)` and `Restore()`, against transparency, before the result composites onto the canvas.

The built-in content effects are:

- `BlurLayerEffect` blurs the layer content with a Gaussian sigma.
- `DropShadowLayerEffect` slots a blurred, offset shadow beneath the layer content.
- `GlowLayerEffect` surrounds the content with a colored glow.
- `InnerShadowLayerEffect` shades the content inside its own edges.
- `ColorMatrixLayerEffect` transforms the content colors through a `ColorMatrix`.

The layer bounds are expanded internally by the effect's reach, so blurred or offset output is not cut off.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 220, Color.White.ToPixel<Rgba32>());

image.Mutate(ctx => ctx.Paint(canvas =>
{
    _ = canvas.SaveLayer(
        new GraphicsOptions(),
        new Rectangle(60, 40, 240, 140),
        new DropShadowLayerEffect(new Point(6, 6), 4F, Color.Black.WithAlpha(0.5F)));

    canvas.FillEllipse(Brushes.Solid(Color.OrangeRed), new(180, 110), new(90, 50));
```

```
    canvas.Restore();
}));
```

An overload accepts an `IPath` region instead of a rectangle. The effect then processes only pixels covered by the path, and its output lands on the path translated by the effect's offset.

Backdrop Effects

A `BackdropLayerEffect` filters the pixels already on the canvas beneath the layer's region. The filter runs when the layer is opened, the filtered result is clipped to the region, and the layer's content then renders above it. This is the CSS `backdrop-filter` model.

The built-in backdrop effects are:

- `BackdropBlurLayerEffect` blurs the backdrop, the classic frosted-glass base.
- `BackdropAcrylicLayerEffect` combines a blur with a tint color for acrylic panels.
- `BackdropDropShadowLayerEffect` casts a shadow from the backdrop content.
- `BackdropColorMatrixLayerEffect` transforms backdrop colors through a `ColorMatrix`. Prebuilt variants cover brightness, contrast, grayscale, hue rotation, inversion, opacity, sepia, and saturation.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = Image.Load<Rgba32>("photo.jpg");

image.Mutate(ctx => ctx.Paint(canvas =>
{
    Rectangle panel = new(70, 46, 220, 128);

    _ = canvas.SaveLayer(
        new GraphicsOptions(),
        panel,
        new BackdropAcrylicLayerEffect(10F, Color.White.WithAlpha(0.3F)));

    // The backdrop inside the panel is already blurred and tinted.
    // Content drawn here renders above the filtered backdrop.
    canvas.Draw(Pens.Solid(Color.White, 2), panel);
    canvas.Restore();
}));
```

Content effects and backdrop effects answer different questions. Use a content effect when the drawing inside the layer needs the treatment. Use a backdrop effect when the pixels behind a panel need the treatment and the panel content must stay sharp.

Custom Effects

Both effect families are open for extension. Pass the `LayerEffect` base constructor an operation that implements your effect through image processing, then override the members that describe its behavior:

- Override `Reach` when the effect pushes content beyond its source region. Layer and processing bounds expand by this distance, so the output is not cut off.
- Override `WriteBackOptions` and `WriteBackOffset` when the result must composite back with specific graphics options or at an offset, as a drop shadow does.
- Override `IsPassThrough` so a no-op configuration skips processing entirely.

Derive from `BackdropLayerEffect` in the same way to filter the pixels behind a region. Backdrop effects clip to their region, so their reach is always zero.

The operation can implement any per-pixel logic. `ProcessPixelRowsAsVector4(...)` overloads that take a `PixelRowOperation<Point>` supply each row's start coordinate, so position-dependent effects such as vignettes and scanlines are possible too.

This posterize effect quantizes each color channel with its own per-pixel logic. ImageSharp has no such processor, so the operation implements the math directly:

```
using System.Numerics;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

// Quantizes each color channel to a fixed number of levels.
public sealed class PosterizeLayerEffect : LayerEffect
{
    private readonly int levels;

    public PosterizeLayerEffect(int levels)
        : base(ctx => ctx.ProcessPixelRowsAsVector4(row =>
        {
            float steps = levels - 1;
            for (int i = 0; i < row.Length; i++)
            {
                Vector4 pixel = row[i];
                Vector3 quantized = new Vector3(
```

```
                    MathF.Round(pixel.X * steps),
                    MathF.Round(pixel.Y * steps),
                    MathF.Round(pixel.Z * steps)) / steps;
                row[i] = new Vector4(quantized, pixel.W);
            }
        }))
        => this.levels = levels;

    // Quantizing to 256 or more levels leaves 8-bit content unchanged.
    public override bool IsPassThrough => this.levels >= 256;
}
```

An effect authored this way runs its operation on every backend. On a WebGPU canvas, that means a CPU round trip for the effect step. The next section removes that cost.

GPU Shader Effects

The WebGPU package ships shader-backed versions of the built-in set: `WebGPUGaussianBlurLayerEffect`, `WebGPUDropShadowLayerEffect`, `WebGPUGlowLayerEffect`, `WebGPUInnerShadowLayerEffect`, and `WebGPUColorMatrixLayerEffect`, with matching backdrop variants. These execute as WGSL shader passes, so the effect stays on the GPU. Each carries the equivalent CPU effect as a built-in fallback, so one instance renders correctly on both backends.

Custom GPU effects follow the same pattern. Derive from `WebGPUShaderLayerEffect` for layer content, or `WebGPUBackdropShaderLayerEffect` for backdrops. The base constructor takes complete WGSL source, a `WebGPUShaderUniformLayout` describing the named uniform values, and the CPU fallback: either an ImageSharp processing operation or an equivalent `LayerEffect`.

The Shader Contract

The source must define the entry point `fn layer_effect(position: vec2<f32>) -> vec4<f32>`. The position uses effect-local pixel-center coordinates. The return value is associated-alpha RGBA in `[0, 1]`.

ImageSharp generates a prelude around your source. It provides these functions for reading the effect input:

Function	Behavior
`layer_load(position: vec2<i32>) -> vec4<f32>`	Loads one texel with associated alpha. Positions outside the valid input region return transparent black.

Function	Behavior
`layer_load_unassociated(position: vec2<i32>) -> vec4<f32>`	Loads one texel with the alpha unassociated, for math on straight color values.
`layer_sample(position: vec2<f32>) -> vec4<f32>`	Samples with bilinear filtering at a pixel-center position, honoring the pass's border wrapping modes.

The prelude also declares two uniform structures:

- `imagesharp_framework` carries the geometry of the input: `imagesharp_input_size`, `imagesharp_valid_min`, and `imagesharp_valid_max`, all `vec2<i32>`.
- `imagesharp_uniforms` carries the fields you declare through `WebGPUShaderUniformLayout`. Each `WebGPUShaderUniform` declares a name, a type (`Float32`, `Int32`, `UInt32`, `Vector2`, `Vector3`, `Vector4`, or `Matrix4x4`), and an element count for arrays. The layout is capped at 64 KiB.

Passes

Each source defines one complete shader pass. The protected `AddShaderPass(...)` method adds an ordered invocation and assigns its uniform values through a `WebGPUShaderUniformBuilder`, with typed setters such as `SetFloat32(...)`, `SetVector2(...)`, and `SetMatrix4x4(...)`. An overload sets the horizontal and vertical `BorderWrappingMode` for the pass's filtered samples. Each pass receives the preceding pass's output as its input. The built-in Gaussian blur runs as two separable passes this way.

A Complete Dual-Backend Effect

This GPU version of the posterize effect implements the same math in WGSL and reuses the CPU class as its fallback. One type renders on both backends:

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;

// Posterizes on the GPU, falling back to PosterizeLayerEffect on the CPU.
public sealed class WebGPUPosterizeLayerEffect : WebGPUShaderLayerEffect
{
    private const string ShaderSource = """
        fn layer_effect(position: vec2<f32>) -> vec4<f32> {
            let source = layer_load_unassociated(vec2<i32>(position));
            let steps = imagesharp_uniforms.levels - 1.0;
            let quantized = round(source.rgb * steps) / steps;
```

```
            // Return associated-alpha RGBA.
            return vec4<f32>(quantized * source.a, source.a);
        }
        """;

    public WebGPUPosterizeLayerEffect(int levels)
        : base(
            new PosterizeLayerEffect(levels),
            ShaderSource,
            new WebGPUShaderUniformLayout([new WebGPUShaderUniform("levels",
WebGPUShaderUniformType.Float32, 1)]))
        => this.AddShaderPass(uniforms => uniforms.SetFloat32("levels", levels));
}
```

Shader pipelines compile on first use. Call `WebGPUDeviceContext.Precompile(...)` during startup when the first frame must not pay that cost.

Practical Guidance

- Use a content effect when the drawing inside the layer needs the treatment.
- Use a backdrop effect when the pixels behind a region need the treatment while the content on top stays sharp.
- Derive from the shader base classes for effects that render on GPU canvases, and supply the CPU fallback for everything else.
- Declare an accurate `Reach` for spreading effects, so nothing is clipped.
- Precompile custom shader effects when startup can absorb the cost and the first frame cannot.

Related Topics

- Clipping, Regions, and Layers
- Canvas Drawing
- WebGPU

Images, Masks, and Processing

ImageSharp.Drawing can draw images through the canvas, use images as brushes, and run ImageSharp processors inside drawing regions. These features look similar because they all produce pixels from images, but they model different intent.

Use `DrawImage(...)` when you want to place a rectangular source image into a rectangular destination. Use an image brush when the image should behave like a fill for arbitrary geometry. Use `Apply(...)` when you need normal ImageSharp processors to affect the pixels visible at a specific point in the canvas timeline.

Draw an Image

`DrawImage(...)` samples a source rectangle from an image and places it into a destination rectangle on the canvas. The source rectangle is expressed in source-image coordinates. The destination rectangle is expressed in canvas coordinates and is affected by the current transform and clip state.

The optional resampler is used when the selected source pixels have to be scaled into the destination. Bicubic is the default, so pass a sampler only when your output needs a different tradeoff such as sharper edges, smoother downsampling, or a specific product policy.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> image = new(480, 300, Color.White.ToPixel<Rgba32>());

DrawingOptions clipInside = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
};

EllipsePolygon clip = new(240, 150, 340, 190);

image.Mutate(ctx => ctx.Paint(canvas =>
```

```
{
    _ = canvas.Save(clipInside, clip);

    // The selected source pixels are scaled into the destination rectangle.
    canvas.DrawImage(source, new Rectangle(20, 10, 280, 180), new RectangleF(70, 54, 340,
190), KnownResamplers.Bicubic);
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.Black, 3), clip);
}));
```

Keep the source image alive until the canvas has replayed the command. With `Paint(...)`, that means the source must remain alive until `Mutate(...)` completes.

Choose the source rectangle in source-image coordinates and the destination rectangle in canvas coordinates. That separation is useful when you want to crop from a large source image while placing the selected pixels into a fixed layout region.

Use an Image as a Brush

Use `ImageBrush<TPixel>` when an image should fill any path as a texture. This is different from `DrawImage(...)`: the brush supplies sampled image pixels, while the supplied path controls coverage. That makes image brushes useful for clipped portraits, textured text, patterned fills, masks, thumbnails inside arbitrary shapes, and repeated decorative elements.

An image brush references its source image. Keep the source alive until canvas replay has completed. If the same image is reused by several brushes or commands, own that lifetime outside the `Paint(...)` callback rather than disposing it in the middle of drawing.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> image = new(420, 260, Color.White.ToPixel<Rgba32>());

StarPolygon star = new(x: 210, y: 130, prongs: 5, innerRadii: 62, outerRadii: 118);
RectangleF sourceRegion = new(0, 0, source.Width, source.Height);
ImageBrush<Rgba32> brush = new(source, sourceRegion, new Point(-120, -70));

image.Mutate(ctx => ctx.Paint(canvas =>
```

```
{
    // The star path controls coverage; the brush supplies the sampled image pixels.
    canvas.Fill(brush, star);
    canvas.Draw(Pens.Solid(Color.DarkSlateGray, 3), star);
}));
```

An image brush is best when the same texture should fill a shape, text path, or repeated decorative element. For one-off rectangular placement, `DrawImage(...)` is usually easier to reason about.

Apply Processors Inside a Shape

`Apply(...)` runs normal ImageSharp processors inside a rectangle, path, or path builder. It is a replay barrier: commands before it affect the pixels being processed, and commands after it do not.

That makes `Apply(...)` a timeline tool, not just a clipping tool. Put it immediately after the pixels that should be processed. Draw crisp outlines, labels, or foreground objects after the barrier so they are not blurred, pixelated, color-adjusted, or otherwise processed with the background.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> image = new(520, 320, Color.White.ToPixel<Rgba32>());

RectangleF destination = new(40, 38, 440, 244);
EllipsePolygon redaction = new(300, 168, 150, 96);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.DrawImage(source, source.Bounds, destination, KnownResamplers.Bicubic);

    // Apply scopes the processor to the path; pixels outside the ellipse stay unchanged.
    canvas.Apply(redaction, region => region.Pixelate(10));
    canvas.Draw(Pens.Solid(Color.OrangeRed, 3), redaction);
}));
```

On GPU-backed canvases, `Apply(...)` requires the affected pixels to be read back, processed by the CPU pipeline, and written back before presentation. Keep regions as small as the effect

allows.

The placement of `Apply(...)` matters. Commands recorded before it contribute pixels to the processor input; commands recorded after it are drawn over the processed result. This makes it possible to blur or pixelate an image region, then draw a crisp outline or label on top.

Practical Guidance

Use `DrawImage(...)` when an image should be sampled from a source rectangle and placed into a destination rectangle. Use an image brush when the image should behave like a fill pattern inside arbitrary geometry. Those two APIs can produce similar-looking results, but they model different intent: placement versus shading.

The source image must remain alive until the canvas has replayed the command. In `Paint(...)`, that means through the end of the mutation pipeline. With manually-created canvases, it means until the canvas is disposed. Source rectangles are expressed in source-image coordinates; destination rectangles are expressed in canvas coordinates, so cropping and placement can be reasoned about independently.

`Apply(...)` is a timeline decision. Put it exactly where the processor should observe the image: commands before it contribute pixels to the processor input, and commands after it draw over the processed result. Keep processor regions tight, especially on GPU-backed canvases where readback may be required.

Transforms and Composition

`DrawingOptions` carries the transform, graphics options, and shape options used by canvas commands. Use it when drawing state should change for a group of operations.

The safest way to think about transforms and composition is as scoped canvas state. Save the options that should affect a group, draw the affected commands, then restore the previous state before drawing labels, guides, or other unaffected output.

`DrawingOptions` is not a styling object like a brush or pen. It describes how later drawing commands are interpreted by the canvas: where their geometry lands, how their coverage is rasterized, how their pixels combine with existing pixels, and how fill or clip geometry is interpreted. That is why the same options object can affect fills, strokes, text, images, clips, layers, and image-processing barriers.

Transform Drawing

`DrawingOptions.Transform` is applied to vector output before rasterization. Paths, shapes, text glyph geometry, generated stroke outlines, and clip paths are prepared with the active transform before the backend receives the command. The source geometry still starts in the local coordinate system you wrote in the code; the transform is part of the saved canvas state that converts that local geometry into final drawing space.

For strokes, the pen first generates an outline from the source path in local geometry space, then the active transform is applied to that generated outline. That means a scaled drawing state affects the visible stroke as well as the path it follows. If you need a shape to move or rotate while keeping a screen-constant outline width, draw the fill inside transformed state, restore, then draw the outline separately in parent coordinates.

Why Matrix4×4?

ImageSharp.Drawing is a 2D drawing library, but it uses `Matrix4x4` for transforms so the same drawing state can represent both ordinary 2D affine transforms and projective transforms.

For normal drawing, construct the value from `Matrix3x2`. That keeps rotation, scale, skew, and translation code familiar:

```
Matrix4x4 transform = new(Matrix3x2.CreateRotation(angle, center));
```

When more than one 2D operation is needed, compose the `Matrix3x2` expression first and wrap the final result in `Matrix4x4`. Keeping the 2D operations together makes order explicit and avoids hand-written matrix values for ordinary scale, rotate, skew, and translate cases.

Use the full `Matrix4x4` form when you need transforms that cannot be expressed by `Matrix3x2`, such as perspective-style projection. The canvas, path, text, brush, image, and WebGPU paths all carry the same transform type, so code can move between CPU drawing, retained backend scenes, and GPU rendering without changing the public drawing model.

```
using System.Numerics;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 260, Color.White.ToPixel<Rgba32>());

DrawingOptions rotated = new()
{
    Transform = new(Matrix3x2.CreateRotation(0.32F, new Vector2(210, 130)))
};

Rectangle panel = new(92, 70, 236, 120);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    _ = canvas.Save(rotated);

    // Both the fill and stroke use the saved transform.
    canvas.Fill(Brushes.Solid(Color.LightSkyBlue), panel);
    canvas.Draw(Pens.Solid(Color.MidnightBlue, 5), panel);
    canvas.Restore();

    canvas.Draw(Pens.Dot(Color.Gray, 2), panel);
}));
```

Transforms also apply to clipped drawing. When you save transformed options with clip paths, the command geometry and clip geometry are prepared so the backend receives consistent clipped output.

Blend and Composite

`GraphicsOptions` answers four separate questions for each command:

- `Antialias` controls coverage at geometry edges. When enabled, edge pixels can receive fractional coverage for smoother vector output. When disabled, each pixel is fully covered or not covered, decided at the pixel center.
- `BlendPercentage` scales the strength of the drawing operation. `1F` applies the command at full strength, `0F` makes it invisible, and values in between behave like operation opacity.
- `ColorBlendingMode` controls how source and destination color channels are combined where the command draws. `Normal` uses ordinary alpha blending. The full W3C blend mode set is supported: separable modes such as `Multiply`, `Screen`, `Overlay`, `SoftLight`, `ColorDodge`, and `Difference`, and the non-separable `Hue`, `Saturation`, `Color`, and `Luminosity` modes. `Add` and `Subtract` are also available.
- `AlphaCompositionMode` controls how source and destination alpha are combined using Porter-Duff composition rules. The default `SrcOver` draws the new source over existing pixels. Modes such as `Src`, `Clear`, `DestIn`, and `DestOut` are useful for replacement, erasing, masks, and cutouts. `Plus` adds source and destination, clamped to the supported range, which is useful for additive light effects.

Those settings are per-command canvas state when you use `Save(...)`. If two shapes are drawn under a saved `GraphicsOptions`, each one blends independently with the destination. Use `SaveLayer(...)` when several commands should first render together into an isolated layer and then blend back as one group.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(360, 240, Color.White.ToPixel<Rgba32>());

DrawingOptions multiply = new()
{
    GraphicsOptions = new()
    {
        ColorBlendingMode = PixelColorBlendingMode.Multiply,
        AlphaCompositionMode = PixelAlphaCompositionMode.SrcOver,
        BlendPercentage = 0.85F
    }
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
```

```
    canvas.Fill(Brushes.Solid(Color.DarkBlue), new Rectangle(24, 88, 312, 60));

    _ = canvas.Save(multiply);

    // The saved GraphicsOptions affect commands recorded until Restore.
    canvas.Fill(Brushes.Solid(Color.HotPink), new Rectangle(100, 32, 110, 176));
    canvas.FillEllipse(Brushes.Solid(Color.Red.WithAlpha(0.5F)), new(194, 120),
new(124, 92));
    canvas.Restore();
}));
```

The example uses `Multiply` to darken overlapping colors while leaving alpha composition as `SrcOver`, so the new shapes still draw over the existing background. `BlendPercentage` reduces the strength of the whole operation without changing the source color values in the code.

Use `SaveLayer(...)` when the blend should apply to a group as a single composited result. Use plain `Save(...)` when each command should blend independently. This distinction is important for group opacity: two semi-transparent shapes drawn independently will overlap each other; the same shapes drawn inside a layer can be composited once as a single group.

Antialiasing

Antialiasing is about edge coverage, not color choice. With antialiasing enabled, partially covered edge pixels receive partial coverage so diagonal and curved edges look smooth. With antialiasing disabled, each pixel is either fully covered or not covered. Coverage is decided by sampling the geometry at the pixel center, matching classic aliased rasterizers. Thin fill features that would otherwise fall between pixel centers are recovered, so hairline geometry does not disappear.

Turn antialiasing off when exact binary coverage matters, such as low-resolution masks, hit-test masks, generated sprite masks, or pixel-art-style output. Leave it on for normal vector graphics, text, badges, diagrams, and annotations.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(80, 80, Color.Black.ToPixel<Rgba32>());
DrawingOptions aliased = new()
{
    GraphicsOptions = new()
    {
        Antialias = false
```

```
    }
};

image.Mutate(ctx => ctx.Paint(aliased, canvas =>
{
    // With antialiasing disabled, integer rectangle corners render as full covered pixels.
    canvas.Fill(Brushes.Solid(Color.White), new Rectangle(10, 10, 44, 28));
}));
```

Practical Guidance

- Treat transforms and graphics options as scoped canvas state.
- Compose ordinary 2D transforms with `Matrix3x2`, then wrap the final value in `Matrix4x4`.
- Draw diagnostic bounds outside transformed state when you need parent-coordinate references.
- Use `SaveLayer(...)` for group opacity or group blending; use `Save(...)` for per-command state.
- Leave antialiasing enabled for normal vector graphics and disable it only for exact pixel coverage.

Drawing Text

ImageSharp.Drawing exposes a high-performance text drawing API that is unusually rich for a 2D image library. It combines the text engine from SixLabors.Fonts with the canvas drawing model, so shaped text, fallback fonts, color fonts, bidirectional layout, wrapping, alignment, rich runs, filled glyphs, stroked glyphs, decorations, path text, and glyph geometry all flow through `DrawingCanvas.DrawText(...)`.

Use the Fonts docs for font loading and text-layout details. This page focuses on placing that text onto an image.

At the simple end, text is one call. At the advanced end, the same model can draw a multilingual paragraph with per-run fonts, brushes, pens, decorations, and layout options, or turn glyphs into paths for clipping and compositing.

Draw Simple Text

Simple text drawing still uses the full Fonts shaping pipeline. The text is shaped, positioned from `RichTextOptions.Origin`, and then painted through the same brush and pen model as other canvas drawing. Pass a brush to fill glyphs, a pen to outline glyphs, or both when the text needs a filled face and a stroked edge.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(640, 240, Color.White.ToPixel<Rgba32>());

Font font = SystemFonts.CreateFont("Arial", 46);
RichTextOptions options = new(font)
{
    Origin = new(48, 72)
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.DrawText(options, "Hello from ImageSharp.Drawing", Brushes.Solid(Color.Black),
```

```
pen: null);
}));
```

Even in simple examples, treat `RichTextOptions` as part of the drawing contract. If you later measure the same string, use the same font, wrapping, alignment, culture, fallback, and feature settings so the measured layout matches the rendered pixels.

Draw Rich Text

`RichTextOptions.TextRuns` lets one string carry multiple visual styles without manually splitting and positioning each span. Runs can change font, brush, pen, decorations, and other text features while the layout engine still wraps and aligns the text as one paragraph.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(760, 260, Color.White.ToPixel<Rgba32>());

Font body = SystemFonts.CreateFont("Arial", 34);
Font emphasis = SystemFonts.CreateFont("Arial", 40, FontStyle.Bold);
const string text = "Rich text can mix fill, outline, and decoration in one layout.";

RichTextOptions options = new(body)
{
    Origin = new(48, 48),
    WrappingLength = 664,
    LineSpacing = 1.15F,
    TextRuns =
    [
        new RichTextRun
        {
            Start = 0,
            End = 9,
            Font = emphasis,
            Brush = Brushes.Solid(Color.MidnightBlue),
            Pen = Pens.Solid(Color.Gold, 1.5F)
        },

        new RichTextRun
        {
```

```
            Start = 18,
            End = 22,
            Brush = Brushes.Solid(Color.DarkRed),
            TextDecorations = TextDecorations.Underline,
            UnderlinePen = Pens.Solid(Color.DarkRed, 2)
        },

        new RichTextRun
        {
            Start = 24,
            End = 31,
            Brush = Brushes.Solid(Color.DarkGreen),
            Pen = Pens.Solid(Color.LightGreen, 1)
        },

        new RichTextRun
        {
            Start = 37,
            End = 47,
            Brush = Brushes.Solid(Color.DarkGoldenrod),
            TextDecorations = TextDecorations.Overline,
            OverlinePen = Pens.Solid(Color.DarkGoldenrod, 2)
        }
    ]
};

image.Mutate(ctx => ctx.Paint(canvas =>
{

    // Runs style spans; DrawText still shapes, wraps, and aligns the paragraph as
one layout.
    canvas.DrawText(options, text, Brushes.Solid(Color.Black), pen: null);
}));
```

Run indices are counted in grapheme clusters, not UTF-16 code units. `Start` is inclusive and `End` is exclusive, so each run covers the `[Start, End)` grapheme range. For plain ASCII those values match character positions; for emoji, combining marks, and complex scripts, count grapheme clusters as shown in the Fonts Unicode docs.

Draw Prepared Text

Use TextBlock when the same text will be measured, wrapped, inspected, or drawn more than once. `TextBlock` keeps the prepared text layout work in the Fonts layer, and `DrawingCanvas.DrawText(...)` places that prepared block onto the canvas.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(640, 260, Color.White.ToPixel<Rgba32>());

Font font = SystemFonts.CreateFont("Arial", 32);
RichTextOptions options = new(font)
{
    Origin = new(0, 0),
    HorizontalAlignment = HorizontalAlignment.Center,
    LineSpacing = 1.15F
};

TextBlock block = new("Prepared text can be measured and drawn with the same
shaping.", options);
TextMetrics metrics = block.Measure(wrappingLength: 520);
Rectangle layoutBox = new(60, 48, 520, (int)MathF.Ceiling(metrics.Advance.Height + 24));

image.Mutate(ctx => ctx.Paint(canvas =>
{

    // TextBlock owns shaping and text options; DrawText supplies canvas placement
and wrapping.
    canvas.Draw(Pens.Solid(Color.LightGray, 1), layoutBox);
    canvas.DrawText(block, new PointF(60, 60), 520, Brushes.Solid(Color.DarkSlateBlue),
pen: null);
}));
```

For manual line flow, choose the `TextBlock` API based on the coordinate space you want to draw from:

- Use `TextBlock.GetLineLayouts(...)` when the text still behaves as one stacked block. Each returned `LineLayout` is positioned in block coordinates, including the cumulative advance of the lines before it, so it is ready to draw relative to the block origin.
- Use `TextBlock.EnumerateLineLayouts()` when each line is placed independently. Each `LineLayout` is line-local, as if it were the first line in the block, and the caller supplies the final canvas position or path when calling `DrawingCanvas.DrawText(...)`.

The line-local enumerator is the right fit for text that flows through different columns, separate frames, or different paths. See Prepared Text with TextBlock for the Fonts-side coordinate model.

Wrap and Align Text

`RichTextOptions` inherits the core Fonts text options and adds ImageSharp.Drawing-specific rich text behavior.

Wrapping and alignment happen before pixels are drawn. `WrappingLength` determines where line breaking can happen. `TextAlignment` aligns lines within the paragraph. `HorizontalAlignment` and `VerticalAlignment` position the laid-out paragraph relative to `Origin`. Keeping those roles separate avoids the common mistake of manually subtracting measured widths and then fighting wrapped or fallback text.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(640, 260, Color.White.ToPixel<Rgba32>());

Font font = SystemFonts.CreateFont("Arial", 34);
RichTextOptions options = new(font)
{
    Origin = new(48, 42),
    WrappingLength = 544,
    HorizontalAlignment = HorizontalAlignment.Center,
    LineSpacing = 1.15F
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.DrawText(
        options,
        "Wrapped text can be measured and rendered with the same options.",
        Brushes.Solid(Color.MidnightBlue),
        pen: null);
}));
```

Center Text in a Region

Use `WrappingLength`, `HorizontalAlignment`, and `VerticalAlignment` when text should align within a known layout region. For centered alignment, `Origin` is the center anchor for the laid-out text, and `WrappingLength` sets the width used for line breaking.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(520, 220, Color.White.ToPixel<Rgba32>());

Font font = SystemFonts.CreateFont("Arial", 36);
Rectangle layoutBounds = new(40, 56, 440, 108);
PointF layoutCenter = new(
    layoutBounds.Left + (layoutBounds.Width / 2F),
    layoutBounds.Top + (layoutBounds.Height / 2F));

RichTextOptions options = new(font)
{
    Origin = layoutCenter,
    WrappingLength = layoutBounds.Width,
    HorizontalAlignment = HorizontalAlignment.Center,
    VerticalAlignment = VerticalAlignment.Center,
    TextAlignment = TextAlignment.Center
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Draw(Pens.Solid(Color.LightGray, 1), layoutBounds);

    // The origin is the center anchor because both horizontal and vertical alignment
are centered.
    canvas.DrawLine(Pens.Dash(Color.Gray, 1), new PointF(layoutCenter.X, layoutBounds.Top),
new PointF(layoutCenter.X, layoutBounds.Bottom));
    canvas.DrawLine(Pens.Dash(Color.Gray, 1), new PointF(layoutBounds.Left, layoutCenter.Y),
new PointF(layoutBounds.Right, layoutCenter.Y));
    canvas.DrawText(options, "Centered by layout options", Brushes.Solid(Color.Black),
pen: null);
}));
```

Draw Text Along a Path

Text can also follow an `IPath`. In this mode the path acts as the text baseline, so path direction matters: reversing the path reverses the flow direction. Use open paths for natural baselines. Closed shapes can work, but they should be chosen deliberately because the baseline continues around the contour.

Path text is still shaped text. The font, runs, fallback, culture, and decoration options come from the text options; the path only changes where the shaped glyphs are placed.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(640, 260, Color.White.ToPixel<Rgba32>());

Font font = SystemFonts.CreateFont("Arial", 30);
RichTextOptions options = new(font)
{
    Origin = new(0, 0)
};

PathBuilder builder = new();
builder.AddCubicBezier(
    new(52, 168),
    new(186, 42),
    new(420, 44),
    new(588, 172));

IPath path = builder.Build();

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Draw(Pens.Dot(Color.LightGray, 2), path);
    canvas.DrawText(options, "Text can follow path geometry", path,
Brushes.Solid(Color.DarkSlateBlue), pen: null);
}));
```

Use Text as Geometry

Use `TextBuilder.GeneratePaths(...)` when the glyph outlines themselves should become drawing geometry. The returned paths can be filled, stroked, used as clips, or combined with image drawing.

Generating paths changes the problem from text layout to geometry. Once glyph outlines become paths, they can be clipped, filled with image brushes, stroked with pens, transformed, or

combined with other paths. Use this when text is part of a graphic effect or mask. Use `DrawText(...)` when you simply want text rendered as text.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Drawing.Text;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> image = new(640, 240, Color.White.ToPixel<Rgba32>());

RectangleF imageArea = new(0, 0, image.Width, image.Height);
Font font = SystemFonts.CreateFont("Arial", 104, FontStyle.Bold);
TextOptions glyphOptions = new(font)
{
    Origin = new(42, 150)
};

IPathCollection letters = TextBuilder.GeneratePaths("MASK", glyphOptions);
IPath[] glyphClips = [.. letters];
DrawingOptions clipOptions = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.DarkSlateBlue));
    canvas.Save(clipOptions, glyphClips);

    // The generated glyph paths clip the photo to the visible letter shapes.
    canvas.DrawImage(source, source.Bounds, imageArea, KnownResamplers.Bicubic);
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.White, 2), letters);
}));
```

Practical Guidance

Use `RichTextOptions` as the drawing contract for canvas text. If text is measured before it is drawn, the measurement and drawing passes should use the same font, origin model, wrapping length, alignment, culture, fallback, feature tags, and text runs. Otherwise the final pixels can differ from the measured result even when the string is identical.

Prefer the layout options over manual coordinate math. Centering text in a region is a layout problem: set the origin to the region anchor, specify wrapping, and use horizontal, vertical, and text alignment so the layout engine accounts for line height, wrapping, shaping, and fallback metrics. Manual width subtraction is fragile as soon as the string localizes, wraps, or uses a fallback face.

Style ranges and placeholders use grapheme-indexed `[start, end)` ranges. This matters for emoji, combining marks, complex scripts, and any text where one visible unit is not one UTF-16 `char`. Use `TextBlock` when the same shaped text needs to be measured, inspected, hit-tested, or rendered repeatedly. Use generated text paths when text becomes geometry for fills, clips, strokes, or masks.

WebGPU

ImageSharp.Drawing.WebGPU provides GPU-backed drawing targets for the same `DrawingCanvas` API used by the CPU image pipeline.

Use the WebGPU package when the destination is naturally GPU-owned: an interactive window, a native surface owned by another UI toolkit, or an offscreen GPU texture. Use regular ImageSharp.Drawing when the destination is an `Image<TPixel>` that you will process, inspect, encode, or save on the CPU.

What WebGPU Is

WebGPU is a modern, explicit GPU API standardized for portable GPU acceleration. It gives applications access to adapters, devices, queues, textures, buffers, shaders, and presentation surfaces. It is conceptually similar to Vulkan, Metal, and Direct3D 12, but it exposes a portable WebGPU programming model. For the broader standard, implementation status, learning resources, and community material, see webgpu.org.

In ImageSharp.Drawing, WebGPU is a native .NET rendering backend, not a browser-only feature. The `SixLabors.ImageSharp.Drawing.WebGPU` package creates or attaches to native WebGPU targets, records `DrawingCanvas` work, lowers that work into GPU scenes, and renders into WebGPU textures.

The important shift is the destination:

- CPU ImageSharp.Drawing draws into CPU image memory.
- ImageSharp.Drawing.WebGPU draws into GPU textures and presentation surfaces.

That affects application design. A CPU image pipeline is best when you need direct pixels after each step. A WebGPU pipeline is best when output should stay on the GPU, be redrawn repeatedly, or be presented directly to a surface.

Public Type Map

The WebGPU API is organized around ownership:

Type	Owns	Use it when
`WebGPUEnvironment`	Process-level environment configuration	You need to check availability or configure the adapter preference before creating

Type	Owns	Use it when
	and support probes	WebGPU objects
`WebGPUWindow`	A native window, surface, frame loop, and presentation cycle	ImageSharp.Drawing should own the application window
`WebGPUExternalSurface`	A WebGPU surface attached to caller-owned native handles	Another toolkit or host application owns the window or drawable
`WebGPURenderTarget`	An offscreen GPU texture	You need GPU drawing without a visible window, or readback into ImageSharp
`WebGPUSurfaceFrame`	One acquired presentable frame	You are drawing one visible frame and must dispose it to render and present

Start by choosing the output target. You normally do not create WebGPU devices, queues, command encoders, or shader modules yourself.

Installation

Install the WebGPU package alongside ImageSharp.Drawing.

The package restores the managed WebGPU interop and native WebGPU runtime dependencies it needs. Applications still need a machine and driver stack capable of creating a WebGPU adapter, device, queue, and compute pipeline.

Package Manager | .NET CLI | PackageReference | Paket CLI

```
PM > Install-Package SixLabors.ImageSharp.Drawing.WebGPU -Version VERSION_NUMBER
```

How Rendering Works

The WebGPU backend keeps the public drawing model the same. You still draw with `DrawingCanvas`, brushes, pens, paths, text, images, clips, layers, and retained backend scenes.

The replay target changes:

1. Canvas commands are recorded in order.
2. The canvas seals command ranges into the replay timeline.
3. The WebGPU backend prepares retained GPU scene data for those ranges.
4. Rendering creates frame-scoped resources and dispatches GPU work into the target texture.
5. Surface frames are presented when the frame is disposed.

Disposal is part of correctness. A manually-created WebGPU canvas must be disposed to replay its recorded work. A `WebGPUSurfaceFrame` must be disposed to render and present the frame.

Choosing a Target

Use `WebGPUWindow` when ImageSharp.Drawing should own the window and event loop.

Use `WebGPUExternalSurface` when another application, UI framework, game engine, or native toolkit owns the window and exposes native handles.

Use `WebGPURenderTarget` when you want offscreen GPU drawing, render-to-texture workflows, tests, benchmarks, or readback into an `Image<TPixel>`.

Use `WebGPUEnvironment` before any of those when you need predictable startup behavior, diagnostics, or fallback to CPU rendering.

Shared Concepts

Texture format controls the GPU target and, for readback, the matching ImageSharp pixel type:

WebGPU format	Natural ImageSharp pixel type
`Rgba8Unorm`	`Rgba32`
`Bgra8Unorm`	`Bgra32`
`Rgba8Snorm`	`NormalizedByte4`
`Rgba16Float`	`RgbaHalf`

Targets that store associated (premultiplied) alpha map to the `P`-suffixed pixel types instead, such as `Rgba32P` and `RgbaHalfP`. See Pixel Formats for the associated-alpha family.

Present mode controls how visible frames wait for the display:

- `Fifo` is v-synced and is the safest default.
- `Immediate` presents as soon as possible and can tear.

- `Mailbox` keeps newer frames over older queued frames when supported.

CPU/GPU synchronization is the expensive boundary. Reading a render target back into an `Image<TPixel>` copies GPU texture data into CPU memory. Running normal ImageSharp processors through `Apply(...)` on a GPU-backed canvas can require GPU readback, CPU processing, and upload before drawing continues.

Layer effects are the GPU-friendly alternative for common treatments. The WebGPU package ships shader-backed versions of the built-in `LayerEffect` and `BackdropLayerEffect` set, such as `WebGPUGaussianBlurLayerEffect` and `WebGPUBackdropAcrylicLayerEffect`. These execute as shader passes, so a `SaveLayer(...)` effect stays on the GPU. An equivalent `Apply(...)` processor round-trips through CPU memory instead. Each carries its CPU equivalent as a fallback, so the same instance renders correctly on a CPU canvas. The effect model, the shader authoring contract, and a complete dual-backend example are on the dedicated page. See Layer and Backdrop Effects.

When Not to Use WebGPU

WebGPU is not automatically better for every drawing workload. Prefer the normal CPU path when:

- you are generating static images on a server
- you need direct CPU pixel access after most operations
- you immediately encode the result to PNG, JPEG, WebP, or another image format
- the deployment environment has unreliable GPU or native WebGPU support
- the drawing workload is small enough that GPU setup or readback costs dominate

Prefer WebGPU when:

- the target is already a GPU surface
- the scene is interactive or redrawn repeatedly
- the output can stay on the GPU
- you need a native window or host surface
- the drawing workload benefits from GPU-side batching and rasterization

Related Topics

- WebGPU Environment and Support
- WebGPU Window Rendering
- WebGPU External Surfaces
- WebGPU Offscreen Render Targets

- Canvas Drawing
- Transforms and Composition

Practical Guidance

Choose the target first, because the target owns the lifetime rules. Probe support before constructing production WebGPU targets. Dispose canvases and frames promptly so recorded drawing is submitted. Keep source images, image brushes, fonts, paths, and retained backend scenes alive until every canvas or frame that references them has been disposed.

WebGPU Environment and Support

`WebGPUEnvironment` configures and probes the library-managed WebGPU environment. Use it before constructing windows, external surfaces, or render targets when startup must be predictable.

The environment represents the process-level WebGPU runtime used by the public target types. It is responsible for acquiring the adapter, device, and queue used by ImageSharp.Drawing.WebGPU. Because the environment initializes on first use, set options and error callbacks before creating any WebGPU object.

Configure Before First Use

`WebGPUEnvironment.Options` is read during first initialization. Changing it later does not reconfigure an existing device.

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;

WebGPUEnvironment.Options = new()
{
    PowerPreference = WebGPUPowerPreference.HighPerformance
};
```

`HighPerformance` is the usual choice for drawing workloads. If an application should prefer a lower-power adapter, configure that once during startup before probing or creating targets.

Probe Availability

Call `ProbeAvailability()` to check whether the library can initialize the WebGPU API, create an instance, acquire an adapter, acquire a device, and get the default queue.

Call `ProbeComputePipelineSupport()` when the drawing backend must prove it can create compute pipelines. This is a stronger check than basic device acquisition.

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;

static bool TryUseWebGPU()
{
    WebGPUEnvironmentError availability = WebGPUEnvironment.ProbeAvailability();
    if (availability != WebGPUEnvironmentError.Success)
```

```
    {
        Console.WriteLine($"WebGPU unavailable: {availability}");
        return false;
    }

    WebGPUEnvironmentError compute = WebGPUEnvironment.ProbeComputePipelineSupport();
    if (compute != WebGPUEnvironmentError.Success)
    {
        Console.WriteLine($"WebGPU compute unavailable: {compute}");
        return false;
    }

    return true;
}
```

`Success` is the only successful result. Other values are stable failure categories such as API initialization failure, adapter timeout, device request failure, queue acquisition failure, or compute-pipeline probe failure. Branch on the enum value rather than parsing diagnostic strings.

Log Native WebGPU Errors

Configure `UncapturedError` to receive native WebGPU validation, device, or internal errors reported outside a specific managed call.

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;

WebGPUEnvironment.UncapturedError = (errorType, message) =>
{
    Console.Error.WriteLine($"{errorType}: {message}");
};
```

The callback can be raised from a native WebGPU callback thread. Keep handlers short and non-blocking. Use it for logging, not for complex UI updates or recovery work.

Fallback Strategy

Applications that can render on either CPU or GPU should decide early:

```
bool useGpu = TryUseWebGPU();

if (useGpu)
{
```

```
    // Construct WebGPUWindow, WebGPUExternalSurface, or WebGPURenderTarget.
}
else
{
    // Fall back to Image<TPixel> and the normal ImageSharp.Drawing path.
}
```

Do not create a WebGPU target first and then probe after failure. Probing first gives better diagnostics and avoids partially initialized rendering paths.

Related Topics

- WebGPU
- WebGPU Window Rendering
- WebGPU External Surfaces
- WebGPU Offscreen Render Targets

Practical Guidance

Configure `WebGPUEnvironment.Options` and `UncapturedError` during application startup. Use `ProbeAvailability()` for basic device readiness and `ProbeComputePipelineSupport()` when WebGPU drawing is required. Treat a non-success result as a normal deployment condition and provide a CPU fallback when the application can still produce useful output.

WebGPU Window Rendering

`WebGPUWindow` is the highest-level WebGPU target. It owns the native window, WebGPU surface, device resources, frame acquisition, and presentation cycle.

Use it when ImageSharp.Drawing should own the application window. If another UI framework, game engine, or host application owns the window, use `WebGPUExternalSurface` instead.

Window Ownership

A `WebGPUWindow` wraps a real native window and exposes a `DrawingCanvas` for each acquired frame. You draw into that frame canvas, and the frame is presented when the frame scope is disposed.

The window owns:

- the platform window
- the WebGPU presentation surface
- the frame acquisition loop
- resize-driven surface reconfiguration
- the drawing backend bound to the window target

That ownership makes it a good fit for demos, tools, visualizers, preview windows, and applications where ImageSharp.Drawing is the main renderer.

Create a Window

`WebGPUWindowOptions` controls the initial title, size, position, visibility, scheduling hints, state, border, present mode, and texture format.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Drawing.Processing.Backends;

WebGPUWindowOptions options = new()
{
    Title = "ImageSharp.Drawing WebGPU",
    Size = new(960, 540),
    PresentMode = WebGPUPresentMode.Fifo,
    Format = WebGPUTextureFormat.Rgba8Unorm
};
```

```
using WebGPUWindow window = new(options);
```

`Size` is the initial client-area size in window coordinates. `FramebufferSize` is the pixel size of the drawable WebGPU surface. On high-DPI displays those can differ; use `RenderScale`, `PointToFramebuffer(...)`, or framebuffer resize events when mapping input to pixels.

Render With Run

The simplest model is `Run(...)`. The window owns the loop, acquires one frame per render callback, and disposes the frame after your callback returns.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Drawing.Processing.Backends;

using WebGPUWindow window = new(new WebGPUWindowOptions { Title = "WebGPU Demo" });

window.Run((WebGPUSurfaceFrame frame) =>
{
    DrawingCanvas canvas = frame.Canvas;
    Rectangle panel = new(64, 72, 320, 180);

    // The frame is presented after Run disposes it at the end of the callback.
    canvas.Clear(Brushes.Solid(Color.White));
    canvas.Fill(Brushes.Solid(Color.CornflowerBlue), panel);
    canvas.FillEllipse(Brushes.Solid(Color.Gold), new(224, 162), new(120, 82));
    canvas.Draw(Pens.Solid(Color.Black, 3), panel);
});
```

Use the elapsed-time overload when animation needs frame timing:

```
window.Run((frame, elapsed) =>
{
    DrawingCanvas canvas = frame.Canvas;
    float radius = 40 + (MathF.Sin((float)elapsed.TotalSeconds) * 12);

    // The radius is converted to an ellipse size because FillEllipse takes width
and height.
    canvas.Clear(Brushes.Solid(Color.White));
    canvas.FillEllipse(Brushes.Solid(Color.MediumSeaGreen), new(120, 120), new(radius * 2,
```

```
radius * 2));
});
```

Drive the Loop Manually

Use `TryAcquireFrame(...)` when the application owns the loop.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Drawing.Processing.Backends;

using WebGPUWindow window = new();

while (!window.IsClosing)
{
    window.DoEvents();

    if (!window.TryAcquireFrame(out WebGPUSurfaceFrame? frame))
    {
        continue;
    }

    using (frame)
    {
        DrawingCanvas canvas = frame.Canvas;
        canvas.Clear(Brushes.Solid(Color.White));
        canvas.Fill(Brushes.Solid(Color.CornflowerBlue), new Rectangle(40, 40, 180, 120));
    }
}
```

`TryAcquireFrame(...)` can return `false` when no drawable frame is available right now. That can happen for transient surface states such as timeout, outdated surface, lost surface, zero-sized framebuffer, or device recovery. Treat `false` as "skip this render attempt and try again later."

Window Events and State

`WebGPUWindow` exposes events for update, resize, framebuffer resize, closing, focus, move, state change, and file drop. Use `FramebufferResized` when WebGPU pixel dimensions matter. Use `Resized` when UI layout in client coordinates matters.

Useful properties include:

- `Title`, `ClientSize`, `FramebufferSize`, and `RenderScale`
- `Position`, `IsVisible`, `WindowState`, and `WindowBorder`
- `FramesPerSecond`, `UpdatesPerSecond`, and `IsEventDriven`
- `PresentMode` and `Format`
- `PointToClient(...)`, `PointToScreen(...)`, and `PointToFramebuffer(...)`

Changing `PresentMode` reconfigures the surface. Changing `Format` is an initial creation choice; choose the target format in `WebGPUWindowOptions`.

Frame Lifetime

A frame owns one acquired presentable texture. Drawing commands are recorded through `frame.Canvas`. Disposing the frame disposes that canvas, replays the drawing timeline, submits GPU work, presents the surface texture, and releases per-frame resources.

If `Run(...)` owns the loop, it disposes the frame for you. If you call `TryAcquireFrame(...)`, dispose every frame you acquire.

Related Topics

- WebGPU
- WebGPU Environment and Support
- WebGPU External Surfaces
- WebGPU Offscreen Render Targets

Practical Guidance

Use `WebGPUWindow` when ImageSharp.Drawing owns the render loop. Start with `Run(...)`; move to `TryAcquireFrame(...)` only when you need explicit event, update, and render scheduling. Keep frame drawing short, dispose frames promptly, and use framebuffer coordinates when mapping input or layout to actual GPU pixels.

WebGPU External Surfaces

`WebGPUExternalSurface` attaches ImageSharp.Drawing.WebGPU to a native drawable owned by another application or UI toolkit.

Use it when you already have a window, view, swapchain host, or platform drawable and ImageSharp.Drawing should render into that existing surface. Unlike `WebGPUWindow`, this type does not own the native window or event loop.

Ownership Model

The host application owns:

- the native window or drawable
- the native handles passed to `WebGPUSurfaceHost`
- resize notifications
- event processing
- render scheduling
- the lifetime of the underlying UI object

`WebGPUExternalSurface` owns the WebGPU surface resources attached to those handles. It can acquire frames, create frame canvases, and present rendered output, but it never releases the native handles you supplied.

Create a Surface Host

Create a `WebGPUSurfaceHost` with the factory method matching the host platform or toolkit:

- `Glfw(...)`
- `Sdl(...)`
- `Win32(...)`
- `X11(...)`
- `Cocoa(...)`
- `UIKit(...)`
- `Wayland(...)`
- `WinRT(...)`
- `Android(...)`
- `Vivante(...)`
- `EGL(...)`

For Win32:

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;

WebGPUSurfaceHost host = WebGPUSurfaceHost.Win32(hwnd, hinstance);
```

Pass valid native handles for the lifetime of the external surface. The exact handles depend on the platform. For example, X11 needs a display pointer and window id, Wayland needs display and surface pointers, and UIKit needs the window plus framebuffer objects.

Create the External Surface

The framebuffer size is the drawable size in pixels, not necessarily the logical UI size.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing.Backends;

WebGPUExternalSurfaceOptions options = new()
{
    PresentMode = WebGPUPresentMode.Fifo,
    Format = WebGPUTextureFormat.Rgba8Unorm,
    AlphaMode = WebGPUCompositeAlphaMode.Auto
};

using WebGPUExternalSurface surface = new(host, new Size(1280, 720), options);
```

Use a custom `Configuration` overload when the drawing backend should be bound to a specific ImageSharp configuration.

`AlphaMode` controls how the native compositor interprets alpha in the presented surface. `Auto` lets WebGPU select the composition mode. `Opaque` ignores the alpha channel and presents every pixel as opaque. `Premultiplied` presents color components that have already been multiplied by alpha, which allows the surface to blend over other native content.

Handle Resizes

The host application must notify the external surface when the drawable framebuffer changes size:

```
void OnFramebufferResized(int width, int height)
{
```

```
    surface.Resize(new Size(width, height));
}
```

Zero-sized framebuffers are ignored. This matters for minimized windows, hidden views, and platform states where the drawable temporarily has no size.

Render a Frame

Call `TryAcquireFrame(...)` from the host render loop. Dispose each acquired frame to submit and present.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Drawing.Processing.Backends;

void Render()
{
    if (!surface.TryAcquireFrame(out WebGPUSurfaceFrame? frame))
    {
        return;
    }

    using (frame)
    {
        DrawingCanvas canvas = frame.Canvas;

        // The host owns when Render is called; the frame owns presentation.
        canvas.Clear(Brushes.Solid(Color.White));
        canvas.Fill(Brushes.Solid(Color.Orange), new Rectangle(48, 48, 320, 160));
    }
}
```

Use the overload that accepts `DrawingOptions` when the whole frame should start with a specific transform, graphics options, or shape options.

External Surface Failure Modes

`TryAcquireFrame(...)` can return `false` when a drawable frame is not available. Common causes include a zero-sized framebuffer, an outdated or lost surface, timeout, or device recovery.

The correct response is usually to skip that render attempt, keep processing host events, and try again on the next render tick. Recreate the external surface only when the host application has

replaced the native drawable or invalidated the handles.

Related Topics

- WebGPU
- WebGPU Environment and Support
- WebGPU Window Rendering
- WebGPU Offscreen Render Targets

Practical Guidance

Use `WebGPUExternalSurface` when ImageSharp.Drawing is a renderer inside somebody else's windowing model. Keep native handles valid, forward framebuffer resize events, acquire at most one frame per render, dispose every acquired frame, and treat a failed frame acquisition as a normal transient condition.

WebGPU Offscreen Render Targets

`WebGPURenderTarget` owns an offscreen WebGPU texture. Use it when you want GPU drawing without a visible window, or when a GPU-rendered result must later be read back into an ImageSharp image.

Offscreen render targets are useful for render-to-texture workflows, GPU-generated assets, tests, benchmarks, previews, and pipelines that draw on the GPU before handing the final result back to CPU code.

Create a Render Target

The simplest constructor uses the default `Rgba8Unorm` format:

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;

using WebGPURenderTarget target = new(640, 360);
```

Specify a format when the target must match another GPU workflow or readback pixel type:

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;

using WebGPURenderTarget target = new(WebGPUTextureFormat.Bgra8Unorm, 640, 360);
```

Pass a `PixelAlphaRepresentation` when the target must store associated (premultiplied) alpha, which is the working representation of most GPU pipelines:

```
using SixLabors.ImageSharp.Drawing.Processing.Backends;
using SixLabors.ImageSharp.PixelFormats;

using WebGPURenderTarget target = new(
    WebGPUTextureFormat.Rgba8Unorm,
    PixelAlphaRepresentation.Associated,
    640,
    360);
```

The target exposes `Width`, `Height`, `Bounds`, `Format`, and `AlphaRepresentation`. The bounds are always rooted at `(0, 0)` in target pixel coordinates.

Draw Offscreen

Create a canvas, draw into it, and dispose the canvas to replay and submit the recorded work to the offscreen texture.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Drawing.Processing.Backends;

using WebGPURenderTarget target = new(640, 360);

using (DrawingCanvas canvas = target.CreateCanvas())
{
    // Canvas disposal replays the recorded drawing work into the GPU texture.
    canvas.Fill(Brushes.Solid(Color.White));
    canvas.FillEllipse(Brushes.Solid(Color.LightSkyBlue), new(320, 180), new(260, 140));
    canvas.DrawEllipse(Pens.Solid(Color.DarkSlateBlue, 4), new(320, 180), new(260, 140));
}
```

Use `CreateCanvas(DrawingOptions)` when the whole render pass should start with non-default drawing state.

Read Back to ImageSharp

`ReadbackImage()` creates a new CPU image from the current GPU texture contents.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing.Backends;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> image = target.ReadbackImage<Rgba32>();
image.Save("webgpu-output.png");
```

The typed readback pixel type must match the target format:

Target format	Typed readback
`Rgba8Unorm`	`ReadbackImage<Rgba32>()`
`Bgra8Unorm`	`ReadbackImage<Bgra32>()`

Target format	Typed readback
`Rgba8Snorm`	`ReadbackImage<NormalizedByte4>()`
`Rgba16Float`	`ReadbackImage<RgbaHalf>()`

The non-generic `ReadbackImage()` chooses the natural ImageSharp pixel type from the render target format and alpha representation. A target created with `PixelAlphaRepresentation.Associated` reads back into the matching associated-alpha pixel type instead: `Rgba32P`, `Bgra32P`, `NormalizedByte4P`, or `RgbaHalfP`.

Read Back Into an Existing Buffer

Use `ReadbackInto(...)` when you already own the destination pixel buffer:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing.Backends;
using SixLabors.ImageSharp.PixelFormats;

using Image<Rgba32> destination = new(target.Width, target.Height);
target.ReadbackInto(destination.Frames.RootFrame.PixelBuffer.GetRegion());
```

If the destination region is smaller than the render target, the matching top-left portion is read back. This lets callers read into bounded regions without forcing an intermediate full-size image.

Readback Cost

Readback is a synchronization point. The GPU work must be visible to the CPU, and the texture data must be copied into CPU memory. That cost is fine for final export, tests, snapshots, or occasional thumbnails. It is usually the wrong thing to do every frame in an interactive GPU render loop.

If the next stage is also GPU-owned, keep the result in a WebGPU render target or surface instead of reading it back to ImageSharp immediately.

Related Topics

- WebGPU
- WebGPU Environment and Support
- WebGPU Window Rendering

- WebGPU External Surfaces

Practical Guidance

Use `WebGPURenderTarget` when the output should be GPU-rendered but not immediately presented to a window. Dispose the canvas before readback. Pick the texture format from the final consumer, and avoid readback in tight frame loops unless CPU pixels are genuinely required.

Migrating from System.Drawing

If you are coming from `System.Drawing`, the biggest adjustment is moving from a `Graphics` object over a `Bitmap` to an ImageSharp image pipeline with `Paint(...)` and `DrawingCanvas`.

The drawing concepts still map cleanly. `Graphics` becomes `DrawingCanvas`, `Brush` and `Pen` become ImageSharp.Drawing brushes and pens, `GraphicsPath` becomes `PathBuilder` or `IPath`, and text moves to the Fonts-powered `DrawText(...)` APIs.

Treat migration as a behavior-matching exercise first. Keep the same image size, geometry, colors, alpha, transform order, clipping behavior, and font choice while translating the API shape. Once the output is equivalent, simplify the ImageSharp.Drawing code to use higher-level shapes, text layout, and image processing where they make the intent clearer.

For core image loading, saving, pixel formats, and raw pixel access, see the ImageSharp Migrating from System.Drawing guide. This page focuses on drawing code.

Core Type Mapping

`System.Drawing` concept	ImageSharp.Drawing equivalent
`Bitmap`	`Image<TPixel>`
`Graphics`	`DrawingCanvas` inside `Paint(...)`, or a canvas created from an image frame
`System.Drawing.Color`	`SixLabors.ImageSharp.Color`, or a concrete pixel type such as `Rgba32`
`SolidBrush` / `TextureBrush`	`Brushes.Solid(...)`, image brushes, pattern brushes, gradient brushes
`Pen`	`SixLabors.ImageSharp.Drawing.Processing.Pen`, usually through `Pens.Solid(...)`
`Rectangle` / `RectangleF`	`Rectangle` for rectangle fill, stroke, and clear helpers; `RectangleF` for APIs that explicitly accept floating-point bounds such as image destination rectangles
`GraphicsPath`	`PathBuilder`, `Path`, `IPath`, and built-in shape types when geometry must be reused

`System.Drawing` concept	ImageSharp.Drawing equivalent
`Matrix`	`Matrix4x4`, commonly constructed from `Matrix3x2`
`Graphics.DrawImage(...)`	`DrawingCanvas.DrawImage(...)`
`Graphics.DrawString(...)`	`DrawingCanvas.DrawText(...)` with `RichTextOptions`

Graphics vs Paint Pipelines

In `System.Drawing`, drawing usually starts by creating a `Graphics` object from a `Bitmap`:

System.Drawing:

```
using System.Drawing;

using Bitmap bitmap = new(420, 240);
using Graphics graphics = Graphics.FromImage(bitmap);
using SolidBrush brush = new(Color.CornflowerBlue);

graphics.Clear(Color.White);
graphics.FillRectangle(brush, new Rectangle(40, 40, 260, 110));
```

In ImageSharp.Drawing, draw inside an ImageSharp mutation pipeline:

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

image.Mutate(context => context.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.CornflowerBlue), new Rectangle(40, 40, 260, 110));
}));
```

Use `Mutate(...)` when you want to update an image in place. Use `Clone(...)` when the old code created a separate output bitmap while keeping the source unchanged. The `Paint(...)` processor

owns the canvas lifetime for this common case: commands recorded inside the callback are replayed into the image at the correct point in the ImageSharp processing pipeline.

Brushes and Pens

`System.Drawing` separates filled shapes and stroked outlines through `Brush` and `Pen`. ImageSharp.Drawing keeps the same drawing vocabulary, but the objects belong to the ImageSharp.Drawing pipeline rather than the GDI+ object model.

A brush supplies color, gradient, pattern, or image samples for covered pixels. A pen describes how to turn a source line, path, or shape into stroke geometry: width, dash pattern, joins, caps, and miter behavior all affect that generated outline. The generated outline is then filled by the pen brush. That distinction matters when migrating dashed strokes, image-filled outlines, or paths where cap and join behavior changes the visible shape.

System.Drawing:

```
using System.Drawing;

using SolidBrush fill = new(Color.FromArgb(255, 47, 128, 237));
using Pen stroke = new(Color.FromArgb(255, 27, 63, 114), 4);

graphics.FillRectangle(fill, new RectangleF(48, 42, 280, 126));
graphics.DrawRectangle(stroke, 48, 42, 280, 126);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.FromPixel(new Rgba32(47, 128, 237, 255))), new
Rectangle(48, 42, 280, 126));
    canvas.Draw(Pens.Solid(Color.FromPixel(new Rgba32(27, 63, 114, 255)), 4), new
Rectangle(48, 42, 280, 126));
}));
```

Paths and Shapes

`GraphicsPath` maps to `PathBuilder` when you are constructing custom geometry. Build the path in the same coordinate space as the original `GraphicsPath`, then fill or stroke it with ImageSharp.Drawing brushes and pens.

Keep open and closed figures deliberate. A closed figure represents an area boundary, so fill rules, joins, and holes are part of the shape contract. An open figure is usually a stroke path, where caps and joins define the visible ends and corners. For direct migrations of simple rectangles, ellipses, arcs, pies, lines, and Beziers, prefer the canvas helpers. Rectangles use `Fill(brush, Rectangle)` and `Draw(pen, Rectangle)` overloads; ellipses, arcs, pies, lines, and Beziers have named helpers. Use shape objects such as `EllipsePolygon`, `RectanglePolygon`, or custom paths when the geometry is reused for fill, stroke, clipping, measurement, or composition.

System.Drawing:

```
using System.Drawing;
using System.Drawing.Drawing2D;

using GraphicsPath triangle = new();
triangle.StartFigure();
triangle.AddLine(80, 180, 160, 48);
triangle.AddLine(160, 48, 240, 180);
triangle.CloseFigure();

using SolidBrush fill = new(Color.Gold);
using Pen stroke = new(Color.DarkGoldenrod, 4);

graphics.FillPath(fill, triangle);
graphics.DrawPath(stroke, triangle);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    PathBuilder builder = new();
    builder.MoveTo(new PointF(80, 180));
    builder.LineTo(new PointF(160, 48));
    builder.LineTo(new PointF(240, 180));
    builder.CloseFigure();
```

```
    IPath triangle = builder.Build();

    canvas.Fill(Brushes.Solid(Color.Gold), triangle);
    canvas.Draw(Pens.Solid(Color.DarkGoldenrod, 4), triangle);
}));
```

For common one-off geometry, use the canvas helpers that match the `Graphics` method you are replacing:

System.Drawing:

```
using System.Drawing;

using SolidBrush fill = new(Color.MediumSeaGreen);

graphics.FillEllipse(fill, new RectangleF(70, 72, 220, 96));
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    // ImageSharp.Drawing ellipse helpers take center and size, not top-left bounds.
    canvas.FillEllipse(Brushes.Solid(Color.MediumSeaGreen), new(180, 120), new(220, 96));
}));
```

Use an explicit polygon when the ellipse is part of the drawing model rather than a one-off command. For example, clipping needs an `IPath`, so `new EllipsePolygon(...)` is the right shape for the clipping example below.

Transforms and Canvas State

`System.Drawing.Graphics` stores transform state on the `Graphics` object. ImageSharp.Drawing stores transform state in `DrawingOptions`, which can be saved onto the canvas state stack.

Translate transform code by preserving operation order. The transformed coordinate system affects subsequent fills, strokes, text, clips, and image placement until the saved state is restored. ImageSharp.Drawing uses `Matrix4x4` for canvas state so the same model can represent 2D affine

and projective transforms across CPU and GPU backends; for normal migration work, build the value from `Matrix3x2` so the six affine numbers stay familiar.

System.Drawing:

```
using System.Drawing;
using System.Drawing.Drawing2D;

using SolidBrush fill = new(Color.HotPink);
using Pen stroke = new(Color.White, 3);
using Matrix transform = new(1.2F, 0, 0, 0.8F, 210, 120);

GraphicsState state = graphics.Save();
graphics.Transform = transform;
graphics.FillRectangle(fill, new RectangleF(-70, -24, 140, 48));
graphics.DrawRectangle(stroke, -70, -24, 140, 48);
graphics.Restore(state);
```

ImageSharp.Drawing:

```
using System.Numerics;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    DrawingOptions options = new()
    {
        Transform = new(
            Matrix3x2.CreateScale(1.2F, 0.8F) *
            Matrix3x2.CreateTranslation(210, 120))
    };

    _ = canvas.Save(options);
    canvas.Fill(Brushes.Solid(Color.HotPink), new Rectangle(-70, -24, 140, 48));
    canvas.Draw(Pens.Solid(Color.White, 3), new Rectangle(-70, -24, 140, 48));

    canvas.Restore();
}));
```

ImageSharp.Drawing uses `Matrix4x4` for canvas transforms so the same drawing state can represent normal 2D affine transforms and projective transforms. For normal 2D drawing, construct it from `Matrix3x2`.

Image Composition

If your `System.Drawing` code uses `Graphics.DrawImage(...)`, use `DrawImage(...)` inside `Paint(...)` when the image placement belongs with the rest of the drawing commands.

Keep source and destination rectangles explicit. The source rectangle selects pixels from the input image; the destination rectangle defines where those pixels land on the canvas. If you do not pass a resampler, ImageSharp.Drawing uses the drawing API default, which is the right choice for ordinary image placement. Choose a specific resampler only when the migration requires a known sampling policy.

System.Drawing:

```
using System.Drawing;

using Bitmap source = new("photo.jpg");
using Bitmap output = new(640, 360);
using Graphics graphics = Graphics.FromImage(output);

using Pen stroke = new(Color.White, 4);

graphics.Clear(Color.White);
graphics.DrawImage(source, new RectangleF(32, 32, 320, 220));
graphics.DrawRectangle(stroke, 32, 32, 320, 220);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> output = new(640, 360, Color.White.ToPixel<Rgba32>());

output.Mutate(context => context.Paint(canvas =>
{
    canvas.DrawImage(source, source.Bounds, new RectangleF(32, 32, 320, 220));
    canvas.Draw(Pens.Solid(Color.White, 4), new Rectangle(32, 32, 320, 220));
}));
```

Keep source images alive until the canvas has replayed. Inside `Paint(...)`, replay is owned by the processing operation. If you create and manage a canvas yourself, dispose it before disposing

source images used by drawing commands.

Clipping

`Graphics.SetClip(...)` maps to saving canvas state with clip paths. Restore the state when the clipped drawing is complete.

For equivalent `SetClip(...)` behavior, use `BooleanOperation.Intersection`. ImageSharp.Drawing clip paths are combined through `ShapeOptions.BooleanOperation`, and the default operation is not the same as intersecting the current drawing area with the supplied clip.

System.Drawing:

```
using System.Drawing;
using System.Drawing.Drawing2D;

using GraphicsPath clip = new();
clip.AddEllipse(60, 40, 260, 160);

GraphicsState state = graphics.Save();
graphics.SetClip(clip);
graphics.FillRectangle(Brushes.CornflowerBlue, new Rectangle(20, 20, 360, 200));
graphics.Restore(state);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    DrawingOptions clipInside = new()
    {
        ShapeOptions = new()
        {
            BooleanOperation = BooleanOperation.Intersection
        }
    };

    // SetClip-style behavior keeps only the intersection with the ellipse.
    _ = canvas.Save(clipInside, new EllipsePolygon(190, 120, 260, 160));
```

```
    canvas.Fill(Brushes.Solid(Color.CornflowerBlue), new Rectangle(20, 20, 360, 200));

    canvas.Restore();
}));
```

Text

`Graphics.DrawString(...)` handles simple text drawing. ImageSharp.Drawing uses SixLabors.Fonts through `DrawText(...)`, so wrapping, alignment, shaping, fallback, and rich text options are part of the normal text pipeline.

Use the same font file and layout rectangle when checking output parity. `RichTextOptions.Origin` is the anchor used by the layout options, `WrappingLength` defines the available line width, `TextAlignment` aligns lines within that wrapping width, and `HorizontalAlignment` / `VerticalAlignment` place the laid-out block relative to the origin. This keeps text positioning declarative instead of relying on manual string measurement.

System.Drawing:

```
using System.Drawing;
using System.Drawing.Text;

using PrivateFontCollection collection = new();
collection.AddFontFile("Inter.ttf");

using Font font = new(collection.Families[0], 32);
using StringFormat format = new()
{
    Alignment = StringAlignment.Center
};

graphics.DrawString(
    "Fast text layout for generated graphics",
    font,
    Brushes.Black,
    new RectangleF(48, 48, 320, 120),
    format);
```

ImageSharp.Drawing:

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    FontCollection collection = new();
    FontFamily family = collection.Add("Inter.ttf");
    Font font = family.CreateFont(32);

    RichTextOptions options = new(font)
    {
        Origin = new PointF(48, 48),
        WrappingLength = 320,
        HorizontalAlignment = HorizontalAlignment.Center
    };

    canvas.DrawText(options, "Fast text layout for generated graphics",
Brushes.Solid(Color.Black), pen: null);
}));
```

Practical Migration Strategy

For most `System.Drawing` drawing migrations:

1. Move bitmap load/save work to ImageSharp.
2. Replace `Graphics.FromImage(...)` blocks with `image.Mutate(context => context.Paint(canvas => ...))`.
3. Replace `SolidBrush`, `TextureBrush`, and gradient brushes with ImageSharp.Drawing brushes.
4. Replace `System.Drawing.Pen` with `Pens.Solid(...)` or a custom ImageSharp.Drawing pen.
5. Replace `GraphicsPath` with `PathBuilder`, `Path`, or built-in shape types.
6. Replace `Graphics` transform state with saved canvas state and `Matrix4x4` values constructed from `Matrix3x2`.
7. Replace `SetClip(...)` with `Save(options, clipPaths)` and `Restore()`.
8. Replace `DrawString(...)` with `DrawText(...)`, `RichTextOptions`, and the Fonts layout APIs when wrapping or shaping matters.

You do not have to migrate all drawing code at once. Start with one rendering workflow, match the output, then simplify the code once the ImageSharp.Drawing model is in place.

Practical Guidance

- Keep source and destination geometry equivalent while translating examples.
- Replace `Graphics` state with explicit canvas `Save(...)` and `Restore()` scopes.
- Use ImageSharp.Drawing brushes and pens instead of carrying `System.Drawing` object lifetimes across.
- Move text layout decisions into `RichTextOptions` and Fonts APIs rather than manually positioning strings.
- Validate output on non-Windows environments if the migration goal is cross-platform rendering.

Migrating from SkiaSharp

If you are coming from SkiaSharp, the biggest adjustment is the rendering model. SkiaSharp code is usually centered on an `SKCanvas` supplied by the destination you are drawing to: a bitmap, raster surface, GPU surface, document, or picture recorder. ImageSharp.Drawing works inside the ImageSharp processing pipeline and records ordered drawing work through `DrawingCanvas` before replaying it to the active backend.

That difference is useful. The same drawing code can target normal CPU-backed images, retained backend scenes, and WebGPU-backed surfaces while keeping the same shape, brush, pen, text, and image composition model.

Start by matching behavior, not by chasing the shortest code. Keep the same canvas size, rectangles, colors, alpha, stroke widths, transform order, clipping, font files, and image sampling choices while translating the drawing model. Once the output matches, ImageSharp.Drawing usually lets you simplify because geometry, styling, text layout, and image processing are expressed as separate concepts.

Core Type Mapping

SkiaSharp concept	ImageSharp.Drawing equivalent
`SKBitmap` / `SKImage` / `SKSurface`	`Image<TPixel>` for CPU images, or a WebGPU surface/render target for GPU output
`SKCanvas`	`DrawingCanvas` inside `Paint(...)`, or a canvas created from an image frame or backend
`SKPaint` fill	`Brush`, usually `Brushes.Solid(...)`, gradient brushes, image brushes, or pattern brushes
`SKPaint` stroke	`Pen`, usually `Pens.Solid(...)` or a custom `Pen` with stroke options
`SKColor`	`Color`, or a concrete pixel type such as `Rgba32` when working directly with pixels
`SKRect` / `SKRoundRect`	`Rectangle` for rectangle fill, stroke, and clear helpers; `RectangleF` for APIs that explicitly accept floating-point bounds such as image destination rectangles; shape types when geometry must be reused

SkiaSharp concept	ImageSharp.Drawing equivalent
`SKPath`	`PathBuilder`, `Path`, `IPath`, and built-in shapes when geometry must be reused
`SKMatrix`	`Matrix4x4` transforms, commonly constructed from `Matrix3x2`
`SKImageFilter` / `SKMaskFilter`	`Apply(...)` with ImageSharp processors for region-scoped effects
`SKTextBlob` / text drawing	`RichTextOptions`, `TextBlock`, Fonts shaping, and `DrawText(...)`

Drawing Targets and Paint Pipelines

In SkiaSharp, you draw through the `SKCanvas` provided by the current destination. A canvas backed by a raster bitmap or raster surface writes to pixels visible to the CPU. A GPU-surface canvas targets GPU work that is flushed or submitted later. A document or picture-recorder canvas records drawing commands instead of exposing writable pixels.

ImageSharp.Drawing gives you one ordered canvas API for these destination styles. Inside `Paint(...)`, the canvas records drawing work at that point in the ImageSharp pipeline and replays it into the active backend when the processor completes.

For simple bitmap code, that often looks like this:

SkiaSharp:

```
using SkiaSharp;

using SKBitmap bitmap = new(420, 240);
using SKCanvas canvas = new(bitmap);
using SKPaint paint = new()
{
    Color = SKColors.CornflowerBlue,
    IsAntialias = true
};

canvas.Clear(SKColors.White);
canvas.DrawRect(SKRect.Create(40, 40, 260, 110), paint);
```

In ImageSharp.Drawing, draw inside an ImageSharp mutation pipeline:

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

image.Mutate(context => context.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.CornflowerBlue), new Rectangle(40, 40, 260, 110));
}));
```

Use `Image.Mutate(...)` when you want to modify an existing image. Use `Image.Clone(...)` when your old SkiaSharp code created a new output from an existing source while leaving the source unchanged.

Paint Becomes Brush and Pen

SkiaSharp uses `SKPaint` as a general drawing state object. The same type can represent fill, stroke, antialiasing, shaders, blend modes, filters, text settings, and more.

ImageSharp.Drawing splits those concepts into smaller objects:

- `Brush` describes how an area is filled.
- `Pen` describes how outlines are stroked.
- `DrawingOptions` controls antialiasing, transforms, blending, and shape behavior.
- `RichTextOptions` controls text layout and shaping.

SkiaSharp:

```
using SkiaSharp;

using SKPaint fill = new()
{
    Color = SKColor.Parse("#2f80ed"),
    IsAntialias = true,
    Style = SKPaintStyle.Fill
};
```

```
using SKPaint stroke = new()
{
    Color = SKColor.Parse("#1b3f72"),
    IsAntialias = true,
    StrokeWidth = 4,
    Style = SKPaintStyle.Stroke
};

canvas.DrawRect(SKRect.Create(48, 42, 280, 126), fill);
canvas.DrawRect(SKRect.Create(48, 42, 280, 126), stroke);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.ParseHex("#2f80ed")), new Rectangle(48, 42, 280, 126));
    canvas.Draw(Pens.Solid(Color.ParseHex("#1b3f72"), 4), new Rectangle(48, 42, 280, 126));
}));
```

This is usually the cleanest migration path: create brushes and pens where SkiaSharp code previously configured fill and stroke paints. Avoid looking for a single `SKPaint` replacement. In ImageSharp.Drawing, fill style belongs to the brush, stroke geometry belongs to the pen, graphics state belongs to `DrawingOptions`, and text layout belongs to `RichTextOptions`.

Paths and Shapes

SkiaSharp path code usually builds an `SKPath`, then fills or strokes it. ImageSharp.Drawing uses `PathBuilder` for incremental construction and `IPath` for the finished geometry.

Preserve whether each figure is open or closed. Closed figures define fillable areas and produce closed stroke joins; open figures are usually stroked outlines where cap behavior is visible at the ends. If the original Skia path relies on winding for holes, keep the same winding model or explicitly choose an `IntersectionRule` that matches the original fill type.

For direct migrations of simple `SKCanvas` calls, use the canvas helpers first. Rectangles use `Fill(brush, Rectangle)` and `Draw(pen, Rectangle)` overloads; ellipses, arcs, pies, lines, and Beziers

have named helpers. Move to explicit shape objects when the same geometry is reused for fill, stroke, clipping, measurement, or composition.

SkiaSharp:

```
using SkiaSharp;

using SKPath triangle = new();
triangle.MoveTo(80, 180);
triangle.LineTo(160, 48);
triangle.LineTo(240, 180);
triangle.Close();

using SKPaint fill = new()
{
    Color = SKColors.Gold,
    IsAntialias = true,
    Style = SKPaintStyle.Fill
};

using SKPaint stroke = new()
{
    Color = SKColors.DarkGoldenrod,
    IsAntialias = true,
    StrokeWidth = 4,
    Style = SKPaintStyle.Stroke
};

canvas.DrawPath(triangle, fill);
canvas.DrawPath(triangle, stroke);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    PathBuilder builder = new();
    builder.MoveTo(new PointF(80, 180));
    builder.LineTo(new PointF(160, 48));
    builder.LineTo(new PointF(240, 180));
    builder.CloseFigure();
```

```
    IPath triangle = builder.Build();

    canvas.Fill(Brushes.Solid(Color.Gold), triangle);
    canvas.Draw(Pens.Solid(Color.DarkGoldenrod, 4), triangle);
}));
```

For common one-off geometry, prefer the canvas helper that matches the original `SKCanvas` call:

SkiaSharp:

```
using SkiaSharp;

using SKPaint fill = new()
{
    Color = SKColors.MediumSeaGreen,
    IsAntialias = true,
    Style = SKPaintStyle.Fill
};

canvas.DrawOval(SKRect.Create(70, 72, 220, 96), fill);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    // ImageSharp.Drawing ellipse helpers take center and size, not top-left bounds.
    canvas.FillEllipse(Brushes.Solid(Color.MediumSeaGreen), new(180, 120), new(220, 96));
}));
```

Use an explicit shape when the geometry is data, not just a drawing call. For example, an `EllipsePolygon` can be filled, stroked, clipped against, transformed, measured, or reused in several commands.

Transforms and Canvas State

SkiaSharp commonly uses `Save()`, `Restore()`, `Translate()`, `Scale()`, and `RotateDegrees()` on the canvas. ImageSharp.Drawing exposes the same scoped-state idea through `Save(...)`, `Restore()`, and `Matrix4x4` transforms.

Translate the transform in the same order that Skia applied it. The saved state affects subsequent geometry, strokes, text, clips, and image placement until it is restored. For ordinary 2D affine transforms, construct the ImageSharp.Drawing transform from `Matrix3x2`; the resulting `Matrix4x4` keeps the public canvas model consistent across CPU, retained scene, and WebGPU targets.

SkiaSharp:

```
using SkiaSharp;

using SKPaint fillPaint = new()
{
    Color = SKColors.HotPink,
    IsAntialias = true,
    Style = SKPaintStyle.Fill
};

using SKPaint strokePaint = new()
{
    Color = SKColors.White,
    IsAntialias = true,
    StrokeWidth = 3,
    Style = SKPaintStyle.Stroke
};

canvas.Save();
canvas.Translate(210, 120);
canvas.Scale(1.2F, 0.8F);
canvas.DrawRect(SKRect.Create(-70, -24, 140, 48), fillPaint);
canvas.DrawRect(SKRect.Create(-70, -24, 140, 48), strokePaint);
canvas.Restore();
```

ImageSharp.Drawing:

```
using System.Numerics;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
```

```
    DrawingOptions options = new()
    {
        Transform = new(
            Matrix3x2.CreateScale(1.2F, 0.8F) *
            Matrix3x2.CreateTranslation(210, 120))
    };

    _ = canvas.Save(options);
    canvas.Fill(Brushes.Solid(Color.HotPink), new Rectangle(-70, -24, 140, 48));
    canvas.Draw(Pens.Solid(Color.White, 3), new Rectangle(-70, -24, 140, 48));

    canvas.Restore();
}));
```

ImageSharp.Drawing uses `Matrix4x4` because the same transform model works across CPU rendering, retained backend scenes, and WebGPU output. For normal 2D drawing, construct it from `Matrix3x2` so the affine values stay familiar.

Image Composition

SkiaSharp image composition often uses `DrawImage(...)` or `DrawBitmap(...)`. In ImageSharp.Drawing, use `DrawImage(...)` inside `Paint(...)` when the operation belongs with the rest of the drawing commands.

Keep the source and destination rectangles explicit while migrating. The source rectangle is in source-image coordinates; the destination rectangle is in canvas coordinates after the current transform. If the old Skia code used a specific sampling option, choose the matching ImageSharp resampler. Otherwise, the drawing API default is appropriate for normal resized placement.

SkiaSharp:

```
using SkiaSharp;

using SKBitmap source = SKBitmap.Decode("photo.jpg");
using SKBitmap output = new(640, 360);
using SKCanvas canvas = new(output);

using SKPaint strokePaint = new()
{
    Color = SKColors.White,
    IsAntialias = true,
    StrokeWidth = 4,
    Style = SKPaintStyle.Stroke
```

```
};

canvas.Clear(SKColors.White);
canvas.DrawBitmap(source, SKRect.Create(32, 32, 320, 220));
canvas.DrawRect(SKRect.Create(32, 32, 320, 220), strokePaint);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("photo.jpg");
using Image<Rgba32> output = new(640, 360, Color.White.ToPixel<Rgba32>());

output.Mutate(context => context.Paint(canvas =>
{
    canvas.DrawImage(source, source.Bounds, new RectangleF(32, 32, 320, 220));
    canvas.Draw(Pens.Solid(Color.White, 4), new Rectangle(32, 32, 320, 220));
}));
```

Keep source images alive until the canvas has replayed. Inside `Paint(...)`, replay is owned by the processing operation. If you create and manage a canvas yourself, dispose it before disposing source images used by drawing commands.

Region Effects

SkiaSharp often applies blur, masking, or filters through paint filters or image filters. ImageSharp.Drawing uses `Apply(...)` to run normal ImageSharp processors inside a rectangle or path.

SkiaSharp:

```
using SkiaSharp;

using SKPaint shadowPaint = new()
{
    Color = SKColors.Black.WithAlpha(89),
    ImageFilter = SKImageFilter.CreateBlur(10, 10),
    IsAntialias = true,
    Style = SKPaintStyle.Fill
};
```

```
using SKPaint panelFillPaint = new()
{
    Color = SKColors.White,
    IsAntialias = true,
    Style = SKPaintStyle.Fill
};

using SKPaint panelStrokePaint = new()
{
    Color = SKColors.LightGray,
    IsAntialias = true,
    StrokeWidth = 1,
    Style = SKPaintStyle.Stroke
};

canvas.DrawRect(SKRect.Create(70, 72, 280, 110), shadowPaint);
canvas.DrawRect(SKRect.Create(62, 58, 280, 110), panelFillPaint);
canvas.DrawRect(SKRect.Create(62, 58, 280, 110), panelStrokePaint);
```

ImageSharp.Drawing:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.Black.WithAlpha(0.35F)), new Rectangle(70, 72,
280, 110));

    // Blur a larger region so the softened shadow can spread beyond the source rectangle.
    canvas.Apply(new Rectangle(60, 62, 300, 130), region => region.GaussianBlur(10));

    canvas.Fill(Brushes.Solid(Color.White), new Rectangle(62, 58, 280, 110));
    canvas.Draw(Pens.Solid(Color.LightGray, 1), new Rectangle(62, 58, 280, 110));
}));
```

On GPU-backed canvases, `Apply(...)` may require readback into the CPU ImageSharp pipeline. Keep the affected region tight, just as you would keep Skia image filters scoped to the area that actually needs the effect.

Text

SkiaSharp text drawing can start simple, but richer layout usually involves `SKTextBlob`, font managers, shaping, and manual measurement. ImageSharp.Drawing uses SixLabors.Fonts directly, so advanced text layout is part of the normal drawing API.

The most common positioning difference is baseline versus layout origin. Skia's simple `DrawText(...)` overloads position text by baseline. ImageSharp.Drawing positions text through `RichTextOptions`, where `Origin` is interpreted by the chosen alignment and wrapping settings. When you need a top-left equivalent for Skia baseline code, account for font metrics during migration, then prefer `RichTextOptions` alignment once the output is confirmed.

SkiaSharp:

```
using SkiaSharp;

using SKTypeface typeface = SKTypeface.FromFile("Inter.ttf");
using SKFont font = new(typeface, 32);
using SKPaint paint = new()
{
    Color = SKColors.Black,
    IsAntialias = true
};

SKFontMetrics metrics;
font.GetFontMetrics(out metrics);

canvas.DrawText("Fast text layout for generated graphics", 48, 48 - metrics.Ascent,
font, paint);
```

ImageSharp.Drawing:

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;

image.Mutate(context => context.Paint(canvas =>
{
    FontCollection collection = new();
    FontFamily family = collection.Add("Inter.ttf");
    Font font = family.CreateFont(32);

    RichTextOptions options = new(font)
    {
        Origin = new PointF(48, 48)
```

```
    };

    canvas.DrawText(options, "Fast text layout for generated graphics",
Brushes.Solid(Color.Black), pen: null);
}));
```

For manual line flow, measurement, caret movement, or rich spans, use the Fonts docs alongside the Drawing text guide.

Practical Migration Strategy

For most SkiaSharp migrations:

1. Move bitmap load/save work to ImageSharp.
2. Replace `SKCanvas` drawing blocks with `image.Mutate(context => context.Paint(canvas => ...))`.
3. Replace fill `SKPaint` objects with `Brush` instances.
4. Replace stroke `SKPaint` objects with `Pen` instances.
5. Replace `SKPath` construction with `PathBuilder`, `Path`, or built-in shape types.
6. Replace canvas transform calls with saved canvas state and `Matrix4x4` values constructed from `Matrix3x2`.
7. Replace image filters with `Apply(...)` where a normal ImageSharp processor gives the same effect.
8. Move text code to `RichTextOptions`, `TextBlock`, and the Fonts layout APIs when measurement or wrapping matters.

You do not need to migrate everything at once. ImageSharp.Drawing is usually easiest to adopt by moving one rendering workflow at a time: generate the same output image, replace the paint/path/text concepts with the closest Drawing equivalents, then simplify once the new model is in place.

Practical Guidance

- Keep examples behavior-equivalent while migrating; change API shape first, then simplify.
- Move bitmap processing to core ImageSharp and canvas drawing to ImageSharp.Drawing.
- Replace mutable paint objects with explicit brushes, pens, and drawing options.
- Use saved canvas state for transforms, clipping, and scoped graphics options.
- Verify text output with real fonts and wrapping because SkiaSharp and Fonts use different layout models.

Recipes

These pages are the quick-start side of the ImageSharp.Drawing docs. They focus on practical drawing tasks that combine canvas commands, brushes, pens, images, text, clipping, and processors.

Each recipe is intentionally complete enough to show the shape of a real workflow: create or load an image, set up reusable drawing objects outside the canvas callback, record the drawing commands, then save the result. After using a recipe, follow the related conceptual pages to understand the canvas state, lifetime, and composition behavior behind it.

Drawing recipes are easiest to adapt when you keep three things separate: geometry, styling, and canvas state. Geometry decides where drawing can happen. Brushes, pens, and text options decide what is drawn. Canvas state decides which later commands are transformed, clipped, layered, or processed.

When adapting a recipe, change one of those layers at a time. If the layout is wrong, inspect the geometry and coordinate system before changing brushes. If the colors or texture are wrong, inspect the brush or pen before changing the shape. If later drawing is unexpectedly clipped, blurred, transformed, or transparent, inspect the canvas state scope around `Save(...)`, `Restore()`, `SaveLayer(...)`, and `Apply(...)`.

Common Tasks

- Add a Text Watermark for anchored, semi-transparent text over an image.
- Clip an Image to a Shape for avatar crops, badges, and shaped image fills.
- Draw a Badge or Label for small generated graphics with shapes, strokes, and text.
- Add Callouts and Annotations for overlays, markers, outlines, and dashed guides.
- Create a Soft Shadow for shadowed panels and grouped drawing effects.

How to Adapt a Recipe

- Keep images, brushes, pens, fonts, and paths alive until the `Paint(...)` operation has completed.
- Create reusable geometry before the callback when more than one command needs the same shape.
- Use `Save(...)` and `Restore()` when a clip, transform, or drawing option should affect only part of the recipe.

- Put `Apply(...)` after the drawing commands that should be processed and before any crisp outlines or labels.
- Choose final output dimensions before positioning text, watermarks, badges, or annotations.
- Prefer `RichTextOptions` alignment and wrapping over manual string measurements.

Practical Guidance

Drawing recipes become easier to maintain when the drawing model stays explicit. Keep source images, image brushes, fonts, paths, and retained backend scenes alive until canvas replay has completed. With `Paint(...)`, that means until the processing callback has finished; with manually-created canvases, it means until the root canvas is disposed.

Use state scopes to make composition readable. `Save(...)` is for clipping, transforms, and options that affect later commands. `SaveLayer(...)` is for a group that should blend back as one result. `Apply(...)` is a timeline barrier for ImageSharp processors, so place it after the pixels that should be processed and before crisp outlines or labels. Effects such as blur need expanded processing regions because the result spreads outside the original shape.

For text-heavy recipes, prefer layout options over guessed coordinates. Wrapping, horizontal alignment, vertical alignment, and text alignment keep examples robust when labels change, localize, or use fallback fonts.

Related Topics

- Canvas Drawing
- Brushes and Pens
- Clipping, Regions, and Layers
- Images, Masks, and Processing
- Drawing Text

Add a Text Watermark

Use `DrawText(...)` with alignment options when a watermark should stay anchored to an image edge. The text layout options keep the placement declarative, so you do not need to measure the string manually.

Anchor the watermark by choosing an origin near the desired edge, then set horizontal and vertical alignment relative to that origin. This keeps the code stable when the watermark text changes length or the image size changes.

Watermark placement should normally happen after the image has reached its final export size and orientation. If you resize after drawing the watermark, the text will be resampled with the image and may become soft. If you draw before `AutoOrient()`, the anchor can land in the wrong visual corner.

Readability is usually the hard part. A watermark that looks fine on one photo can disappear over another. Combining a semitransparent fill with a subtle contrasting stroke gives the text a chance to remain readable over both light and dark regions without making it dominate the image.

Think of watermark styling as an accessibility problem, not only a branding problem. The fill alpha controls how strongly the watermark competes with the photo. The outline pen protects glyph edges against local contrast changes. The font size and wrapping length should be chosen for the final export dimensions, not for the original camera image size.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = Image.Load<Rgba32>("photo.jpg");

Font font = SystemFonts.CreateFont("Arial", 36, FontStyle.Bold);
RichTextOptions options = new(font)
{
    Origin = new(image.Width - 64, image.Height - 64),
    HorizontalAlignment = HorizontalAlignment.Right,
    VerticalAlignment = VerticalAlignment.Bottom
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
```

```
    // Alignment anchors the watermark to the bottom-right corner without measuring the
text first.
    canvas.DrawText(
        options,
        "© Six Labors",
        Brushes.Solid(Color.White.WithAlpha(0.72F)),
        Pens.Solid(Color.Black.WithAlpha(0.45F), 2));
}));

image.Save("watermarked.jpg");
```

Use a subtle fill alpha and a darker outline when the watermark must remain readable over mixed image content. If the watermark can contain user-supplied text, set `WrappingLength` and use alignment rather than assuming a fixed string width.

For repeated export workflows, create the font and text options once per image size, then draw inside the `Paint(...)` callback. Use wrapping when the watermark can contain user or tenant names that may be longer than expected.

Related Topics

- Drawing Text
- Images, Masks, and Processing

Practical Guidance

Use alignment options to anchor watermarks instead of manual text-size guesses. Normalize orientation and resize before positioning watermarks for export, then recreate text options when the image size, font size, wrapping, or origin changes. Use a fill alpha and outline that remain readable on both light and dark image regions.

Clip an Image to a Shape

Use `Save(DrawingOptions, params IPath[])` with `BooleanOperation.Intersection` when later drawing should be limited to a shape. This is useful for avatars, shaped thumbnails, masked hero images, and photo badges.

The important idea is that clipping is canvas state. Once saved, the clip applies to every later command until `Restore()` is called. Draw the clipped image while that state is active, then restore before drawing borders, labels, shadows, or other elements that should sit outside the mask.

Think about the image placement and the mask separately. The clip path decides where pixels are allowed to appear. The `DrawImage(...)` destination rectangle decides how the source image is cropped and scaled into the canvas. Matching the destination rectangle to the shape bounds gives predictable avatar-style crops; using a larger rectangle intentionally zooms or pans the source behind the mask.

Clipping is also stateful, so restore as soon as the clipped drawing is complete. If you forget to restore, later borders, shadows, and labels will be clipped too, which often looks like missing drawing rather than a clipping bug.

The clip path and destination rectangle do not need to be identical, but they should be chosen deliberately. Equal bounds give a simple fit. A larger destination rectangle zooms the source behind the mask. An offset destination rectangle pans the source without moving the mask. That separation is what lets one avatar shape support several crop choices.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> source = Image.Load<Rgba32>("portrait.jpg");
using Image<Rgba32> image = new(360, 360, Color.Transparent.ToPixel<Rgba32>());

PointF avatarCenter = new(180, 180);
SizeF avatarSize = new(300, 300);
EllipsePolygon avatar = new(avatarCenter, avatarSize);
RectangleF destination = new(30, 30, 300, 300);
DrawingOptions clipInside = new()
{
    ShapeOptions = new()
    {
```

```
        // Intersection keeps the image draw inside the avatar path instead of
subtracting it.
        BooleanOperation = BooleanOperation.Intersection
    }
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Save(clipInside, avatar);

    // The active clip limits the photo to the ellipse while DrawImage handles resizing.
    canvas.DrawImage(source, source.Bounds, destination, KnownResamplers.Bicubic);
    canvas.Restore();

    canvas.Draw(Pens.Solid(Color.White, 8), avatar);
    canvas.Draw(Pens.Solid(Color.DarkSlateGray.WithAlpha(0.4F), 2), avatar);
}));

image.Save("avatar.png");
```

Keep the source image alive until the drawing operation has replayed. The `Paint(...)` pipeline handles the canvas lifetime for this example.

Use a destination rectangle that matches the visible shape bounds when you want predictable cropping. Use a larger destination rectangle when the source image should intentionally bleed beyond the shape, for example to zoom into a face inside an avatar.

Related Topics

- Clipping, Regions, and Layers
- Images, Masks, and Processing
- Troubleshooting

Practical Guidance

Save clipped state only around the commands that should be constrained, then restore before drawing borders, labels, or shadows that should sit outside the clip. Keep the source image alive until canvas replay has finished. Match the destination rectangle to the visible shape unless an intentional zoom or bleed is required.

Draw a Badge or Label

Small generated badges usually combine a filled shape, an outline, and centered text. Define the badge bounds once, then use the same geometry for fill and stroke so the border exactly follows the filled area.

This pattern works well for status chips, Open Graph badges, generated labels, and small UI assets. Keep the badge geometry, gradient, and text layout separate: the same rectangle controls fill and stroke, while `RichTextOptions` controls how the label sits inside the shape.

Generated badges tend to be consumed by other layout systems, so stable output dimensions matter. Decide the canvas size and badge bounds first, then fit text inside that region with wrapping and centered alignment. If labels can vary by localization, tenant name, or status text, leave more horizontal padding than the ideal English sample appears to need.

The same geometry should usually drive the fill and the stroke. That avoids one-pixel mismatches where a border no longer follows the filled shape. For more complex badge shapes, build a custom path once and reuse it for the gradient fill, outline, clipping, and any hit-test or layout calculations.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 180, Color.Transparent.ToPixel<Rgba32>());

Rectangle badge = new(24, 36, 372, 108);
Font font = SystemFonts.CreateFont("Arial", 38, FontStyle.Bold);
PointF gradientStart = new(24, 36);
PointF gradientEnd = new(396, 144);
RichTextOptions textOptions = new(font)
{
    Origin = new(210, 90),
    WrappingLength = 320,
    HorizontalAlignment = HorizontalAlignment.Center,
    VerticalAlignment = VerticalAlignment.Center,
    TextAlignment = TextAlignment.Center
};

LinearGradientBrush fill = new(
```

```
    gradientStart,
    gradientEnd,
    GradientRepetitionMode.None,
    new ColorStop(0F, Color.DeepSkyBlue),
    new ColorStop(1F, Color.MediumBlue));

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(fill, badge);
    canvas.Draw(Pens.Solid(Color.White.WithAlpha(0.9F), 4), badge);

    // The text anchor is the badge center, and wrapping keeps long labels inside the shape.
    canvas.DrawText(textOptions, "ACTIVE", Brushes.Solid(Color.White), pen: null);
}));

image.Save("badge.png");
```

Use a path type that matches the badge geometry you want. `RectanglePolygon`, `EllipsePolygon`, `RegularPolygon`, `StarPolygon`, and custom `PathBuilder` paths can all be filled and stroked through the same canvas calls.

If the label can vary, set `WrappingLength` smaller than the badge width and use centered alignment. That gives long values room to wrap instead of spilling into the border.

Related Topics

- Primitive Drawing Helpers
- Brushes and Pens
- Drawing Text

Practical Guidance

Build badge geometry once and reuse it for fill and stroke. Keep source dimensions stable when generated badges are consumed by layout systems. Set text wrapping shorter than the badge width when labels can vary, and use centered alignment instead of manual text offsets.

Add Callouts and Annotations

Annotations are overlays that explain or identify parts of an existing image. In ImageSharp.Drawing they are built from the same primitives as any other drawing: fills, strokes, text, lines, clips, and regions. The useful part is not the style; it is the workflow for keeping annotation geometry tied to the pixels it describes.

An annotation usually has three pieces:

- a target region in image coordinates;
- a visual marker such as a fill, outline, arrow, or leader line;
- a label laid out in a predictable rectangle.

Compute those pieces after the image has the size and orientation that will be exported. If you resize, crop, or auto-orient after drawing annotations, the overlay will be transformed with the pixels and may no longer point at the intended feature.

Coordinate Workflow

Keep annotation geometry in final image coordinates. If the target was detected in source-image coordinates, map it after any crop, resize, or orientation step before drawing.

```
using SixLabors.ImageSharp;

static Rectangle ScaleRectangle(Rectangle source, Size sourceSize, Size destinationSize)
{
    float scaleX = (float)destinationSize.Width / sourceSize.Width;
    float scaleY = (float)destinationSize.Height / sourceSize.Height;

    return new Rectangle(
        (int)MathF.Round(source.X * scaleX),
        (int)MathF.Round(source.Y * scaleY),
        (int)MathF.Round(source.Width * scaleX),
        (int)MathF.Round(source.Height * scaleY));
}
```

Use the same mapping for every point that belongs to the annotation: highlight bounds, leader start, leader end, label origin, and panel bounds. That keeps the annotation coherent when the output size changes.

Highlight a Region

A rectangular highlight is the simplest annotation. Fill the target with a translucent brush, then stroke the same rectangle so the boundary is clear.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = Image.Load<Rgba32>("photo.jpg");

Rectangle regionOfInterest = new(92, 64, 220, 140);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.Gold.WithAlpha(0.22F)), regionOfInterest);
    canvas.Draw(Pens.Dash(Color.Gold, 5), regionOfInterest);
}));

image.Save("highlighted.jpg");
```

Use a rectangle overload when the marker is just a one-off rectangular highlight. Use a reusable path or polygon when the same geometry must be filled, stroked, clipped, measured, or passed through a geometry operation.

Add a Leader and Label

Labels are normal text drawing. Use `RichTextOptions` so wrapping and placement are explicit. Use a text stroke when the label must remain readable over arbitrary image content.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = Image.Load<Rgba32>("photo.jpg");

Rectangle target = new(92, 64, 220, 140);
PointF targetEdge = new(target.Right, target.Top + (target.Height / 2F));
PointF labelOrigin = new(target.Right + 28, target.Top + 12);
```

```
Font font = SystemFonts.CreateFont("Arial", 24, FontStyle.Bold);
RichTextOptions labelOptions = new(font)
{
    Origin = labelOrigin,
    WrappingLength = 220
};

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Fill(Brushes.Solid(Color.Gold.WithAlpha(0.22F)), target);
    canvas.Draw(Pens.Dash(Color.Gold, 5), target);

    // The leader line connects the label to the target in the same image coordinate system.
    canvas.DrawLine(
        Pens.Solid(Color.Gold, 3),
        new PointF(labelOrigin.X - 12, labelOrigin.Y + 12),
        targetEdge);

    // The outline pen makes the text readable over mixed light and dark pixels.
    canvas.DrawText(
        labelOptions,
        "Region of interest",
        Brushes.Solid(Color.White),
        Pens.Solid(Color.Black, 1.5F));
}));

image.Save("annotated.jpg");
```

Draw the leader before the label so the label remains crisp and unobstructed. When a label can contain user-supplied text, set `WrappingLength` and choose an origin that leaves room for multiple lines.

Use a Local Panel Region

When a callout contains several items, use `CreateRegion(...)` so the panel has local coordinates. The parent canvas still uses image coordinates; the child region uses `(0, 0)` at the panel origin.

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = Image.Load<Rgba32>("photo.jpg");
```

```
Rectangle target = new(92, 64, 220, 140);
Rectangle panelBounds = new(348, 52, 260, 116);
Font titleFont = SystemFonts.CreateFont("Arial", 22, FontStyle.Bold);
Font bodyFont = SystemFonts.CreateFont("Arial", 16);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    canvas.Draw(Pens.Solid(Color.Gold, 4), target);
    canvas.DrawLine(
        Pens.Solid(Color.Gold, 3),
        new PointF(target.Right, target.Top + (target.Height / 2F)),
        new PointF(panelBounds.Left, panelBounds.Top + 34));

    using DrawingCanvas panel = canvas.CreateRegion(panelBounds);

    panel.Fill(Brushes.Solid(Color.Black.WithAlpha(0.72F)));
    panel.Draw(Pens.Solid(Color.Gold, 2), new Rectangle(0, 0,
panelBounds.Width, panelBounds.Height));

    // Text inside the region is positioned relative to the panel, not the source image.
    panel.DrawText(
        new RichTextOptions(titleFont) { Origin = new(14, 12), WrappingLength =
panelBounds.Width - 28 },
        "Inspection note",
        Brushes.Solid(Color.White),
        pen: null);

    panel.DrawText(
        new RichTextOptions(bodyFont) { Origin = new(14, 48), WrappingLength =
panelBounds.Width - 28 },
        "The highlighted area is drawn in parent coordinates; this panel uses
local coordinates.",
        Brushes.Solid(Color.WhiteSmoke),
        pen: null);
}));

image.Save("annotation-panel.jpg");
```

Region canvases are useful for labels, inset panels, badges, and legends because the panel layout can be written once without repeatedly adding the parent offset.

Clip an Annotation to a Shape

Use `Save(DrawingOptions, params IPath[])` when a marker should be constrained to a non-rectangular target. The clip uses `ShapeOptions.BooleanOperation`, so set `Intersection` for "draw only inside this shape" behavior.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;

DrawingOptions insideShape = new()
{
    ShapeOptions = new()
    {
        BooleanOperation = BooleanOperation.Intersection
    }
};

EllipsePolygon target = new(220, 140, 180, 96);

canvas.Save(insideShape, target);
canvas.Fill(Brushes.ForwardDiagonal(Color.Gold.WithAlpha(0.5F), Color.Transparent), new
Rectangle(120, 82, 200, 116));
canvas.Restore();

canvas.Draw(Pens.Solid(Color.Gold, 4), target);
```

The fill and outline are separate on purpose: the hatch is clipped to the target, then the outline is drawn after `Restore()` so it remains crisp.

Practical Guidance

- Normalize orientation, crop, and resize before computing annotation geometry.
- Keep target geometry in image coordinates, and use regions only for local panel layout.
- Use primitive rectangle, line, and text APIs for one-off callouts.
- Use paths or polygons when annotation geometry must be reused for clipping, fill, stroke, or measurement.
- Draw translucent markers before crisp outlines and labels.
- Set text wrapping instead of assuming label text will fit on one line.

Related Topics

- Canvas Drawing
- Primitive Drawing Helpers

- Brushes and Pens
- Clipping, Regions, and Layers
- Drawing Text

Create a Soft Shadow

For the common case, pass a `DropShadowLayerEffect` to `SaveLayer(...)`. The effect slots a blurred, offset shadow beneath the layer content when the layer is restored, and the layer bounds expand automatically to keep the soft edge. See Layer and Backdrop Effects for the layer effect model.

This recipe shows the manual pattern instead. It is useful when you need full control over the shadow geometry and processing region, or when the shadow must come from a normal ImageSharp processor.

Draw the shadow shape first, then apply a blur to the shadow region before drawing the foreground object. `Apply(...)` is a replay barrier, so only commands recorded before the barrier are processed.

The blur region should be larger than the original shadow shape. Gaussian blur spreads pixels outward, so using the exact shape bounds clips the soft edge. A good rule of thumb is to expand the processing rectangle by at least the blur radius on every side.

The order is what makes the effect work. First draw the shadow pixels. Then use `Apply(...)` to blur only the region that contains the shadow. Then draw the crisp foreground panel. If the panel were drawn before the blur, it would be blurred with the shadow. If the blur region were too small, the soft edge would be cut off.

Use `Apply(...)` for this pattern when a normal ImageSharp processor should affect an already-recorded part of the drawing. Use `SaveLayer(...)` for a different problem: grouping several commands so the group composites back to the parent as one result.

A shadow is usually easier to reason about as three separate decisions: the shadow geometry, the blur processing area, and the foreground geometry. The shadow geometry is often the foreground shape offset by a few pixels. The blur processing area should cover the shadow plus the blur spread. The foreground geometry should be drawn after the blur so it remains sharp.

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

using Image<Rgba32> image = new(420, 240, Color.White.ToPixel<Rgba32>());

Rectangle shadowOffsetBounds = new(70, 72, 280, 110);
Rectangle blurBounds = new(60, 62, 300, 130);
```

```
Rectangle panelBounds = new(62, 58, 280, 110);

image.Mutate(ctx => ctx.Paint(canvas =>
{
    // The shadow is intentionally offset from the panel before the blur spreads it outward.
    canvas.Fill(Brushes.Solid(Color.Black.WithAlpha(0.35F)), shadowOffsetBounds);

    // Apply seals earlier drawing commands before the blur is replayed, and the expanded
region preserves the feathered edge.
    canvas.Apply(blurBounds, region => region.GaussianBlur(10));

    canvas.Fill(Brushes.Solid(Color.White), panelBounds);
    canvas.Draw(Pens.Solid(Color.LightGray, 1), panelBounds);
}));

image.Save("shadow.png");
```

Keep the blur region tight. On CPU canvases this reduces the amount of image data processed, and on GPU-backed canvases it reduces readback and upload work. If you need a shadow around a complex path, use the path bounds expanded by the blur radius as the processing region, then draw the foreground path after the barrier.

Use `SaveLayer(...)` instead when the foreground and shadow need to be composed as one group over existing content. Use `Apply(...)` when you want a normal ImageSharp processor, such as blur, pixelation, or color adjustment, to affect only part of the drawing timeline.

Related Topics

- Canvas Drawing
- Images, Masks, and Processing
- Transforms and Composition

Practical Guidance

Expand blur regions so feathered pixels are not clipped, but keep them as small as the effect allows. Draw crisp foreground content after the blur barrier. Use layers instead when the shadow and foreground must compose as one group.

Troubleshooting

This page collects common issues you can hit when moving from simple drawing samples to full ImageSharp.Drawing pipelines. Most problems come from three areas: canvas replay lifetime, clipping and fill-rule choices, or text layout state.

If the issue is WebGPU-specific, start with the WebGPU section below and then check the dedicated WebGPU, environment, window, external surface, and render target pages.

Nothing Appears on the Image

If you are drawing through `image.Mutate(ctx => ctx.Paint(...))`, the processing pipeline owns the canvas lifetime and replays the recorded drawing commands for you.

If you create a canvas manually, make sure the canvas is disposed before you inspect the destination image. Canvas drawing is recorded and replayed in order, so pending commands are not visible until the root canvas replays them. `Flush()` only seals queued commands into the replay timeline; it does not render them by itself.

```
using Image<Rgba32> image = new(400, 240, Color.White.ToPixel<Rgba32>());

using (DrawingCanvas canvas = image.CreateCanvas())
{
    canvas.Fill(Brushes.Solid(Color.CornflowerBlue), new Rectangle(40, 40, 180, 100));

    // Disposing the canvas replays the recorded drawing commands onto the image.
}

image.Save("output.png");
```

When you use images as drawing sources, keep those source images alive until the canvas has replayed. `DrawImage` and `ImageBrush<TPixel>` record the drawing operation; they do not make the source image safe to dispose before replay.

Clipping Removes the Wrong Area

`ShapeOptions.BooleanOperation` controls how the clip shape combines with the current drawing region. The default value is `Difference`, which subtracts the supplied shape from the current region. For the usual "draw only inside this shape" behavior, set it to `BooleanOperation.Intersection`.

```
DrawingOptions options = new()
{
    ShapeOptions = new()
    {
        // Intersect keeps the part of subsequent drawing inside the clip shape.
        BooleanOperation = BooleanOperation.Intersection
    }
};

PointF clipCenter = new(200, 120);
SizeF clipSize = new(260, 160);
EllipsePolygon clip = new(clipCenter, clipSize);

canvas.Save(options, clip);
canvas.Fill(Brushes.Solid(Color.HotPink), new Rectangle(0, 0, 400, 240));
canvas.Restore();
```

Use `Save(...)` for scoped clipping and state changes. Call `Restore()` when the scoped operation is complete so later drawing returns to the previous state.

Holes or Overlaps Fill Unexpectedly

The fill rule controls how overlapping contours inside a complex polygon are interpreted. ImageSharp.Drawing defaults to `IntersectionRule.NonZero`, which matches the default used by SVG and web canvas APIs. With `NonZero`, contour winding order is meaningful, so holes are normally expressed by reversing the winding of the inner contour.

Use `IntersectionRule.EvenOdd` when you want parity-based filling where each crossing toggles between inside and outside. This can be convenient for imported geometry that does not carry reliable winding direction.

```
DrawingOptions options = new()
{
    ShapeOptions = new()
    {
        // EvenOdd treats alternating contours as filled and unfilled regions.
        IntersectionRule = IntersectionRule.EvenOdd
    }
};

_ = canvas.Save(options);
```

```
canvas.Fill(Brushes.Solid(Color.MediumSeaGreen), complexPolygon);
canvas.Restore();
```

Text Is Not Centered Where Expected

For region-based text layout, use the text alignment options instead of manually subtracting measured text sizes. The `Origin` is the layout anchor, `WrappingLength` defines the line width, and `HorizontalAlignment` / `VerticalAlignment` place the text block relative to that anchor.

`TextAlignment` controls how wrapped lines are aligned inside the paragraph. `HorizontalAlignment` controls how the resulting paragraph bounds are positioned relative to `Origin`.

Styled Text Affects the Wrong Characters

Rich text runs use grapheme indices, not UTF-16 code unit indices. `Start` is inclusive and `End` is exclusive, so the affected range is `[Start, End)`.

This matters for emoji, combining marks, flags, and other user-perceived characters that can contain multiple Unicode scalar values. See the Fonts Unicode page for the same indexing model.

Processors Run Before Earlier Drawing

Canvas operations are ordered, but image processors operate at replay barriers. A processor such as blur, opacity, or a mask operation includes drawing that was recorded before the `Apply(...)` call.

```
canvas.Fill(Brushes.Solid(Color.Black), shadowShape);

// Apply seals the shadow geometry before the blur processor is applied.
canvas.Apply(x => x.GaussianBlur(8));
```

This is most useful when you mix vector drawing with ImageSharp processors in the same canvas sequence.

Images, Brushes, or Masks Stop Working After Disposal

Drawing commands can be replayed later than the point where the command is recorded. Keep any source `Image<TPixel>` used by `DrawImage`, masks, or `ImageBrush<TPixel>` alive until the root

canvas has been disposed.

The canvas does not own images passed into it. Dispose those images after the drawing scope that uses them has completed.

WebGPU Produces a Blank Frame

Probe WebGPU support before creating GPU-backed drawing resources. WebGPU depends on the runtime environment, adapter, device, texture format, and native surface or offscreen target.

For window or surface rendering, acquire a frame, draw into its canvas, and dispose the frame. Disposing the frame completes the drawing scope and presents it to the surface.

```
if (!surface.TryAcquireFrame(out WebGPUSurfaceFrame? frame))
{
    return;
}

using (frame)
{
    DrawingCanvas canvas = frame.CreateCanvas();

    // Drawing commands are presented when the frame is disposed.
    canvas.Clear(Brushes.Solid(Color.White));
    canvas.Fill(Brushes.Solid(Color.SteelBlue), new Rectangle(40, 40, 180, 120));
}
```

Resize the `WebGPUExternalSurface` when the framebuffer size changes. If you need to read pixels back to the CPU, use a pixel type that matches the target texture format, for example `Rgba32` with an `Rgba8Unorm` target.

A Good Debugging Order

1. Confirm the root canvas scope is disposed before checking the output.
2. Check source image lifetimes when using image brushes, masks, or `DrawImage`.
3. Check `ShapeOptions.BooleanOperation` when clipping.
4. Check `ShapeOptions.IntersectionRule` and contour winding for complex polygons.
5. Check text layout options before doing manual measurement math.
6. Check grapheme-based `[Start, End)` indices for rich text runs.
7. Probe WebGPU availability and surface frame acquisition before drawing GPU content.

Related Topics

- Canvas Drawing
- Clipping, Regions, and Layers
- Paths and Shapes
- Drawing Text
- WebGPU
- WebGPU Environment and Support
- WebGPU Window Rendering
- WebGPU External Surfaces
- WebGPU Offscreen Render Targets

Practical Guidance

- Start with lifetime and replay issues before debugging visual details.
- Reduce complex examples to one shape, one clip, or one text block to isolate state.
- Check canvas ordering around `Apply(...)` whenever processors see unexpected pixels.
- Check font availability and grapheme ranges before changing text drawing code.

ImageSharp.Web

ImageSharp.Web is Six Labors' high-performance ASP.NET Core image middleware for on-the-fly processing and caching. It sits in front of one or more image providers, parses URL commands, runs the matching ImageSharp processors, and stores the result so repeated requests are inexpensive after the first hit.

The current package targets .NET 8 and is built on top of ImageSharp. The middleware is intentionally modular: you can change how commands are parsed, where source images come from, how cache keys are built, where processed images are stored, and whether image requests must be signed.

The practical model is a web request pipeline. A provider resolves the original image, a parser turns the request into commands, processors transform the image, an encoder writes the response, and a cache stores the result so the next matching request can avoid the expensive work. Most configuration choices are about one of those stages.

Use ImageSharp.Web when image variants are determined by HTTP requests: responsive thumbnails, CDN-backed transformations, signed URLs, tenant-specific providers, or cached format conversion. Use core ImageSharp directly when processing is an offline job, queue worker, or application workflow that is not naturally request-driven.

License

ImageSharp.Web is licensed under the terms of the Six Labors Split License, Version 1.0. See https://sixlabors.com/pricing for commercial licensing details.

> **IMPORTANT**
>
> Projects that directly reference ImageSharp.Web require a valid Six Labors license key at build time from version 3.3.0 in the previous major and 4.0.0 in the current major. Enforcement applies to direct dependencies only. This is enforcement of the existing license terms, not a license change.

Install ImageSharp.Web

ImageSharp.Web is distributed on NuGet with preview and nightly builds available on Feedz.

Package Manager | .NET CLI | PackageReference | Paket CLI

```
PM > Install-Package SixLabors.ImageSharp.Web -Version VERSION_NUMBER
```

Prerelease versions installed via the Visual Studio NuGet Package Manager require the "include prerelease" checkbox to be checked.

How to use the license file

By default, the build searches from each project directory for `sixlabors.lic`. Place the supplied file in the directory that contains the project file, or in a subdirectory below it. Use the file as supplied; it contains the complete license string required by the build.

> **IMPORTANT**
>
> Assembly-scoped community keys can be committed to a public repository. Contributors can use the included key to build the named assemblies without obtaining their own keys. Unrestricted community keys and commercial keys must remain private. Use environment variables or repository secrets for those keys.

To keep the file elsewhere, including a repository root above the project directory, set `SixLaborsLicenseFile` in your project file or a shared props file:

```
<PropertyGroup>
  <!-- Set the location of the supplied license file. -->
  <SixLaborsLicenseFile>path/to/sixlabors.lic</SixLaborsLicenseFile>
</PropertyGroup>
```

To use an environment variable instead, set this project property:

```
<PropertyGroup>
  <!-- Read the complete license string from the environment. -->
  <SixLaborsLicenseKey>$(SIXLABORS_LICENSE_KEY)</SixLaborsLicenseKey>
</PropertyGroup>
```

Set `SIXLABORS_LICENSE_KEY` to the complete contents of your issued license, not only its `Key` field. Use the original value without escaping. Replace the entire example value below with your issued

license contents.

PowerShell:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
$env:SIXLABORS_LICENSE_KEY = 'Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

Bash:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
export SIXLABORS_LICENSE_KEY='Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

If you pass the license directly through `-p:SixLaborsLicenseKey`, escape every semicolon as `%253B`. MSBuild treats semicolons as property separators even inside a quoted argument. The pipe between assembly signatures stays unchanged.

PowerShell:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
dotnet publish "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
```

Bash:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
dotnet publish "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
```

You do not need to reference the licensing package directly. Projects that directly reference a Six Labors package carry its build-time validation.

Community keys for public projects

For an open-source or source-available application, provide the exact `AssemblyName` of every project that directly references a Six Labors package, including test projects. Names are case-

sensitive.

For example, if `MyProject.csproj` sets `<AssemblyName>MyProject.Core</AssemblyName>`, enter `MyProject.Core`. If it does not set `AssemblyName`, enter `MyProject`, the project filename without `.csproj`. If a shared props file sets the name, use that value. Do not include `.dll` or `.csproj`.

Existing community license holders can request one early replacement with an assembly-scoped key. These upgrade requests are approved automatically, without another eligibility review. You do not need to wait for your current key to enter its renewal window.

Apply at licensing.sixlabors.com. Select open-source or source-available, use the same email address, and provide the assembly names. Later requests follow the normal renewal window: the final 90 days before the expiry date in your license. Commercial licenses are not eligible for this early replacement.

Security support

The latest major version of each library is eligible for security updates. Each major remains eligible for 12 months after the first stable release of its successor. A later major release does not shorten an existing support window.

Install the latest available patch or minor release within a supported major to receive security fixes. Older releases within that major are not maintained separately. This policy covers security fixes only. It does not include feature backports or extend Microsoft's support for the underlying .NET version. Security fixes remain at Six Labors' discretion.

Start Here

- Getting Started covers the minimal ASP.NET Core setup and the default provider and cache behavior.
- Configuration and Pipeline explains what `AddImageSharp()` registers, the middleware's default auto-orientation behavior, web-focused encoder defaults, ICC profile handling, and how to replace or reorder the moving parts.
- Processing Commands documents the built-in resize, auto-orient, format, quality, and background-color commands, including which ones are implicit by default.
- Image Providers covers filesystem, Azure Blob Storage, and AWS S3 source images.
- Image Caches covers the default physical cache, cloud cache backends, cache keys, and cache lifetime.
- Securing Requests explains HMAC signing and preset-only parsing.
- Tag Helpers covers Razor integration and automatic HMAC generation.
- Extensibility walks through custom processors, parsers, providers, caches, and converters.

- Troubleshooting covers the most common middleware-order, provider, cache, and signing problems.

Implicit Usings

Set `UseImageSharp` in your project file to automatically import the most common ImageSharp and ImageSharp.Web namespaces:

```
<PropertyGroup>
  <UseImageSharp>true</UseImageSharp>
</PropertyGroup>
```

When enabled, ImageSharp.Web adds implicit `global using` directives for:

- `SixLabors.ImageSharp`
- `SixLabors.ImageSharp.Processing`
- `SixLabors.ImageSharp.Web`

You can turn this off by removing the property or setting it to `false`.

How to Use These Docs

- Start with getting started and processing commands to understand the default request pipeline.
- Read configuration, providers, and caches before deploying beyond a single local filesystem setup.
- Read security before exposing arbitrary command URLs to clients.
- Use extensibility only after choosing which pipeline stage actually owns the behavior you need.

Getting Started

ImageSharp.Web is easiest to understand as a request pipeline: match a source image, parse commands, process with ImageSharp, cache the result, then serve the cached bytes on later requests. This page shows the smallest setup first and then explains what the default registration gives you.

Minimal ASP.NET Core Setup

```
using SixLabors.ImageSharp.Web;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddImageSharp();

WebApplication app = builder.Build();

app.UseImageSharp();
app.UseStaticFiles();

app.Run();
```

`app.UseImageSharp()` must appear before `app.UseStaticFiles()`. If static files run first, requests such as `/images/photo.jpg` or `/images/photo.jpg?width=400` will be served directly from disk and ImageSharp.Web will never see them.

Treat the middleware order as part of the image contract for your application. Anything registered before ImageSharp.Web can short-circuit the request before image processing happens. Anything registered after it will normally see the processed response only when ImageSharp.Web chooses not to handle the request.

What the Default Registration Includes

`AddImageSharp()` wires up the core middleware services plus a sensible default pipeline:

- `QueryCollectionRequestParser` reads commands from the query string.
- `PhysicalFileSystemProvider` resolves source images from the web root by default.
- `PhysicalFileSystemCache` stores processed output under `wwwroot/is-cache` by default.
- `UriRelativeLowerInvariantCacheKey` and `SHA256CacheHash` create hashed cache filenames.

- `ResizeWebProcessor`, `FormatWebProcessor`, `BackgroundColorWebProcessor`, `QualityWebProcessor`, and `AutoOrientWebProcessor` provide the built-in command set.
- A default `OnParseCommandsAsync` callback that inserts `autoorient=true` when the request does not already specify `autoorient`.
- A middleware-specific ImageSharp `Configuration` with web-oriented `JpegEncoder`, `PngEncoder`, and `WebpEncoder` defaults.

With that setup in place, requests like these are processed automatically:

```
/images/photo.jpg?width=400
/images/photo.jpg?width=400&height=250&rmode=crop
/images/logo.png?bgcolor=white&format=jpg&quality=85
```

That default configuration is intentionally opinionated for web output. Processed JPEGs use quality `75` with progressive, interleaved `YCbCrRatio420` encoding, processed PNGs use `BestCompression` with adaptive filtering, and processed WebP output uses quality `75` with `BestQuality` encoding method.

The default command path is opinionated too: ImageSharp.Web transparently adds `autoorient=true` unless the request already contains an `autoorient` value. That means processed output is EXIF-normalized by default, which is especially important for WebP delivery where browser orientation support is inconsistent.

When you keep the default middleware configuration and do not return custom `DecoderOptions` from `OnBeforeLoadAsync`, the middleware also decodes with `ColorProfileHandling.Convert`. That normalizes embedded ICC profiles for web-oriented re-encoding instead of blindly carrying source color encodings through the pipeline.

If you later replace `ImageSharpMiddlewareOptions.Configuration`, you also replace those encoder defaults. If you replace `OnParseCommandsAsync`, you replace the default auto-orientation injection unless you explicitly preserve it. See Configuration and Pipeline for both patterns.

A Useful Default Mental Model

With the default `PhysicalFileSystemProvider`, the provider itself still uses `ProcessingBehavior.CommandOnly`, but the default middleware callback inserts `autoorient=true` when no `autoorient` command is present. In practice that means:

- `/images/photo.jpg` is intercepted, auto-oriented, cached, and served by ImageSharp.Web.
- `/images/photo.jpg?width=400` is also intercepted and processed by ImageSharp.Web.

That default favors display correctness over passthrough behavior, especially for formats such as WebP where browser EXIF-orientation support is unreliable.

If you want passthrough behavior that only processes URLs that already contain commands, you must replace `OnParseCommandsAsync` or otherwise bypass the middleware for those paths. `ProcessingBehavior.CommandOnly` by itself is not enough while the default auto-orientation callback is active.

Configure the Physical Provider and Cache

If your source images or cache should live somewhere other than the default web root locations, configure the provider and cache options explicitly:

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Caching;
using SixLabors.ImageSharp.Web.Providers;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddImageSharp(options =>
{
    options.BrowserMaxAge = TimeSpan.FromDays(7);
    options.CacheMaxAge = TimeSpan.FromDays(30);
})
.Configure<PhysicalFileSystemProviderOptions>(options =>
{
    options.ProviderRootPath = "assets";
})
.Configure<PhysicalFileSystemCacheOptions>(options =>
{
    options.CacheRootPath = "cache";
    options.CacheFolder = "imagesharp";
    options.CacheFolderDepth = 8;
});
```

Relative paths are resolved against the application content root. If your app does not define a web root, set both `ProviderRootPath` and `CacheRootPath` explicitly.

Keep source storage and cache storage conceptually separate. The provider root is where original images come from. The cache root is disposable derived output and can usually be cleared, rebuilt, or moved to cheaper storage without losing source assets. In clustered deployments, choose cache storage that matches your invalidation and sharing requirements.

Next Steps

- Configuration and Pipeline
- Processing Commands
- Image Providers
- Image Caches
- Securing Requests

Practical Guidance

- Put `UseImageSharp()` before middleware that would otherwise serve source image files directly.
- Keep source storage separate from derived cache storage.
- Configure provider and cache roots explicitly when the app has no web root or runs in a container.
- Read the security page before exposing free-form transformation URLs publicly.

Configuration and Pipeline

ImageSharp.Web is assembled through `AddImageSharp()` and the returned `IImageSharpBuilder`. Most applications never need to replace every piece, but it helps to know what is there so you can change the correct layer without over-customizing the whole pipeline.

Request Pipeline Model

For a processed image request, the middleware does four separate jobs:

- parse and normalize the requested commands;
- resolve the source image through a provider;
- look up or write the processed result through a cache;
- decode, process, and encode the image when the cache does not already contain a valid result.

Those layers are intentionally separate. Providers are source-of-truth readers, caches store derived output, processors transform the decoded image, and encoders decide the response format. Most customization should replace one layer at a time instead of rebuilding the whole middleware configuration.

What `AddImageSharp()` Registers

The default registration wires up:

- `QueryCollectionRequestParser` for query-string command parsing.
- `PhysicalFileSystemCache` for processed-output storage.
- `UriRelativeLowerInvariantCacheKey` and `SHA256CacheHash` for cache naming.
- `PhysicalFileSystemProvider` for source image resolution.
- The built-in resize, format, quality, background-color, and auto-orient processors.
- The built-in command converters for numbers, booleans, strings, arrays, lists, colors, and enums.

That gives you a fully working middleware out of the box, but every one of those pieces can be swapped or extended.

Configure Middleware Options

`ImageSharpMiddlewareOptions` controls the shared middleware behavior:

- `Configuration` is the underlying ImageSharp `Configuration`.
- By default, that `Configuration` is not raw `Configuration.Default`; ImageSharp.Web installs web-oriented JPEG, PNG, and WebP encoders into it.
- `OnParseCommandsAsync` defaults to a callback that injects `autoorient=true` when the request does not already contain `autoorient`.
- `MemoryStreamManager` controls pooled response streams.
- `UseInvariantParsingCulture` controls whether command parsing is culture-invariant.
- `BrowserMaxAge`, `CacheMaxAge`, and `CacheHashLength` control cache behavior.

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Caching;
using SixLabors.ImageSharp.Web.Providers;

builder.Services.AddImageSharp(options =>
{
    options.UseInvariantParsingCulture = true;
    options.BrowserMaxAge = TimeSpan.FromDays(7);
    options.CacheMaxAge = TimeSpan.FromDays(30);
    options.CacheHashLength = 16;
})
.Configure<PhysicalFileSystemProviderOptions>(options =>
{
    options.ProviderRootPath = "assets";
    options.ProcessingBehavior = ProcessingBehavior.CommandOnly;
})
.Configure<PhysicalFileSystemCacheOptions>(options =>
{
    options.CacheRootPath = "cache";
    options.CacheFolder = "imagesharp";
    options.CacheFolderDepth = 8;
});
```

If you do not need to change format registrations or encoder defaults, leave `ImageSharpMiddlewareOptions.Configuration` alone. Replacing it opts you out of the middleware's built-in web defaults.

Likewise, if you do not need custom command augmentation, leave `OnParseCommandsAsync` alone. Replacing it opts you out of the default EXIF-normalization behavior unless you chain the existing callback yourself.

Default Orientation Behavior

The default `OnParseCommandsAsync` callback inserts `autoorient=true` when the request does not already specify `autoorient`.

That makes EXIF normalization part of the default middleware behavior rather than an opt-in query-string feature. The main reason is web delivery: some browsers still ignore EXIF orientation in formats such as WebP, so relying on the encoded metadata alone does not produce consistent display results.

Two details matter in practice:

- `autoorient=false` still disables the behavior for that request because the middleware only inserts the command when it is absent.
- Replacing `OnParseCommandsAsync` with your own delegate removes the built-in insertion unless you invoke the previous delegate.

With the out-of-the-box local filesystem setup, that also means commandless image URLs are usually processed and cached instead of falling through to static files unchanged.

Default Encoder and ICC Behavior

The default middleware `Configuration` is a cloned ImageSharp configuration with web-oriented encoder registrations:

- `JpegEncoder` uses `Quality = 75`, `Progressive = true`, `Interleaved = true`, and `ColorType = YCbCrRatio420`.
- `PngEncoder` uses `CompressionLevel = BestCompression` and `FilterMethod = Adaptive`.
- `WebpEncoder` uses `Quality = 75` and `Method = BestQuality`.

Those registrations are used whenever the middleware saves processed output in JPEG, PNG, or WebP format, whether the format came from the source image or from the `format` command.

If `OnBeforeLoadAsync` returns `null`, the middleware also creates fallback `DecoderOptions` for you. The ICC-profile behavior depends on whether you kept the default configuration:

- With the default middleware configuration, `DecoderOptions.ColorProfileHandling` is `ColorProfileHandling.Convert`.
- If you replace `options.Configuration`, the fallback changes to `ColorProfileHandling.Compact`.

`Compact` only removes canonical sRGB ICC profile data. It does not convert non-sRGB source images. That distinction matters most when you transcode or resize JPEGs that arrive with CMYK or other non-sRGB profiles.

Customize Encoders Without Losing ICC Conversion

If you want your own encoder registrations but still want the middleware to decode with `ColorProfileHandling.Convert`, clone the current configuration, replace the encoders you care about, then return explicit decoder options:

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Formats;
using SixLabors.ImageSharp.Formats.Jpeg;
using SixLabors.ImageSharp.Web;

builder.Services.AddImageSharp(options =>
{
    Configuration configuration = options.Configuration.Clone();

    configuration.ImageFormatsManager.SetEncoder(JpegFormat.Instance, new JpegEncoder
    {
        Quality = 82,
        Progressive = true,
        Interleaved = true,
        ColorType = JpegColorType.YCbCrRatio420
    });

    options.Configuration = configuration;

    options.OnBeforeLoadAsync = (_, _) => Task.FromResult<DecoderOptions?>(new()
    {
        Configuration = configuration,
        ColorProfileHandling = ColorProfileHandling.Convert
    });
});
```

Use this pattern when you want to keep ImageSharp.Web's ICC-conversion behavior but need different encoder quality, chroma subsampling, format registrations, allocator settings, or other base ImageSharp customization.

Change Individual Pipeline Pieces

The builder methods let you replace only the layer you actually need to change:

- `SetRequestParser<TParser>()` replaces the request parser.

- `SetCache<TCache>()` replaces the backend cache.
- `SetCacheKey<TCacheKey>()` and `SetCacheHash<TCacheHash>()` change cache naming.
- `AddProvider<TProvider>()`, `InsertProvider<TProvider>()`, `RemoveProvider<TProvider>()`, and `ClearProviders()` manage source providers.
- `AddProcessor<TProcessor>()`, `RemoveProcessor<TProcessor>()`, and `ClearProcessors()` manage the processing command set.
- `AddConverter<TConverter>()`, `RemoveConverter<TConverter>()`, and `ClearConverters()` manage typed command parsing.
- `Configure<TOptions>(...)` binds or mutates option objects for any registered provider, cache, or parser.

For example, if you want to keep the default middleware but remove format conversion:

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Processors;

builder.Services.AddImageSharp()
    .RemoveProcessor<FormatWebProcessor>();
```

Use Presets Instead of Free-Form Query Strings

`PresetOnlyQueryCollectionRequestParser` is the built-in alternative to the normal query parser. Instead of reading every query-string command, it reads a single `preset` key and expands that to a predefined command set.

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Commands;

builder.Services.AddImageSharp()
    .SetRequestParser<PresetOnlyQueryCollectionRequestParser>()
    .Configure<PresetOnlyQueryCollectionRequestParserOptions>(options =>
    {
        options.Presets["thumb"] = "width=160&height=160&rmode=crop";
        options.Presets["card"] = "width=640&height=360&rmode=crop&format=webp&quality=75";
    });
```

That turns requests like `/images/photo.jpg?preset=thumb` into a controlled, named command set without exposing arbitrary query-string processing.

Middleware Callbacks

`ImageSharpMiddlewareOptions` also exposes targeted callbacks for app-specific customization:

- `OnParseCommandsAsync` runs after a provider has matched the request and after the command set has been sanitized, but before the source image is resolved.
- `OnBeforeLoadAsync` can return custom `DecoderOptions` before the source image is decoded. If it returns `null`, the middleware supplies defaults based on the current `Configuration`.
- `OnBeforeSaveAsync` can adjust the `FormattedImage` after processing but before encoding.
- `OnProcessedAsync` receives an `ImageProcessingContext` after encoding but before the result is cached.
- `OnPrepareResponseAsync` runs after status code and headers are set but before the body is written.

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Middleware;

builder.Services.AddImageSharp(options =>
{
    Func<ImageCommandContext, Task> defaultParse = options.OnParseCommandsAsync;

    options.OnParseCommandsAsync = async context =>
    {
        await defaultParse(context);

        if (!context.Commands.Contains("format"))
        {
            context.Commands["format"] = "webp";
        }

    };

    options.OnPrepareResponseAsync = context =>
    {
        context.Response.Headers["X-ImageSharp"] = "true";
        return Task.CompletedTask;
    };
});
```

These callbacks are often the right tool when you need small workflow adjustments without inventing a custom provider, parser, or processor. If you override `OnParseCommandsAsync`, preserve the existing delegate unless you intentionally want to remove the middleware's default `autoorient=true` insertion.

Related Topics

- Getting Started
- Processing Commands
- Securing Requests
- Extensibility

Practical Guidance

Most ImageSharp.Web configuration changes affect one stage of the request pipeline: command parsing, source resolution, image processing, encoding, caching, or response behavior. Change the stage that owns the behavior you need, and keep the others boring. For example, a provider should resolve source images; it should not also reinterpret resize commands. A parser should shape the command collection; it should not open streams.

Be careful when replacing callbacks. The default `OnParseCommandsAsync` inserts `autoorient=true` when the request does not specify orientation behavior, so replacing it without preserving the existing delegate also changes default output. That can be correct for passthrough scenarios, but it should be a deliberate compatibility decision.

For public URLs, presets are often a better product surface than arbitrary query strings. They limit the transformation vocabulary, simplify HMAC signing, reduce cache explosion, and make generated variants easier to reason about. When you do allow free-form commands, keep the middleware ImageSharp configuration aligned with your encoder and ICC expectations so a URL means the same thing across deployments.

Processing Commands

ImageSharp.Web ships with a small set of built-in processors that cover the most common web-image tasks: resize, EXIF-aware orientation, format conversion, quality control, and alpha flattening. By default those commands come from the query string, but the same processors also work with custom request parsers or Razor tag helpers.

How Command Execution Works

The default `QueryCollectionRequestParser` reads query-string pairs into an ordered `CommandCollection`. A few details are worth knowing:

- If the same command key appears more than once, the last value wins.
- Unknown commands are stripped before HMAC validation and before the processor pipeline runs.
- If `autoorient` is absent, the default middleware callback inserts `autoorient=true` before processing continues.
- Processors run in the order their first recognized command appears in the request, not in a hard-coded global order.
- Values are parsed with invariant culture by default. If you turn that off, parsing follows `CultureInfo.CurrentCulture`.

That ordering rule is important when you add custom processors. A processor should declare the command keys that activate it, and callers should build URLs in the order they want the pipeline to run. For example, resizing before a custom watermark processor is not the same as watermarking before resizing.

Command parsing and HMAC validation are also connected. Unknown commands are removed before the final command collection is validated and executed, so a signed URL only protects the command surface the application actually recognizes. If you need a locked-down public API, combine HMAC with presets or a custom parser that exposes only the transformations you want users to request.

Resize

Resize commands are handled by `ResizeWebProcessor` and map to `ResizeOptions`.

```
/images/photo.jpg?width=300
/images/photo.jpg?width=300&height=200&rmode=crop
/images/photo.jpg?width=300&height=200&rmode=pad&rcolor=limegreen
```

```
/images/photo.jpg?width=300&height=200&rxy=0.37,0.78
/images/photo.jpg?width=300&rsampler=lanczos3&compand=true
```

- `width` and `height` set the target dimensions in pixels. If you provide only one dimension, the original aspect ratio is preserved.
- `rmode` selects the `ResizeMode`. Common values are `crop`, `pad`, `boxpad`, `max`, `min`, `stretch`, and `manual`.
- `ranchor` selects the `AnchorPositionMode`. Valid values are `center`, `top`, `bottom`, `left`, `right`, `topleft`, `topright`, `bottomright`, and `bottomleft`.
- `rxy` supplies an exact focal point as `x,y`, where both values are between `0` and `1`.
- `rcolor` sets the pad color for resize modes that add canvas area.
- `rsampler` selects the resampler. Built-in keywords are `bicubic`, `nearest`, `box`, `mitchell`, `catmull`, `lanczos2`, `lanczos3`, `lanczos5`, `lanczos8`, `welch`, `robidoux`, `robidouxsharp`, `spline`, `triangle`, and `hermite`.
- `orient` defaults to `true` and changes how resize interprets EXIF rotation when mapping dimensions, anchors, and focal points. It does not physically rotate the pixels.
- `compand` toggles linear-light companding during the resize.

`orient` is easy to confuse with `autoorient`. The short version is that `orient` only changes resize math, while `autoorient` actually rotates or flips the decoded image.

For responsive-image URLs, prefer specifying only the dimension that is actually constrained by layout. Use both `width` and `height` only when the output must occupy an exact box, then choose the `rmode` that matches the design: `max` to fit inside, `crop` to fill, `pad` to preserve everything with extra canvas, and `stretch` only when distortion is acceptable.

Auto-Orient

`AutoOrientWebProcessor` applies EXIF orientation to the decoded image before later processors run.

```
/images/photo.jpg?autoorient=true
/images/photo.jpg?autoorient=true&width=300&height=200&rmode=crop
/images/photo.jpg?autoorient=false
```

ImageSharp.Web behaves as though `autoorient=true` were present unless you explicitly provide `autoorient=false`. That means processed output is EXIF-normalized by default, which avoids format- and browser-specific orientation inconsistencies during web delivery.

Use `autoorient=false` only when you intentionally want to preserve the original pixel orientation and rely on EXIF metadata downstream.

Format

`FormatWebProcessor` switches the encoder used for the response and cached output.

```
/images/logo.png?format=jpg
/images/logo.png?format=webp
/images/logo.png?width=300&format=gif
```

Any file extension registered with the active `ImageFormatsManager` can be used here. The exact set therefore depends on the underlying ImageSharp configuration.

The selected format uses the encoder currently registered in `ImageSharpMiddlewareOptions.Configuration`. With the default middleware configuration, that means `format=jpg`, `format=png`, and `format=webp` all use web-oriented encoder settings rather than the raw ImageSharp library defaults.

Quality

`QualityWebProcessor` controls encoder quality for JPEG and WebP output.

```
/images/photo.jpg?quality=90
/images/photo.jpg?format=jpg&quality=42
/images/photo.jpg?format=webp&quality=75
```

Quality values are clamped by the target encoder. For WebP, values below `100` switch the encoder to lossy mode.

When no `quality` command is supplied, the default middleware configuration still encodes JPEG and WebP at quality `75`. If you replace `ImageSharpMiddlewareOptions.Configuration`, you also change those no-query defaults.

Background Color

`BackgroundColorWebProcessor` fills transparent areas with a color.

```
/images/logo.png?bgcolor=FFFF00
/images/logo.png?bgcolor=C1FF0080
/images/logo.png?bgcolor=red
```

```
/images/logo.png?bgcolor=128,64,32
/images/logo.png?bgcolor=128,64,32,16
```

This is most useful when flattening transparent images before converting them to opaque formats such as JPEG:

```
/images/logo.png?bgcolor=white&format=jpg&quality=85
```

If `bgcolor` is omitted and the output format cannot represent alpha, transparent pixels must still be resolved by the encoder or format behavior. Set the background color explicitly when brand colors, UI previews, or predictable JPEG output matter.

Related Topics

- Configuration and Pipeline
- Securing Requests
- Tag Helpers
- Extensibility

Practical Guidance

Command order is part of the processing contract. ImageSharp.Web runs processors in the order their first recognized command appears, so URL generation should be treated like pipeline construction. If a custom watermark should happen after resize, generate the resize command first and the watermark command later.

For normal web delivery, leave auto-orientation enabled unless preserving raw source orientation is intentional. For transparent images converted to opaque formats, specify a background color explicitly so output is predictable across encoders and future defaults.

Before exposing command URLs publicly, decide whether clients should have a free-form transformation API. HMAC protects generated URLs, while presets reduce the command surface itself. Many applications want both: presets for a small public vocabulary and signing to prevent tampering.

Image Providers

Image providers answer one question: where does the source image come from? Every incoming request is offered to the registered providers in order, and the first provider whose `Match` function returns `true` owns the request.

That means provider order matters. If two providers can both match the same path, put the more specific one first or narrow its `Match` predicate so the wrong provider does not claim the request.

Default Physical Filesystem Provider

`PhysicalFileSystemProvider` is the default source provider registered by `AddImageSharp()`.

- It resolves images from the web root by default.
- `PhysicalFileSystemProviderOptions.ProviderRootPath` can be `null`, absolute, or relative to the application content root.
- `PhysicalFileSystemProviderOptions.ProcessingBehavior` still defaults to `ProcessingBehavior.CommandOnly`, but the default middleware callback injects `autoorient=true` when the request does not already contain `autoorient`, so local image requests are usually processed anyway.

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Providers;

builder.Services.AddImageSharp()
    .Configure<PhysicalFileSystemProviderOptions>(options =>
    {
        options.ProviderRootPath = "assets";
        options.ProcessingBehavior = ProcessingBehavior.CommandOnly;
    });
```

If you want a provider fixed to `IWebHostEnvironment.WebRootFileProvider` with no extra options, `WebRootImageProvider` is also available.

If you want truly command-only processing for local files, changing `ProcessingBehavior` is no longer sufficient on its own. You must also replace or suppress the default `OnParseCommandsAsync` behavior that inserts `autoorient=true`.

Provider Matching and Ordering

ImageSharp.Web stops at the first provider whose `Match` function returns `true`. It does not continue searching if that provider later decides the request is invalid, so keep these rules in mind:

- Register more specific providers before more general ones.
- Keep `Match` predicates mutually exclusive whenever possible.
- Use `InsertProvider(...)` when provider precedence matters more than registration order.

Cloud providers in particular usually want a path prefix such as a container or bucket name so they can distinguish their requests cheaply.

Azure Blob Storage

Install the Azure provider package:

```
dotnet add package SixLabors.ImageSharp.Web.Providers.Azure
```

Then configure one or more containers:

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Azure.Providers;

builder.Services.AddImageSharp()
    .ClearProviders()
    .Configure<AzureBlobStorageImageProviderOptions>(options =>
    {
        options.BlobContainers.Add(new AzureBlobContainerClientOptions
        {
            ConnectionString = builder.Configuration["Azure:ConnectionString"]!,
            ContainerName = "public-images"
        });
    })
    .AddProvider<AzureBlobStorageImageProvider>();
```

Requests are matched by container name at the start of the path:

```
/public-images/avatars/jane.png?width=200
```

`AzureBlobStorageImageProvider` uses `ProcessingBehavior.All`, so it can serve both processed and commandless requests.

AWS S3

Install the AWS provider package:

```
dotnet add package SixLabors.ImageSharp.Web.Providers.AWS
```

Then configure one or more buckets:

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.AWS.Providers;

builder.Services.AddImageSharp()
    .ClearProviders()
    .Configure<AWSS3StorageImageProviderOptions>(options =>
    {
        options.S3Buckets.Add(new AWSS3BucketClientOptions
        {
            BucketName = "public-images",
            Region = "us-east-1",
            AccessKey = builder.Configuration["AWS:AccessKey"],
            AccessSecret = builder.Configuration["AWS:SecretKey"]
        });
    })
    .AddProvider<AWSS3StorageImageProvider>();
```

Requests are matched by bucket name at the start of the path:

```
/public-images/avatars/jane.png?width=200
```

If your public URL shape does not naturally include the bucket name, use URL rewriting before ImageSharp.Web or implement a custom provider.

Implementing Your Own Provider

Implement `IImageProvider` when you need a new source backend. Your provider is responsible for three things:

- deciding whether it owns the request via `Match`;
- deciding whether the request is valid via `IsValidRequest(...)`;
- returning an `IImageResolver` that can open the source stream and report source metadata.

If your source already fits an `IFileProvider`-style model, `FileProviderImageProvider` is the easiest base class to start from.

Related Topics

- Getting Started
- Image Caches
- Extensibility
- Troubleshooting

Practical Guidance

- Put providers in the order you want requests to be matched.
- Keep original source storage separate from processed cache storage.
- Return accurate source metadata so stale cache detection works.
- Implement a custom provider only when existing filesystem, Azure, or S3 providers do not match the source model.

Image Caches

ImageSharp.Web caches processed output so that identical requests do not repeatedly decode, process, and re-encode the source image. The cache stores both the encoded bytes and metadata about the source and response so the middleware can detect stale entries and serve correct headers.

Think of the cache as derived output, not source-of-truth storage. It should be safe to clear and rebuild, but it must be configured carefully enough that all application instances agree on keys, freshness, and storage location.

How the Cache Works

For each processed request, the middleware:

- builds a cache key from the request path plus the sanitized command collection;
- hashes that key into a filesystem-safe cache name;
- stores the encoded image plus metadata such as source last-write time, cache write time, content type, browser max-age, and content length;
- reuses the cached result until the source changes or the cache entry ages beyond `ImageSharpMiddlewareOptions.CacheMaxAge`.

Default Physical Cache

`PhysicalFileSystemCache` is the default backend registered by `AddImageSharp()`.

- It stores cached files under the web root by default.
- `PhysicalFileSystemCacheOptions.CacheFolder` defaults to `is-cache`.
- `PhysicalFileSystemCacheOptions.CacheFolderDepth` defaults to `8`, which spreads cached files across nested folders.

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Caching;

builder.Services.AddImageSharp()
    .Configure<PhysicalFileSystemCacheOptions>(options =>
    {
        options.CacheRootPath = "cache";
        options.CacheFolder = "imagesharp";
```

```
        options.CacheFolderDepth = 8;
    });
```

If your app does not define a web root, set `CacheRootPath` explicitly. Relative paths are resolved against the application content root.

Browser Lifetime Versus Backend Lifetime

ImageSharp.Web tracks two different lifetimes:

- `ImageSharpMiddlewareOptions.BrowserMaxAge` controls the `Cache-Control` lifetime sent to clients.
- `ImageSharpMiddlewareOptions.CacheMaxAge` controls how long the processed result stays valid in the backend cache.

If the source provider supplies a source `Cache-Control` max-age, that value overrides `BrowserMaxAge` for the response.

Set browser lifetime based on how long clients may keep a response without revalidation. Set backend cache lifetime based on how long your server-side derived output should be trusted before checking the source again. Those are related, but they are not the same operational decision.

Cache Keys and Hashes

By default, ImageSharp.Web uses:

- `UriRelativeLowerInvariantCacheKey` to turn the request path and command collection into a canonical cache key.
- `SHA256CacheHash` to hash that key into the stored filename.
- `ImageSharpMiddlewareOptions.CacheHashLength` to control how many hash characters are kept.

If you need cache entries to vary by host or some other request detail, swap the key implementation with `SetCacheKey<T>()` or `SetCacheHash<T>()`:

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Caching;

builder.Services.AddImageSharp(options =>
{
    options.CacheHashLength = 16;
})
```

```
.SetCacheKey<UriAbsoluteLowerInvariantCacheKey>()
.SetCacheHash<SHA256CacheHash>();
```

Preserve the v1 Cache Layout

If you are migrating an older installation and want new requests to keep using the v1 cache naming layout, switch to `LegacyV1CacheKey` and keep the folder depth aligned with the hash length:

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Caching;

builder.Services.AddImageSharp(options =>
{
    options.CacheHashLength = 12;
})
.Configure<PhysicalFileSystemCacheOptions>(options =>
{
    options.CacheFolderDepth = 12;
})
.SetCacheKey<LegacyV1CacheKey>();
```

Azure Blob Storage Cache

Install the Azure provider package:

```
dotnet add package SixLabors.ImageSharp.Web.Providers.Azure
```

Then replace the default cache backend with `SetCache<T>()`:

```
using Azure.Storage.Blobs.Models;
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Azure.Caching;

builder.Services.AddImageSharp()
    .Configure<AzureBlobStorageCacheOptions>(options =>
    {
        options.ConnectionString = builder.Configuration["Azure:ConnectionString"]!;
        options.ContainerName = "imagesharp-cache";
        options.CacheFolder = "processed";

        AzureBlobStorageCache.CreateIfNotExists(options, PublicAccessType.None);
```

```
    })
    .SetCache<AzureBlobStorageCache>();
```

Cached objects use the hashed request key as the blob name, and the cache metadata is stored in blob properties alongside the object.

AWS S3 Cache

Install the AWS provider package:

```
dotnet add package SixLabors.ImageSharp.Web.Providers.AWS
```

Then replace the default cache backend with `SetCache<T>()`:

```
using Amazon.S3;
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.AWS.Caching;

builder.Services.AddImageSharp()
    .Configure<AWSS3StorageCacheOptions>(options =>
    {
        options.BucketName = "imagesharp-cache";
        options.Region = "us-east-1";
        options.AccessKey = builder.Configuration["AWS:AccessKey"];
        options.AccessSecret = builder.Configuration["AWS:SecretKey"];
        options.CacheFolder = "processed";

        AWSS3StorageCache.CreateIfNotExists(options, S3CannedACL.Private);
    })
    .SetCache<AWSS3StorageCache>();
```

Cached objects use the hashed request key as the object key, and the response metadata needed by the middleware is stored with the object.

Practical Guidance

Treat the cache as derived output. It should be safe to clear and rebuild, but it must be separate from source storage so cleanup jobs cannot delete originals. In single-instance deployments a physical cache may be enough; in multi-instance deployments, use shared storage when all instances should reuse the same processed variants.

Cache lifetime has two audiences. Browser lifetime controls how long clients may reuse a response without coming back. Backend cache lifetime controls how long the server trusts a generated variant before checking source freshness. Those values should match source update frequency, CDN behavior, and the cost of regeneration.

If you customize cache keys, include every output-affecting request detail. Host, tenant, preset expansion, command values, and source path can all matter depending on the application. Monitor cache growth when public URLs expose many dimensions or quality values, because variant counts can grow faster than source image counts.

Related Topics

- Getting Started
- Image Providers
- Securing Requests
- Extensibility

Securing Requests

Once you let clients describe image transformations in the URL, you usually want some control over who can generate those URLs and which command shapes are allowed. ImageSharp.Web gives you two main tools for that: HMAC signing for request authorization and preset-only parsing for fixed command sets.

Require HMAC Tokens for Command Requests

Set `ImageSharpMiddlewareOptions.HMACSecretKey` to enable request signing:

```
using SixLabors.ImageSharp.Web;

builder.Services.AddImageSharp(options =>
{
    options.HMACSecretKey = Convert.FromBase64String(
        builder.Configuration["ImageSharp:HmacKey"]!);
});
```

Once a non-empty secret key is configured, ImageSharp command URLs must also include a matching `hmac` query parameter. If the token is missing or invalid, the middleware returns HTTP 400.

Use one stable secret across all app instances that must validate the same URLs. Rotating the secret invalidates previously generated signed URLs.

Do not put the HMAC secret in source control or client-side code. Treat it like any other server-side signing secret and load it from configuration, environment variables, or a secret store.

How the Default Token Is Computed

By default, ImageSharp.Web computes HMAC-SHA256 over a lower-invariant relative URL built from:

- the request path base;
- the request path;
- the sanitized command collection.

That behavior is important because the middleware strips unknown commands before validation. The easiest way to stay in sync with the server is to let ImageSharp.Web compute the token for

you instead of re-implementing the canonicalization rules yourself.

If a proxy, CDN, or URL generator rewrites paths or query strings, make sure it preserves the canonical URL shape used to compute the token. A harmless-looking path-base or casing change can invalidate signatures.

Generate Signed URLs on the Server

`RequestAuthorizationUtilities` is the simplest server-side API for generating a valid token:

```
using SixLabors.ImageSharp.Web;

RequestAuthorizationUtilities auth =
    app.Services.GetRequiredService<RequestAuthorizationUtilities>();

string path = "/images/hero.jpg?width=400&format=webp";
string token = auth.ComputeHMAC(path, CommandHandling.Sanitize)!;
string signedPath = $"{path}&{RequestAuthorizationUtilities.TokenCommand}={token}";
```

Use `CommandHandling.Sanitize` unless you have a very specific reason to hash unsanitized commands. That keeps token generation aligned with the middleware's own validation path.

Customize the Hash Algorithm or Canonicalization

If the default HMAC-SHA256 plus lower-invariant relative URL is not the contract you want, override `OnComputeHMAC`:

```
using Microsoft.AspNetCore.Http;
using SixLabors.ImageSharp.Web;

builder.Services.AddImageSharp(options =>
{
    options.OnComputeHMAC = (context, secret) =>
    {
        string uri = CaseHandlingUriBuilder.BuildRelative(
            CaseHandlingUriBuilder.CaseHandling.LowerInvariant,
            context.Context.Request.PathBase,
            context.Context.Request.Path,
            QueryString.Create(context.Commands));
```

```
        return HMACUtilities.ComputeHMACSHA512(uri, secret);
    };
});
```

If you change the canonicalization rules or hash algorithm, every URL generator in your system must use the same logic.

Let Razor Tag Helpers Add the Token

If you are rendering image URLs in Razor, the built-in tag helpers can generate the token automatically once `HMACSecretKey` is configured. See Tag Helpers for the Razor setup and examples.

Use Presets to Limit the Exposed Command Surface

If you do not want clients to submit arbitrary commands at all, switch to `PresetOnlyQueryCollectionRequestParser`:

```
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Commands;

builder.Services.AddImageSharp()
    .SetRequestParser<PresetOnlyQueryCollectionRequestParser>()
    .Configure<PresetOnlyQueryCollectionRequestParserOptions>(options =>
    {
        options.Presets["avatar"] = "width=128&height=128&rmode=crop&format=webp";
        options.Presets["card"] = "width=640&height=360&rmode=crop";
    });
```

That makes requests look like this:

```
/images/user.jpg?preset=avatar
```

Only the named preset is expanded into commands. Other free-form query-string keys are ignored by that parser. You can combine presets with HMAC signing if you want both a small command surface and signed URLs.

Practical Guidance

HMAC signing proves that a URL was generated by code that knows the server-side secret. It does not, by itself, decide whether the command surface is a good product API. For public endpoints, decide first which transformations clients should be allowed to request. If the answer is a small set of known variants, presets are usually clearer and safer than exposing free-form `width`, `height`, `quality`, and `format` combinations.

Keep HMAC keys server-side and load them from configuration or a secret store. Rotating the key invalidates existing signed URLs, so treat rotation as a deployment event and plan cache/CDN behavior around it. Use HTTPS so signed URLs and source paths are not exposed or altered in transit.

Generate signatures with ImageSharp.Web utilities instead of reimplementing canonicalization. The token is computed over the sanitized command collection and relative URL shape; reverse proxies, CDNs, path-base changes, and query-string rewriting can all affect validation. Test the production URL path, including proxy rewriting, before assuming a signed URL generated locally will validate after deployment.

Related Topics

- Configuration and Pipeline
- Processing Commands
- Tag Helpers
- Troubleshooting

Tag Helpers

ImageSharp.Web includes Razor tag helpers so you can build image-processing URLs in strongly typed server-side markup instead of hand-concatenating query strings. The tag helpers also integrate with HMAC signing, which makes them the easiest way to generate safe image URLs in MVC and Razor Pages apps.

Enable the Tag Helpers

Add the ImageSharp namespaces and tag helper registration to `_ViewImports.cshtml`:

```
@using SixLabors.ImageSharp
@using SixLabors.ImageSharp.Processing
@using SixLabors.ImageSharp.Web

@addTagHelper *, SixLabors.ImageSharp.Web
```

That enables both `ImageTagHelper` and `HmacTokenTagHelper`.

Generate Command URLs with `ImageTagHelper`

`ImageTagHelper` targets `<img>` elements and converts `imagesharp-*` attributes into the corresponding query-string commands:

```
<img
    src="images/hero.png"
    imagesharp-width="400"
    imagesharp-height="250"
    imagesharp-rmode="ResizeMode.Crop"
    imagesharp-format="Format.WebP"
    imagesharp-quality="75"
    alt="Hero image" />
```

That renders a `src` roughly like this:

```
images/hero.png?width=400&height=250&rmode=Crop&format=webp&quality=75
```

The built-in typed values mirror the processor APIs:

- Use `ResizeMode` and `AnchorPositionMode` for resize modes and anchors.
- Use `Color` for `imagesharp-rcolor` and `imagesharp-bgcolor`.
- Use `Format` for common output formats such as `Format.Jpg` and `Format.WebP`.
- Use `Resampler` for common resamplers such as `Resampler.Lanczos3` and `Resampler.NearestNeighbor`.

The supported built-in attributes are:

- `imagesharp-width`, `imagesharp-height`, `imagesharp-rmode`, `imagesharp-ranchor`, `imagesharp-rxy`, `imagesharp-rcolor`, `imagesharp-rsampler`, `imagesharp-compand`, and `imagesharp-orient` for resize behavior.
- `imagesharp-autoorient` for explicitly controlling EXIF-based rotation and flipping.
- `imagesharp-format` for output format selection.
- `imagesharp-bgcolor` for flattening transparency.
- `imagesharp-quality` for JPEG and WebP quality.

If the `<img>` element does not already have literal `width` and `height` attributes, `ImageTagHelper` also writes them to the markup from the processing dimensions. That helps avoid layout shift for simple resize scenarios.

Because the middleware injects `autoorient=true` by default, you usually do not need `imagesharp-autoorient="true"` just to get correctly oriented output. The main reason to set the attribute explicitly is to opt out with `imagesharp-autoorient="false"` or to make the behavior explicit in markup.

Local URLs Versus External URLs

`ImageTagHelper` is intended for local application image URLs. It skips `http`, `ftp`, and `data` sources because ImageSharp.Web does not process those through the built-in local-path pipeline.

Automatic HMAC Generation

`HmacTokenTagHelper` runs on `<img src="...">` and appends the `hmac` command automatically when `ImageSharpMiddlewareOptions.HMACSecretKey` is configured and the final `src` contains recognized commands.

That means both of these patterns work:

```
<img src="images/avatar.jpg?width=128&height=128&rmode=Crop" />

<img
    src="images/avatar.jpg"
```

```
    imagesharp-width="128"
    imagesharp-height="128"
    imagesharp-rmode="ResizeMode.Crop" />
```

In the first case, `HmacTokenTagHelper` signs your handwritten command URL. In the second case, `ImageTagHelper` generates the command URL and `HmacTokenTagHelper` signs it afterward.

Extending the Tag Helper

`ImageTagHelper` is unsealed. If you add custom processors and want matching Razor syntax, inherit from it and override `AddProcessingCommands(...)` to append your own commands before the final `src` is emitted.

Related Topics

- Processing Commands
- Securing Requests
- Extensibility

Practical Guidance

- Use tag helpers when Razor should generate command URLs instead of hand-built query strings.
- Let the HMAC tag helper sign generated URLs when request signing is enabled.
- Keep custom tag helper commands aligned with custom processors.
- Inspect the emitted `src` during troubleshooting so command order and token generation are visible.

Extensibility

ImageSharp.Web is designed as a set of replaceable layers rather than one monolithic middleware. Most customizations only need one of those layers, so the first job is choosing the lightest extension point that matches your problem.

Choose the Right Extension Point

- Use `OnParseCommandsAsync` when the request shape is already close to what you want and you only need to add, remove, or normalize commands.
- Use `IRequestParser` when the request syntax changes completely.
- Use `IImageWebProcessor` when you need a new image-processing command.
- Use `ICommandConverter` when your processor needs a custom typed command value.
- Use `IImageProvider` and `IImageResolver` when source images come from a new backend.
- Use `IImageCache` and `IImageCacheResolver` when processed output should be stored in a new backend.
- Use `ICacheKey` or `ICacheHash` when only cache naming needs to change.

Choose the narrowest extension point that owns the behavior. A parser should not open source images. A provider should not parse resize commands. A processor should not decide where cached files live. Keeping those boundaries clean makes security, caching, HMAC validation, and diagnostics much easier to reason about.

Add a Custom Processor

Custom processors are the usual way to introduce a new query-string command. Implement `IImageWebProcessor`, parse your command values from the `CommandCollection`, and mutate the `FormattedImage`:

```
using System.Globalization;
using Microsoft.Extensions.Logging;
using SixLabors.ImageSharp.Processing;
using SixLabors.ImageSharp.Web;
using SixLabors.ImageSharp.Web.Commands;
using SixLabors.ImageSharp.Web.Processors;

public sealed class SepiaWebProcessor : IImageWebProcessor
{
    public IEnumerable<string> Commands { get; } = new[] { "sepia" };
```

```
    public FormattedImage Process(
        FormattedImage image,
        ILogger logger,
        CommandCollection commands,
        CommandParser parser,
        CultureInfo culture)
    {
        if (parser.ParseValue<bool>(commands.GetValueOrDefault("sepia"), culture))
        {
            image.Image.Mutate(x => x.Sepia());
        }

        return image;
    }

    public bool RequiresTrueColorPixelFormat(
        CommandCollection commands,
        CommandParser parser,
        CultureInfo culture) => false;
}
```

Register it with `AddProcessor<T>()`:

```
builder.Services.AddImageSharp()
    .AddProcessor<SepiaWebProcessor>();
```

Processor order is driven by the order of the recognized command keys in the request, so custom processors participate in the same ordering model as the built-in ones.

Processors should be deterministic for the same source image and command collection. If a processor depends on external data, include that data in the command surface or cache key strategy; otherwise cached output can become stale or inconsistent.

Custom Command Converters

The built-in converters already cover integral types, floating-point values, booleans, strings, arrays, lists, colors, and enums. If your processor wants a custom command type, implement `ICommandConverter<T>`, register it with `AddConverter<TConverter>()`, then parse it inside the processor with `CommandParser.ParseValue<T>()`.

This is the right place to centralize parsing rules for custom value syntaxes instead of repeating string parsing inside each processor.

Converters should parse request values into stable typed values. Keep validation messages clear, because parse failures normally surface as client-facing bad requests.

Custom Providers and Caches

Implement a custom provider when your source image is not on disk, in Azure Blob Storage, or in S3. A provider owns request matching and returns a resolver that can:

- open the source stream;
- report source last-write and cache metadata;
- decide whether requests use `ProcessingBehavior.CommandOnly` or are always handled.

When the source maps naturally to an `IFileProvider`, `FileProviderImageProvider` is the easiest base class.

Implement a custom cache when processed images should live somewhere other than the built-in physical filesystem cache or the cloud caches. A cache receives the hashed key, encoded stream, and `ImageCacheMetadata`, then later returns an `IImageCacheResolver` that can reopen the cached entry.

If you only need different cache naming rather than a whole new backend, replace `ICacheKey` or `ICacheHash` instead of writing a new cache.

Providers and caches sit on hot request paths. Keep stream ownership explicit, avoid buffering entire images unless the backend requires it, and make cache metadata decisions consistently so conditional requests and stale entries behave predictably.

Replace the Request Syntax

Implement `IRequestParser` when commands should come from somewhere other than the raw query string, for example:

- route values;
- a compact signed token;
- database-backed presets;
- application-specific aliases.

Your parser returns an ordered `CommandCollection`. That order matters because it is what the middleware uses to decide processor execution order.

Extend Razor Integration

If you add custom processors and want equally natural Razor markup, derive from `ImageTagHelper` and override `AddProcessingCommands(...)` to write your custom command keys into the outgoing URL.

That lets your Razor layer stay strongly typed instead of falling back to raw query-string fragments.

Production Checklist

- Decide whether the extension changes request parsing, processing, source resolution, cache storage, or URL generation before choosing an API.
- Keep custom command names stable; changing them invalidates URLs and cache keys.
- Include any output-affecting external state in commands, presets, or cache-key inputs.
- Preserve HMAC and preset restrictions when replacing request parsing.
- Log enough context to diagnose provider misses, parser failures, cache misses, and processor validation errors.

Related Topics

- Configuration and Pipeline
- Processing Commands
- Image Providers
- Image Caches
- Tag Helpers

Troubleshooting

Most ImageSharp.Web problems come down to one of five layers: middleware order, provider matching, command parsing, request signing, or cache configuration. This page groups the common failures that way so you can check the right layer first.

Query Strings Are Ignored

If `/images/photo.jpg?width=400` behaves the same as `/images/photo.jpg`, check these first:

- `app.UseImageSharp()` must run before `app.UseStaticFiles()`.
- If you replaced `OnParseCommandsAsync`, make sure you did not accidentally remove the default `autoorient=true` insertion or other command mutations you rely on.
- A provider may be matching the request before the provider you expected. Provider order matters.

I Get HTTP 400 After Enabling HMAC

Once `ImageSharpMiddlewareOptions.HMACSecretKey` is configured, requests that include ImageSharp commands must include a valid `hmac` token.

Useful checks:

- Generate the token with `RequestAuthorizationUtilities` instead of recreating the canonicalization logic by hand.
- Use `CommandHandling.Sanitize` when generating the token unless you intentionally need unsanitized commands.
- Make sure all app instances share the same secret key.
- Remember that unknown commands are stripped before validation, so signing a URL with extra application-specific keys will not match unless you remove them first or translate them in a custom request parser.

I Get a 404 or the Original Image Instead of a Processed One

That usually means the source image was never resolved by the expected provider.

Check these cases:

- The file is outside the configured `ProviderRootPath`.
- The request path does not include the expected bucket or container prefix for the AWS or Azure providers.
- The source file extension is not recognized by the active ImageSharp format configuration.
- You switched to `PresetOnlyQueryCollectionRequestParser` and the request uses a missing or misspelled `preset` value.

Physical Cache or Provider Root Path Cannot Be Determined

The physical provider and physical cache default to the web root when their root paths are `null`. If your app does not define a web root, configure both explicitly:

- `PhysicalFileSystemProviderOptions.ProviderRootPath`
- `PhysicalFileSystemCacheOptions.CacheRootPath`

Relative paths are resolved against the application content root.

Tag Helpers Do Nothing

If `imagesharp-*` attributes are not changing the rendered `src`, check these first:

- `_ViewImports.cshtml` must contain `@addTagHelper *, SixLabors.ImageSharp.Web`.
- `ImageTagHelper` is for local application URLs and skips `http`, `ftp`, and `data` sources.
- Automatic HMAC generation only happens when `HMACSecretKey` is configured and the final URL contains recognized commands.

Parsed Values Differ Between Machines

By default, ImageSharp.Web parses commands with invariant culture. If you set `UseInvariantParsingCulture` to `false`, separators and decimal parsing follow `CultureInfo.CurrentCulture`.

That is useful for specialized local workflows, but it also means a value like `0.5` versus `0,5` can behave differently across environments.

Images Stopped Auto-Rotating After I Customized `OnParseCommandsAsync`

The default `OnParseCommandsAsync` callback inserts `autoorient=true` when the request does not already contain `autoorient`.

If you assign your own callback without chaining the previous delegate, you remove that default behavior. Preserve the existing callback first unless you intentionally want to opt out:

```
using SixLabors.ImageSharp.Web.Middleware;

Func<ImageCommandContext, Task> defaultParse = options.OnParseCommandsAsync;

options.OnParseCommandsAsync = async context =>
{
    await defaultParse(context);

    // Your additional command mutations here.
};
```

Colors or Compression Changed After I Replaced `options.Configuration`

Check whether you replaced `ImageSharpMiddlewareOptions.Configuration` with `Configuration.Default.Clone()` or another custom configuration.

That changes two things at once:

- You replace the middleware's built-in JPEG, PNG, and WebP encoder registrations.
- If `OnBeforeLoadAsync` still returns `null`, decode falls back to `ColorProfileHandling.Compact` instead of `ColorProfileHandling.Convert`.

If you want custom encoders and the original ICC-conversion behavior, clone the current middleware configuration, assign it back, and return explicit `DecoderOptions` from `OnBeforeLoadAsync`.

Cached Output Does Not Refresh

ImageSharp.Web keeps using a cached result until one of these changes:

- the source last-write time changes;
- the cache entry ages beyond `CacheMaxAge`;
- the cached entry disappears and the middleware has to regenerate it.

If you are replacing source files in place, make sure the backing store actually updates the source last-write metadata the provider sees.

A Good Debugging Order

When an ImageSharp.Web request misbehaves, this order is usually productive:

1. Check middleware order.
2. Confirm which provider should own the request.
3. Confirm the parsed command set or preset name.
4. Check HMAC generation if signing is enabled.
5. Check cache roots, cache lifetime, and source last-write metadata.

Related Topics

- Getting Started
- Configuration and Pipeline
- Securing Requests
- Extensibility

Practical Guidance

- Check middleware order before provider or cache details.
- Confirm the parsed command collection before debugging processor behavior.
- Validate HMAC with the same canonicalization path used by the middleware.
- Separate source misses from cache misses when diagnosing 404s or stale output.

Introduction

What is Fonts?

Fonts is the high-performance part of the Six Labors stack that handles font loading, text measurement, layout, shaping, and custom text rendering.

If you are new to the library, the easiest way to think about it is in layers: load families, create concrete `Font` instances, then measure or render text with `TextOptions`. The rest of this section is organized around that path so you can start simple and move into shaping, Unicode, fallback, and custom rendering only when you need them.

It supports TrueType and OpenType fonts, including CFF1 and CFF2 outlines, WOFF and WOFF2 web fonts, variable fonts, color fonts, advanced OpenType layout, complex script shaping, and bidirectional text rendering.

Fonts is often used underneath ImageSharp.Drawing, but it is not limited to image rendering. You can also use it for font inspection, text measurement, shaping, and custom rendering pipelines.

The main thing to learn early is the difference between font assets, font instances, and text layout. A font collection tells you what families are available, a `Font` chooses a family/style/size, and `TextOptions` describes how a specific piece of text should be shaped, wrapped, aligned, measured, or rendered. `TextBlock` builds on that by preparing layout once so you can inspect lines, hit-test, move carets, or draw the same shaped text consistently.

The Unicode pages are part of the practical API story, not a side topic. Most real text is not one UTF-16 code unit per visible character, and indexes used for rich text, placeholders, selection, and hit testing need to be understood in terms of graphemes and shaped layout rather than raw `char` positions.

License

Fonts is licensed under the terms of the Six Labors Split License, Version 1.0. See https://sixlabors.com/pricing for commercial licensing details.

> **IMPORTANT**
>
> Projects that directly reference Fonts require a valid Six Labors license key at build time from version 2.2.0 in the previous major and 3.0.0 in the current major. Enforcement applies to direct dependencies only. This is enforcement of the existing license terms, not a license change.

Installation

Fonts is installed via NuGet with nightly builds available on Feedz.

Package Manager | .NET CLI | PackageReference | Paket CLI

```
PM > Install-Package SixLabors.Fonts -Version VERSION_NUMBER
```

Prerelease versions installed via the Visual Studio NuGet Package Manager require the "include prerelease" checkbox to be checked.

How to use the license file

By default, the build searches from each project directory for `sixlabors.lic`. Place the supplied file in the directory that contains the project file, or in a subdirectory below it. Use the file as supplied; it contains the complete license string required by the build.

> **IMPORTANT**
>
> Assembly-scoped community keys can be committed to a public repository. Contributors can use the included key to build the named assemblies without obtaining their own keys. Unrestricted community keys and commercial keys must remain private. Use environment variables or repository secrets for those keys.

To keep the file elsewhere, including a repository root above the project directory, set `SixLaborsLicenseFile` in your project file or a shared props file:

```
<PropertyGroup>
  <!-- Set the location of the supplied license file. -->
  <SixLaborsLicenseFile>path/to/sixlabors.lic</SixLaborsLicenseFile>
</PropertyGroup>
```

To use an environment variable instead, set this project property:

```
<PropertyGroup>
  <!-- Read the complete license string from the environment. -->
  <SixLaborsLicenseKey>$(SIXLABORS_LICENSE_KEY)</SixLaborsLicenseKey>
</PropertyGroup>
```

Set `SIXLABORS_LICENSE_KEY` to the complete contents of your issued license, not only its `Key` field. Use the original value without escaping. Replace the entire example value below with your issued license contents.

PowerShell:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
$env:SIXLABORS_LICENSE_KEY = 'Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

Bash:

```
# Quotes preserve the field separators and the pipe between assembly signatures.
export SIXLABORS_LICENSE_KEY='Id=<id>;Kind=Community;ExpiryDateUtc=<expiry>;Key=Assembly:
<signature>|<signature>'
dotnet build
dotnet publish
```

If you pass the license directly through `-p:SixLaborsLicenseKey`, escape every semicolon as `%253B`. MSBuild treats semicolons as property separators even inside a quoted argument. The pipe between assembly signatures stays unchanged.

PowerShell:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
```

```
dotnet publish "-p:SixLaborsLicenseKey=$($env:SIXLABORS_LICENSE_KEY.Replace(';', '%253B'))"
```

Bash:

```
# Escape field separators only when passing the license through -p:.
dotnet build "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
dotnet publish "-p:SixLaborsLicenseKey=${SIXLABORS_LICENSE_KEY//;/%253B}"
```

You do not need to reference the licensing package directly. Projects that directly reference a Six Labors package carry its build-time validation.

Community keys for public projects

For an open-source or source-available application, provide the exact `AssemblyName` of every project that directly references a Six Labors package, including test projects. Names are case-sensitive.

For example, if `MyProject.csproj` sets `<AssemblyName>MyProject.Core</AssemblyName>`, enter `MyProject.Core`. If it does not set `AssemblyName`, enter `MyProject`, the project filename without `.csproj`. If a shared props file sets the name, use that value. Do not include `.dll` or `.csproj`.

Existing community license holders can request one early replacement with an assembly-scoped key. These upgrade requests are approved automatically, without another eligibility review. You do not need to wait for your current key to enter its renewal window.

Apply at licensing.sixlabors.com. Select open-source or source-available, use the same email address, and provide the assembly names. Later requests follow the normal renewal window: the final 90 days before the expiry date in your license. Commercial licenses are not eligible for this early replacement.

Security support

The latest major version of each library is eligible for security updates. Each major remains eligible for 12 months after the first stable release of its successor. A later major release does not shorten an existing support window.

Install the latest available patch or minor release within a supported major to receive security fixes. Older releases within that major are not maintained separately. This policy covers security fixes only. It does not include feature backports or extend Microsoft's support for the underlying .NET version. Security fixes remain at Six Labors' discretion.

Start Here

If you are new to Fonts, start with Loading Fonts and Collections and then use the pages below to branch into the topics your application needs.

- System Fonts
- Font Metadata and Inspection
- Font Metrics
- Measuring Text
- Prepared Text with TextBlock
- Hit Testing and Caret Movement
- Selection and Bidi Drag
- Text Layout and Options
- OpenType Features
- Text Shaping
- Font Hinting
- Color Fonts
- Unicode, Code Points, and Graphemes
- Fallback Fonts and Multilingual Text
- Variable Fonts
- Custom Rendering
- Recipes
- Troubleshooting

How to Use These Docs

- Start with font loading and measurement before moving into shaping, fallback, and rendering.
- Use the Unicode pages whenever text ranges, caret movement, styling, or placeholders are involved.
- Use `TextBlock` pages when layout must be measured, inspected, interacted with, and rendered consistently.
- Use custom rendering only after the layout model is clear.

Loading Fonts and Collections

The quickest way to get comfortable with Fonts is to separate three ideas: where fonts come from, how a family becomes a concrete font instance, and how that font is later used for measurement or rendering. This page walks through that path before the more advanced layout guides.

The main types you will meet first are:

- `FontCollection` stores the families you load.
- `FontFamily` represents a family and the styles available for it.
- `Font` represents a concrete instance of a family at a given point size, style, and optional variation settings.
- `SystemFonts` gives you access to the fonts installed on the current machine.

Load a single font

Use `FontCollection.Add(...)` when you want to register an individual font file such as a `.ttf`, `.otf`, `.woff`, or `.woff2`.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/SourceSans3-Regular.ttf");
Font font = family.CreateFont(16, FontStyle.Regular);
```

`Font.Size` is expressed in points. Measurement and rendering are then converted to pixels using `TextOptions.Dpi`.

Load from a stream and inspect metadata

The `Add(...)` overloads can also return a `FontDescription`, which is useful when you want to inspect what was loaded.

```
using System.IO;
using SixLabors.Fonts;

FontCollection collection = new();

using FileStream stream = File.OpenRead("fonts/SourceSans3-Regular.ttf");
```

```
FontFamily family = collection.Add(stream, out FontDescription description);

string familyName = description.FontFamilyInvariantCulture;
Font font = family.CreateFont(16);
```

If you only need metadata, use `FontDescription.LoadDescription(...)` or `FontDescription.LoadFontCollectionDescriptions(...)` instead of adding the font to a collection. See Font Metadata and Inspection for more detail.

Load a font collection

Use `AddCollection(...)` for files that contain multiple faces, such as `.ttc` collections.

```
using SixLabors.Fonts;

FontCollection collection = new();
var families = collection.AddCollection("fonts/NotoSansCJK-Regular.ttc");
```

Resolve families by name

Once fonts are loaded, resolve a family with `Get(...)` or `TryGet(...)`.

```
using SixLabors.Fonts;

FontCollection collection = new();
collection.Add("fonts/SourceSans3-Regular.ttf");
collection.Add("fonts/NotoColorEmoji-Regular.ttf");

if (collection.TryGet("Source Sans 3", out FontFamily textFamily) &&
    collection.TryGet("Noto Color Emoji", out FontFamily emojiFamily))
{
    TextOptions options = new(textFamily.CreateFont(16))
    {
        FallbackFontFamilies = [emojiFamily]
    };
}
```

`FallbackFontFamilies` is a list of `FontFamily` instances, not `Font` instances. Fonts are created after the fallback family is selected for a run.

Use system fonts

If you want to work with fonts installed on the current machine, use `SystemFonts`.

```
using SixLabors.Fonts;

Font caption = SystemFonts.CreateFont("Segoe UI", 12);
Font heading = SystemFonts.CreateFont("Segoe UI", 24, FontStyle.Bold);
```

Replace `"Segoe UI"` with any installed family that exists on your machine.

You can also merge the system font set into your own `FontCollection`.

```
using SixLabors.Fonts;

FontCollection collection = new();
collection.AddSystemFonts();
collection.Add("fonts/BrandSans-Regular.ttf");
```

When you need localized family-name lookup, use `AddWithCulture(...)`, `GetByCulture(...)`, or `TryGetByCulture(...)`.

See System Fonts for the fuller system-font API surface, including enumeration, culture-aware lookup, and `SearchDirectories`.

Create variable-font instances

Variable fonts are exposed through `FontVariation` and `KnownVariationAxes`.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/RobotoFlex.ttf");

Font font = family.CreateFont(
    16,
    new FontVariation(KnownVariationAxes.Weight, 700),
    new FontVariation(KnownVariationAxes.OpticalSize, 16));
```

The active variation values become part of the `Font` instance, so the same family can be reused to create multiple design-space instances.

Practical guidance

- Use a private `FontCollection` when output must be stable across machines.
- Use `SystemFonts` for host-dependent behavior such as diagnostics, user font pickers, or "use what is installed here" workflows.
- Create `Font` instances for the size, style, culture, and variation values you actually intend to measure or render.
- Keep font loading separate from per-request or per-frame layout work when the same files are reused.

Next steps

- Use Measuring Text when you need layout metrics before rendering.
- Use System Fonts when you want to inspect or consume the fonts installed on the current machine.
- Use Font Metadata and Inspection when you need names, styles, or version information without loading a font for shaping.
- Use Text Layout and Options to control wrapping, alignment, direction, shaping, fallback fonts, and text runs.
- Use OpenType Features when you want to request fractions, tabular figures, stylistic sets, or other font features explicitly.
- Use Fallback Fonts and Multilingual Text when one family is not enough for your content.
- Use Variable Fonts when you want to work with weight, width, optical size, or custom axes from a single font file.
- Use Custom Rendering if you need to render glyph geometry to your own output surface.

System Fonts

`SystemFonts` are convenient because they let you get moving without shipping font files yourself. They also come with the tradeoff that the available families depend on the machine you are running on, so this page treats portability as part of the topic rather than an afterthought.

Use it when you want to work with platform fonts directly instead of loading files into your own `FontCollection`.

What `SystemFonts` exposes

The main entry points are:

- `SystemFonts.Families` to enumerate installed families
- `SystemFonts.Get(...)` and `SystemFonts.TryGet(...)` to resolve a family by invariant name
- `SystemFonts.CreateFont(...)` to create a `Font` directly
- `SystemFonts.Collection` when you also need access to the searched directories
- `SystemFonts.TryMatchCharacter(...)` to ask the operating system which installed font covers a code point
- `SystemFonts.FallbackResolver` to let text shaping consult the system for unresolved code points; see Fallback Fonts and Multilingual Text

```
using SixLabors.Fonts;

Font caption = SystemFonts.CreateFont("Segoe UI", 12);
Font heading = SystemFonts.CreateFont("Segoe UI", 24, FontStyle.Bold);
```

Replace `"Segoe UI"` with any installed family that exists on your machine.

Enumerate available families

Use `SystemFonts.Families` when you want to inspect what the current environment actually exposes.

```
using System;
using SixLabors.Fonts;

foreach (FontFamily family in SystemFonts.Families)
{
```

```
    Console.WriteLine(family.Name);
}
```

Use culture-aware lookup

Font family names can vary by culture, so `SystemFonts` also exposes the same culture-aware lookup helpers as `FontCollection`.

```
using System.Globalization;
using SixLabors.Fonts;

CultureInfo japanese = CultureInfo.GetCultureInfo("ja-JP");

if (SystemFonts.TryGetByCulture("Yu Gothic", japanese, out FontFamily family))
{
    Font font = family.CreateFont(16);
}
```

You can also create a font directly with the culture-aware `CreateFont(...)` overloads.

Merge system fonts into your own collection

If you want your own custom fonts and the machine fonts in one lookup surface, copy the system font set into a `FontCollection`.

```
using SixLabors.Fonts;

FontCollection collection = new();
collection.AddSystemFonts();
collection.Add("fonts/BrandSans-Regular.ttf");
```

There is also a filtered overload when you only want a subset of the installed fonts.

```
using System;
using SixLabors.Fonts;

FontCollection collection = new();
collection.AddSystemFonts(metric =>
    metric.Description.FontFamilyInvariantCulture.Contains("Noto",
StringComparison.OrdinalIgnoreCase));
```

Search directories

`SystemFonts.Collection` implements `IReadOnlySystemFontCollection`, which exposes `SearchDirectories`.

That is useful for diagnostics and for understanding where the current process looked for fonts.

```
using System;
using SixLabors.Fonts;

foreach (string directory in SystemFonts.Collection.SearchDirectories)
{
    Console.WriteLine(directory);
}
```

Portability considerations

The available system fonts are environment-specific.

- Windows, Linux, macOS, containers, and CI agents will often expose different families.
- A family name that exists on your dev machine may not exist in production.
- If predictable output matters, prefer shipping the fonts you need and loading them into a `FontCollection`.

For file-based loading, see Loading Fonts and Collections. For metadata-only inspection, see Font Metadata and Inspection.

Practical guidance

- Use `SystemFonts` for host-specific behavior and diagnostics.
- Use a private `FontCollection` for deterministic rendering.
- Log `SearchDirectories` when diagnosing missing fonts in containers or CI.
- Resolve by culture-aware names only when the user-facing font name is localized.

Font Metadata and Inspection

Sometimes you need to inspect a font long before you care about laying text out with it. Maybe you are building an importer, a picker, or a diagnostics tool. `FontDescription` is the lightweight part of the API for that job.

Read metadata without loading the font for layout

Use `FontDescription.LoadDescription(...)` when you only need descriptive information from a single font file or stream.

```
using System.Globalization;
using SixLabors.Fonts;
using SixLabors.Fonts.WellKnownIds;

FontDescription description = FontDescription.LoadDescription("fonts/SourceSans3-
Regular.ttf");

string family = description.FontFamilyInvariantCulture;
string fullName = description.FontNameInvariantCulture;
string subfamily = description.FontSubFamilyNameInvariantCulture;
string version = description.GetNameById(CultureInfo.InvariantCulture,
KnownNameIds.Version);
```

This is a better fit than `FontCollection.Add(...)` when you are building font pickers, diagnostics, import tools, or metadata listings.

Work with localized names

`FontDescription` exposes both invariant and culture-aware name accessors:

- `FontNameInvariantCulture`
- `FontFamilyInvariantCulture`
- `FontSubFamilyNameInvariantCulture`
- `FontName(culture)`
- `FontFamily(culture)`
- `FontSubFamilyName(culture)`

```
using System.Globalization;
using SixLabors.Fonts;

FontDescription description = FontDescription.LoadDescription("fonts/SourceSans3-
Regular.ttf");
CultureInfo english = CultureInfo.GetCultureInfo("en-US");

string familyName = description.FontFamily(english);
```

Read additional name-table entries

Use `GetNameById(...)` with `KnownNameIds` when you need more than the basic family and subfamily fields.

Common values include:

- `KnownNameIds.Version`
- `KnownNameIds.PostscriptName`
- `KnownNameIds.Designer`
- `KnownNameIds.Manufacturer`
- `KnownNameIds.LicenseDescription`
- `KnownNameIds.LicenseInfoUrl`
- `KnownNameIds.SampleText`

```
using System.Globalization;
using SixLabors.Fonts;
using SixLabors.Fonts.WellKnownIds;

FontDescription description = FontDescription.LoadDescription("fonts/SourceSans3-
Regular.ttf");

string designer = description.GetNameById(CultureInfo.InvariantCulture,
KnownNameIds.Designer);
string sample = description.GetNameById(CultureInfo.InvariantCulture,
KnownNameIds.SampleText);
```

Inspect font collections

Use `FontDescription.LoadFontCollectionDescriptions(...)` when a file contains multiple faces, such as a `.ttc` collection.

```
using System;
using SixLabors.Fonts;

ReadOnlyMemory<FontDescription> descriptions =
    FontDescription.LoadFontCollectionDescriptions("fonts/NotoSansCJK-Regular.ttc");
```

If you are loading a collection into a `FontCollection`, the `AddCollection(...)` overloads can also return the descriptions that were discovered during the load.

Inspect loaded families and fonts

Once a family has been loaded, there are a few additional inspection helpers worth knowing about:

- `FontFamily.GetAvailableStyles()` lists the styles currently available for that family in the collection
- `FontFamily.TryGetPaths(...)` returns source file paths when the family came from filesystem-backed fonts
- `Font.TryGetPath(...)` returns the backing file path for a concrete font instance when one exists
- `Font.FontMetrics.Description` exposes the same `FontDescription` for the resolved face

```
using System;
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/SourceSans3-Regular.ttf");

foreach (FontStyle style in family.GetAvailableStyles().Span)
{
    Console.WriteLine(style);
}

Font font = family.CreateFont(16);
FontDescription description = font.FontMetrics.Description;
```

What `Style` means

`FontDescription.Style` is the resolved `FontStyle` for that face. Fonts derives it from the face metadata in the font tables, so it is a useful quick check when you want to know whether a face is marked as bold, italic, or both.

For loading fonts into collections, see Loading Fonts and Collections. For working with installed machine fonts, see System Fonts.

If you want the face-level metrics that drive layout and glyph inspection rather than just the descriptive metadata, see Font Metrics.

Practical guidance

- Use metadata inspection before loading untrusted or user-supplied font files into normal collections.
- Store invariant names for stable configuration and localized names for UI.
- Inspect family styles before assuming bold or italic faces are available.
- Use font paths for diagnostics, not as the only identity for a face.

Font Metrics

`FontDescription` tells you what a face is called. `FontMetrics` tells you how that face behaves.

Once you know what a font is, the next question is usually how it behaves. `FontMetrics` is where you inspect the measurements and coverage data that explain line spacing, decoration placement, variation support, and glyph availability.

How to get `FontMetrics`

The most direct route is through a resolved `Font` instance:

```
using System;
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/SourceSans3-Regular.ttf");
Font font = family.CreateFont(16);

FontMetrics metrics = font.FontMetrics;
```

You can also inspect available faces on a family before you create a `Font`.

```
using System;
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/SourceSans3-Regular.ttf");

if (family.TryGetMetrics(FontStyle.Regular, out FontMetrics? metrics))
{
    Console.WriteLine(metrics.Description.FontNameInvariantCulture);
}
```

Description, units, and scale

The core identity and scaling properties are:

- `Description` for the face metadata
- `UnitsPerEm` for the design-space resolution of the font

- `ScaleFactor` for the face-level unit-to-point scaling used by glyph metrics
- `XHeight` and `CapHeight` for the lowercase and capital letter heights in design units, or `0` when the font does not provide them

`UnitsPerEm` is the important anchor for understanding almost every other metric on the typeface. Values like ascenders, underline positions, or glyph advances are stored in font units and should be interpreted relative to that em square.

Horizontal and vertical metrics

`FontMetrics` exposes both `HorizontalMetrics` and `VerticalMetrics`.

Both headers provide the same core fields:

- `Ascender`
- `Descender`
- `LineGap`
- `LineHeight`
- `AdvanceWidthMax`
- `AdvanceHeightMax`

The difference is not in the property names. It is in which layout direction those values are meant to describe.

- `HorizontalMetrics` describes the face when text is laid out in horizontal modes such as `LayoutMode.HorizontalTopBottom` and `LayoutMode.HorizontalBottomTop`.
- `VerticalMetrics` describes the face when text is laid out in vertical modes such as `LayoutMode.VerticalLeftRight`, `LayoutMode.VerticalRightLeft`, `LayoutMode.VerticalMixedLeftRight`, and `LayoutMode.VerticalMixedRightLeft`.

In practical terms:

- use `HorizontalMetrics` for normal Latin-style line layout, UI text, paragraphs, and most measurement scenarios
- use `VerticalMetrics` for vertical text layout, especially CJK-oriented column flow and vertical glyph advance

What the fields mean

`Ascender` and `Descender` define the font's recommended extents above and below the baseline for the layout direction you are inspecting.

`LineGap` is the additional space the font recommends between lines or columns beyond the ascender and descender space.

`LineHeight` is the face's typographic line spacing for that metrics header. If you want the font's default line advance, this is usually the most direct value to start from.

`AdvanceWidthMax` is the maximum glyph advance width in that face.

`AdvanceHeightMax` is the maximum glyph advance height in that face. This matters most for vertical layout. For fonts that do not provide dedicated vertical metrics, this value falls back to the line height.

When to use `HorizontalMetrics`

Reach for `HorizontalMetrics` when you need:

- default line spacing for ordinary left-to-right or right-to-left text
- baseline, ascender, and descender values for UI layout or custom renderers
- a face-level sanity check before measuring or clipping horizontal text
- maximum advance budgeting for horizontally flowing glyphs

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 16);
FontMetrics metrics = font.FontMetrics;

short ascender = metrics.HorizontalMetrics.Ascender;
short descender = metrics.HorizontalMetrics.Descender;
short lineHeight = metrics.HorizontalMetrics.LineHeight;
short maxAdvanceWidth = metrics.HorizontalMetrics.AdvanceWidthMax;
```

When to use `VerticalMetrics`

Reach for `VerticalMetrics` when you need:

- default line or column spacing for vertical layout
- face-level values for custom vertical renderers
- the maximum advance height budget for vertical glyph flow
- inspection of whether a font behaves sensibly in vertical layout

```
using SixLabors.Fonts;
```

```
Font font = SystemFonts.CreateFont("Segoe UI", 16);
FontMetrics metrics = font.FontMetrics;

short verticalAscender = metrics.VerticalMetrics.Ascender;
short verticalLineHeight = metrics.VerticalMetrics.LineHeight;
short maxAdvanceHeight = metrics.VerticalMetrics.AdvanceHeightMax;
```

These values are expressed in font units, not pixels.

Decoration and script-positioning metrics

`FontMetrics` also exposes the face-level metrics that support decoration and typographic adjustments:

- `UnderlinePosition`
- `UnderlineThickness`
- `StrikeoutPosition`
- `StrikeoutSize`
- `SubscriptXSize`
- `SubscriptYSize`
- `SubscriptXOffset`
- `SubscriptYOffset`
- `SuperscriptXSize`
- `SuperscriptYSize`
- `SuperscriptXOffset`
- `SuperscriptYOffset`
- `ItalicAngle`

These are useful when you are building your own renderer, diagnostics, or typography tools and want the font's own recommendations rather than hard-coded values.

Variable-font support

`FontMetrics.TryGetVariationAxes(...)` lets you inspect the variation axes that the resolved face supports.

```
using System;
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic.Variations;

FontCollection collection = new();
```

```
FontFamily family = collection.Add("fonts/RobotoFlex.ttf");
Font font = family.CreateFont(16);

if (font.FontMetrics.TryGetVariationAxes(out ReadOnlyMemory<VariationAxis> axes))
{
    foreach (VariationAxis axis in axes.Span)
    {
        Console.WriteLine($"{axis.Tag}: {axis.Min}..{axis.Max} (default {axis.Default})");
    }
}
```

Each `VariationAxis` gives you:

- `Name`
- `Tag`
- `Min`
- `Max`
- `Default`

The registered tags in `KnownVariationAxes` such as `wght`, `wdth`, `opsz`, `slnt`, and `ital` are useful when you want to relate those exposed axes back to font creation with `FontVariation`.

Code-point coverage

Use `GetAvailableCodePoints()` when you need to know which Unicode scalar values the face can map directly.

```
using System;
using SixLabors.Fonts;
using SixLabors.Fonts.Unicode;

Font font = SystemFonts.CreateFont("Segoe UI", 16);
ReadOnlyMemory<CodePoint> codePoints = font.FontMetrics.GetAvailableCodePoints();
```

This is useful for diagnostics, glyph coverage tooling, fallback decisions, and script-support inspection.

Inspect glyph metrics directly

If you need glyph-level inspection without going through full text layout, use `TryGetGlyphMetrics(...)`.

```
using SixLabors.Fonts;
using SixLabors.Fonts.Unicode;

Font font = SystemFonts.CreateFont("Segoe UI", 16);

if (font.FontMetrics.TryGetGlyphMetrics(
        new CodePoint('A'),
        TextAttributes.None,
        TextDecorations.None,
        LayoutMode.HorizontalTopBottom,
        ColorFontSupport.None,
        out FontGlyphMetrics? glyphMetrics))
{
    float width = glyphMetrics.Width;
    ushort advance = glyphMetrics.AdvanceWidth;
    GlyphType glyphType = glyphMetrics.GlyphType;
}
```

This is the lower-level face inspection API behind the higher-level `Font.TryGetGlyphs(...)` helpers.

When to use `FontMetrics` vs `FontDescription`

Use `FontDescription` when you care about names and face identity.

Use `FontMetrics` when you care about:

- line and em metrics
- underline and strikeout placement
- subscript and superscript recommendations
- variation-axis availability
- code-point coverage
- direct glyph inspection

For face names and other descriptive metadata, see Font Metadata and Inspection. For variable-font usage, see Variable Fonts.

Practical guidance

- Use font metrics when layout, decoration, glyph coverage, or variation axes matter.
- Use font descriptions when the question is identity, naming, style, or version metadata.
- Treat glyph availability as a layout input, not as a guarantee of final script quality.
- Cache metrics-derived decisions with the font face and variation values that produced them.

Measuring Text

Measurement is often the point where text layout stops being abstract and starts affecting a real UI. `TextMeasurer` lets you run the same shaping and layout engine that rendering uses, which means you can decide widths, line breaks, placements, and bounds before anything is drawn.

The measurement APIs come in three layers:

- `TextMeasurer`: one-shot convenience methods for measuring a string. Best for ad-hoc work.
- `TextBlock`: prepares a string once, then measures or renders it repeatedly at different wrapping lengths. See Prepared Text with TextBlock.
- `TextMetrics`: the full measurement object returned by `TextMeasurer.Measure(...)` or `TextBlock.Measure(...)`. Keep this when callers need several measurements, hit testing, carets, or selection geometry from the same laid-out text.

Choose the right measurement

- `MeasureAdvance(...)` returns the logical advance rectangle from layout, including line height and advance.
- `MeasureBounds(...)` returns only the tight rendered glyph ink bounds.
- `MeasureRenderableBounds(...)` returns the union of the logical advance rectangle and the glyph ink bounds.

The important distinction is that glyph geometry and layout geometry are not the same thing. Glyphs can overshoot the logical advance box, and the logical advance box can also include space that no glyph pixels occupy.

Measure a block of text

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 320
};

FontRectangle advance = TextMeasurer.MeasureAdvance("Hello world", options);
FontRectangle bounds = TextMeasurer.MeasureBounds("Hello world", options);
FontRectangle renderable = TextMeasurer.MeasureRenderableBounds("Hello world", options);
```

Replace `"Segoe UI"` with any installed family that exists on your machine.

Use `MeasureAdvance(...)` when you care about layout flow, alignment, wrapping, or line-box size.

Use `MeasureBounds(...)` when you want the pure glyph bounds only.

Use `MeasureRenderableBounds(...)` when you need the full rendered area that combines layout space and glyph overshoot.

Understand bounds and origin

`MeasureBounds(...)` returns absolute glyph bounds only, so the returned `X` and `Y` can be non-zero, and the width and height reflect only where glyph ink exists.

`MeasureRenderableBounds(...)` returns a larger conceptual rectangle when needed: it includes the full logical advance rectangle from layout and then expands that rectangle to also include any glyph ink that extends beyond it.

If you need a rectangle that can safely contain both the typographic layout box and any glyph overshoot, prefer `MeasureRenderableBounds(...)`.

Measure per-entry data

`TextMeasurer` exposes three per-entry collections. Each answers a different layout question and is independent of the others.

```
using System;
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 320
};

ReadOnlyMemory<GraphemeMetrics> graphemes = TextMeasurer.GetGraphemeMetrics("Hello
world", options);
ReadOnlyMemory<WordMetrics> words = TextMeasurer.GetWordMetrics("Hello world", options);
ReadOnlyMemory<GlyphMetrics> glyphs = TextMeasurer.GetGlyphMetrics("Hello world", options);
```

- `GraphemeMetrics` is the unit for text interaction: hit testing, caret positioning, range selection, and UI overlays. Use `Advance` for hit targets and selection geometry; `Bounds` is the rendered ink

only and can be empty or overhang.
- `WordMetrics` describes one Unicode word-boundary segment from UAX #29, including separators. `GraphemeStart` and `StringStart` are inclusive; `GraphemeEnd` and `StringEnd` are exclusive.
- `GlyphMetrics` exposes laid-out glyph entries for rendering diagnostics or glyph-level visualization. Do not use them as character or caret positions: ligatures, decomposition, fallback, emoji, and combining marks mean one grapheme can map to multiple glyph entries.

These APIs measure laid-out output, not raw UTF-16 code units, so do not assume a one-to-one mapping with the original string in the presence of shaping, ligatures, or complex scripts.

If you need a refresher on the difference between UTF-16 code units, `CodePoint` values, and graphemes, see Unicode, Code Points, and Graphemes.

Measure lines

When you care about wrapped text, use `CountLines(...)` and `GetLineMetrics(...)`.

```
using System;
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 320
};

int lineCount = TextMeasurer.CountLines("Hello world from Fonts", options);
ReadOnlyMemory<LineMetrics> lines = TextMeasurer.GetLineMetrics("Hello world from
Fonts", options);
```

Each `LineMetrics` entry includes:

- `Ascender`: the ascender guide position within the line box. This marks where tall glyphs such as `H` or `l` typically rise to.
- `Baseline`: the baseline position within the line box. This is the line most glyphs sit on.
- `Descender`: the descender guide position within the line box. This marks where descending glyph parts such as `g`, `p`, or `y` typically fall to.
- `LineHeight`: the total height of the line box after line spacing has been applied.
- `Start`: the positioned line-box origin in pixel units.
- `Extent`: the positioned line-box size in pixel units.

- `StringIndex`, `GraphemeIndex`, `GraphemeCount`: the source-text range owned by the line. `GraphemeCount` is not a glyph count.

`Start` and `Extent` are full `Vector2` values. Selection and caret APIs use the line box for the cross-axis size, which matches normal text editor and browser behavior: selecting mixed font sizes on the same line paints a consistent line-height rectangle rather than one rectangle per glyph height.

Capture the full measurement with `TextMetrics`

When a single layout pass needs to feed several questions — overall size, per-line metrics, per-grapheme positions, hit testing, carets, and selection — measure once and keep the returned `TextMetrics`.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 320
};

TextMetrics metrics = TextMeasurer.Measure("Hello world", options);

FontRectangle advance = metrics.Advance;
FontRectangle bounds = metrics.Bounds;
FontRectangle renderable = metrics.RenderableBounds;
int lineCount = metrics.LineCount;

ReadOnlySpan<LineMetrics> lines = metrics.LineMetrics;
ReadOnlySpan<GraphemeMetrics> graphemes = metrics.GraphemeMetrics;
ReadOnlySpan<WordMetrics> words = metrics.WordMetrics;
```

Line and grapheme collections are in final layout order; for bidi text and reverse line-order layout modes, that can differ from source order. Word collections are in source order because word-boundary navigation is a logical operation.

`TextMetrics` returns the per-entry collections as `ReadOnlySpan<T>` because the metrics object owns their lifetime. The `TextMeasurer` and `TextBlock` methods return `ReadOnlyMemory<T>` because those snapshots can be stored alongside other layout state. Use `.Span` when drawing.

The same object exposes interaction APIs:

```
TextHit hit = metrics.HitTest(point);
CaretPosition caret = metrics.GetCaretPosition(hit);
ReadOnlyMemory<FontRectangle> selection = metrics.GetSelectionBounds(anchor, focus);
```

See Hit Testing and Caret Movement and Selection and Bidi Drag for the full editor-style interaction surface.

Keep measurement and rendering aligned

Always measure with the same `TextOptions` that you intend to render with. `Dpi`, `LineSpacing`, `WrappingLength`, `TextDirection`, `LayoutMode`, `KerningMode`, `Tracking`, `FeatureTags`, `TextRuns`, and fallback fonts all affect the final layout.

For repeated measurement of the same string at different wrapping lengths, prefer `TextBlock` over calling `TextMeasurer` multiple times — it shapes the text once and varies wrapping per call.

Practical guidance

- Measure advance when you need layout flow; measure bounds when you need ink or selection geometry.
- Keep the same `TextOptions` for measuring, rendering, hit testing, and selection.
- Use `TextBlock` when the same shaped text will be inspected or wrapped more than once.
- For UI text, test with the longest localized strings and fallback fonts, not only the default language.

Prepared Text with TextBlock

`TextMeasurer` is the shortest path from a string to a measurement, but every call shapes the text from scratch. `TextBlock` does the wrapping-independent work once, then lets you measure, render, and inspect the same text repeatedly at different wrapping lengths.

Use `TextBlock` whenever the same string will be measured, wrapped, drawn, or inspected more than once: rich-text editors, layout panels that resize, anything that needs both a measurement pass and a render pass.

Construct once, vary the wrapping length

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    Origin = new System.Numerics.Vector2(20, 30)
};

TextBlock block = new("Hello, world!", options);

TextMetrics narrow = block.Measure(240);
TextMetrics wide = block.Measure(480);
```

`TextOptions.WrappingLength` is ignored by the constructor. Pass the wrapping length to each operation instead, and use `-1` to disable wrapping for that call.

```
TextMetrics unwrapped = block.Measure(-1);
```

Detail APIs

`TextBlock` exposes the same per-entry collections that `TextMetrics` does, for callers that do not need the full measurement object:

```
using System;
using SixLabors.Fonts;

ReadOnlyMemory<LineMetrics> lines = block.GetLineMetrics(320);
```

```
ReadOnlyMemory<GraphemeMetrics> graphemes = block.GetGraphemeMetrics(320);
ReadOnlyMemory<WordMetrics> words = block.GetWordMetrics(320);
ReadOnlyMemory<GlyphMetrics> glyphs = block.GetGlyphMetrics(320);
```

Method-returned collections use `ReadOnlyMemory<T>` because they are snapshots a caller may store with their own layout state. Owner-backed properties such as `TextMetrics.LineMetrics` and `LineLayout.GraphemeMetrics` use `ReadOnlySpan<T>` because the owner already controls the lifetime.

Per-line layout

When the UI needs line-local data, use `GetLineLayouts(...)` or `EnumerateLineLayouts()`. Both produce `LineLayout` instances that mirror the interaction surface of `TextMetrics` for a single line — hit testing, caret positioning, caret movement, word lookup, and selection bounds — but they position those lines in different coordinate spaces.

Block coordinates with `GetLineLayouts`

`GetLineLayouts(...)` lays out the whole block as one unit. Lines stack in their natural flow direction starting from `TextOptions.Origin`, so each successive line's `LineMetrics.Start` includes the cumulative advance of the lines that came before it.

```
using SixLabors.Fonts;

ReadOnlyMemory<LineLayout> layouts = block.GetLineLayouts(320);

foreach (LineLayout line in layouts.Span)
{
    LineMetrics lineMetrics = line.LineMetrics;
    ReadOnlySpan<GraphemeMetrics> lineGraphemes = line.GraphemeMetrics;
    ReadOnlyMemory<GlyphMetrics> lineGlyphs = line.GetGlyphMetrics();
}
```

Use this when the whole block paints into one rectangle and you want the returned geometry to be ready to draw without any further offsetting.

Line-local coordinates with `EnumerateLineLayouts`

`EnumerateLineLayouts()` lays out one line at a time and accepts the wrapping length per call. Each produced line is positioned independently, as if it were the first and only line in the block — its geometry sits at `TextOptions.Origin` regardless of which line index the enumerator is on. The caller is responsible for placing the line into the final layout.

```
using SixLabors.Fonts;

LineLayoutEnumerator enumerator = block.EnumerateLineLayouts();

while (enumerator.MoveNext(wrappingLength: 320))
{
    LineLayout line = enumerator.Current;
}
```

Use this when each line goes into a different column, frame, or shape — flowed text, variable-width columns, virtualized lists, or curved baselines — and the block's natural top-to-bottom stacking does not match the surface you are painting on. The wrapping length can also vary per line.

Picking between them

- Use `GetLineLayouts(...)` when the whole block paints as one stacked unit and you want the returned line positions to be ready to draw against the block origin.
- Use `EnumerateLineLayouts()` when the caller controls where each line lands and the block's stacking is not the layout you want.

Render the prepared block

`RenderTo(...)` draws the block to any `IGlyphRenderer` using the same wrapping-length argument as the measurement methods.

```
using SixLabors.Fonts.Rendering;

block.RenderTo(renderer, wrappingLength: 480);
```

Always render with the same `TextOptions` and wrapping length you measured with. Reusing the prepared block avoids re-shaping the text between the two passes.

When to choose TextBlock over TextMeasurer

Use `TextMeasurer` for one-off measurements where you do not need to keep a measurement object around.

Use `TextBlock` when:

- The same text is laid out repeatedly with different wrapping lengths.

- You want to measure once and render later with the same prepared shaping.
- You need per-line interaction (hit testing, carets, selection) — see Hit Testing and Caret Movement.
- You want to walk the laid-out text line by line without materializing every line up front.

Practical guidance

Use `TextMeasurer` for one-off answers. Use `TextBlock` when the shaped text becomes state: it will be measured more than once, rendered later, inspected line by line, hit-tested, or used for caret and selection behavior. Preparing the block once keeps measurement and rendering tied to the same shaping result.

The line-layout APIs differ by coordinate model. `GetLineLayouts(...)` returns lines positioned as one stacked block, which is what you want when the text paints as a normal paragraph or label. `EnumerateLineLayouts()` returns line-local layouts, which is what you want when another system places each line into columns, frames, paths, or virtualized rows. Choosing the wrong one usually shows up as doubled offsets or lines that are positioned correctly by themselves but not as a block.

Keep the prepared block tied to the `TextOptions` that created it. If font, culture, fallback, feature tags, or direction changes, prepare a new block rather than trying to reuse old layout state.

Hit Testing and Caret Movement

Hit testing resolves a point in laid-out text back to a text position. In Fonts, hit testing is not a yes/no collision test against visible pixels. `HitTest(...)` returns a `TextHit` describing the nearest grapheme, the line it belongs to, the source string index, and whether the point resolved to the leading or trailing side of that grapheme.

Once text has been laid out, applications usually need to translate between pixels, character positions, and editor commands. Fonts exposes a small set of types that own the bidi, grapheme, and hard-break rules so callers do not need to reimplement them: `TextHit`, `CaretPosition`, `CaretPlacement`, and `CaretMovement`.

All positional values returned by these APIs are in pixel units.

How this differs from graphics hit testing

In general graphics APIs, hit testing usually means checking whether a cursor point intersects a shape, path, bounding box, or visual object. Text interaction has a different goal. A text editor must answer "where would the caret go?" even when the point is over whitespace, outside the ink, above the first line, or beyond the end of a wrapped line.

Fonts therefore resolves the point to a text position rather than returning a collision result. The returned `TextHit` is an input to caret placement, selection, word lookup, and movement. If you need geometric picking against custom rendered glyph outlines, use the geometry produced by your renderer for that purpose; do not substitute ink bounds for caret hit testing.

Get a measurement object

Hit testing, caret positioning, and caret movement all operate on a `TextMetrics` (whole-block) or `LineLayout` (single line). Either come from `TextMeasurer` or from a prepared `TextBlock`. See Prepared Text with TextBlock for when to prefer one over the other.

```
using System.Numerics;
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 320,
    Origin = new Vector2(20, 30),
```

```
    TextInteractionMode = TextInteractionMode.Editor
};

TextMetrics metrics = TextMeasurer.Measure("Hello, world!", options);
```

Coordinate space and hit targets

The point passed to `HitTest(...)` is in the same pixel coordinate space as the measured layout. That includes the `TextOptions.Origin`, wrapping length, layout mode, text direction, fallback fonts, and interaction mode that were used when the `TextMetrics` or `LineLayout` was produced.

Fonts uses the logical advance rectangle of each laid-out grapheme as the hit target. It does not hit-test rendered ink bounds. Ink bounds are unsuitable for text interaction because whitespace can have no ink, accents and glyph overhangs can extend outside the advance, and some glyphs draw less than the area users expect to click or select.

For whole-block hit testing, `TextMetrics` first chooses the nearest laid-out line on the cross axis. It then resolves the nearest grapheme on that line's primary axis. In horizontal layout the primary axis is X; in vertical layout the primary axis is Y.

Points outside the text block clamp to the nearest line and grapheme instead of returning no hit. This is intentional for editor-style behavior: clicking to the left, right, above, or below the text can still place a caret at the nearest valid text position.

Choose paragraph or editor mode

`TextOptions.TextInteractionMode` controls how trailing whitespace and terminal hard breaks behave for hit testing, caret movement, and selection.

- `TextInteractionMode.Paragraph` (the default) is the right fit for laid-out paragraphs and rendered text labels. Trailing breaking whitespace at the end of a line is trimmed from the layout, and a hard break that ends the text does not produce a caret stop on a trailing blank line. This matches the way browsers measure and paint static text.
- `TextInteractionMode.Editor` is the right fit for editable text surfaces. Ordinary trailing whitespace stays addressable so typed spaces continue to advance the caret, and a terminal `Enter` produces a blank line whose geometry the caret can land on.

Set this once on the `TextOptions` you measure with. Every interaction API on the resulting `TextMetrics` (and on each `LineLayout`) honors it automatically — there is no per-call switch.

If your application has both rendered paragraph regions and editable regions, use a different `TextOptions` instance for each, with the matching `TextInteractionMode` set on it.

Hit-test a point

`HitTest(point)` maps a pointer position to the nearest grapheme and returns a `TextHit`.

```
using System.Numerics;
using SixLabors.Fonts;

TextHit hit = metrics.HitTest(new Vector2(mouseX, mouseY));

int line = hit.LineIndex;
int grapheme = hit.GraphemeIndex;
int stringIndex = hit.StringIndex;
bool trailing = hit.IsTrailing;
```

`IsTrailing` records which side of the grapheme was hit. For left-to-right text, a point after the grapheme midpoint on the primary axis resolves to the trailing side. For right-to-left text, the visual side is reversed. Prefer `GetCaretPosition(hit)` when you need caret geometry; it applies the side and direction rules for the resolved layout.

`TextHit` is meant to be passed straight back into the interaction APIs — `GetCaretPosition(hit)`, `GetSelectionBounds(anchor, focus)`, `GetWordMetrics(hit)`. Those overloads consume the hit directly and apply the trailing-side and bidi rules internally, so callers do not need to compute the visual side themselves.

The properties are exposed for diagnostics and for cases where you need to point back into your own text — for example, mapping the hit to a position in your source string. `GraphemeInsertionIndex` is the insertion position within the laid-out grapheme array; you rarely need to read it yourself.

Position a caret

A `CaretPosition` is both a drawable line and the navigation token used by the movement APIs.

```
using SixLabors.Fonts;

CaretPosition caret = metrics.GetCaretPosition(hit);

DrawCaret(caret.Start, caret.End);

if (caret.HasSecondary)
{
```

```
    DrawSecondaryCaret(caret.SecondaryStart, caret.SecondaryEnd);
}
```

At bidi run boundaries, one logical insertion position has two visual edges. `CaretPosition.HasSecondary` indicates that case, and `SecondaryStart` / `SecondaryEnd` give the second visual edge. Editor-style callers can choose how to present or navigate the boundary without recomputing bidi affinity.

When initializing a caret without a pointer hit (for example, for a freshly opened editor), use the placement overload:

```
using SixLabors.Fonts;

CaretPosition start = metrics.GetCaret(CaretPlacement.Start);
CaretPosition end = metrics.GetCaret(CaretPlacement.End);
```

Move a caret

`MoveCaret(...)` applies an editor-style movement to a caret and returns the new caret. The library owns the grapheme, line, and hard-break rules; callers should not perform their own grapheme arithmetic.

```
using SixLabors.Fonts;

CaretPosition caret = metrics.GetCaret(CaretPlacement.Start);

caret = metrics.MoveCaret(caret, CaretMovement.Next);
caret = metrics.MoveCaret(caret, CaretMovement.NextWord);
caret = metrics.MoveCaret(caret, CaretMovement.LineEnd);
caret = metrics.MoveCaret(caret, CaretMovement.TextStart);
```

`CaretMovement` covers the standard editor commands:

- `Previous` and `Next` move through grapheme insertion positions.
- `PreviousWord` and `NextWord` move through Unicode word boundaries.
- `LineStart` and `LineEnd` are the Home/End-style operations.
- `TextStart` and `TextEnd` are the whole-block equivalents.
- `LineUp` and `LineDown` move to the previous or next visual line.

All movement operations work in logical order and the returned `CaretPosition` is placed at the correct visual edge for the resolved bidi layout. In a right-to-left run, `Next` advances the caret one

grapheme forward in the source text — visually that lands on the *left* edge of the next glyph, matching how browsers and native text editors behave. `LineStart` and `LineEnd` resolve to the visual edges that match the line's text direction (logical start of an RTL paragraph is on the right). Callers should not adjust for direction themselves; pass the returned `CaretPosition` straight back into `MoveCaret(...)` and the library tracks the bidi state.

At bidi run boundaries one logical insertion position has two visual edges. `MoveCaret(...)` returns a `CaretPosition` with `HasSecondary == true` in that case so editor-style callers can present both edges or pick whichever fits the surrounding caret state.

`LineUp` and `LineDown` preserve the caret's original requested position on the line. Repeated vertical movement keeps that position even when an intermediate line is shorter and the visible caret has to clamp to that line's end.

```
CaretPosition caret = metrics.GetCaret(CaretPlacement.Start);
caret = metrics.MoveCaret(caret, CaretMovement.LineEnd);

// Repeated LineDown remembers the original line position.
CaretPosition next = metrics.MoveCaret(caret, CaretMovement.LineDown);
CaretPosition after = metrics.MoveCaret(next, CaretMovement.LineDown);
```

This matches normal rich-text editor behavior: moving down through a short line does not permanently lose the user's original horizontal or vertical line position.

Look up a word

For double-click or word-based selection, pass the hit (or caret) directly to `GetWordMetrics(...)`. This uses the grapheme that was hit, so clicking the trailing side of the final grapheme of a word still selects that word rather than the following separator segment.

```
using SixLabors.Fonts;

TextHit hit = metrics.HitTest(doubleClickPosition);
WordMetrics word = metrics.GetWordMetrics(hit);
```

A Unicode word-boundary segment includes its separators. `can't stop` produces three segments: `can't`, the space, and `stop`. Higher-level editor commands can decide whether to stop on separator boundaries or skip over them.

Per-line interaction

`LineLayout` mirrors the interaction surface for a single line. Use it when the caller already knows interaction is line-local; otherwise prefer `TextMetrics` so cross-line behavior (such as `LineDown` or wrapping selection) works correctly.

```
using SixLabors.Fonts;

ReadOnlyMemory<LineLayout> layouts = block.GetLineLayouts(320);

foreach (LineLayout line in layouts.Span)
{
    TextHit hit = line.HitTest(point);
    CaretPosition caret = line.GetCaretPosition(hit);
    WordMetrics word = line.GetWordMetrics(hit);
}
```

Hard line breaks

Hard line breaks at the end of non-empty lines are trimmed with other trailing breaking whitespace. Hard line breaks that own a blank line remain in the metrics so source ranges, hit testing, caret movement, and selection painting still cover that line. Consumers that inspect graphemes individually can use `GraphemeMetrics.IsLineBreak` to identify these cases.

In `TextInteractionMode.Editor`, a terminal hard break also produces a blank line at the end of the text so the caret can land on it after the user types `Enter`. In `TextInteractionMode.Paragraph` that trailing blank line is omitted, matching paragraph-style layout.

For more on the underlying measurement model and the `TextMetrics` shape, see Measuring Text. For the full selection API, see Selection and Bidi Drag.

Practical guidance

- Use `TextMetrics` for interaction that can cross line boundaries.
- Use `LineLayout` only when the caller already knows the interaction is line-local.
- Choose `TextInteractionMode.Editor` for editable text and `Paragraph` for display layout.
- Keep hit testing, caret movement, and selection tied to the same measured layout.
- Hit-test the measured layout, not rendered glyph ink bounds.
- Treat `TextHit` as a resolved text interaction position, not proof that the pointer was inside visible ink.
- Expect clamping for points outside the text block.

Selection and Bidi Drag

Once you can hit-test a point and place a caret, the next step is painting selection ranges. Fonts returns selection geometry as a list of rectangles in visual order so editor-style UIs can paint browser-shaped selections without reimplementing bidi or line-box rules.

For the underlying types — `TextHit`, `CaretPosition`, `CaretPlacement`, and `CaretMovement` — see Hit Testing and Caret Movement.

The shape of a selection

`GetSelectionBounds(...)` returns `ReadOnlyMemory<FontRectangle>`. Use `.Span` when drawing, and store the memory itself if the selection needs to be retained alongside other layout state.

```
using System;
using SixLabors.Fonts;

ReadOnlyMemory<FontRectangle> selection = metrics.GetSelectionBounds(anchor, focus);

foreach (FontRectangle rectangle in selection.Span)
{
    FillSelectionRectangle(rectangle);
}
```

A single logical selection can be visually discontinuous inside one line when it crosses bidi runs. Returning multiple rectangles allows browser-style selection where the unselected visual gap stays unpainted.

Do not sort, union, or merge the returned rectangles unless the UI explicitly wants a different visual.

Pointer selection

For pointer drags, hit-test both endpoints and pass the hits to the selection API. The `TextHit` overload converts both endpoints to logical insertion indices for you.

```
using System.Numerics;
using SixLabors.Fonts;

TextHit anchor = metrics.HitTest(new Vector2(downX, downY));
```

```
TextHit focus = metrics.HitTest(new Vector2(moveX, moveY));

ReadOnlyMemory<FontRectangle> selection = metrics.GetSelectionBounds(anchor, focus);
```

This keeps trailing-edge and bidi handling inside the library.

Keyboard selection

For keyboard selection, keep an anchor caret fixed and move the focus caret. Shift+Right-style behavior updates only the focus caret.

```
using SixLabors.Fonts;

CaretPosition anchor = metrics.GetCaret(CaretPlacement.Start);
CaretPosition focus = anchor;

focus = metrics.MoveCaret(focus, CaretMovement.Next);

ReadOnlyMemory<FontRectangle> selection = metrics.GetSelectionBounds(anchor, focus);
```

Selecting whole words via keyboard is the same shape: move the focus by `NextWord` or `PreviousWord`.

Word selection

For double-click word selection, find the word containing the hit and ask for its selection bounds.

```
using SixLabors.Fonts;

TextHit hit = metrics.HitTest(doubleClickPosition);
WordMetrics word = metrics.GetWordMetrics(hit);

ReadOnlyMemory<FontRectangle> selection = metrics.GetSelectionBounds(word);
```

The `GraphemeMetrics` overload selects exactly one grapheme, which is useful for caret-region overlays:

```
using SixLabors.Fonts;
```

```
GraphemeMetrics grapheme = metrics.GraphemeMetrics[index];
ReadOnlyMemory<FontRectangle> selection = metrics.GetSelectionBounds(grapheme);
```

Bidi drag selection

Consider a left-to-right paragraph whose source text is:

```
Tall ערב שלום
```

The right-to-left run can paint with Arabic before Hebrew. When a user drags from the left edge of `Tall` toward the Hebrew word, the visual selection can become split:

```
[Tall ] [שלום] ערב
```

Application code should not manually decide which physical edge of the Hebrew glyph means "before" or "after". The hit-test result already carries the logical insertion index, and the selection result is already split into the visual rectangles that should be painted.

```
using SixLabors.Fonts;

TextHit anchor = metrics.HitTest(mouseDown);
TextHit focus = metrics.HitTest(mouseMove);

ReadOnlyMemory<FontRectangle> rectangles = metrics.GetSelectionBounds(anchor, focus);
```

Just paint every rectangle. The library produces the correct visual gaps.

Hard line breaks

Hard line breaks that end non-empty lines are trimmed with trailing breaking whitespace. Hard line breaks that own a blank line remain in the metrics and contribute their own selection rectangle so the blank line still highlights when the selection crosses it.

For text with two hard breaks in the middle:

```
Tall عرب שלום

Small مرحبا שלום
```

A full selection paints three visual rows: the first text line, the blank line, and the second text line. The line break that ends a non-empty line does not add a separate painted box; the line break that owns the blank line does. Callers should not special-case this — paint the rectangles `GetSelectionBounds` returns.

Consumers that inspect individual graphemes can use `GraphemeMetrics.IsLineBreak` to identify the blank-line hard breaks that remain in the metrics.

In `TextInteractionMode.Editor`, a hard break that ends the text produces an additional blank line so a selection can extend past the final newline; `TextInteractionMode.Paragraph` omits that trailing blank line. See Hit Testing and Caret Movement for the full mode comparison.

Per-line selection

`LineLayout` exposes the same selection overloads when the caller knows the selection is line-local:

```
using SixLabors.Fonts;

LineLayout line = layouts.Span[lineIndex];

ReadOnlyMemory<FontRectangle> selection = line.GetSelectionBounds(anchor, focus);
ReadOnlyMemory<FontRectangle> wordSelection = line.GetSelectionBounds(word);
```

Use the full `TextMetrics` overloads for selections that can cross line boundaries; use `LineLayout` only when interaction is bounded to one line.

Stable line-box geometry

Per-line selection uses the line-box height rather than per-glyph height, which matches normal text editor and browser behavior: selecting mixed font sizes on the same line paints a consistent line-height rectangle rather than one rectangle per glyph height. The selection geometry stays visually stable across mixed fonts and font sizes.

For a wider tour of the measurement model and how line metrics are derived, see Measuring Text.

Practical guidance

- Paint the selection rectangles returned by the API instead of reconstructing selection geometry yourself.

- Keep anchor and focus as logical text positions; let the metrics map them into visual rectangles.
- Use editor interaction mode when selections must include terminal blank lines.
- Test mixed LTR/RTL selections with real strings, not only simple Latin text.

Text Layout and Options

Once you have a `Font`, `TextOptions` becomes the center of almost everything else. It is where you tell Fonts how text should flow, wrap, align, shape, and render, so getting comfortable with this type pays off quickly.

The same options type is used by both `TextMeasurer` and `TextRenderer`, which makes it easy to keep measurement and rendering in sync.

Core units

`Font.Size` is expressed in points. `TextOptions.Dpi` controls how that size is converted into pixels for measurement and rendering. The default DPI is `72`.

`WrappingLength` is expressed in pixels and defines when text wraps. `Origin` sets the rendering origin used by the layout engine.

```
using System.Numerics;
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    Dpi = 96,
    Origin = new Vector2(20, 40),
    WrappingLength = 480
};
```

Replace `"Segoe UI"` with any installed family that exists on your machine in the `SystemFonts` examples on this page.

Wrapping, flow, and direction

These properties control how text is broken into lines and laid out:

- `WrappingLength`
- `WordBreaking`
- `MaxLines`
- `TextEllipsis`
- `TextHyphenation`

- `TextDirection`
- `LayoutMode`

`WordBreaking` supports `Standard`, `BreakAll`, `KeepAll`, and `BreakWord`. `MaxLines` limits how many lines are laid out; use `-1` for unlimited lines. `TextDirection` supports left-to-right, right-to-left, and automatic detection. `LayoutMode` supports horizontal and vertical layouts, including mixed vertical modes that rotate horizontal glyphs.

`VisibleBounds` culls rendering to a visible region in pixel units. Whole lines that fall outside the region are skipped, and the remaining lines are positioned as if every line had been rendered. This is useful for scrolling views and large virtualized text. Leave the value `null` to disable culling.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 320,
    WordBreaking = WordBreaking.BreakWord,
    TextDirection = TextDirection.Auto,
    LayoutMode = LayoutMode.HorizontalTopBottom
};
```

Ellipsis and hyphenation markers

`TextEllipsis` controls whether a marker is inserted when `MaxLines` hides remaining text. `TextEllipsis.Standard` inserts the standard ellipsis marker, `TextEllipsis.Custom` uses `CustomEllipsis`, and `TextEllipsis.None` clips to the line limit without adding a marker.

`TextHyphenation` controls the marker used when wrapping selects a soft-hyphen break opportunity. `TextHyphenation.Standard` inserts the standard hyphenation marker, `TextHyphenation.Custom` uses `CustomHyphen`, and `TextHyphenation.None` allows the soft-hyphen break without drawing a marker.

```
using SixLabors.Fonts;
using SixLabors.Fonts.Unicode;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 220,
    MaxLines = 2,
```

```
    TextEllipsis = TextEllipsis.Standard,
    TextHyphenation = TextHyphenation.Custom,

    // CustomHyphen is used only when wrapping chooses a soft-hyphen break.
    CustomHyphen = new CodePoint('-')
};
```

Set `CustomEllipsis` or `CustomHyphen` only when the matching option is `Custom`. Standard markers still depend on glyph coverage in the selected font or fallback families.

Alignment and justification

`TextAlignment` expresses logical alignment within the text box using `Start`, `End`, and `Center`, and it respects the active text direction. `TextJustification` controls whether additional spacing is distributed between words or between characters.

`HorizontalAlignment` and `VerticalAlignment` give you physical alignment controls for the layout box itself.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    WrappingLength = 320,
    TextAlignment = TextAlignment.Start,
    TextJustification = TextJustification.InterWord,
    HorizontalAlignment = HorizontalAlignment.Left,
    VerticalAlignment = VerticalAlignment.Top
};
```

Spacing, hinting, and shaping controls

Fonts exposes several knobs that directly affect glyph layout:

- `LineSpacing` multiplies the line height.
- `TabWidth` controls tab stops in space units.
- `KerningMode` enables, disables, or lets the engine decide about font-provided kerning during shaping.
- `Tracking` adds uniform spacing after each rendered grapheme. It is measured in em, so `0.02F` adds 2% of the current em size; it is not a multiplier like `LineSpacing`.

- `HintingMode` is separate from shaping and controls grid fitting of TrueType and CFF outlines for the current size and DPI.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    LineSpacing = 1.2F,
    TabWidth = 4,
    KerningMode = KerningMode.Standard,
    Tracking = 0.02F,
    HintingMode = HintingMode.Standard
};
```

For a deeper explanation of GSUB/GPOS shaping, bidi analysis, fallback runs, and font hinting, see Text Shaping and Font Hinting.

Font weight and style synthesis

`TextOptions.FontWeight` requests a visual weight on the OpenType and CSS scale, from `FontWeight.Thin` (100) to `FontWeight.Black` (900). `TextRun.FontWeight` overrides the request for a range of text. Leave the value `null` to use the weight implied by the font.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FontWeight = FontWeight.SemiBold
};
```

How the request is satisfied depends on the font:

- A variable font with a registered `wght` axis is instanced at the requested weight. Other configured axes are preserved.
- A static font keeps its own outlines. If the request is `SemiBold` (600) or heavier and the face weight is lighter than 600, Fonts thickens the outlines with a synthetic bold operation. This mirrors browser behavior, where 600 is the boundary between regular and bold weights.
- Color glyphs keep their authored weight, because thickening independently colored layers would distort them.

Fonts also synthesizes an oblique slant. If a run requests an italic style that the resolved face does not provide, glyph outlines are sheared horizontally. This mirrors the CSS `font-synthesis: style` behavior in web browsers.

Fallback fonts and color fonts

Use `FallbackFontFamilies` when a single font cannot cover every glyph you need.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily textFamily = collection.Add("fonts/NotoSans-Regular.ttf");
FontFamily arabicFamily = collection.Add("fonts/NotoSansArabic-Regular.ttf");
FontFamily emojiFamily = collection.Add("fonts/NotoColorEmoji-Regular.ttf");

TextOptions options = new(textFamily.CreateFont(16))
{
    FallbackFontFamilies = [arabicFamily, emojiFamily],
    ColorFontSupport = ColorFontSupport.ColrV1 | ColorFontSupport.Svg
};
```

`ColorFontSupport` controls which color-font technologies are honored during layout and rendering: `ColrV0`, `ColrV1`, and `Svg`.

For a fuller discussion of multilingual text, fallback ordering, and script coverage, see Fallback Fonts and Multilingual Text.

OpenType feature tags

`FeatureTags` lets you request additional OpenType features during shaping. The property type is `IReadOnlyList<Tag>`, which means you can use either `KnownFeatureTags` enum values or parse raw four-character tags with `Tag.Parse(...)`.

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FeatureTags =
    [
        KnownFeatureTags.Ligatures,
```

```
        KnownFeatureTags.TabularFigures,

        // 'ss01' is the first of OpenType's stylistic sets (ss01..ss20),
        // which a font can use to expose an alternate glyph design.
        Tag.Parse("ss01")
    ]
};
```

Use `KnownFeatureTags` values when the feature already has a named constant. Use `Tag.Parse(...)` when you need a raw tag that is not otherwise surfaced in your code.

See OpenType Features for a fuller guide to common feature tags and when to request them explicitly.

Text runs

`TextRuns` lets you override layout attributes for subranges of text. A `TextRun` can replace the font and apply `TextAttributes` or `TextDecorations`.

`TextRun.Start` is inclusive and `TextRun.End` is exclusive. Both are grapheme indices, not UTF-16 code-unit indices.

```
using SixLabors.Fonts;

const string text = "Title: 1234";

Font baseFont = SystemFonts.CreateFont("Segoe UI", 18);
Font emphasisFont = SystemFonts.CreateFont("Segoe UI", 18, FontStyle.Bold);

TextOptions options = new(baseFont)
{
    TextRuns =
    [
        new TextRun
        {
            Start = 7,
            End = 11,
            Font = emphasisFont,
            TextDecorations = TextDecorations.Underline
        }
    ]
};
```

For plain ASCII text, grapheme indices often line up with character positions. For emoji, combining marks, and complex scripts, calculate ranges in graphemes rather than assuming one UTF-16 code unit equals one visible character.

See Unicode, Code Points, and Graphemes for a fuller explanation of `char`, `CodePoint`, and grapheme units.

Inline placeholders

Use `TextPlaceholder` when the text layout must reserve space for an inline object that your renderer will draw separately, such as an icon, emoji image, inline control, or attachment. Placeholders participate in measurement, wrapping, bidi ordering, and line-height calculation, but they do not consume text from the source string.

Add a placeholder through a zero-length `TextRun`. The run's `Start` and `End` values must be the same grapheme index, because the placeholder is inserted at that point rather than replacing text.

```
using SixLabors.Fonts;

const string text = "Pay now";

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    TextRuns =
    [
        new TextRun
        {
            // Placeholder runs are zero-length insertion points: [Start, End).
            Start = 4,
            End = 4,

            // The placeholder reserves inline space; the caller draws the object itself.
            Placeholder = new TextPlaceholder(
                width: 28,
                height: 20,
                alignment: TextPlaceholderAlignment.Middle,
                baselineOffset: 14)
        }
    ]
};

FontRectangle bounds = TextMeasurer.MeasureAdvance(text, options);
```

`TextPlaceholderAlignment` controls how the placeholder box aligns with the surrounding line. `Baseline` uses the supplied baseline offset directly, while `AboveBaseline`, `BelowBaseline`, `Top`, `Bottom`, and `Middle` align the placeholder against the surrounding line box.

Practical guidance

Treat `TextOptions` as the complete layout contract for a string. Font, culture, DPI, wrapping length, line spacing, direction, layout mode, fallback families, feature tags, text runs, and placeholders all participate in shaping and measurement. If you measure with one set of options and render with another, the result can move, wrap, or shape differently.

Use grapheme indexes for `TextRun` ranges and placeholder insertion points. A placeholder is an insertion into the layout flow, not a replacement for characters in the source string, so its run is zero-length: `[Start, End)` with the same value for both ends. That keeps source text ranges stable while still reserving inline space for an object that your renderer draws separately.

When text must fit inside a known region, set wrapping and alignment explicitly. Avoid measuring a string manually and then adjusting coordinates by hand; that bypasses the layout engine exactly where shaping, fallback, bidi order, and line metrics matter most.

OpenType Features

Fonts already applies the OpenType features that are required for correct shaping and layout. `TextOptions.FeatureTags` is where you ask for the extra typographic touches a font may support, such as tabular figures, fractions, stylistic alternates, or discretionary ligatures.

That makes it a typography control, not a substitute for the shaping engine.

How `FeatureTags` works

`TextOptions.FeatureTags` is an `IReadOnlyList<Tag>`.

You can populate it with:

- named values from `KnownFeatureTags`
- raw four-character tags parsed with `Tag.Parse(...)`

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FeatureTags =
    [
        KnownFeatureTags.Fractions,
        KnownFeatureTags.TabularFigures,

        // 'ss01' is the first of OpenType's stylistic sets (ss01..ss20),
        // which a font can use to expose an alternate glyph design.
        Tag.Parse("ss01")
    ]
};
```

A requested feature only has an effect if the font actually supports it.

When to use feature tags

Use explicit feature tags for discretionary typographic behavior such as:

- fractions

- tabular figures
- oldstyle figures
- discretionary ligatures
- stylistic sets
- small capitals
- case-sensitive punctuation
- vertical alternates

Do not think of `FeatureTags` as a way to manually replace the shaping engine. Core script shaping, bidi handling, and other required layout behavior are already handled by Fonts.

Common feature examples

Fractions:

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FeatureTags = [KnownFeatureTags.Fractions]
};
```

Tabular figures for aligned numeric columns:

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FeatureTags = [KnownFeatureTags.TabularFigures]
};
```

Oldstyle figures plus discretionary ligatures:

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
```

```
{
    FeatureTags =
    [
        KnownFeatureTags.OldstyleFigures,
        KnownFeatureTags.DiscretionaryLigatures
    ]
};
```

Raw stylistic-set tag:

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{

    // 'ss01' is the first of OpenType's stylistic sets (ss01..ss20),
    // which a font can use to expose an alternate glyph design.
    FeatureTags = [Tag.Parse("ss01")]
};
```

Named tags vs raw tags

Prefer the `KnownFeatureTags` enum when the feature already has a named constant in the library. Use `Tag.Parse(...)` for raw feature tags that you know exist in the target font but that you want to specify directly in your code.

`Tag.Parse(...)` expects a four-character tag such as `"liga"`, `"frac"`, or `"ss01"`.

Feature tags and layout

Feature requests participate in shaping, so they affect both measurement and rendering. If you want the measured result to match the rendered result, use the same `TextOptions` instance for both `TextMeasurer` and `TextRenderer`.

Vertical layout

Some OpenType features are especially relevant in vertical layout, such as `KnownFeatureTags.VerticalAlternates`, `KnownFeatureTags.VerticalAlternatesAndRotation`, and `KnownFeatureTags.VerticalAlternatesForRotation`.

Those work alongside `LayoutMode`; they do not replace it.

For the surrounding layout controls, see Text Layout and Options. For the broader shaping pipeline, see Text Shaping.

Practical guidance

- Treat feature tags as shaping inputs that affect both measurement and rendering.
- Prefer known feature tags where available and raw four-character tags for font-specific features.
- Validate requested features with the actual production font.
- Be careful combining features that intentionally choose competing glyph forms.

Text Shaping

Text shaping converts Unicode text into glyph IDs and positions. It takes the source string, the selected fonts, script and direction information, OpenType tables, fallback rules, and requested typographic features, then produces the glyph run that measurement and rendering use.

The input is text. The output is not text anymore. It is an ordered set of glyph IDs, glyph advances, offsets, bidi levels, source indexes, and font metrics. That output is what lets Fonts measure, wrap, hit-test, and render text consistently.

Why Shaping Exists

Unicode stores text as code points. Fonts store drawable shapes as glyphs. The relationship between the two is many-to-many:

- One code point can map to one glyph, as with many Latin letters.
- Several code points can become one glyph, as with ligatures or composed forms.
- One code point can become multiple glyphs, as with decomposition and fallback behavior.
- A glyph can move because of kerning, mark positioning, cursive attachment, vertical layout, or script rules.
- The visual order can differ from the logical string order in bidirectional text.

For simple English text in a basic font, shaping may look almost invisible. The same pipeline still matters because kerning, ligatures, fallback fonts, line breaks, and source indexes all depend on the shaped result.

Shaping and Rendering

Shaping decides which glyphs should be used and where those glyphs should be placed. Rendering draws those glyphs onto a target surface.

In Fonts, shaping and rendering are connected but separate responsibilities. Fonts prepares the glyph run and layout data. A renderer then consumes that data to draw paths, color glyph layers, SVG glyphs, or another representation through `TextRenderer` and `IGlyphRenderer`. ImageSharp.Drawing provides a renderer for drawing text into images, but Fonts itself also supports custom renderers.

This separation matters when you build your own renderer. Do not map characters to glyphs inside the renderer. Let Fonts shape the text once, then render the glyphs it gives you.

What Fonts Produces

After shaping, Fonts has enough information to answer layout questions in visual terms while still mapping back to the original string.

The shaped data records:

- which font supplied each glyph
- the glyph ID or glyph sequence that should be rendered
- the source code point and grapheme indexes
- the resolved bidi run and embedding level
- glyph advance and positioning data
- glyph bounds and line metrics
- text-run attributes and decorations

This is why shaped text affects more than drawing. It changes measured width, line wrapping, caret movement, hit testing, selection, and text bounds.

The Fonts Shaping Pipeline

Fonts shapes text during normal measurement and rendering. `TextMeasurer`, `TextBlock`, and `TextRenderer` all use the same `TextOptions` contract.

At a high level, Fonts does the following:

1. Builds the resolved `TextRun` list for the string. If no runs are supplied, the whole string uses `TextOptions.Font`. If runs are supplied, Fonts orders them, fills gaps with the default font, and trims overlaps.
2. Runs Unicode bidirectional analysis using `TextDirection` and `TextBidiMode`.
3. Walks the text by grapheme and then code point so source indexes remain meaningful for caret, selection, and hit testing.
4. Maps code points to glyph IDs with the current text-run font.
5. Applies right-to-left mirrored forms and vertical alternates where the layout requires them.
6. Applies OpenType GSUB substitutions through the appropriate script shaper.
7. Applies GPOS positioning for kerning, marks, cursive attachment, and related placement behavior.
8. Retries unmapped code points with the configured fallback font families, then with families from `FontFallbackResolver` when one is set.
9. Updates final glyph positions for every font involved in the shaped result.

The first shaping result is independent of wrapping width. Line composition and alignment happen after shaping, so the same shaped text can be measured, wrapped, rendered, or

inspected without changing which glyphs were chosen.

Script Shapers

Fonts uses the OpenType Layout shaping model. It reads Unicode script data and the OpenType script tags available in the font, then chooses the script shaper for the run.

Specialized shapers handle scripts whose glyph selection or ordering has rules beyond the default feature plan:

- Arabic-family scripts, including Arabic, Syriac, Nko, Mongolian, Mandaic, Manichaean, Phags Pa, and Psalter Pahlavi.
- Hebrew.
- Thai and Lao.
- Hangul.
- Indic scripts such as Devanagari, Bengali, Gujarati, Gurmukhi, Kannada, Malayalam, Oriya, Tamil, Telugu, and Khmer.
- Myanmar when the font exposes the modern `mym2` shaping model.
- Universal Shaping Engine scripts such as Balinese, Brahmi, Chakma, Javanese, Tibetan, Sinhala, Tai Tham, and other complex scripts covered by the USE model.

Other scripts use the default shaper, which still applies common OpenType behavior such as composition, localized forms, required ligatures, standard ligatures, contextual alternates, mark positioning, kerning, and directional alternates.

Fonts does not use HarfBuzz, Graphite, Apple Advanced Typography, Uniscribe, or platform text APIs. It implements its shaping behavior directly in managed code using the font data it loads.

OpenType Features

Fonts applies required features automatically. You do not need to request core script behavior with `FeatureTags`.

The default feature plan includes common features such as:

- `ccmp` for glyph composition and decomposition
- `locl` for localized forms
- `rlig` for required ligatures
- `mark` and `mkmk` for mark positioning
- `calt`, `clig`, `liga`, `rclt`, and `curs` for horizontal text where applicable
- `kern` unless kerning is disabled
- `vert` for vertical glyph alternates where applicable

- directional features such as `ltra`, `ltrm`, `rtla`, and `rtlm`
- `rvrn` for required variation alternates

`FeatureTags` is for optional features you want to request from the font, such as tabular figures, fractions, stylistic sets, discretionary ligatures, or small capitals. Feature support depends on the font.

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    TextDirection = TextDirection.Auto,
    KerningMode = KerningMode.Standard,
    FeatureTags =
    [
        KnownFeatureTags.Fractions,
        KnownFeatureTags.TabularFigures
    ]
};

FontRectangle bounds = TextMeasurer.MeasureAdvance("9/2", options);
```

Fractions are a good example of a feature that changes the glyph plan. Fonts handles the required `frac`, `numr`, and `dnom` feature assignment around fraction sequences when fraction features are requested.

Direction and Bidi

Bidirectional text is handled before font substitution and positioning. Fonts resolves directional runs, records the bidi run for each shaped glyph, and uses that information during line layout.

Use `TextDirection.Auto` unless your input has an external direction contract. Use `TextBidiMode` when you need override behavior rather than normal Unicode bidi resolution.

Mirrored forms, such as paired punctuation in right-to-left runs, are part of the shaping result. Fonts first relies on font support and also uses Unicode mirror data where needed.

Fallback Fonts

Fallback is not just a missing-glyph replacement step at the end of rendering. It participates in shaping because each font has its own glyph coverage, metrics, OpenType tables, script tags, and mark positioning behavior.

When the primary text-run font cannot map every code point, Fonts retries unresolved text against `FallbackFontFamilies`. The fallback font supplies the glyphs and positions for the code points it covers. A configured `FontFallbackResolver` is consulted last, for code points that no listed family can shape. See Fallback Fonts and Multilingual Text.

For multilingual text, emoji, and complex scripts, validate fallback with real production strings. A font can contain the individual code points but still lack the OpenType data needed for correct shaping.

Text Runs and Placeholders

Use `TextRuns` when a range of text needs a different font, style attributes, decorations, or a placeholder.

Text runs are indexed by grapheme range, not raw UTF-16 code unit count. Fonts resolves the run list before shaping, so font selection and attributes are known when code points are mapped to glyphs.

Placeholder runs are inserted into the layout stream without consuming source text. That makes them useful for inline objects while preserving source text indexes for the surrounding content.

Measurement and Rendering

Shaping changes advance widths and glyph positions, so measurement and rendering must use the same options. Do not measure with one font, direction, feature set, fallback list, or wrapping policy and render with another.

For one-off layout, use `TextMeasurer`. Use `TextBlock` when the shaped result becomes state: you need to measure it more than once, render it later, inspect its lines, hit-test it, or support caret and selection behavior.

Shaping Text Directly

Fonts exposes the shaper as a public API. Use `TextShaper` with a `TextShapingBuffer` when you need glyph IDs and positions without line wrapping. This is the path for building your own line layout, caching shaped runs, or feeding glyphs to an external renderer.

The buffer is caller-owned and reusable. You set the text and its properties on the buffer, shape the buffer against a font, then read the glyphs back. Each shaping call replaces the glyphs. A buffer instance is not thread safe.

`TextShaper.Shape(...)` treats the text as unwrapped logical lines separated by hard breaks. It resolves mixed-direction text within each line and produces each line's glyphs in visual order. `TextShapingBuffer.LineEnds` records where each hard-delimited line ends.

`TextShaper.ShapeRun(...)` shapes one directional run that you have already selected. The glyphs come back in reading order.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 16);

TextShapingBuffer buffer = new()
{
    TextDirection = TextDirection.Auto
};
buffer.Add("Hello world");

TextShaper.Shape(font, buffer);

float penX = 0;
foreach (ShapedGlyph glyph in buffer.Glyphs)
{
    // Draw the glyph at penX + glyph.Offset.X, then advance the pen.
    penX += glyph.AdvanceWidth;
}
```

Each `ShapedGlyph` records:

- `GlyphId`: the glyph identifier within the font. Glyph ID 0 is the font's missing glyph.
- `StringIndex`: the index of the first input `char` the glyph represents. Glyphs produced from the same input chars share this value.
- `GraphemeIndex`: the zero-based grapheme index in the input text.
- `AdvanceWidth` and `AdvanceHeight`: the scaled advances after positioning features have been applied.
- `Offset`: the scaled placement offset relative to the pen position. The offset does not contribute to the advance.

Advances and offsets are scaled to the size of the font used for shaping, in a Y-up coordinate system. DPI and device coordinates are renderer concerns and do not enter shaping.

Both methods accept optional feature tags:

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

TextShaper.Shape(font, buffer, [KnownFeatureTags.DiscretionaryLigatures]);
```

Measuring and Rendering Shaped Glyphs

Glyph IDs and positions are specific to the font and layout mode used while shaping. Pass that same font and layout mode in the `GlyphOptions` used to measure or render the shaped buffer.

`TextMeasurer` and `TextRenderer` accept a shaped buffer directly. They also accept raw glyph IDs with explicit positions for callers that manage their own layout.

```
GlyphOptions glyphOptions = new()
{
    Font = font,
    Dpi = 96
};

FontRectangle advance = TextMeasurer.MeasureAdvance(buffer, glyphOptions);
```

Use the shaper API when you need control over line composition. Use `TextMeasurer`, `TextRenderer`, or `TextBlock` when you want Fonts to handle wrapping and alignment for you.

Practical Guidance

- Treat `TextOptions` as the shaping contract.
- Use `TextDirection.Auto` for natural text unless a protocol or UI explicitly supplies direction.
- Use `FeatureTags` for optional typography, not for required script shaping.
- Use `FallbackFontFamilies` for multilingual text and test fallback with realistic content.
- Use `TextRuns` for known range-level font or attribute changes.
- Keep measurement, rendering, hit testing, and selection on the same shaped options.
- Validate complex scripts with the actual fonts and strings your application will ship.

Related Topics

- Text Layout and Options
- OpenType Features

- Fallback Fonts and Multilingual Text
- Unicode, Code Points, and Graphemes
- Prepared Text with TextBlock

Font Hinting

Font hinting adjusts outline glyphs for a raster grid. TrueType fonts carry bytecode programs that move outline points at a given size and DPI. CFF fonts declare stem hints and alignment zones that a rasterizer fits to the grid. Fonts supports both models.

Why Hinting Exists

Outline fonts are scalable. Raster images are not. When a glyph is small, its outline has to be represented by a limited number of pixels. Without adjustment, similar stems can round to different widths, horizontal features can fall between pixel rows, and small counters or serifs can lose definition.

Hinting changes the scaled outline before rasterization. It can improve:

- stem thickness
- counter shape
- baseline alignment
- x-height consistency
- serif and bar visibility
- mark attachment stability

The effect is most visible for small UI text at ordinary screen DPI. At larger sizes or high-resolution outputs, the outline has more pixels available and hinting usually has less visible effect.

Hinting Modes

`TextOptions.HintingMode` controls how Fonts applies hinting:

- `HintingMode.None` leaves glyph outlines unhinted.
- `HintingMode.Standard` fits outlines to the pixel grid on the vertical axis only. Horizontal placement stays smooth.
- `HintingMode.Full` fits outlines and glyph origins to the pixel grid on both axes. This reproduces the classic bi-level grid fitting that font hints were originally authored for.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 11);
TextOptions options = new(font)
{
```

```
    Dpi = 96,
    HintingMode = HintingMode.Standard
};
```

The active font size and `Dpi` matter because hinting targets a specific pixels-per-em scale.

Standard Hinting

Standard hinting models FreeType's v40 subpixel hinting behavior. For TrueType fonts, it preserves full vertical instruction processing while intentionally disabling horizontal hinting. For CFF fonts, it fits the vertical axis only from the declared hints.

This mode is designed for modern antialiased text rendering, where horizontal subpixel placement remains smooth and glyph advances keep their fractional shaped values. It gives small text the vertical alignment benefits of hinting while avoiding legacy horizontal snapping that can make spacing less consistent.

Full Hinting

Full hinting reproduces the classic rasterizer pipeline. It produces the sharpest results at small sizes, at the cost of shape fidelity and spacing regularity.

Under full hinting, Fonts:

- rounds the em size to a whole number of pixels, with a minimum of one pixel
- executes the complete horizontal and vertical TrueType instruction set, or fits CFF outlines on both axes
- aligns glyph origins and text decorations to the pixel grid on both axes
- replaces fractional advances on the flow axis with whole-pixel advances

For TrueType fonts, the whole-pixel advance comes from the font's `hdmx` device record when the font carries one. Otherwise Fonts uses the hinted advance that instruction execution produced, or the design advance rounded to whole pixels for glyphs the interpreter could not hint. CFF fonts carry no device advance records, so the design advance rounds to whole pixels.

Whole-pixel advances change spacing. Fractional positioning adjustments on the flow axis, such as pair kerning, are dropped because the classic pipeline never applied them. Inter-glyph spacing can vary by up to half a pixel from the unhinted design. Measurement uses the same advances as rendering, so measured layout and rendered output stay in agreement.

Two cases keep their shaped fractional advances: subscript and superscript runs, and upright glyphs in vertical layout.

The TrueType Pipeline

Fonts uses a TrueType bytecode interpreter. When hinting is active, Fonts:

- executes the font program from `fpgm` to initialize TrueType function definitions
- scales the Control Value Table from `cvt` for the current size and DPI
- executes the `prep` program to establish the graphics state for glyph programs
- applies `cvar` deltas to control values for variable TrueType fonts before hinting
- provides normalized variation coordinates for TrueType variation-aware instructions
- adds the four TrueType phantom points used for horizontal and vertical metrics during glyph hinting
- executes each glyph's TrueType instructions against the resolved outline
- leaves the outline unhinted if a glyph has no instructions, hinting is inhibited by the font program, or instruction execution fails

Some fonts are known to require hinting for correct rendering, and some are known to render badly with it. Fonts maintains compatibility lists that force such TrueType fonts to full hinting or to no hinting, regardless of the requested mode.

The CFF Pipeline

CFF outlines carry no instructions. Instead, the font declares the regularities in its design:

- horizontal and vertical stem hints in each glyph's charstring
- hint masks and counter masks that control which stems apply in each region of the glyph
- alignment zones, standard stem widths, and related values in the Private DICT

When hinting is active, Fonts collects these declared hints while it evaluates the charstring, then fits the outline through a hint map. Standard hinting fits the vertical axis only, from the declared horizontal stem zones and alignment-zone flats. Full hinting fits both axes.

Choosing a Mode

Use `HintingMode.Standard` when rendering small UI text to a raster target and you want grid-fitted outlines with smooth, fractional spacing.

Use `HintingMode.Full` when you want the sharpest possible small text and accept the classic-rasterizer tradeoffs: whole-pixel spacing, dropped fractional kerning, and altered glyph shapes. It is also the mode to compare output against classic bi-level rasterizers.

Use `HintingMode.None` when you want raw outline behavior, when you are rendering large display text, or when the text is being treated as artwork rather than screen UI.

There is no universal best setting. Hinting is a raster-quality tradeoff: it can make small text clearer, but it can also move outlines away from their pure scaled design.

Custom Renderers

The requested hinting mode is part of `GlyphRendererParameters`. A renderer that caches glyph geometry can include the parameters in its cache key, so outlines fitted under one mode are never reused under another.

Common Misunderstandings

Hinting does not:

- fix missing glyphs
- enable ligatures or OpenType features
- choose fallback fonts
- reorder complex scripts
- resolve bidirectional text
- change Unicode indexing or grapheme behavior

Those are layout and shaping concerns. For those, see Text Shaping.

Further Reading

The Raster Tragedy is a useful deeper discussion of why rasterizing outline text is difficult and why hinting can matter for small text.

Related Topics

- Text Layout and Options
- Text Shaping
- Font Metrics
- Custom Rendering

Color Fonts

Color fonts are one of the clearest signs of how much richer modern text rendering has become. Instead of a single monochrome outline, a glyph can carry layers, gradients, or even SVG content, and Fonts exposes that support explicitly through `ColorFontSupport`.

Fonts has comprehensive support for the major OpenType color-font technologies it exposes publicly:

- `ColorFontSupport.ColrV0` for layered solid-color glyphs defined by COLR and CPAL tables
- `ColorFontSupport.ColrV1` for paint-graph glyphs with gradients, transforms, and richer composition
- `ColorFontSupport.Svg` for color glyphs stored in the OpenType SVG table

Enable or restrict color-font support

`TextOptions.ColorFontSupport` controls which color-font technologies are honored during layout and rendering.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/NotoColorEmoji-Regular.ttf");
Font font = family.CreateFont(32);

TextOptions options = new(font)
{
    ColorFontSupport = ColorFontSupport.ColrV1 | ColorFontSupport.ColrV0
| ColorFontSupport.Svg
};
```

`TextOptions` enables all three by default, so you usually only need to set this property when you want to disable color glyphs or restrict the allowed formats.

Force monochrome output

Set `ColorFontSupport.None` when you want color-font-capable text to fall back to monochrome outline rendering.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/NotoColorEmoji-Regular.ttf");
Font font = family.CreateFont(32);

TextOptions options = new(font)
{
    ColorFontSupport = ColorFontSupport.None
};
```

Font palettes

COLR glyphs take their colors from the font's CPAL palette table. Many color fonts ship more than one palette, such as a default and a dark-background variant. `TextOptions.FontPalette` selects the palette that resolves the colors. Leave the value `null` to keep the font's default palette.

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI Emoji", 32);
TextOptions options = new(font)
{
    // Resolve COLR glyph colors with the font's second palette.
    // Palette indexes are zero-based; null keeps the font's default.
    FontPalette = new FontPalette(1)
};
```

A palette selection can also override individual entries. Each `FontPaletteOverride` replaces the palette color at one index:

```
// Start from the default palette, then replace entry 2 with opaque red.
FontPalette palette = new(0,
[
    new FontPaletteOverride(2, new GlyphColor(255, 0, 0, 255))
]);
```

`TextRun.FontPalette` replaces the selection over a range of text. `GlyphOptions.FontPalette` applies the same selection when rendering or measuring by glyph ID.

What happens in custom renderers

When a resolved glyph is a painted color glyph, Fonts streams it through `IGlyphRenderer` as one or more layers.

That means custom renderers should pay attention to:

- `GlyphRendererParameters.GlyphType`
- `BeginLayer(...)`
- `BeginGroup(...)` and `EndGroup()`
- `Paint`
- `FillRule`
- `ClipBounds`

Depending on the font technology in use, the `Paint` passed to `BeginLayer(...)` may be:

- `SolidPaint`
- `LinearGradientPaint`
- `RadialGradientPaint`
- `SweepGradientPaint`

COLR v1 paint graphs can also form composite groups. Fonts wraps such content in `BeginGroup(...)` and `EndGroup()` calls. Layers inside a group compose as one isolated unit. The finished group then blends onto the content below it with the `CompositeMode` given to `BeginGroup(...)`. A clip for the glyph arrives through `GlyphRendererParameters.ClipBounds`.

If your renderer ignores paint information, the glyph can still be drawn, but it will no longer preserve the font's intended color presentation.

Inspect color glyphs directly

If you need to inspect a glyph without running full text layout, use `Font.TryGetGlyphs(...)` with explicit color support.

```
using SixLabors.Fonts;
using SixLabors.Fonts.Unicode;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/NotoColorEmoji-Regular.ttf");
Font font = family.CreateFont(32);

if (font.TryGetGlyphs(
        new CodePoint(0x1F600), // 😀 GRINNING FACE
        ColorFontSupport.ColrV1 | ColorFontSupport.ColrV0 | ColorFontSupport.Svg,
        out Glyph? glyph))
```

```
{
    bool isPainted = glyph.GlyphMetrics.GlyphType == GlyphType.Painted;
}
```

COLR vs SVG in practice

At a high level:

- COLR v0 uses layered shapes with palette colors
- COLR v1 extends that model with richer paint graphs, gradients, transforms, and clipping
- SVG glyphs carry SVG-authored painted content

Fonts resolves those technologies into a common painted-glyph rendering flow, which is why custom renderers can consume them through the same layer and paint callbacks.

Measurement and rendering stay aligned

Color-font support is part of text layout, not just final painting. If you measure text with one `ColorFontSupport` configuration and render with another, you can create drift between the measured and rendered result.

Use the same `TextOptions` instance for both `TextMeasurer` and `TextRenderer` when you want a guaranteed match.

For renderer implementation details, see Custom Rendering. For fallback across multiple families, see Fallback Fonts and Multilingual Text.

Practical guidance

- Use the same `ColorFontSupport` setting when measuring and rendering.
- Test the actual emoji or color glyph set you intend to support; technologies vary by font.
- Decide fallback order deliberately when both monochrome and color families can cover the same text.
- Custom renderers should handle painted glyph callbacks even if most text is outline-based.

Unicode, Code Points, and Graphemes

Text handling gets easier once you stop treating every `char` as a whole character. Fonts exposes the text-unit levels it actually uses during layout so you can reason about indexing, fallback, shaping, and glyph coverage with the same vocabulary as the library.

The text-unit levels

- `char`: a single UTF-16 code unit in a .NET `string`
- `CodePoint`: a Unicode scalar value
- grapheme: a user-perceived text element, represented by a `ReadOnlySpan<char>` returned from `SpanGraphemeEnumerator`

In everyday text, those levels often line up for simple ASCII. Once you move beyond that, they diverge quickly.

What is a `char`?

In .NET, `string` is UTF-16. That means a single `char` is just one UTF-16 code unit.

A `char` is not always a full Unicode character:

- BMP scalars such as `A` fit in one `char`
- supplementary-plane scalars such as many emoji use two `char` values as a surrogate pair
- combining sequences can use multiple `char` values to represent what a user sees as one text element

So if you index raw `char` positions, you are working at the storage level, not the text-semantics level.

What is a `CodePoint`?

In strict Unicode terminology:

- a code point is any value in the range `U+0000` through `U+10FFFF`
- a Unicode scalar value is any code point except the surrogate range `U+D800` through `U+DFFF`

CodePoint represents Unicode scalar values. Despite the type name, it intentionally excludes standalone surrogate code points because those are UTF-16 encoding artifacts, not meaningful text values to shape or render.

That makes it the right unit when you want to talk about valid Unicode text values directly.

Useful `CodePoint` members include:

- `Value`
- `Utf16SequenceLength`
- `Utf8SequenceLength`
- `IsAscii`
- `IsBmp`
- `Plane`
- `ReplacementChar`

This is also the unit used by glyph-probing APIs such as `Font.TryGetGlyphs(...)`.

What is a grapheme?

A grapheme cluster is the closest thing to a user-perceived text element.

Examples:

- `A` is one grapheme
- `A` followed by a combining acute accent is still one grapheme
- many emoji sequences joined with zero-width joiners are one grapheme
- a flag emoji made from two regional indicators is one grapheme

Fonts exposes grapheme enumeration through `SpanGraphemeEnumerator`, which implements the Unicode grapheme cluster algorithm from UAX #29.

This is why `TextRun.Start` and `TextRun.End` are grapheme indices rather than raw `char` indices.

Enumerate `CodePoint` values

The Unicode enumeration helpers live in `SixLabors.Fonts.Unicode`.

```
using System;
using SixLabors.Fonts.Unicode;

// 'A' + combining acute accent (U+0301) renders as a single accented A grapheme,
// followed by a space and the grinning-face emoji (U+1F600).
```

```
string text = "Á 😀";

foreach (CodePoint codePoint in text.AsSpan().EnumerateCodePoints())
{
    Console.WriteLine(
        $"U+{codePoint.Value:X}: UTF-16 length {codePoint.Utf16SequenceLength}");
}
```

`EnumerateCodePoints()` returns a `SpanCodePointEnumerator`. It yields `CodePoint` values, which means the enumeration surface is Unicode scalar values. Invalid UTF-16 sequences are surfaced as `CodePoint.ReplacementChar`.

Count helpers are also available:

```
using SixLabors.Fonts.Unicode;

// 'A' + combining acute (U+0301), space, grinning-face emoji (U+1F600).
// 4 code points: 'A', U+0301, ' ', U+1F600.
int count = "Á 😀".GetCodePointCount();
```

Enumerate graphemes

Use grapheme enumeration when you need units that better match what a reader sees.

```
using System;
using SixLabors.Fonts.Unicode;

// Same text as before, but graphemes group the accented A into one cluster.
string text = "Á 😀";
int index = 0;

foreach (ReadOnlySpan<char> grapheme in text.AsSpan().EnumerateGraphemes())
{
    Console.WriteLine($"{index++}: {grapheme.ToString()}");
}
```

`EnumerateGraphemes()` returns a `SpanGraphemeEnumerator`.

Count helpers are available here too:

```
using SixLabors.Fonts.Unicode;
```

```
// 3 graphemes: the accented A, the space, and the emoji.
int count = "Á 😀".GetGraphemeCount();
```

Enumerate word-boundary segments

Use word enumeration when the surface needs to reason about whole words — caret movement that jumps a word at a time, double-click word selection, search-as-you-type tokenization. Word segmentation follows the Unicode Word Boundary Algorithm in UAX #29.

```
using System;
using SixLabors.Fonts.Unicode;

string text = "Don't stop.";

foreach (WordSegment word in text.AsSpan().EnumerateWordSegments())
{
    Console.WriteLine(
        $"[{word.Utf16Offset}..{word.Utf16Offset + word.Utf16Length}]
'{word.Span.ToString()}'");
}
```

The output for the example above is:

```
[0..5] 'Don't'
[5..6] ' '
[6..10] 'stop'
[10..11] '.'
```

UAX #29 segments include separators — the space between `Don't` and `stop` is its own segment, and the trailing `.` is another. Higher-level editor commands can decide whether to stop on those segments or skip past them; the raw enumerator stays aligned with the standard.

`EnumerateWordSegments()` returns a `SpanWordEnumerator`. Each `WordSegment` exposes:

- `Span` — the UTF-16 slice of the segment.
- `Utf16Offset` and `Utf16Length` — UTF-16 indices into the original text.
- `CodePointOffset` and `CodePointCount` — code-point indices into the original text.

This is the same Unicode word-boundary model used by `TextMetrics.WordMetrics`, `MoveCaret(CaretMovement.NextWord)`, and `GetWordMetrics(hit)`. Use the enumerator when you need word boundaries against raw text without going through a full layout pass; use the metrics APIs

when you need positioned word geometry as well. See Hit Testing and Caret Movement for the layout-aware side.

Which unit should you use?

Use `char` when:

- you are working with raw .NET string storage
- you truly need UTF-16 code-unit offsets

Use `CodePoint` when:

- you are inspecting Unicode scalar values
- you are probing glyph availability with `TryGetGlyphs(...)`
- you care about Unicode values, planes, or encoded sequence lengths

Use graphemes when:

- you are slicing visible text ranges
- you are working with `TextRun.Start` and `TextRun.End`
- you want indices that align better with user-visible text elements

Relation to layout

Fonts uses additional Unicode logic internally during layout, including line-breaking and script/shaping data. But the public text-unit APIs you will use most often are:

- `EnumerateCodePoints()`
- `EnumerateGraphemes()`
- `GetCodePointCount()`
- `GetGraphemeCount()`
- `CodePoint`

If you are debugging a `TextRun` range, a missing glyph, or a mismatch between visible text and string indices, start by checking whether you are reasoning in `char`, `CodePoint`, or grapheme units.

Practical guidance

Use grapheme indexes for user-visible ranges: styling, selection, caret movement, placeholder insertion, and rich text runs. That is the unit closest to what a person thinks of as one visible text element, even when it is made from multiple code points.

Use code points when the question is about Unicode scalar values: probing glyph availability, inspecting script coverage, or understanding encoded sequence length. Use UTF-16 indexes only when interoperating with raw .NET string storage or APIs that explicitly require `char` offsets.

Never assume visible characters, code points, and UTF-16 code units have the same count. That assumption is the root cause of most off-by-one text range bugs in emoji, combining marks, and complex scripts.

Fallback Fonts and Multilingual Text

Real text rarely stays inside one script or one font. User names, emoji, CJK text, math, and symbols all show up in the same application, so fallback is what turns a nice Latin-only demo into a text stack that survives real-world input.

Fonts handles that through `TextOptions.FallbackFontFamilies`.

When the primary `Font` does not contain a glyph for part of the text, the layout engine searches the fallback families in order and uses the first family that can supply the missing glyphs.

Use families, not fonts

Fallback is configured with `FontFamily` instances, not `Font` instances.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily latin = collection.Add("fonts/NotoSans-Regular.ttf");
FontFamily arabic = collection.Add("fonts/NotoSansArabic-Regular.ttf");
FontFamily emoji = collection.Add("fonts/NotoColorEmoji-Regular.ttf");

TextOptions options = new(latin.CreateFont(16))
{
    FallbackFontFamilies = [arabic, emoji]
};
```

The primary font still controls the default point size and layout options. When a fallback family is selected, Fonts creates the matching font instance for that run automatically.

Order matters

Fallback families are searched in the order you provide them.

- Put script-specific fonts before more general fallback fonts.
- Put emoji fonts after your normal text families unless you explicitly want them to win earlier.
- Keep the fallback list as small and intentional as possible so the selection stays predictable.

Resolver-based fallback

`FallbackFontFamilies` is a fixed list. `TextOptions.FontFallbackResolver` adds a dynamic last step behind it. Shaping consults the resolver only for code points that neither the primary font nor any fallback family can shape. It asks at most once per distinct unresolved code point per shaping operation. Leave the value `null` to render such code points as the missing-glyph outline.

`SystemFonts.FallbackResolver` is a built-in resolver that selects from the fonts installed on the current machine:

```
using SixLabors.Fonts;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    // Ask the operating system for a family when no configured font
    // can shape a code point.
    FontFallbackResolver = SystemFonts.FallbackResolver
};
```

Installed fonts differ between machines, so output rendered with the system resolver can differ too. Pin `FallbackFontFamilies` for output that must stay identical everywhere, and use the resolver as a safety net behind them.

Implement `IFontFallbackResolver` for a custom policy. The resolver receives the code point, the requested family, the style, and the culture. It returns the family that contains a glyph for the code point.

Mixed-script example

This pattern works well for text that mixes Latin, Arabic, and emoji:

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily latin = collection.Add("fonts/NotoSans-Regular.ttf");
FontFamily arabic = collection.Add("fonts/NotoSansArabic-Regular.ttf");
FontFamily emoji = collection.Add("fonts/NotoColorEmoji-Regular.ttf");

string text = "Status: ready 😀 مرحبا";

TextOptions options = new(latin.CreateFont(18))
{
    FallbackFontFamilies = [arabic, emoji],
    TextDirection = TextDirection.Auto,
```

```
    ColorFontSupport = ColorFontSupport.ColrV1 | ColorFontSupport.Svg
};
```

`TextDirection.Auto` lets the layout engine determine whether a run should flow left-to-right or right-to-left. `ColorFontSupport` matters when one of your fallback families is a color emoji font.

Fallback is not the same as explicit styling

Use fallback fonts when the goal is "use another family if the current one cannot render this text".

Use `TextRuns` when the goal is "this specific range should use a different font even if the base font could render it".

```
using SixLabors.Fonts;

const string text = "Latin title العربية";

FontCollection collection = new();
FontFamily latin = collection.Add("fonts/NotoSans-Regular.ttf");
FontFamily arabic = collection.Add("fonts/NotoSansArabic-Regular.ttf");

TextOptions options = new(latin.CreateFont(18))
{
    FallbackFontFamilies = [arabic],
    TextRuns =
    [
        new TextRun
        {
            Start = 12,
            End = 19,
            Font = arabic.CreateFont(18)
        }
    ]
};
```

The fallback list helps with missing glyphs. `TextRuns` gives you deliberate control over which grapheme ranges use which fonts.

Wrapping and script behavior

Multilingual text often benefits from layout settings beyond just fallback families:

- `TextDirection.Auto` for mixed LTR and RTL content
- `WordBreaking.KeepAll` or `WordBreaking.BreakWord` for CJK-heavy text
- `LayoutMode` for vertical scripts or mixed vertical presentation

If a script needs shaping support, make sure the selected font actually supports that script. Fallback can only help if one of the supplied families contains the needed glyphs and shaping data.

Common pitfalls

- A fallback family will not be used if the primary font already has a glyph for that Unicode scalar value, even if you would prefer the fallback font's design.
- `TextRuns` use grapheme indices, not UTF-16 code-unit indices.
- Emoji color layers are only used if `ColorFontSupport` allows them.
- Mixing many broad-coverage fonts can make fallback order hard to reason about.

If layout still looks wrong after fallback is configured, see Troubleshooting.

Practical guidance

- Put the preferred design family first, then add fallbacks in the order you want missing glyphs to be searched.
- Use `TextRuns` when a specific grapheme range must use a specific font rather than normal fallback.
- Validate fallback with real content, especially emoji, RTL text, CJK text, and combining marks.
- Remember that fallback solves missing glyphs; it does not guarantee matching style, metrics, or shaping behavior.

Variable Fonts

Variable fonts let one font file behave more like a design space than a single static face. Once that idea clicks, `FontVariation` becomes a practical way to ask for weight, width, slant, or optical-size variants without switching families.

Create a variable-font instance

Use `FontFamily.CreateFont(...)` with one or more `FontVariation` values.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/RobotoFlex.ttf");

Font font = family.CreateFont(
    16,
    new FontVariation(KnownVariationAxes.Weight, 700),
    new FontVariation(KnownVariationAxes.Width, 85),
    new FontVariation(KnownVariationAxes.OpticalSize, 16));
```

The tag must be exactly four characters. Common registered axis tags are available in `KnownVariationAxes`, but custom axes can also be addressed directly.

Use a prototype font

If you already have a base `Font`, you can derive a new instance from it.

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/RobotoFlex.ttf");

Font baseFont = family.CreateFont(16);
Font bolderFont = new(
    baseFont,
    new FontVariation(KnownVariationAxes.Weight, 700));
```

This is useful when you want to keep the same family, size, and requested style while changing only the variation coordinates.

Inspect supported axes

You can query the variable axes exposed by the current font through `FontMetrics.TryGetVariationAxes(...)`.

```
using System;
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic.Variations;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/RobotoFlex.ttf");
Font font = family.CreateFont(16);

if (font.FontMetrics.TryGetVariationAxes(out ReadOnlyMemory<VariationAxis> axes))
{
    foreach (VariationAxis axis in axes.Span)
    {
        Console.WriteLine($"{axis.Tag}: {axis.Min}..{axis.Max} (default {axis.Default})");
    }
}
```

Each `VariationAxis` exposes:

- `Name`
- `Tag`
- `Min`
- `Max`
- `Default`

That makes it possible to build UI controls or configuration validation based on the actual font rather than on hard-coded assumptions.

Registered and custom axes

`KnownVariationAxes` includes the registered tags most users expect:

- `Weight` (`wght`)
- `Width` (`wdth`)
- `OpticalSize` (`opsz`)
- `Italic` (`ital`)
- `Slant` (`slnt`)

Fonts also supports arbitrary four-character axis tags:

```
using SixLabors.Fonts;

FontCollection collection = new();
FontFamily family = collection.Add("fonts/SomeVariableFont.ttf");

Font font = family.CreateFont(
    16,
    new FontVariation("GRAD", 50),
    new FontVariation("XTRA", 420));
```

How values behave

`FontVariation` follows CSS `font-variation-settings` semantics. Variation values are clamped to the axis range defined by the font.

That means:

- valid tags must be four characters long
- out-of-range values are constrained by the font
- different fonts can expose different axis sets and ranges

The registered weight axis also has a higher-level entry point. Setting `TextOptions.FontWeight` instances a variable font at the requested weight through its `wght` axis, while preserving every other configured axis. See Text Layout and Options for the weight and style synthesis rules.

Non-variable fonts

Applying `FontVariation` values to a non-variable font is harmless but has no effect. If you need to know whether a font is actually variable, check `TryGetVariationAxes(...)` before building variation-driven UI or configuration.

When to use variable fonts

Variable fonts are especially useful when you want to:

- tune weight or width continuously instead of switching discrete files
- match optical size to the rendered point size
- reduce the number of separate font files you need to ship
- keep a single family while exploring many design-space instances

If you run into unexpected results, see Troubleshooting.

Practical guidance

- Inspect available axes before exposing variation controls.
- Store axis tags and values with the chosen font family so output can be reproduced.
- Use optical size intentionally; it is not just another scale factor.
- Fall back gracefully when a configured axis is missing from a replacement font.

Custom Rendering

> **NOTE**
>
> If you want to draw text onto images, ImageSharp.Drawing already provides the rendering layer for you. This page is for cases where you want to render glyphs to your own surface or extract geometry for another system.

Most developers meet Fonts through ImageSharp.Drawing, where the rendering surface is already handled for you. This page is for the next step down: when you want Fonts to do the shaping and glyph decomposition, but you want to decide how those glyphs are painted or exported.

Custom rendering in Fonts is built around `IGlyphRenderer`. `TextRenderer.RenderTo(...)` performs layout and shaping, then sends the result to your renderer as glyphs, layers, figures, and path commands.

When to use it

Custom rendering is useful when you want to:

- draw text into a game engine or UI toolkit
- export outlines to SVG, PDF, or another vector format
- capture glyph geometry for hit testing or diagnostics
- consume color-font layers and paints yourself

Rendering flow

For monochrome outline glyphs, the path callbacks are delivered inside `BeginGlyph(...)` / `EndGlyph()`:

1. `BeginText(...)`
2. `BeginGlyph(...)`
3. `BeginFigure()`, `MoveTo(...)`, `LineTo(...)`, `QuadraticBezierTo(...)`, `CubicBezierTo(...)`, `ArcTo(...)`, `EndFigure()`
4. `EndGlyph()`
5. `SetDecoration(...)` for any decorations requested by `EnabledDecorations()`
6. `EndText()`

Painted color glyphs add `BeginLayer(...)` / `EndLayer()` around each painted layer between `BeginGlyph(...)` and `EndGlyph()`. COLR v1 composite content adds `BeginGroup(...)` / `EndGroup()` around layers that must compose as one isolated unit before they blend onto the content below.

`BeginGlyph(...)` receives `GlyphRendererParameters`, which identify the glyph instance being rendered, including the glyph ID, the glyph's `CodePoint` value, font style, point size, DPI, layout mode, hinting mode, and active `TextRun`. Return `false` from `BeginGlyph(...)` if you want to skip rendering that glyph. The parameters include everything that changes glyph geometry, so a renderer can use them as a cache key.

A minimal renderer

```
using System.Collections.Generic;
using System.Numerics;
using SixLabors.Fonts;
using SixLabors.Fonts.Rendering;

public sealed class RecordingGlyphRenderer : IGlyphRenderer
{
    public List<Vector2> Points { get; } = new();

    public void BeginText(in FontRectangle bounds)
    {
    }

    public void EndText()
    {
    }

    public bool BeginGlyph(in FontRectangle bounds, in GlyphRendererParameters parameters)
=> true;

    public void EndGlyph()
    {
    }

    public void BeginLayer(Paint? paint, FillRule fillRule)
    {
    }

    public void EndLayer()
    {
    }
```

```
    public void BeginGroup(CompositeMode mode)
    {
    }

    public void EndGroup()
    {
    }

    public void BeginFigure()
    {
    }

    public void MoveTo(Vector2 point) => this.Points.Add(point);

    public void LineTo(Vector2 point) => this.Points.Add(point);

    public void QuadraticBezierTo(Vector2 secondControlPoint, Vector2 point)
    {
        this.Points.Add(secondControlPoint);
        this.Points.Add(point);
    }

    public void CubicBezierTo(Vector2 secondControlPoint, Vector2 thirdControlPoint,
Vector2 point)
    {
        this.Points.Add(secondControlPoint);
        this.Points.Add(thirdControlPoint);
        this.Points.Add(point);
    }

    public void ArcTo(float radiusX, float radiusY, float rotation, bool largeArc, bool
sweep, Vector2 point)
        => this.Points.Add(point);

    public void EndFigure()
    {
    }

    public TextDecorations EnabledDecorations() => TextDecorations.None;

    public void SetDecoration(TextDecorations textDecorations, Vector2 start, Vector2 end,
float thickness)
    {
    }
}
```

Render text to that surface with `TextRenderer`.

```
using SixLabors.Fonts;
using SixLabors.Fonts.Rendering;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    ColorFontSupport = ColorFontSupport.ColrV1 | ColorFontSupport.Svg
};

RecordingGlyphRenderer renderer = new();
TextRenderer.RenderTo(renderer, "Hello world", options);
```

Replace `"Segoe UI"` with any installed family that exists on your machine.

Layers, paints, and color fonts

`BeginLayer(...)` is where Fonts communicates how the current glyph layer should be filled:

- `paint` may be `null` when a painted layer does not specify paint information
- `SolidPaint` represents a single color
- `LinearGradientPaint`, `RadialGradientPaint`, and `SweepGradientPaint` are used for richer color-font layers
- `fillRule` tells you how the path should be filled
- `clipBounds` provides an optional clip quad for the layer

If your renderer only supports monochrome output, you can ignore `paint` when a painted layer is delivered and fill that layer with your own brush. If you want color-font output, honor both `ColorFontSupport` in `TextOptions` and the `Paint` information delivered to `BeginLayer(...)`.

See Color Fonts for a fuller guide to `ColorFontSupport`, painted glyphs, and the different color-font technologies that Fonts can surface.

Decorations

Decorations are opt-in. Return the decorations you care about from `EnabledDecorations()` and Fonts will call `SetDecoration(...)` after the glyph geometry has been emitted.

```
public TextDecorations EnabledDecorations()
    => TextDecorations.Underline | TextDecorations.Strikeout;
```

This makes it possible to render underline, overline, or strikeout using the same backend as the glyph outlines.

Practical guidance

- Implement only the renderer callbacks your backend can honor correctly.
- Keep layout in Fonts and rendering in your backend; do not recompute shaping inside the renderer.
- Honor layer and paint callbacks when color-font output matters.
- Use decoration callbacks instead of drawing underlines from guessed metrics.

Recipes

These pages are the quick-start side of the Fonts docs. They are meant for the moment when you know roughly what you want to do and would rather start from a short working example than read the full conceptual guide first.

Each recipe focuses on one common decision: sizing text to fit, choosing fonts, inspecting font assets, enabling OpenType features, or checking glyph coverage before rendering. Use them as starting points, then follow the linked conceptual articles when you need to understand layout coordinates, Unicode behavior, fallback, shaping, or custom rendering in more detail.

Text recipes are especially sensitive to hidden inputs. The font file, culture, DPI, fallback list, OpenType features, wrapping length, and text direction are all part of the layout. If a recipe measures text and your application later renders with different options, the result can be wider, taller, or shaped differently.

- Fit Text to a Target Width
- Inspect Font Files and Collections
- List System Fonts and Resolve by Culture
- Use OpenType Features for Numbers and Fractions
- Check Glyph Coverage Before Choosing Fallbacks

How to Adapt a Recipe

- Keep font loading separate from per-text measurement or rendering work when the same font is reused.
- Treat text indexes as grapheme-aware unless a page explicitly discusses code points or UTF-16 units.
- Use `TextOptions` for layout decisions such as origin, wrapping, alignment, DPI, culture, and fallback fonts.
- Use `TextBlock` when you need prepared layout, line inspection, hit testing, caret movement, or selection.

Practical Guidance

Text output is only stable when the font assets and layout inputs are stable. If output must match across developer machines, CI, containers, and production servers, ship the required fonts and load them into a private `FontCollection` instead of relying on system fonts. Use the same

`TextOptions` for measurement, hit testing, and rendering so the layout engine answers every question from the same contract.

Fallback and shaping should be tested with the actual content your product supports: localized strings, emoji sequences, RTL text, CJK text, combining marks, and OpenType features. Cache font collections and prepared text at boundaries where the font files and layout options are known not to change; stale layout state is worse than no cache because it can look correct for simple strings and fail on real content.

Use the conceptual guides when you need the bigger picture. Use these recipes when you want a practical starting point quickly.

Fit Text to a Target Width

Fitting text into a fixed width is one of those jobs that sounds simple until you decide how aggressively you want to shrink, wrap, or restyle it. This recipe covers the straightforward single-line measurement loop many apps start with.

For single-line text, the usual pattern is:

1. start with a candidate font size
2. measure with `TextMeasurer.MeasureAdvance(...)`
3. reduce the size until the width fits

```
using SixLabors.Fonts;

const string text = "SixLabors.Fonts";
const float targetWidth = 240;

FontFamily family = SystemFonts.Get("Segoe UI");
float fontSize = 32;
FontRectangle bounds = default;

while (fontSize > 6)
{
    Font font = family.CreateFont(fontSize, FontStyle.Bold);
    TextOptions options = new(font)
    {
        WrappingLength = -1
    };

    bounds = TextMeasurer.MeasureAdvance(text, options);
    if (bounds.Width <= targetWidth)
    {
        break;
    }

    fontSize -= 1;
}

Font fittedFont = family.CreateFont(fontSize, FontStyle.Bold);
```

This is a simple and predictable approach for titles and short labels. If you need more control, you can reduce in larger steps first and then refine more precisely near the final size.

For multiline text, also set `WrappingLength` and measure with the same layout options you plan to render with.

The important rule is that fitting and rendering must use the same layout inputs. Font family, style, size, DPI, culture, wrapping length, fallback fonts, OpenType features, and text direction can all affect measured advance. If any of those differ between the fitting pass and the final drawing pass, the text can still overflow or wrap differently.

For interactive systems, consider a two-stage search: probe coarse sizes first, then refine around the best candidate. That keeps the recipe easy to adapt without turning every label fit into a long linear measurement loop.

> **NOTE**
>
> This example is intentionally naive. It remeasures from scratch on each iteration to keep the recipe easy to follow. Production layout engines would usually cache measurements, font instances, or intermediate fit results instead of doing a full linear probe every time.

See Measuring Text and Text Layout and Options for the fuller discussion.

Practical guidance

- Fit with the same options you will use to render.
- Define a minimum readable size before starting the search.
- Use wrapping or truncation when shrinking would make the text unusable.
- Cache fit results when the same string, font family, and target width repeat often.

Inspect Font Files and Collections

This recipe is a good starting point when you have a font file in hand and want to learn what it contains before you add it to your app's normal font collection with `FontDescription`.

Read a single font file

```
using System.Globalization;
using SixLabors.Fonts;
using SixLabors.Fonts.WellKnownIds;

FontDescription description = FontDescription.LoadDescription("fonts/SourceSans3-
Regular.ttf");

string family = description.FontFamilyInvariantCulture;
string fullName = description.FontNameInvariantCulture;
string subfamily = description.FontSubFamilyNameInvariantCulture;
string version = description.GetNameById(CultureInfo.InvariantCulture,
KnownNameIds.Version);
```

This is useful for import tools, font pickers, diagnostics, and file-inspection utilities.

Use invariant names when you need stable storage, configuration, or logs. Use culture-specific names when presenting font choices to people, because many families expose localized names that are more useful in UI than the invariant English metadata.

Inspect a font collection such as a `.ttc`

```
using System;
using SixLabors.Fonts;

ReadOnlyMemory<FontDescription> descriptions =
    FontDescription.LoadFontCollectionDescriptions("fonts/NotoSansCJK-Regular.ttc");

foreach (FontDescription description in descriptions.Span)
{
    Console.WriteLine(description.FontNameInvariantCulture);
}
```

If you do want to load the collection afterward, use `FontCollection.AddCollection(...)`.

Inspection does not add the font to a collection. That separation is useful for upload validation and tooling: you can reject, categorize, or display font metadata before deciding whether the file should participate in normal font resolution.

For the broader metadata API, see Font Metadata and Inspection.

Practical guidance

- Inspect uploaded font files before adding them to an application collection.
- Use collection inspection for `.ttc` and `.otc` files because one file can contain multiple faces.
- Store invariant names for stable configuration and localized names for display.
- Load the font only after metadata inspection says it belongs in the normal resolution path.

List System Fonts and Resolve by Culture

This recipe is useful when you want a quick picture of what the current machine can actually provide through `SystemFonts`, whether for diagnostics, UI pickers, or culture-aware name resolution.

System fonts are environment-dependent. A font that exists on a developer workstation may be missing from a container, CI agent, Linux server, or customer machine. For predictable rendering, ship the fonts you require and load them into a private `FontCollection`. Use `SystemFonts` when the goal is to use what the host operating system already provides.

List installed families

```
using System;
using SixLabors.Fonts;

foreach (FontFamily family in SystemFonts.Families)
{
    Console.WriteLine(family.Name);
}
```

Show the searched directories

```
using System;
using SixLabors.Fonts;

foreach (string directory in SystemFonts.Collection.SearchDirectories)
{
    Console.WriteLine(directory);
}
```

Resolve a family by culture-aware name

```
using System.Globalization;
using SixLabors.Fonts;
```

```
CultureInfo japanese = CultureInfo.GetCultureInfo("ja-JP");

if (SystemFonts.TryGetByCulture("Yu Gothic", japanese, out FontFamily family))
{
    Font font = family.CreateFont(16);
}
```

This is especially useful when a family's localized name differs from the invariant name you would use elsewhere.

Culture-aware lookup is about names, not shaping. After you resolve a family, still use the correct `TextOptions.Culture`, fallback families, and layout settings for the text you are measuring or rendering.

For the fuller system-font API surface, see System Fonts.

Practical guidance

- Use system font enumeration for diagnostics, not for deterministic rendering guarantees.
- Log search directories when investigating missing fonts in production.
- Prefer private font collections for document generation, tests, and server-rendered assets.
- Treat culture-aware resolution as name lookup; shaping still depends on `TextOptions` and font support.

Use OpenType Features for Numbers and Fractions

This recipe shows the most common way people first encounter discretionary OpenType features: asking fonts through `TextOptions.FeatureTags` to align figures more neatly or substitute fraction glyphs for number-heavy text.

Align numeric columns with tabular figures

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FeatureTags = [KnownFeatureTags.TabularFigures]
};
```

This is useful for scoreboards, tables, counters, and any UI where digits should line up cleanly.

Request diagonal fractions

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FeatureTags = [KnownFeatureTags.Fractions]
};
```

This only has an effect if the font actually provides the requested feature.

Feature requests are not guaranteed substitutions. Fonts decide which features they expose and which scripts, languages, and glyph sequences those features apply to. If output must be exact, test with the production font files rather than assuming a tag will be honored everywhere.

Combine multiple features

```
using SixLabors.Fonts;
using SixLabors.Fonts.Tables.AdvancedTypographic;

Font font = SystemFonts.CreateFont("Segoe UI", 18);
TextOptions options = new(font)
{
    FeatureTags =
    [
        KnownFeatureTags.TabularFigures,
        KnownFeatureTags.OldstyleFigures,

        // 'ss01' is the first of OpenType's stylistic sets (ss01..ss20),
        // which a font can use to expose an alternate glyph design.
        Tag.Parse("ss01")
    ]
};
```

Use the same `TextOptions` for both `TextMeasurer` and `TextRenderer` so the measured result matches the rendered result.

OpenType features can change glyph choice, advance widths, ligature formation, and mark placement. That means they are layout inputs, not just visual decoration applied after measuring.

For the fuller feature model, see OpenType Features.

Practical guidance

- Verify that the production font actually exposes the requested feature tags.
- Use the same feature tags for measurement and rendering.
- Be careful combining mutually exclusive numeric features such as figure styles.
- Prefer `KnownFeatureTags` for standard features and `Tag.Parse(...)` for font-specific stylistic sets.

Check Glyph Coverage Before Choosing Fallbacks

Before you wire up fallback families, it helps to know what your primary font can already cover. This recipe shows a quick way to probe individual scalar values with `Font.TryGetGlyphs(...)` or scan a string so you can make fallback decisions based on actual glyph coverage instead of guesswork.

Check individual code points

```
using SixLabors.Fonts;
using SixLabors.Fonts.Unicode;

Font font = SystemFonts.CreateFont("Segoe UI", 16);

bool hasLatinA = font.TryGetGlyphs(new CodePoint('A'), out _);
bool hasOmega = font.TryGetGlyphs(new CodePoint(0x03A9), out _); // Ω GREEK CAPITAL
LETTER OMEGA
bool hasEmoji = font.TryGetGlyphs(new CodePoint(0x1F600), out _); // 😀 GRINNING FACE
```

Scan a whole string for missing glyphs

```
using System.Collections.Generic;
using SixLabors.Fonts;
using SixLabors.Fonts.Unicode;

string text = "Hello 123 مرحبا 😀";
Font font = SystemFonts.CreateFont("Segoe UI", 16);
List<CodePoint> missing = new();

foreach (CodePoint codePoint in text.AsSpan().EnumerateCodePoints())
{
    if (!font.TryGetGlyphs(codePoint, out _))
    {
        missing.Add(codePoint);
    }
}
```

This is a simple way to decide whether you need `FallbackFontFamilies` before you measure or render the text.

If you want a broader face-level view instead of checking a specific string, use `Font.FontMetrics.GetAvailableCodePoints()`.

Glyph coverage is only the first question. A font can contain glyphs for individual code points but still lack the shaping behavior, marks, variation sequences, or color glyph data needed for the text to look right in a real script. Use coverage checks to choose candidate fallback families, then measure or render with the same `TextOptions` you will use in production.

Emoji and complex scripts are the usual cases where this distinction matters. A visible emoji can be a grapheme made from several code points, and Arabic, Indic, or Southeast Asian scripts can require shaping features that are not captured by a one-code-point probe.

For the conceptual fallback guidance, see Fallback Fonts and Multilingual Text. For face-level coverage inspection, see Font Metrics.

Practical guidance

- Use coverage checks to choose fallback candidates, not to prove final rendered quality.
- Test grapheme clusters such as emoji sequences as whole strings with production layout options.
- Prefer face-level coverage inspection when building diagnostics or font picker tooling.
- Keep fallback order intentional so broad-coverage fonts do not hide preferred design choices.

Troubleshooting

When text does not measure or render the way you expect, the underlying cause is usually one of a few things: family resolution, font-file validity, fallback, shaping assumptions, or misunderstanding the different measurement APIs. This page starts with those common failure modes.

A font family cannot be found

If `Get(...)` or `SystemFonts.CreateFont(...)` fails, you may see `FontFamilyNotFoundException`.

Typical causes:

- the family name does not match the font's actual family name
- the font was never added to your `FontCollection`
- you are relying on a system font that is not installed on the current machine
- you loaded the font with a culture-specific family name and are resolving it with a different culture

Safer patterns are:

- use `TryGet(...)` instead of `Get(...)` when probing
- inspect `FontDescription` after loading a file
- prefer application-owned font files over machine-specific `SystemFonts` when portability matters

A font file loads poorly or throws

Invalid or unsupported font data can surface as:

- `InvalidFontFileException`
- `InvalidFontTableException`
- `MissingFontTableException`

If you hit one of these:

- verify the file is a real font and not an incomplete download
- prefer loading from a stable local file or stream
- if the font is a collection, use `AddCollection(...)` or `LoadFontCollectionDescriptions(...)`

Text renders with missing glyphs

If some characters do not render as expected:

- make sure the selected font actually contains the script you need
- add script-specific families to `FallbackFontFamilies`
- enable color-font support if the missing content is emoji
- use `TryGetGlyphs(...)` when you need to probe a specific `CodePoint` value directly

Fallback can only help if one of the supplied families actually contains the required glyphs.

Fallback fonts are not being used

The most common reason is that the primary font already contains a glyph for that Unicode scalar value, so fallback never activates.

If you want a specific range to use a different font even when the primary font could render it, use `TextRuns` instead of relying on fallback.

Fallback order also matters. Fonts searches `FallbackFontFamilies` in order and uses the first suitable family it finds.

RTL or complex-script text looks wrong

Check these first:

- use a font that actually supports the script
- set `TextDirection` to `TextDirection.Auto` or explicitly choose the correct direction
- avoid assuming simple one-character-per-glyph behavior
- verify your fallback families cover the script, not just isolated characters

Arabic, Indic, Thai, Hebrew, and similar scripts depend on shaping, not just raw Unicode coverage.

Measurements look larger or smaller than expected

This is usually a measurement-choice issue:

- `MeasureAdvance(...)` is the logical layout box
- `MeasureBounds(...)` is pure glyph ink bounds

- `MeasureRenderableBounds(...)` is the union of the two

It is normal for these values to differ. Italics, accents, and decorative forms often extend outside the advance box, while line height can add space that no glyph pixels occupy.

Text run indices look wrong

`TextRun.Start` and `TextRun.End` are grapheme indices, not UTF-16 code-unit indices.

That matters for:

- emoji
- combining marks
- ligatures
- many non-Latin scripts

If a text run seems offset or slices the wrong part of the string, re-check the range in grapheme terms.

See Unicode, Code Points, and Graphemes for the distinction between raw `char` positions, `CodePoint` values, and grapheme indices.

Variable font changes do nothing

Usually one of these is true:

- the font is not actually variable
- the axis tag is wrong
- the value is outside the font's supported range

Use `font.FontMetrics.TryGetVariationAxes(...)` to inspect the actual axes and ranges exposed by the font. `FontVariation` tags must be exactly four characters.

System font behavior differs by machine

`SystemFonts` is convenient, but it is not deterministic across environments. Different machines can have different installed families, versions, and script coverage.

If you need repeatable output across CI, servers, containers, and user machines, ship your own fonts and load them through `FontCollection`.

Debugging checklist

- Confirm the font file or system family is actually available in the current environment.
- Confirm measurement and rendering use the same `TextOptions`.
- Check whether indexes are grapheme indexes, code-point indexes, or UTF-16 indexes.
- Inspect fallback coverage before assuming a missing glyph is a renderer problem.

PolygonClipper

PolygonClipper is Six Labors' high-performance focused geometry library for polygon boolean operations, contour normalization, and stroke-outline generation in managed .NET. It is designed for real 2D geometry workloads: non-convex shapes, holes, multiple contours, overlapping edges, and inputs that need canonicalized output.

The current package targets .NET 8. If you already use ImageSharp.Drawing, you may already be relying on PolygonClipper indirectly: ImageSharp.Drawing uses it internally for boolean operations against paths and for stroke geometry generation.

Under the hood, the boolean-operation pipeline is based on a Martinez-Rueda sweep-line approach for complex polygon clipping, while normalization uses a separate Vatti/Clipper2-inspired cleanup path for resolving self-intersections and overlaps into positive-winding output. You do not need to understand those algorithms to use the library well, but it helps explain why PolygonClipper is comfortable with complex contour topology.

The library works with geometry, not pixels. Coordinates are numeric vertices, contours are rings, and polygons are collections of rings with explicit hierarchy for holes. That makes PolygonClipper useful before rendering, export, hit testing, path cleanup, or any workflow where you need the region itself rather than a raster mask.

Most users should begin with the static boolean, normalization, and stroking entry points. They take ordinary polygon inputs and return new polygon geometry, so the result can be inspected, transformed, rendered, serialized, or handed to another geometry pipeline.

License

PolygonClipper is licensed under the terms of the Six Labors Split License, Version 1.0. See https://sixlabors.com/pricing for commercial licensing details.

> **IMPORTANT**
>
> Starting with PolygonClipper 1.0.0, projects that directly depend on SixLabors.PolygonClipper require a valid Six Labors license at build time. This enforcement applies to direct dependencies only. See License Enforcement Changes and a New Subscription Tier for details.

Installation

PolygonClipper is installed via NuGet with nightly builds available on Feedz.

Package Manager | .NET CLI | PackageReference | Paket CLI

```
PM > Install-Package SixLabors.PolygonClipper -Version VERSION_NUMBER
```

Prerelease versions installed via the Visual Studio NuGet Package Manager require the "include prerelease" checkbox to be checked.

How to use the license file

By default, the build searches from each project directory for `sixlabors.lic`. Place the supplied file in the directory that contains the project file, or in a subdirectory below it. Use the file as supplied; it already contains the complete license string required by the build.

> **IMPORTANT**
>
> Do not commit `sixlabors.lic` or a license key to public repositories such as open source projects. Use environment variables or repository secrets instead, and let contributors apply for their own independent keys at https://licensing.sixlabors.com/.

If you want to keep the file somewhere else, including a repository root that sits above the project directory, set `SixLaborsLicenseFile` in your project file or a shared props file:

```
<PropertyGroup>
  <SixLaborsLicenseFile>path/to/sixlabors.lic</SixLaborsLicenseFile>
</PropertyGroup>
```

If you do not want to store the license on disk, pass the license string directly from an environment variable or secret store. When extracting the value from `sixlabors.lic`, use the full file contents, not only the `Key` field:

```
<PropertyGroup>
  <SixLaborsLicenseKey>$(SIXLABORS_LICENSE_KEY)</SixLaborsLicenseKey>
</PropertyGroup>
```

You can also pass the key to common .NET CLI commands.

PowerShell:

```
dotnet build -p:SixLaborsLicenseKey="$env:SIXLABORS_LICENSE_KEY"
dotnet publish -p:SixLaborsLicenseKey="$env:SIXLABORS_LICENSE_KEY"
```

Bash and other shells that expand environment variables with `$NAME`:

```
dotnet build -p:SixLaborsLicenseKey="$SIXLABORS_LICENSE_KEY"
dotnet publish -p:SixLaborsLicenseKey="$SIXLABORS_LICENSE_KEY"
```

Build as normal after the file or property is configured. If the license is missing or invalid, the build fails with a clear error. You do not need to reference the licensing package directly; it is carried by Six Labors libraries.

Start Here

- Getting Started walks through building a polygon from contours and vertices, then running a first boolean operation.
- Polygons, Contours, and Holes explains the library's core data model and how hierarchy is represented.
- Boolean Operations covers `Intersection`, `Union`, `Difference`, and `Xor`, including subject-versus-clip semantics.
- Normalization and Winding explains when to use `Normalize(...)` to resolve self-intersections and overlaps into positive-winding output.
- Stroking covers `PolygonStroker`, `StrokeOptions`, joins, caps, and open-versus-closed path behavior.

How to Use These Docs

- Start with contours and polygons before choosing a boolean or stroking operation.
- Use boolean operations when combining two regions.
- Use normalization when cleaning one messy region.
- Use stroking when a line or path needs to become filled outline geometry.

Getting Started

The fastest way to get comfortable with PolygonClipper is to think in terms of three building blocks:

- a `Vertex` is one 2D point;
- a `Contour` is one ring of vertices;
- a `Polygon` is a collection of contours.

From there, most applications either run a boolean operation with `PolygonClipper` or generate stroke-outline geometry with `PolygonStroker`.

PolygonClipper does not attach units or coordinate-system meaning to vertices. Your application decides whether a vertex represents pixels, points, millimeters, tiles, or world coordinates. The important part is to use one consistent coordinate space for all inputs to a single operation.

Build Two Input Polygons

This example creates two rectangles, then intersects them:

```
using System;
using SixLabors.PolygonClipper;

static Contour Rectangle(double x, double y, double width, double height)
{
    Contour contour = new(4);
    contour.Add(new Vertex(x, y));
    contour.Add(new Vertex(x + width, y));
    contour.Add(new Vertex(x + width, y + height));
    contour.Add(new Vertex(x, y + height));
    return contour;
}

Polygon subject = new();
subject.Add(Rectangle(0, 0, 80, 60));

Polygon clip = new();
clip.Add(Rectangle(40, 20, 80, 60));

Polygon result = PolygonClipper.Intersection(subject, clip);
```

```
Console.WriteLine($"Contours: {result.Count}");
Console.WriteLine($"Vertices: {result.VertexCount}");
```

You do not need to repeat the first vertex at the end of a contour for normal polygon operations. Contours are treated as implicitly closed.

The example builds contours clockwise, but real inputs often arrive from drawing tools, path importers, or GIS-style data with mixed orientation. Use Normalization and Winding when you need to turn messy single-polygon input into clean positive-winding output before a downstream system consumes it.

Prefer the Static Entry Points

Most applications should call the static methods:

- `PolygonClipper.Intersection(...)`
- `PolygonClipper.Union(...)`
- `PolygonClipper.Difference(...)`
- `PolygonClipper.Xor(...)`
- `PolygonClipper.Normalize(...)`
- `PolygonStroker.Stroke(...)`

Those are the recommended entry points in the source and route work through internal reusable instances. The instance constructors are there for advanced manual flows, but they are not the usual starting point.

Inspect the Result

Returned polygons can contain multiple contours, including holes:

```
for (int i = 0; i < result.Count; i++)
{
    Contour contour = result[i];

    Console.WriteLine(
        $"Contour {i}: Count={contour.Count}, Parent={contour.ParentIndex}, Depth=
{contour.Depth}, Holes={contour.HoleCount}");
}
```

That contour hierarchy is one of the main things PolygonClipper preserves for you. If you want to understand how parent contours, holes, and winding fit together, the next page to read is

Polygons, Contours, and Holes.

Do not assume that one operation returns one contour. Intersections can split a region into multiple islands, differences can create holes, and normalization can reorganize self-intersecting input. Production callers should usually iterate the returned polygon rather than indexing directly into the first contour.

Practical Guidance

- Keep all vertices in one coordinate system for a given operation.
- Do not repeat the first vertex at the end of ordinary boolean-operation contours.
- Prefer the static entry points unless you are building an advanced manual flow.
- Iterate every returned contour and inspect hierarchy when exporting or rendering the result.

Polygons, Contours, and Holes

PolygonClipper's public model is intentionally small. Most of the time you only need to understand three types:

- `Vertex` for 2D coordinates and basic vector math;
- `Contour` for one ring of vertices;
- `Polygon` for a collection of contours.

That small model is enough to describe simple shapes, complex multi-contour shapes, and polygons with holes.

The important mental model is that contours are topology, not styling. PolygonClipper does not know about brushes, pens, fill colors, or pixels. It returns geometry that another layer can render, serialize, hit-test, or combine with more geometry.

Regions, Not Drawing Commands

PolygonClipper treats polygons as filled regions bounded by contours. It is not a path recorder and it does not preserve the original drawing commands that produced the contour data. After normalization, clipping, or boolean operations, the output may contain different contours because the library is describing the resulting region, not the editing history.

This is the same distinction that matters in rendering systems: a path is a set of geometric edges, while a filled region is the area those edges enclose under a fill rule. PolygonClipper operates on the region model.

A `Contour` Is One Ring

A `Contour` is a sequence of vertices. For clipping and normalization, it is treated as implicitly closed, so the library always considers an edge between the last vertex and the first vertex.

That means this is a complete rectangle contour:

```
using SixLabors.PolygonClipper;

Contour contour = new(4);
contour.Add(new Vertex(0, 0));
contour.Add(new Vertex(80, 0));
```

```
contour.Add(new Vertex(80, 60));
contour.Add(new Vertex(0, 60));
```

There is no need to append `(0, 0)` again at the end unless you are deliberately feeding the stroker a contour you want treated as explicitly closed.

Avoid duplicate closing vertices in boolean inputs unless your data source naturally includes them and you have chosen to preserve them. Repeating the first vertex usually adds no information for region operations.

A `Polygon` Is a Collection of Contours

A `Polygon` is simply a list of contours:

```
using SixLabors.PolygonClipper;

Polygon polygon = new();
polygon.Add(contour);
```

That is enough for a single simple region. As soon as you need holes or multiple disjoint regions, you add more contours.

Hole Hierarchy Is Represented Explicitly

Contours can participate in a parent-child hierarchy:

- `ParentIndex` points to the owning contour when a contour is a hole or nested child;
- `HoleCount` and `GetHoleIndex(...)` let an outer contour enumerate its direct holes;
- `Depth` records how deeply nested the contour is;
- `IsExternal` is `true` when `ParentIndex` is `null`.

If you already know the hierarchy of your input data, you can represent it directly:

```
using SixLabors.PolygonClipper;

Polygon polygon = new(2);

Contour outer = new(4);
outer.Add(new Vertex(0, 0));
outer.Add(new Vertex(100, 0));
outer.Add(new Vertex(100, 100));
outer.Add(new Vertex(0, 100));
```

```
Contour hole = new(4);
hole.Add(new Vertex(25, 25));
hole.Add(new Vertex(75, 25));
hole.Add(new Vertex(75, 75));
hole.Add(new Vertex(25, 75));

polygon.Add(outer);
polygon.Add(hole);

hole.ParentIndex = 0;
hole.Depth = 1;
outer.AddHoleIndex(1);
```

When you do not already know the hierarchy, boolean operations and normalization will compute it for the returned polygon.

If you construct hierarchy yourself, keep `ParentIndex`, `Depth`, and hole indexes consistent. Those values are part of how consumers understand which contours are exterior regions and which contours subtract from a parent region.

Orientation Helpers

`Contour` also exposes orientation helpers:

- `IsCounterClockwise()`
- `IsClockwise()`
- `Reverse()`
- `SetClockwise()`
- `SetCounterClockwise()`

Those are useful when you are inspecting or preparing contours, but you do not need to normalize orientation by hand for every workflow. If your real goal is to resolve messy self-overlapping input into canonical positive-winding output, use `PolygonClipper.Normalize(...)`.

Bounding Boxes and Translation

Both polygons and contours can answer a few practical geometry questions without running a boolean operation:

- `GetBoundingBox()` returns a `Box2`
- `Translate(x, y)` offsets the geometry in place

```
using SixLabors.PolygonClipper;

Box2 bounds = polygon.GetBoundingBox();
polygon.Translate(10, 20);
```

Those helpers are especially useful when you are staging input, culling broad regions, or preparing geometry for a later clip or stroke pass.

Practical Guidance

- Store source geometry in one consistent coordinate space before clipping.
- Treat returned polygons as region results, not as a promise to preserve input contour order.
- Inspect `Depth` and `ParentIndex` when exporting to formats that need exterior and hole rings separately.
- Use bounding boxes for broad-phase rejection before expensive geometry work when you have many polygons.

Boolean Operations

Boolean operations are the center of PolygonClipper. They let you combine or subtract polygon regions without dropping down into segment-level geometry code.

The public entry points are the static methods on `PolygonClipper`:

- `Intersection(subject, clip)`
- `Union(subject, clip)`
- `Difference(subject, clip)`
- `Xor(subject, clip)`

These are also the recommended entry points in the source, because they route work through internal reusable instances.

Boolean operations answer region questions. They do not preserve the input contour list as a drawing history. After an operation, the returned polygon describes the resulting filled area, and that may require new contours, fewer contours, or a different hierarchy.

Choose the Right Operation

The four operations have different semantics:

- `Intersection` keeps only the area shared by both inputs.
- `Union` keeps the area covered by either input.
- `Difference` subtracts the clip polygon from the subject polygon.
- `Xor` keeps the non-overlapping parts of both inputs and removes the shared overlap.

If `Xor` is not a familiar term, it helps to think of it as "union, but with the overlapping middle cut away."

A few quick cases make the behavior easier to picture:

- if the two polygons do not touch, `Xor` gives the same result as `Union`;
- if the two polygons are identical, `Xor` returns an empty result;
- if one polygon sits inside the other, `Xor` keeps the outer region and removes the shared inner region.

`Difference` is the one where argument order matters most. `Difference(a, b)` is not the same as `Difference(b, a)`.

Run a Boolean Operation

```
using SixLabors.PolygonClipper;

Polygon result = PolygonClipper.Union(subject, clip);
Polygon overlap = PolygonClipper.Intersection(subject, clip);
Polygon remaining = PolygonClipper.Difference(subject, clip);
Polygon exclusive = PolygonClipper.Xor(subject, clip);
```

The returned `Polygon` may contain:

- more than one contour;
- hole relationships;
- different contour counts than either input.

That is normal. Boolean operations work with regions, not one-contour-in one-contour-out assumptions.

Subject and Clip Inputs

Both input polygons can contain:

- multiple contours;
- holes;
- disjoint islands;
- non-convex shapes.

That is one of the main reasons to use PolygonClipper instead of writing one-off rectangle or convex-shape code.

Implementation Note

PolygonClipper's boolean operations are built on a Martinez-Rueda sweep-line pipeline. For most users, the practical takeaway is simply that the library is designed for complex polygon inputs rather than only simple convex cases.

Inspect Returned Hierarchy

The result can include parent-child contour relationships:

```
for (int i = 0; i < result.Count; i++)
{
    Contour contour = result[i];

    Console.WriteLine(
        $"Contour {i}: Parent={contour.ParentIndex}, Depth={contour.Depth},
Holes={contour.HoleCount}");
}
```

If you care about preserving hole structure or exporting contours to another renderer, inspect that hierarchy instead of assuming every returned contour is a top-level exterior ring.

Practical Guidance

Boolean operations combine regions. They are not a promise to preserve input contour order, input contour count, or drawing history. The result may contain several disjoint islands, holes, or a hierarchy that neither input had in exactly the same form. Production code should iterate the returned polygon and inspect hierarchy rather than assuming the first contour is the only contour that matters.

Keep both inputs in the same coordinate system and units. `Difference(subject, clip)` is especially sensitive to naming because argument order changes the result: the clip is removed from the subject, not the other way around. Clear variable names make call sites much easier to audit.

Use normalization when the problem is "clean this one messy region" rather than "combine these two regions." Imported or user-authored self-overlapping geometry is often easier to reason about after normalization, but there is no need to normalize every polygon before every boolean operation. Preserve hierarchy metadata when exporting to renderers or file formats that distinguish exterior rings from holes.

Used by ImageSharp.Drawing

If you use ImageSharp.Drawing, this part of PolygonClipper may already be in your rendering pipeline. ImageSharp.Drawing converts path geometry into PolygonClipper polygons and uses these boolean operations internally when combining clipped path regions.

That makes PolygonClipper a good fit both as a standalone geometry library and as the lower-level model behind higher-level drawing systems.

Normalization and Winding

Boolean operations combine two polygons. Normalization is different: it cleans up one polygon by resolving self-intersections and overlaps into a canonical positive-winding result.

That makes `PolygonClipper.Normalize(...)` the right tool when your input geometry is already yours, but its contours are messy enough that you want a cleaner region description before export, rendering, or further processing.

Think of normalization as converting drawn or imported edges into filled-region geometry. It is not a visual simplifier and it is not a general-purpose path optimizer. The result describes the same filled area using contours that downstream geometry code can reason about more consistently.

When to Use `Normalize(...)`

Normalization is useful when:

- a contour self-intersects;
- multiple contours overlap and you want one clean regional result;
- you want positive-winding output for a downstream system that depends on winding semantics;
- you want hierarchy and overlap resolution without performing a two-input boolean operation.

Normalize a Self-Intersecting Input

```
using SixLabors.PolygonClipper;

Contour contour = new(4);
contour.Add(new Vertex(0, 0));
contour.Add(new Vertex(80, 80));
contour.Add(new Vertex(0, 80));
contour.Add(new Vertex(80, 0));

Polygon input = new();
input.Add(contour);

Polygon normalized = PolygonClipper.Normalize(input);
```

The output may have a different contour count and different contour hierarchy than the input. That is expected. Normalization is free to split or reorganize the input region as needed to produce clean positive-winding output.

That also means normalization should happen at a clear boundary in your pipeline. Normalize imported or user-authored geometry before you cache, export, or combine it with other trusted geometry. Avoid normalizing repeatedly after every small edit unless your application specifically needs canonical output at each step.

Positive Winding Matters

The source describes normalization in terms of positive fill semantics. In practice, that means the result is intended for consumers that care about winding-consistent filled regions rather than raw overlapping edges.

This is especially useful when you are moving polygon data into a renderer, exporter, or geometry pipeline that expects contours to describe filled regions cleanly.

Positive winding does not mean every original contour keeps its original orientation. It means the returned polygon is organized around positive filled-region semantics after overlaps and self-intersections have been resolved.

Implementation Note

Normalization is a separate pipeline from the two-input boolean operations. In PolygonClipper it follows a Vatti/Clipper2-inspired approach focused on turning overlapping or self-intersecting contour input into a canonical positive-winding result.

Normalization Is Not Required for Every Workflow

You do not need to call `Normalize(...)` before every boolean operation. The boolean APIs already process complex polygon inputs.

Reach for normalization when your goal is specifically:

- cleaning up one polygon rather than combining two;
- resolving self-overlap into a canonical result;
- preparing output for systems that rely on positive-winding contour semantics.

Practical Guidance

- Normalize at import, export, or cache boundaries rather than after every small edit.
- Use normalization for one messy polygon; use boolean operations for combining two regions.
- Expect contour count and hierarchy to change when self-intersections are resolved.
- Preserve positive-winding semantics when passing output to renderers or geometry systems that depend on winding.

Stroking

Stroking in PolygonClipper means turning a path-like input into filled outline geometry. Instead of drawing centerlines directly, `PolygonStroker` emits a polygon that represents the area the stroke would cover.

That makes it useful both for standalone geometry workflows and for renderers that want stroke outlines as polygons.

Use the Static Entry Point

Most callers should use the static `PolygonStroker.Stroke(...)` method:

```
using SixLabors.PolygonClipper;

Polygon outline = PolygonStroker.Stroke(input, width: 12);
```

Like the boolean-operation APIs, this is the recommended entry point in the source and uses internal reusable instances.

Stroke an Open Polyline

```
using SixLabors.PolygonClipper;

Contour polyline = new();
polyline.Add(new Vertex(0, 0));
polyline.Add(new Vertex(60, 20));
polyline.Add(new Vertex(120, 0));

Polygon source = new();
source.Add(polyline);

Polygon outline = PolygonStroker.Stroke(source, 12);
```

In this case the contour is treated as open, so the emitted geometry includes end caps.

Control Joins, Caps, and Output Cleanup

`StrokeOptions` lets you control the shape of the generated outline:

```
using SixLabors.PolygonClipper;

StrokeOptions options = new()
{
    LineJoin = LineJoin.Round,
    LineCap = LineCap.Round,
    MiterLimit = 4,
    ArcDetailScale = 1,
    NormalizeOutput = true
};

Polygon outline = PolygonStroker.Stroke(source, 12, options);
```

The main knobs are:

- `LineJoin` for outer corners;
- `LineCap` for open-path ends;
- `MiterLimit` for clamping long outer miters;
- `ArcDetailScale` for the smoothness-versus-vertex-count tradeoff on round joins and caps;
- `NormalizeOutput` when you want overlaps and self-intersections in the emitted stroke geometry resolved before returning.

`NormalizeOutput` defaults to `false` for throughput. When you leave it off, render the returned geometry with a non-zero winding fill rule.

Open Versus Closed Stroke Input

For stroking, PolygonClipper distinguishes between contours that should behave like open polylines and contours that should behave like closed loops.

If the last vertex returns to the first vertex, or is extremely close to it, the stroker treats the contour as closed and does not emit end caps. Otherwise it treats the contour as open and emits caps.

That means these two inputs are interpreted differently:

- a contour whose endpoints are clearly different behaves like an open path;
- a contour whose last vertex returns to its first behaves like a closed path.

Width Semantics

Most callers use a positive width:

```
Polygon outline = PolygonStroker.Stroke(source, 8);
```

Negative widths are supported for advanced scenarios. They flip the emitted side orientation while preserving the width magnitude.

Used by ImageSharp.Drawing

ImageSharp.Drawing also uses PolygonClipper for stroke geometry generation. Its higher-level stroke options are mapped down to PolygonClipper's `LineJoin`, `LineCap`, miter, and normalization settings before outline polygons are generated.

Practical Guidance

Use stroking when a path or polyline needs to become filled outline geometry. The result is a polygon region, not a rendering command, so it can be inspected, clipped, exported, or rendered by another system.

Decide whether the input is open or closed before choosing caps. Open inputs emit end caps; closed inputs do not. Stroke width is expressed in the same coordinate units as the source geometry, so scaling source coordinates without scaling stroke width changes the visual result.

Join, cap, miter, and cleanup options should match the renderer or exporter that will consume the outline. Inspect the returned polygon as geometry: complex strokes can produce multiple contours and holes, especially around self-overlap, sharp joins, or closed paths.